



US012386617B2

(12) **United States Patent**
Pal et al.

(10) **Patent No.:** **US 12,386,617 B2**

(45) **Date of Patent:** **Aug. 12, 2025**

(54) **GATHERING PAYLOAD FROM ARBITRARY
REGISTERS FOR SEND MESSAGES IN A
GRAPHICS ENVIRONMENT**

(58) **Field of Classification Search**

CPC .. G06F 9/30036; G06F 9/3818; G06F 9/3854;
G06F 9/3858; G06F 9/3887; G06F
9/3888

See application file for complete search history.

(71) Applicant: **Intel Corporation**, Santa Clara, CA
(US)

(56) **References Cited**

U.S. PATENT DOCUMENTS

2005/0053012 A1* 3/2005 Moyer G06F 9/30109
370/254
2008/0059760 A1* 3/2008 Sachs G06F 9/3012
712/7

(Continued)

FOREIGN PATENT DOCUMENTS

DE 102022119733 A1 3/2023

Primary Examiner — Jacob Petranek

(74) *Attorney, Agent, or Firm* — JAFFERY WATSON
HAMILTON & DESANCTIS LLP

(72) Inventors: **Supratim Pal**, Folsom, CA (US);
Chandra Gurram, Folsom, CA (US);
Fan-Yin Tzeng, Fremont, CA (US);
Subramaniam Maiyuran, Gold River,
CA (US); **Guei-Yuan Lueh**, San Jose,
CA (US); **Timothy R. Bauer**,
Hillsboro, CA (US); **Vikranth**
Vemulapalli, Folsom, CA (US);
Wei-Yu Chen, San Jose, CA (US)

(73) Assignee: **INTEL CORPORATION**, Santa Clara,
CA (US)

(*) Notice: Subject to any disclaimer, the term of this
patent is extended or adjusted under 35
U.S.C. 154(b) by 813 days.

(21) Appl. No.: **17/481,448**

(22) Filed: **Sep. 22, 2021**

(65) **Prior Publication Data**

US 2023/0088743 A1 Mar. 23, 2023

(51) **Int. Cl.**
G06F 9/30 (2018.01)
G06F 9/38 (2018.01)
(Continued)

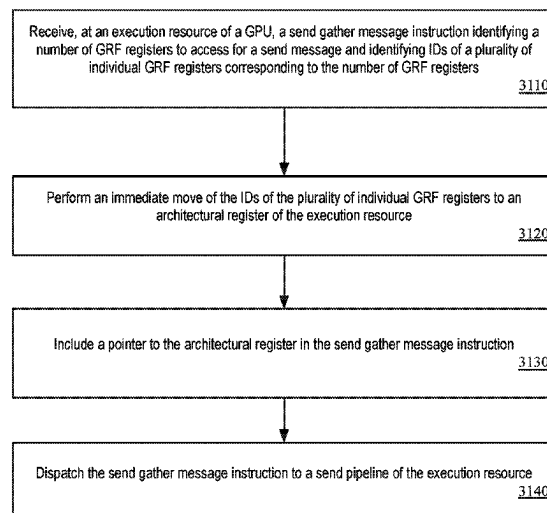
(52) **U.S. Cl.**
CPC **G06F 9/3012** (2013.01); **G06F 9/30036**
(2013.01); **G06F 9/30105** (2013.01);
(Continued)

(57) **ABSTRACT**

An apparatus to facilitate gathering payload from arbitrary registers for send messages in a graphics environment is disclosed. The apparatus includes processing resources comprising execution circuitry to receive a send gather message instruction identifying a number of registers to access for a send message and identifying IDs of a plurality of individual registers corresponding to the number of registers; decode a first phase of the send gather message instruction; based on decoding the first phase, cause a second phase of the send gather message instruction to bypass an instruction decode stage; and dispatch the first phase subsequently followed by dispatch of the second phase to a send pipeline. The apparatus can also perform an immediate move of the IDs of the plurality of individual registers to an architectural register of the execution circuitry and include a pointer to the architectural register in the send gather message instruction.

20 Claims, 51 Drawing Sheets

3100



- (51) **Int. Cl.**
G06F 12/02 (2006.01)
G06T 1/20 (2006.01)
- (52) **U.S. Cl.**
CPC **G06F 9/3826** (2013.01); **G06F 9/3851**
(2013.01); **G06F 9/3854** (2023.08); **G06F**
9/3858 (2023.08); **G06F 9/3887** (2013.01);
G06F 9/3888 (2023.08); **G06F 9/38885**
(2023.08); **G06F 12/0223** (2013.01); **G06T**
1/20 (2013.01)

(56) **References Cited**

U.S. PATENT DOCUMENTS

2008/0222392 A1 * 9/2008 Fuchs G06F 9/3828
712/205
2012/0254588 A1 * 10/2012 Adrian G06F 9/30018
712/E9.021
2013/0212353 A1 * 8/2013 Mimar G06F 9/30036
712/4
2014/0317382 A1 * 10/2014 Segelken G06F 9/30174
712/205
2018/0165210 A1 * 6/2018 Sethuraman G06F 12/0875
2023/0088743 A1 3/2023 Pal et al.

* cited by examiner

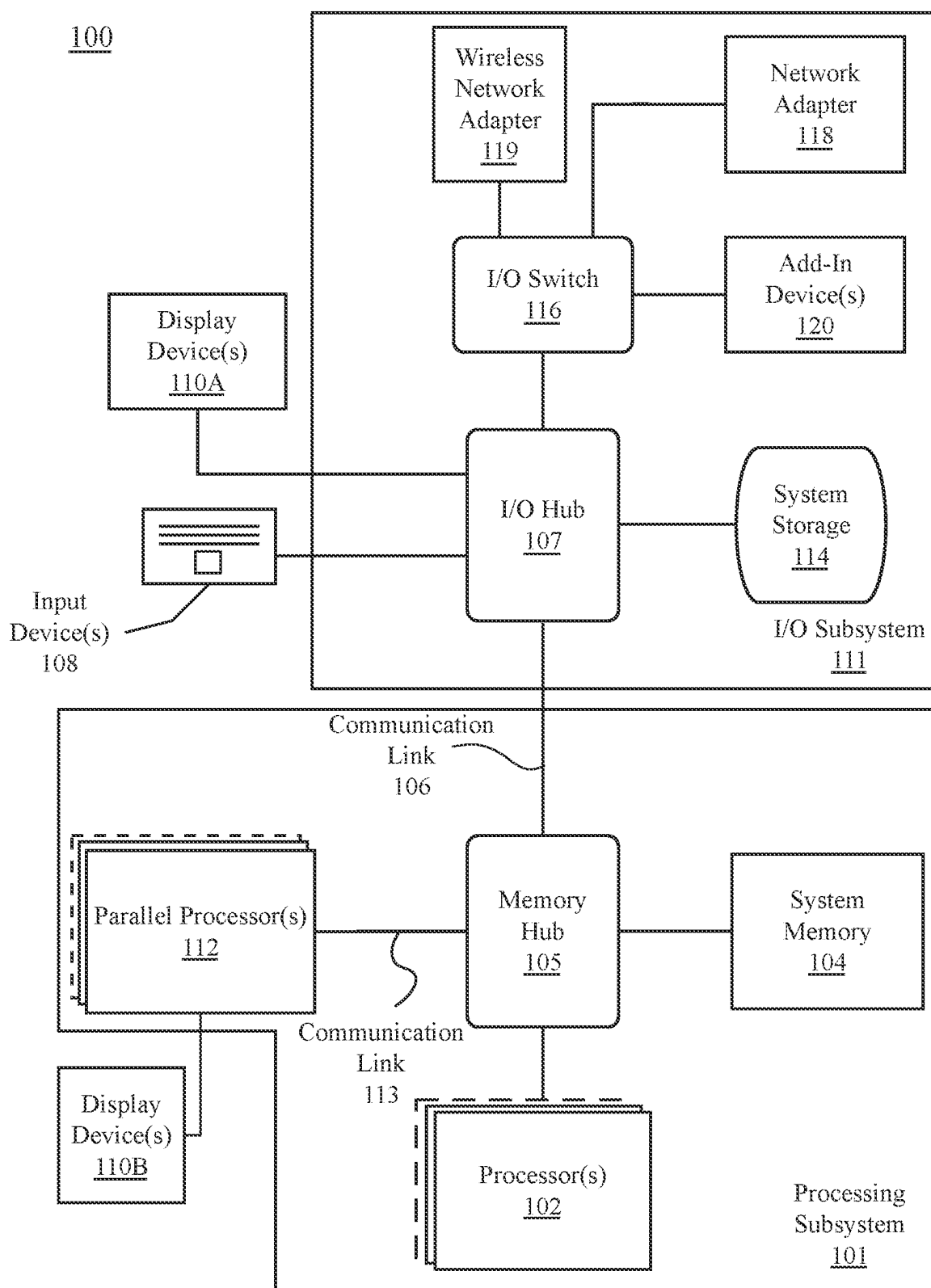


FIG. 1

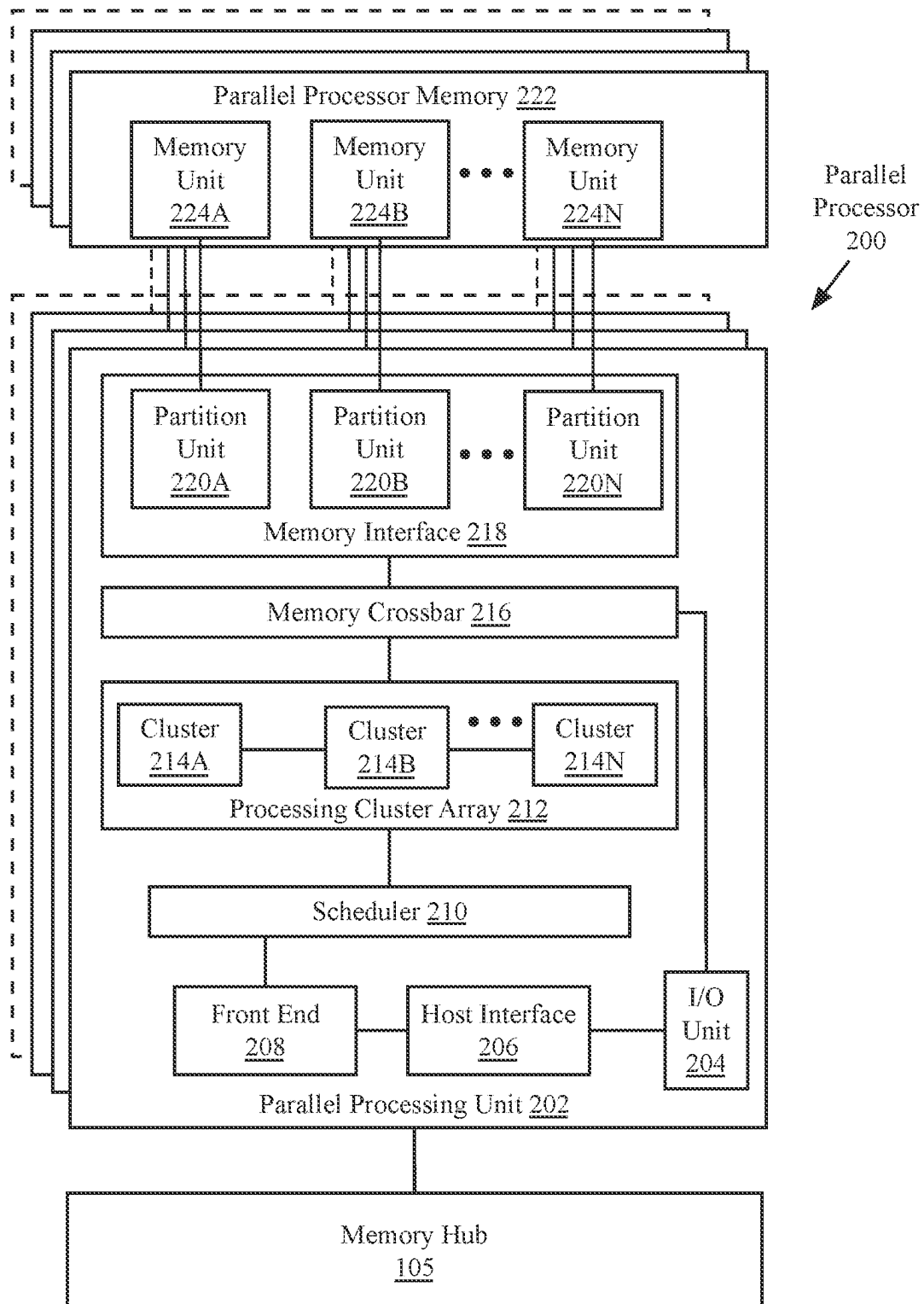


FIG. 2A

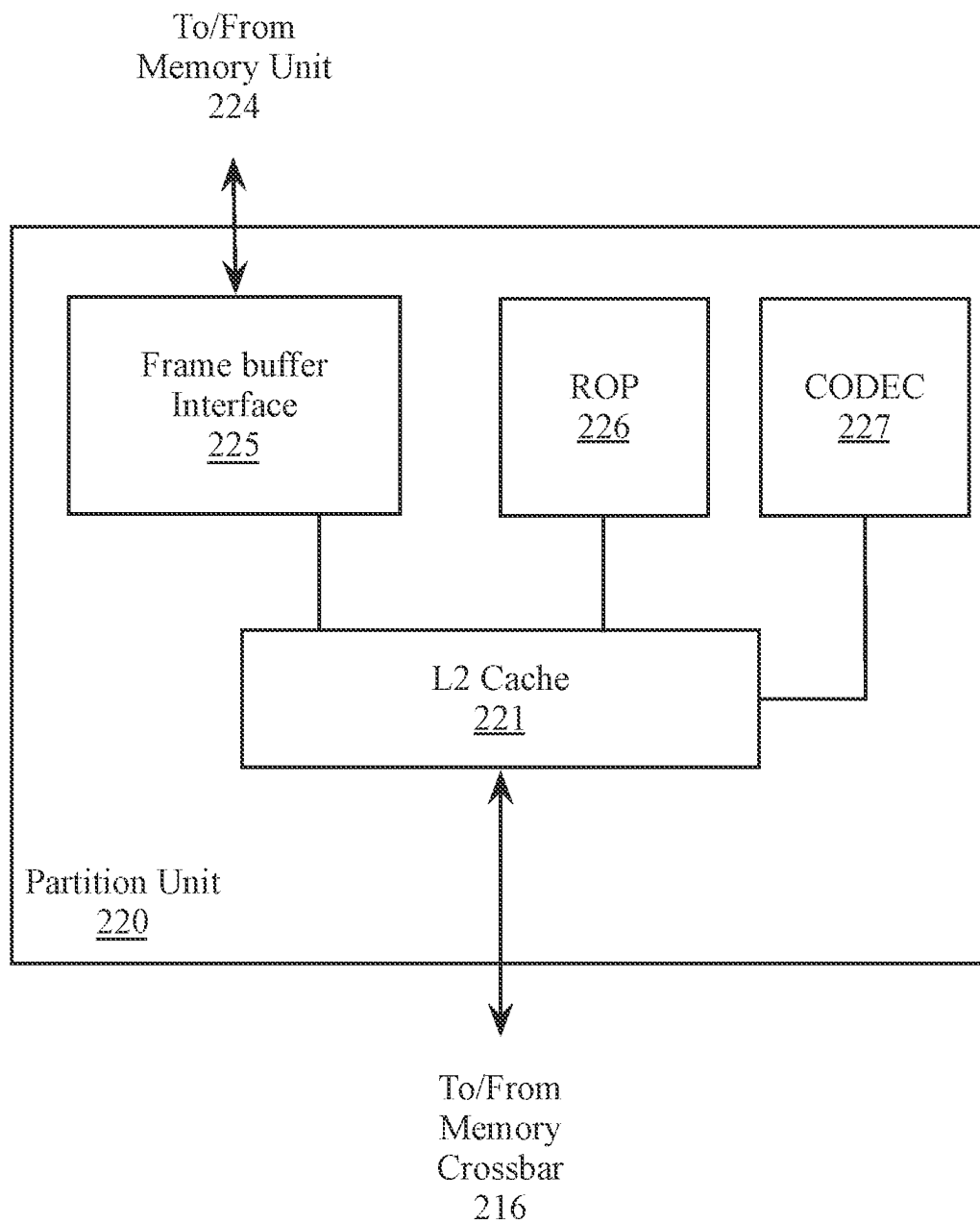


FIG. 2B

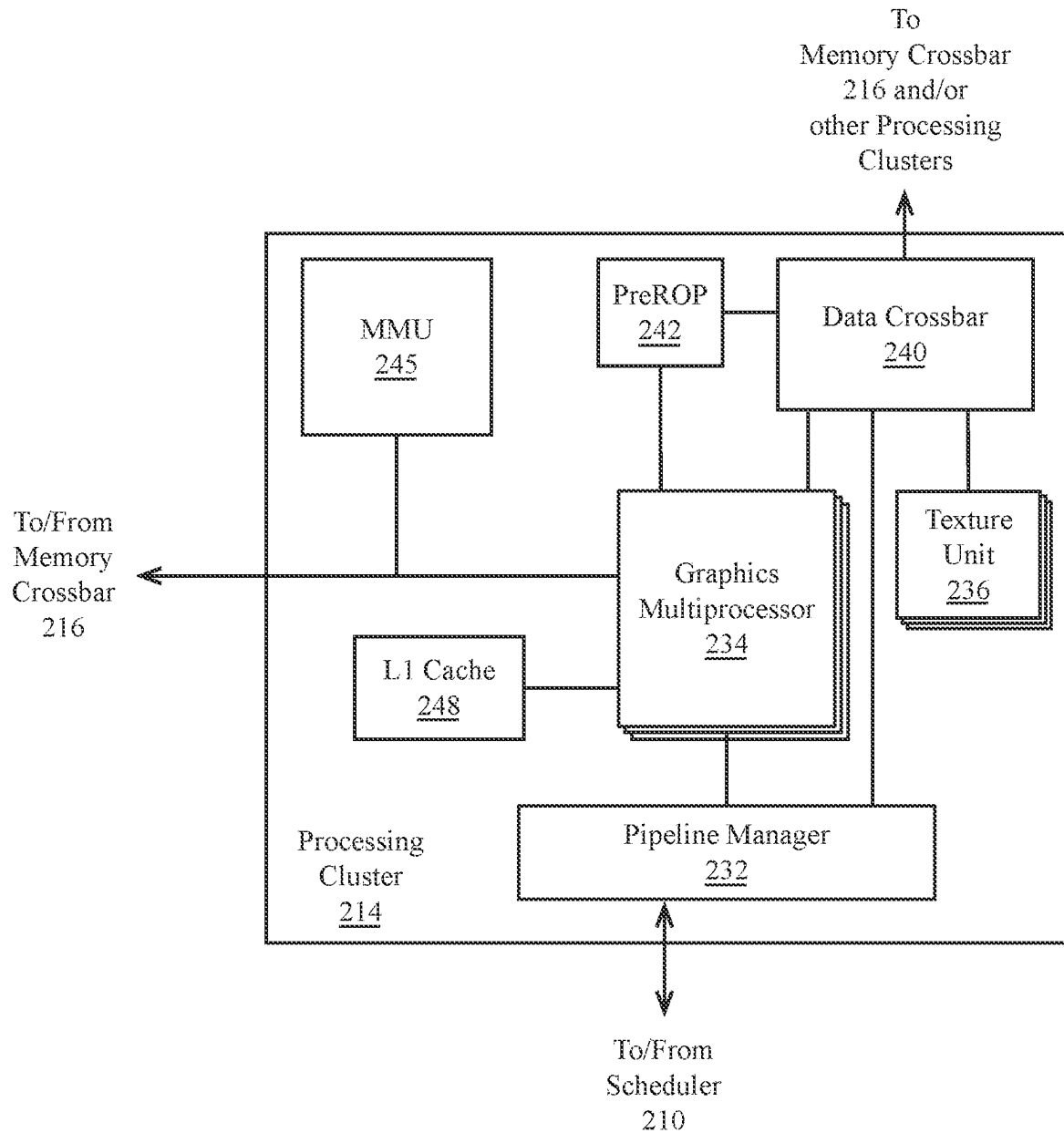


FIG. 2C

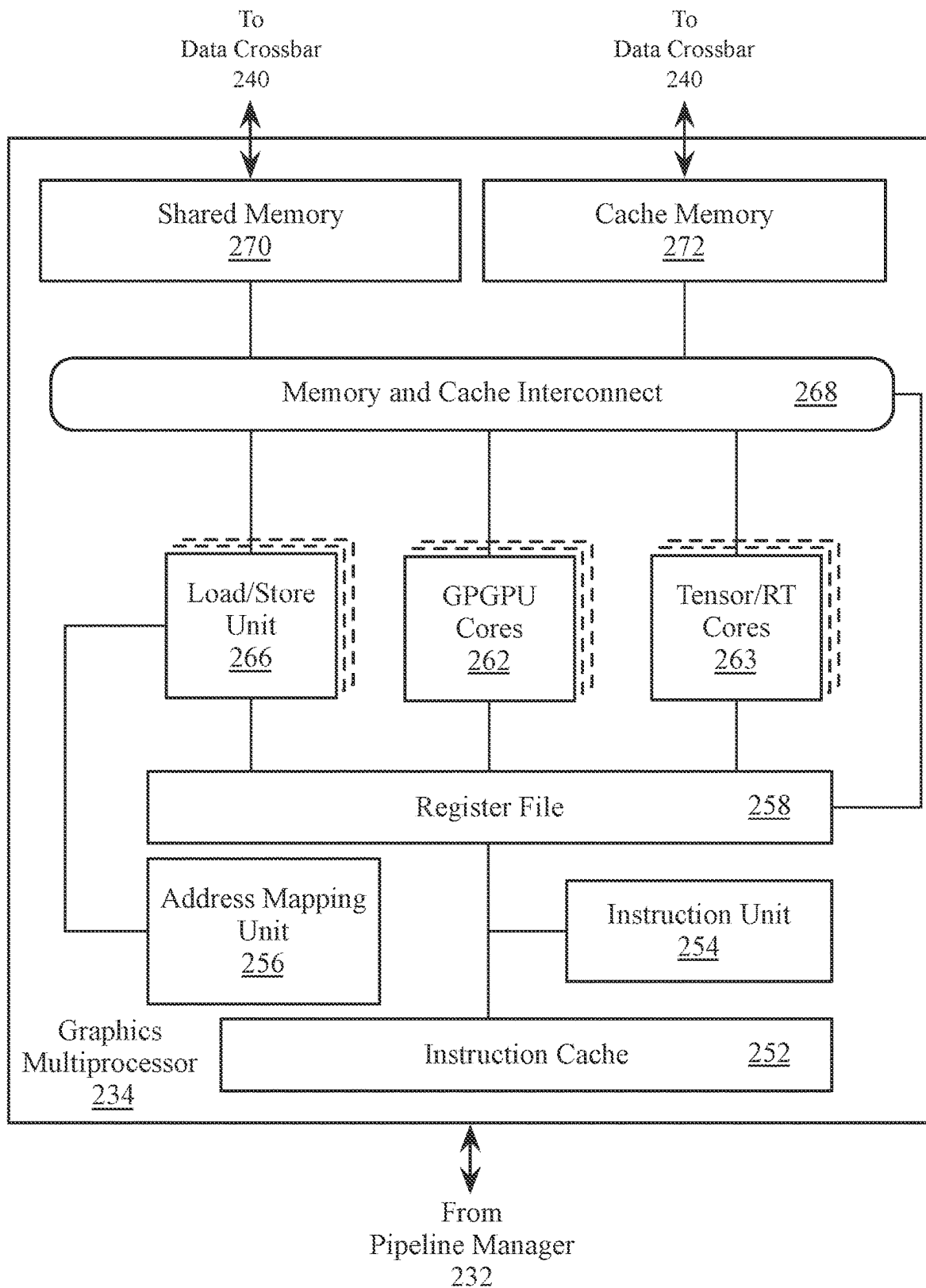


FIG. 2D

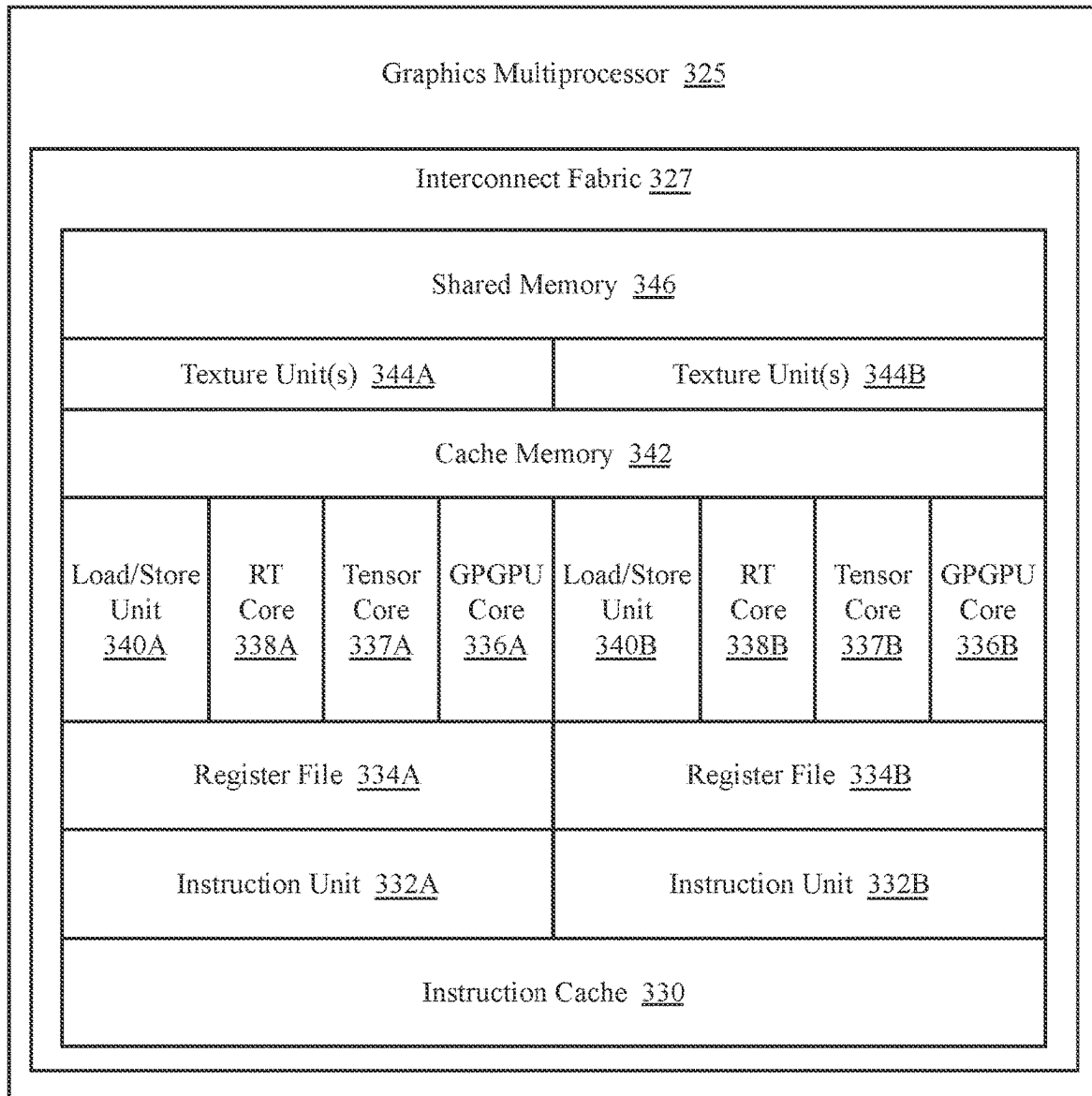


FIG. 3A

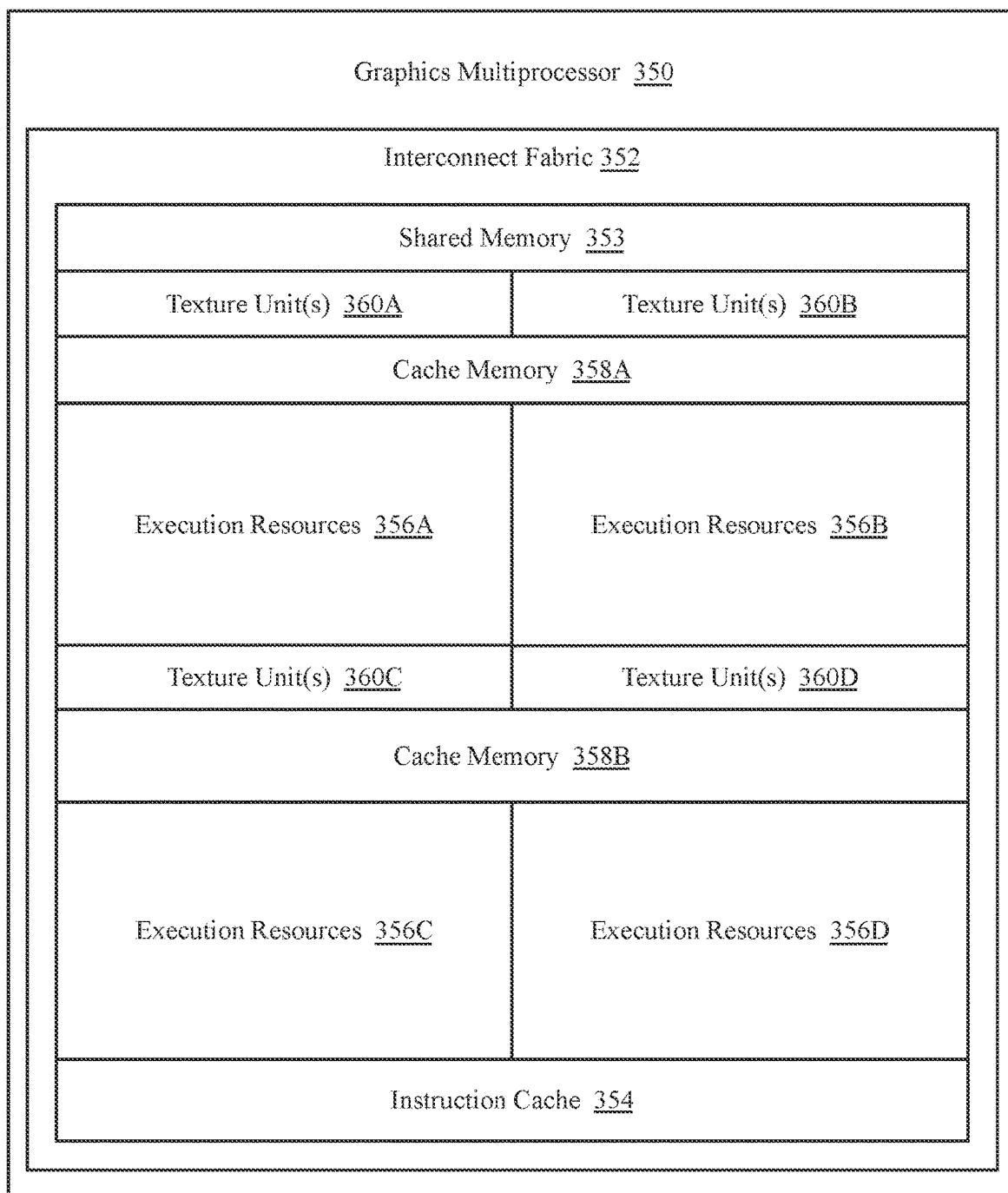


FIG. 3B

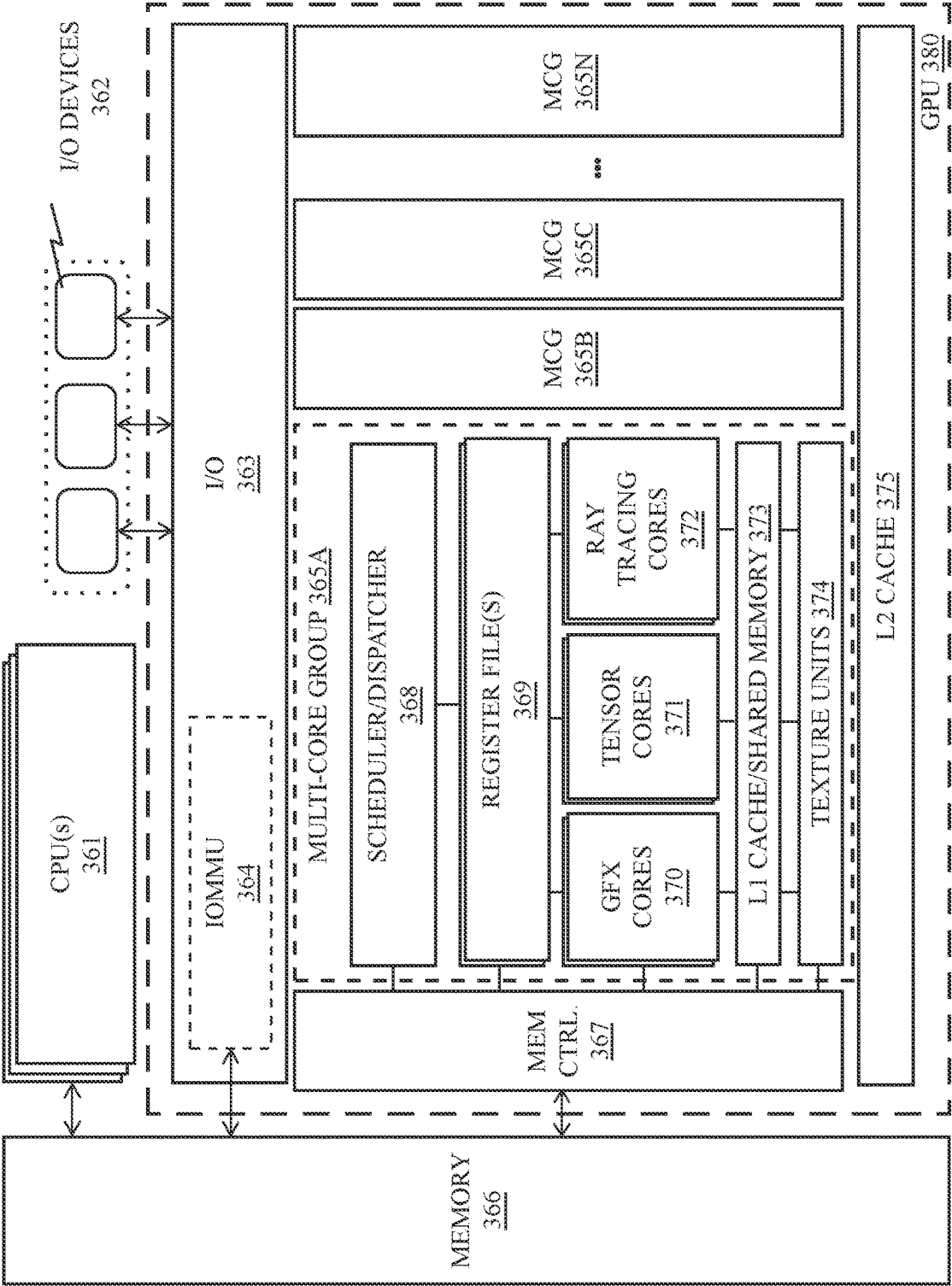


FIG. 3C

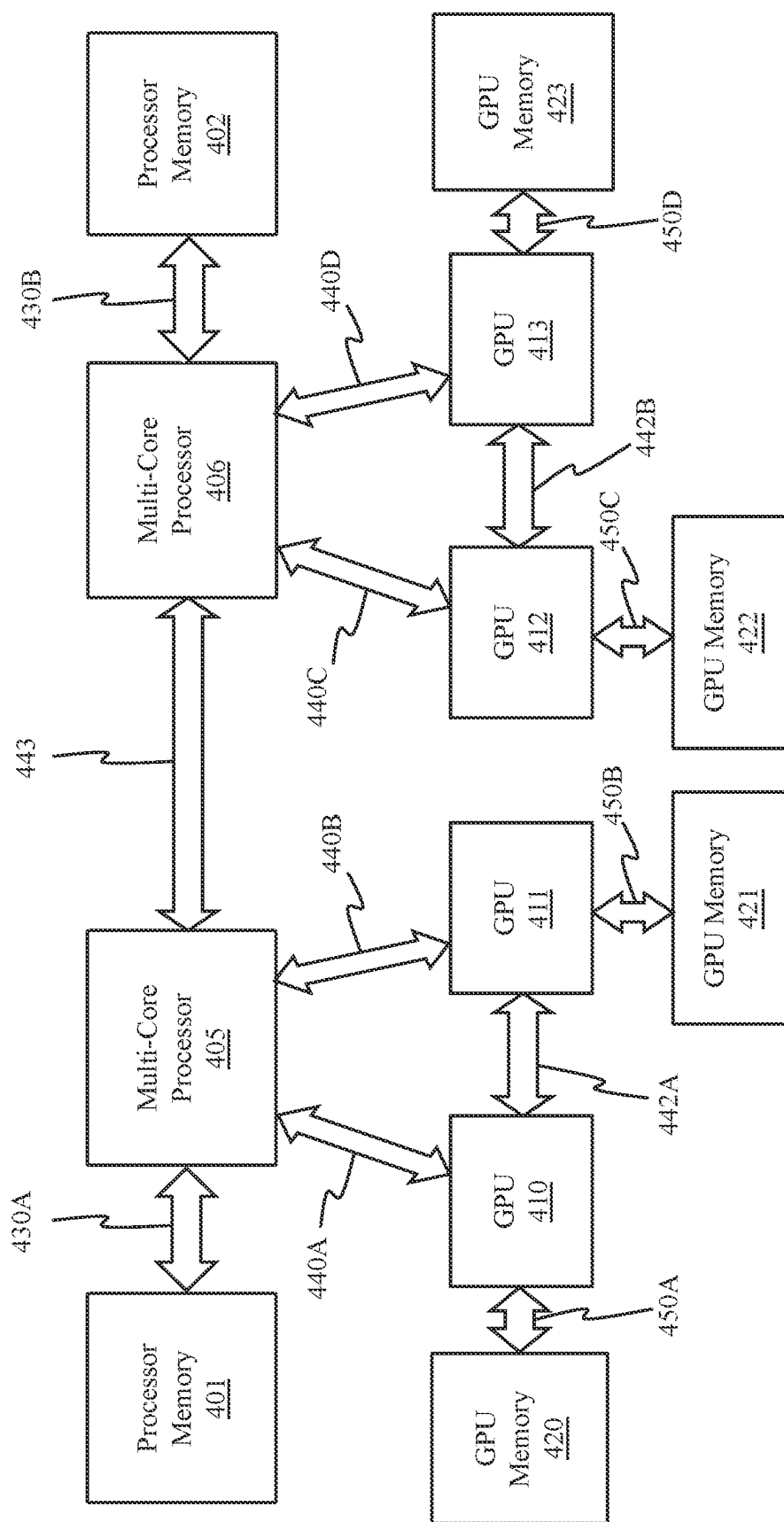


FIG. 4A

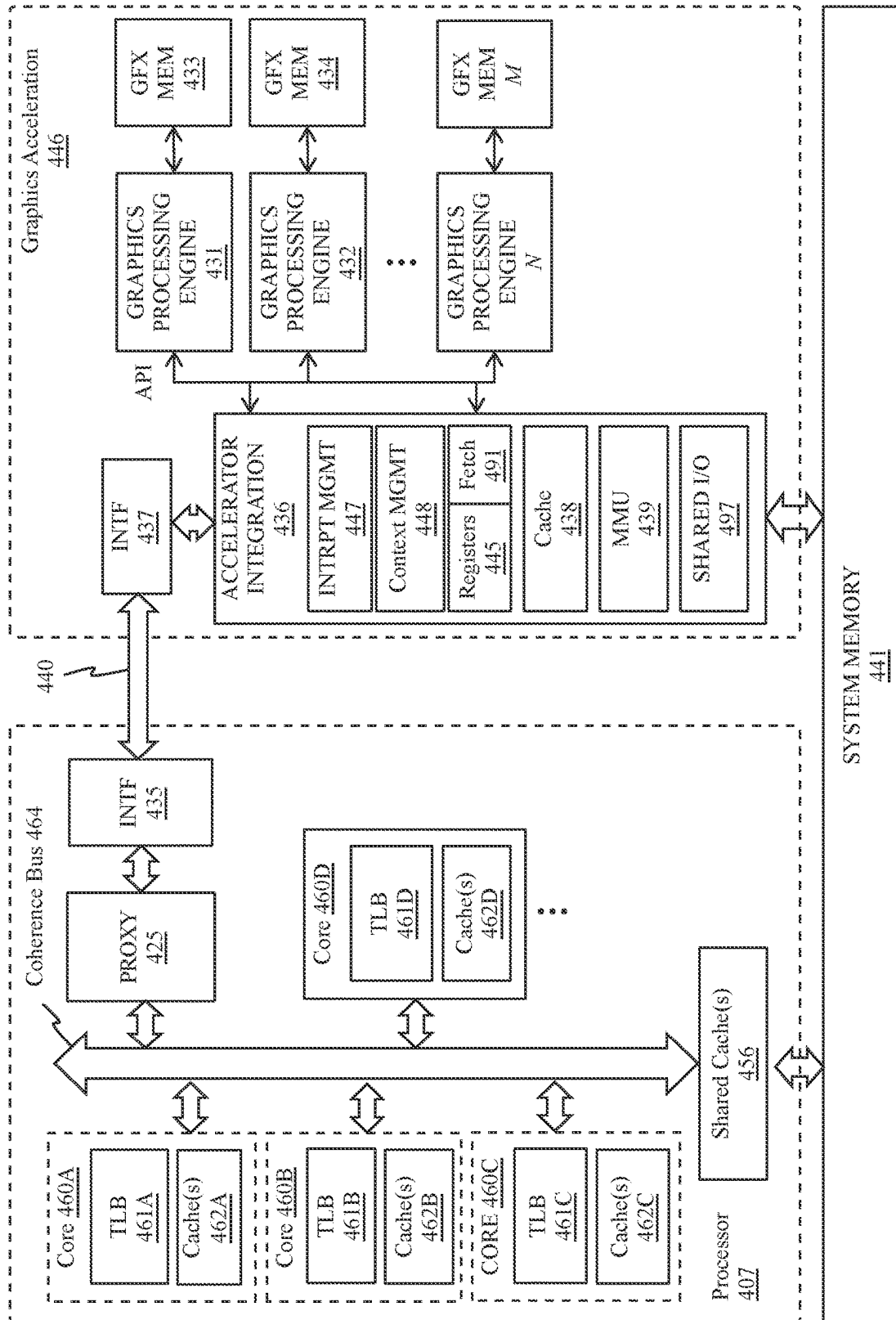


FIG. 4B

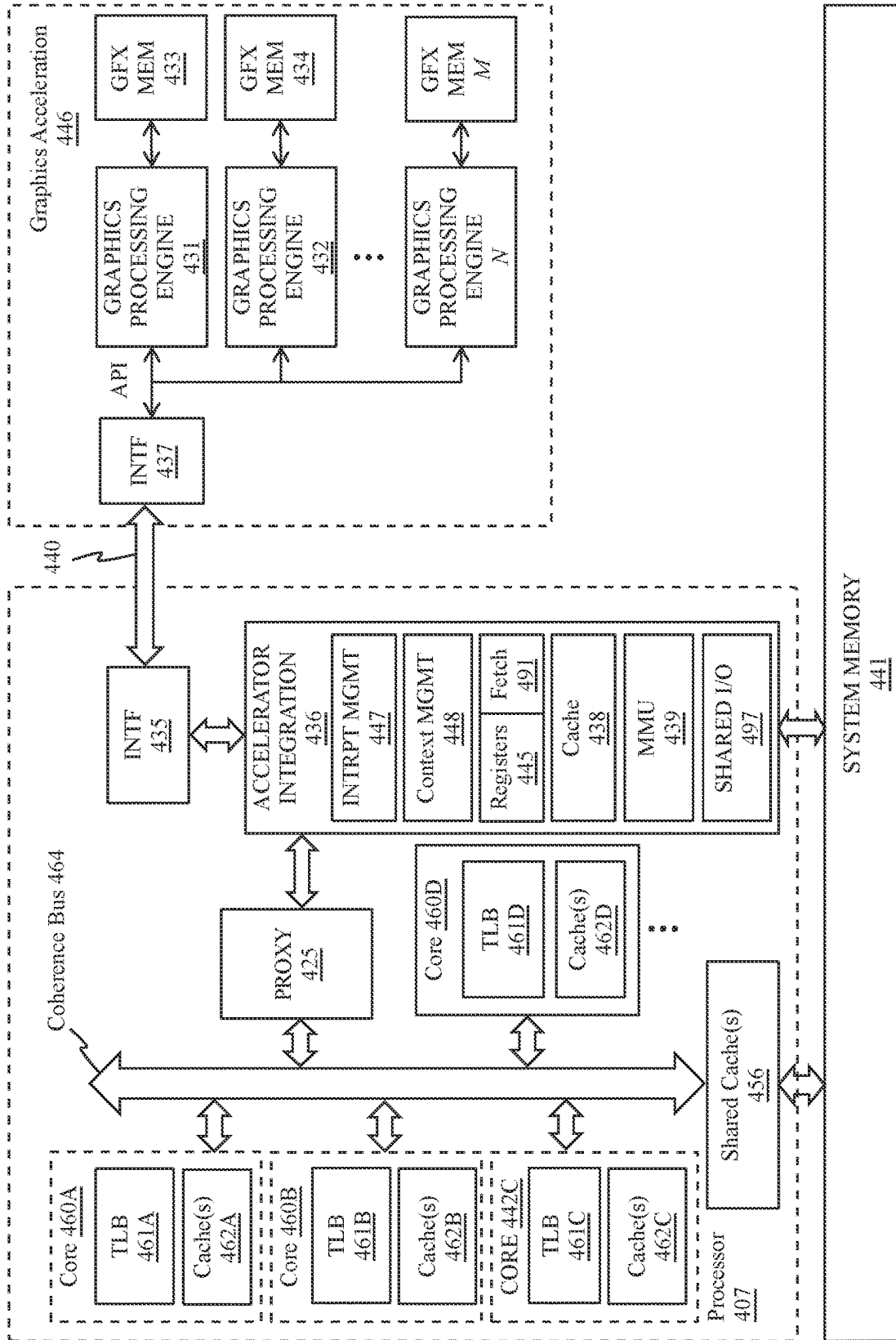


FIG. 4C

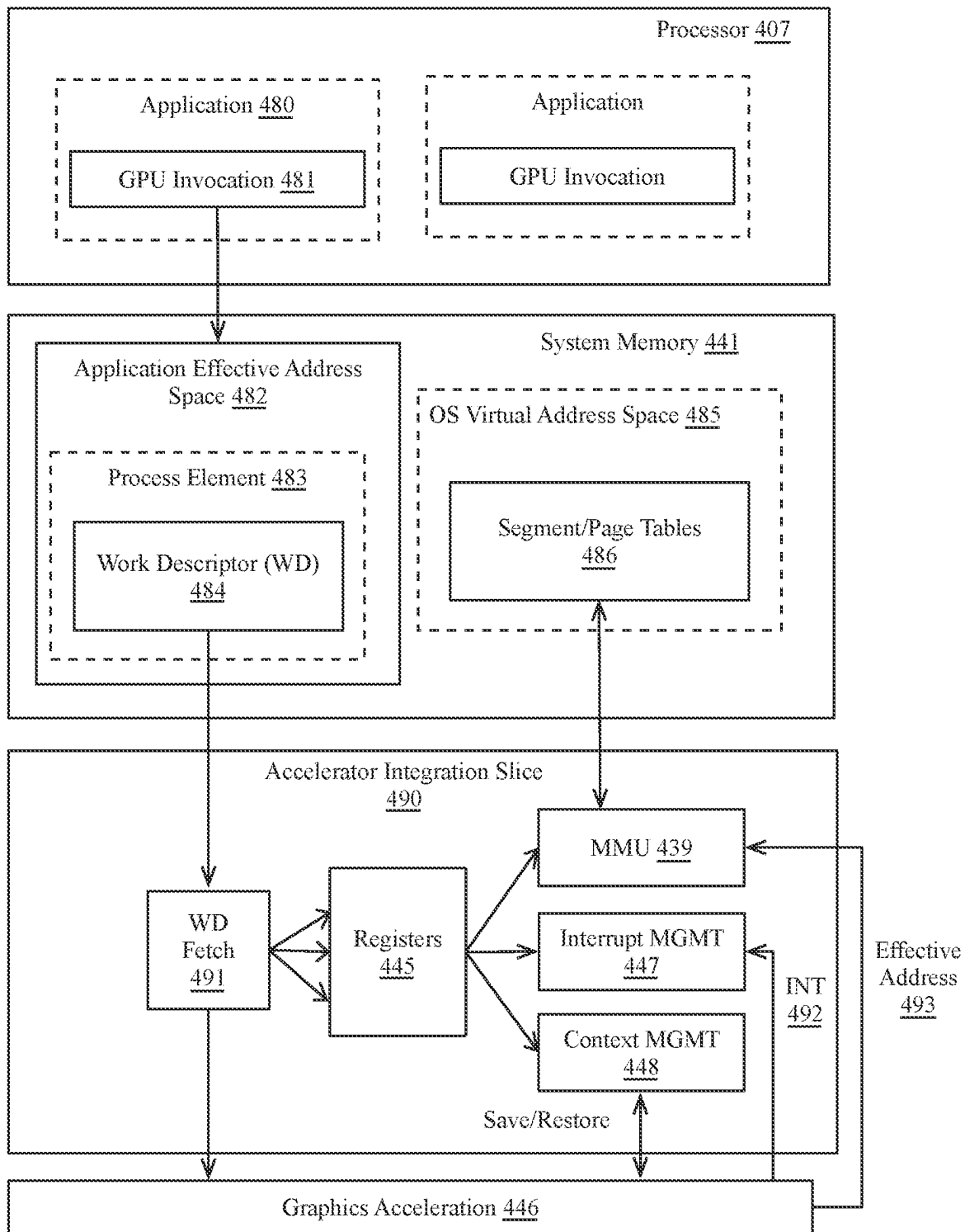


FIG. 4D

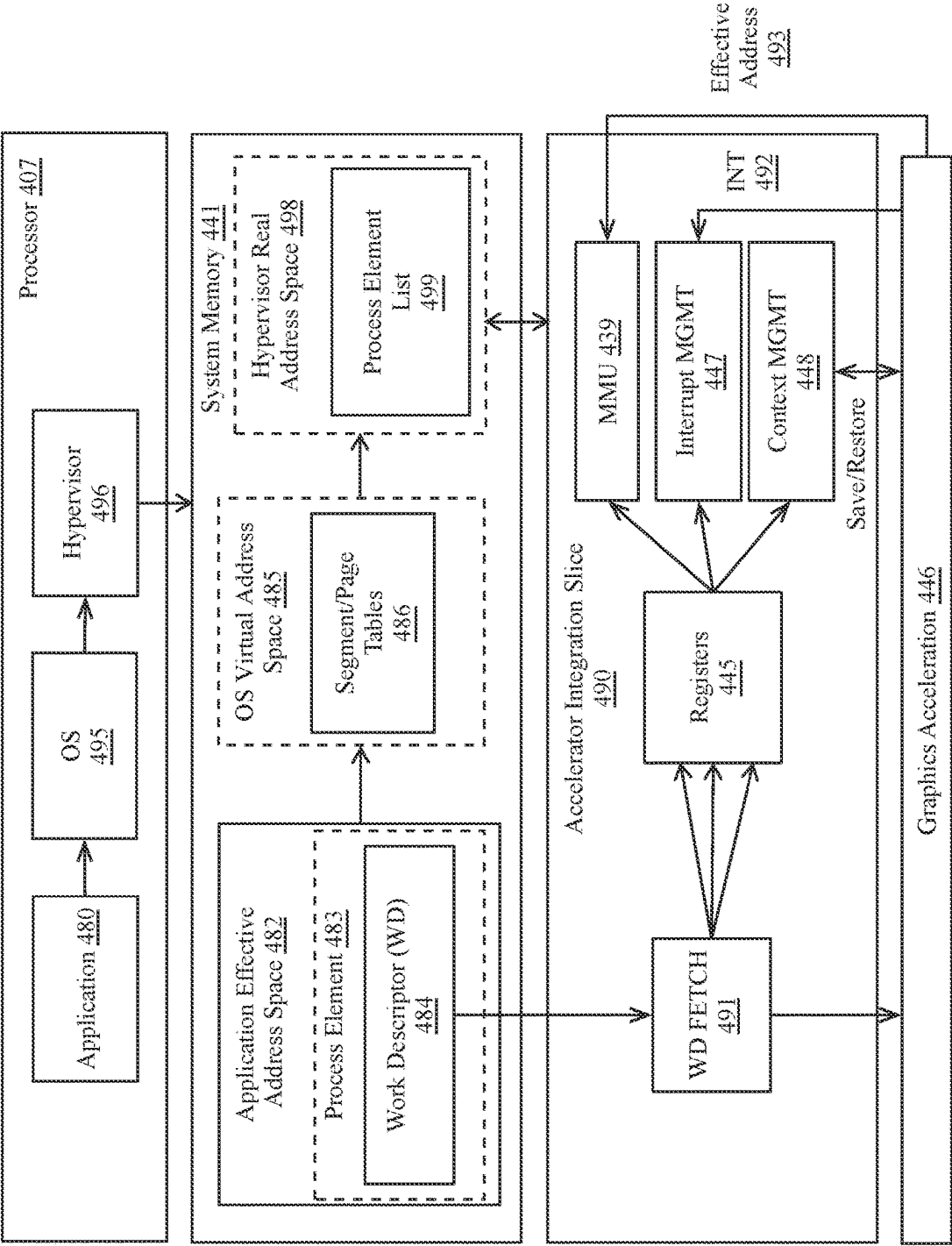


FIG. 4E

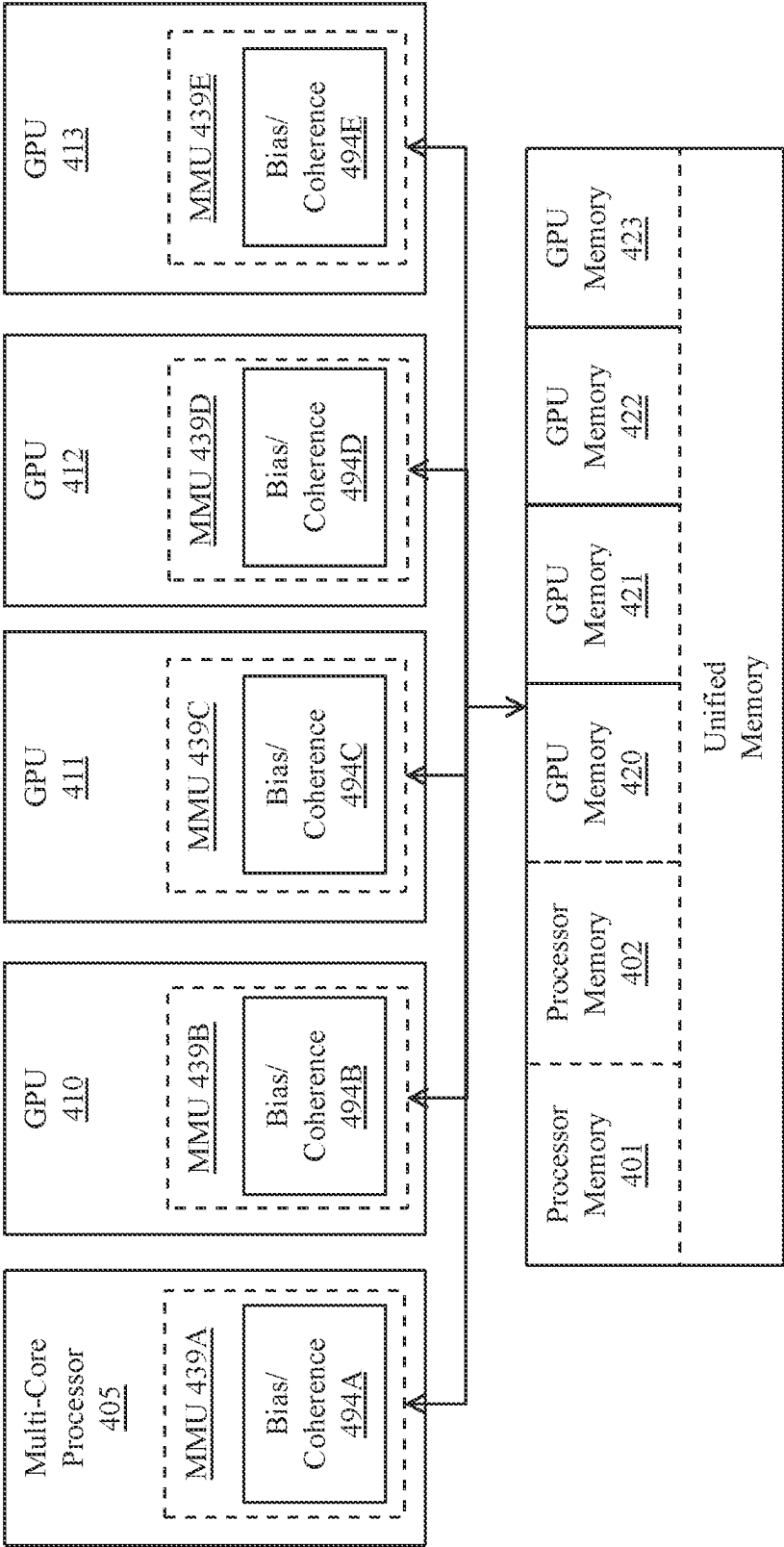


FIG. 4F

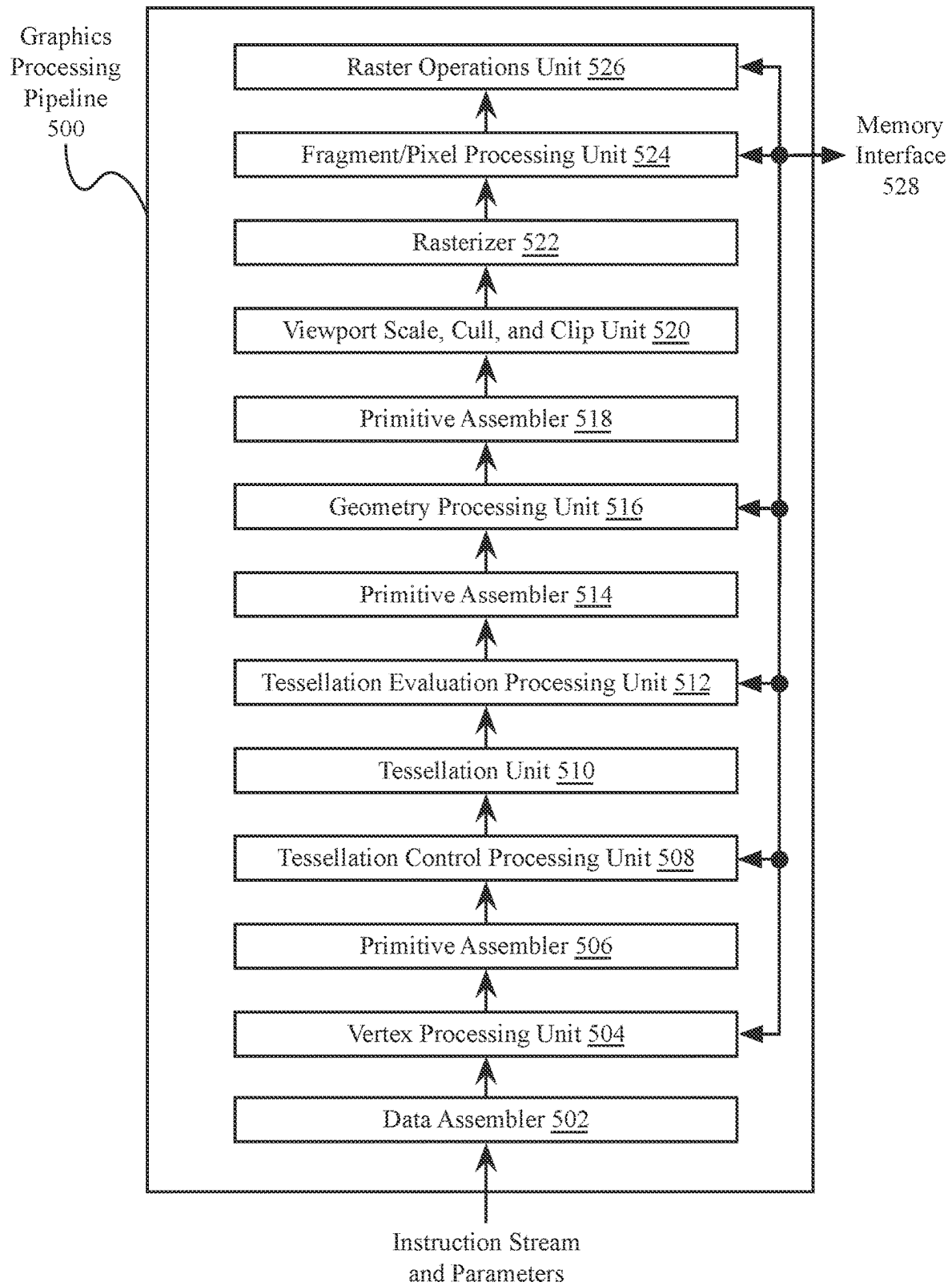


FIG. 5

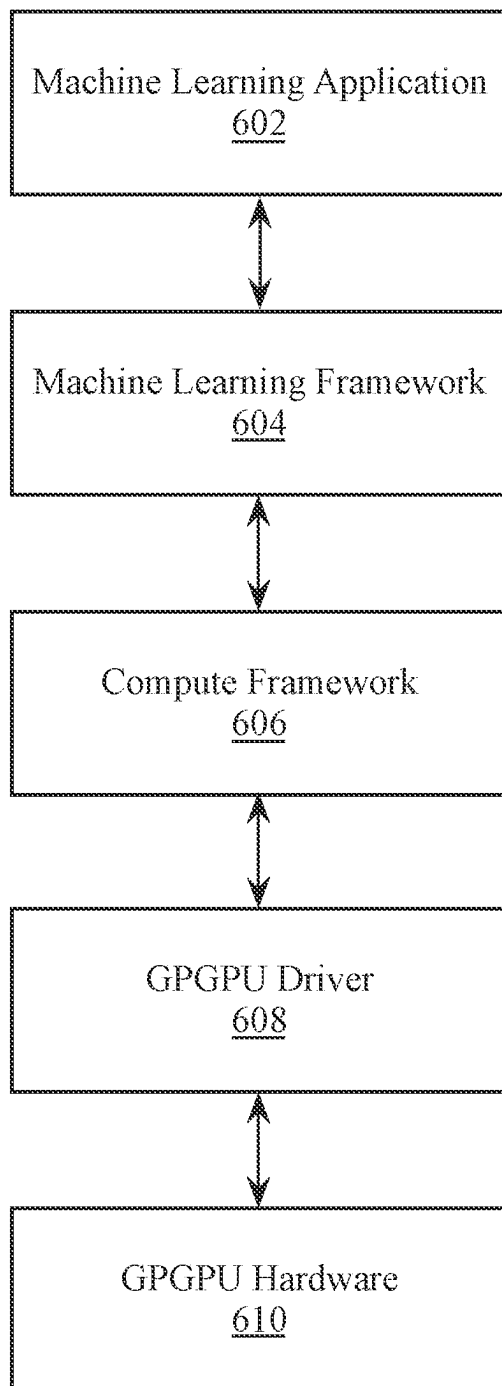
600

FIG. 6

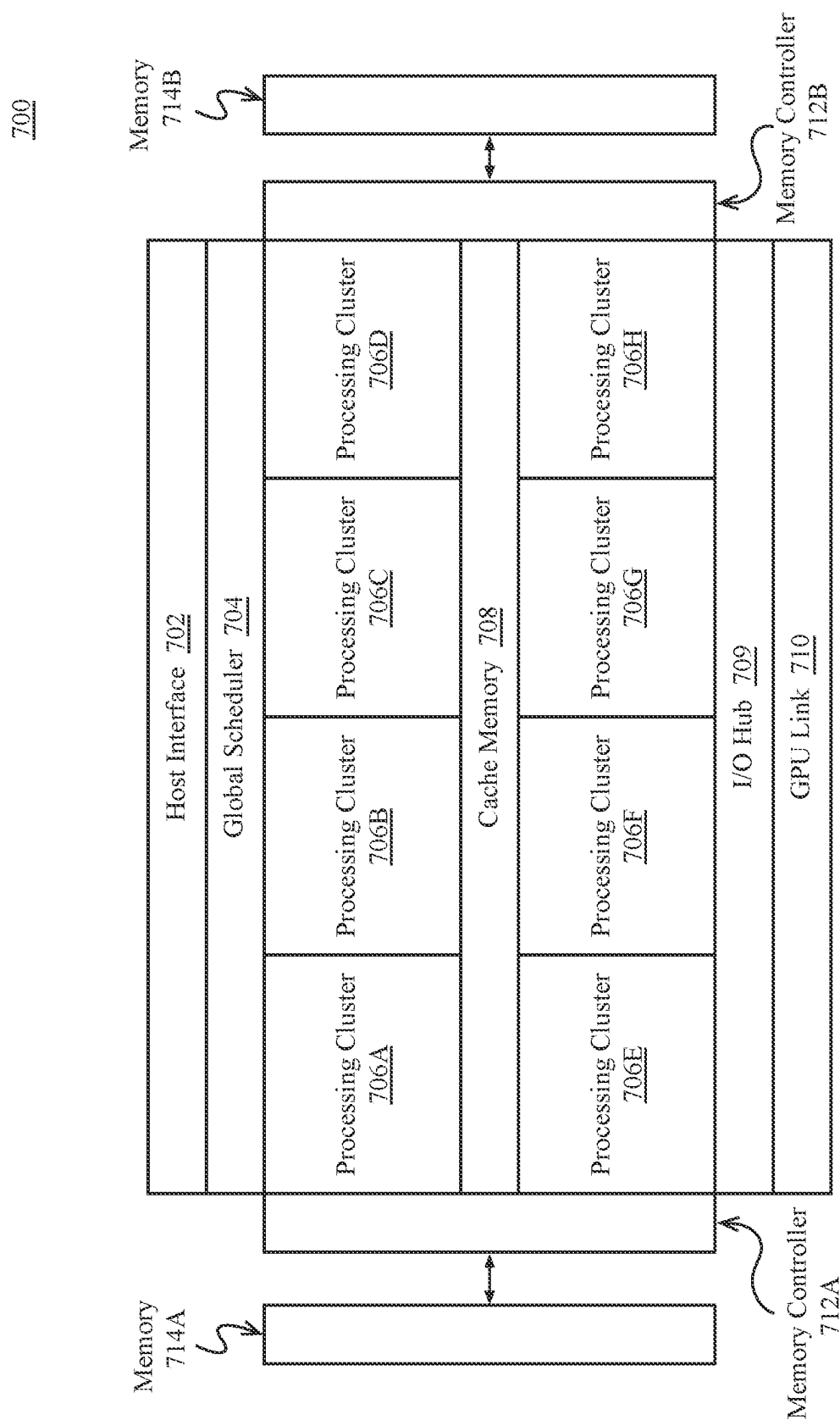


FIG. 7

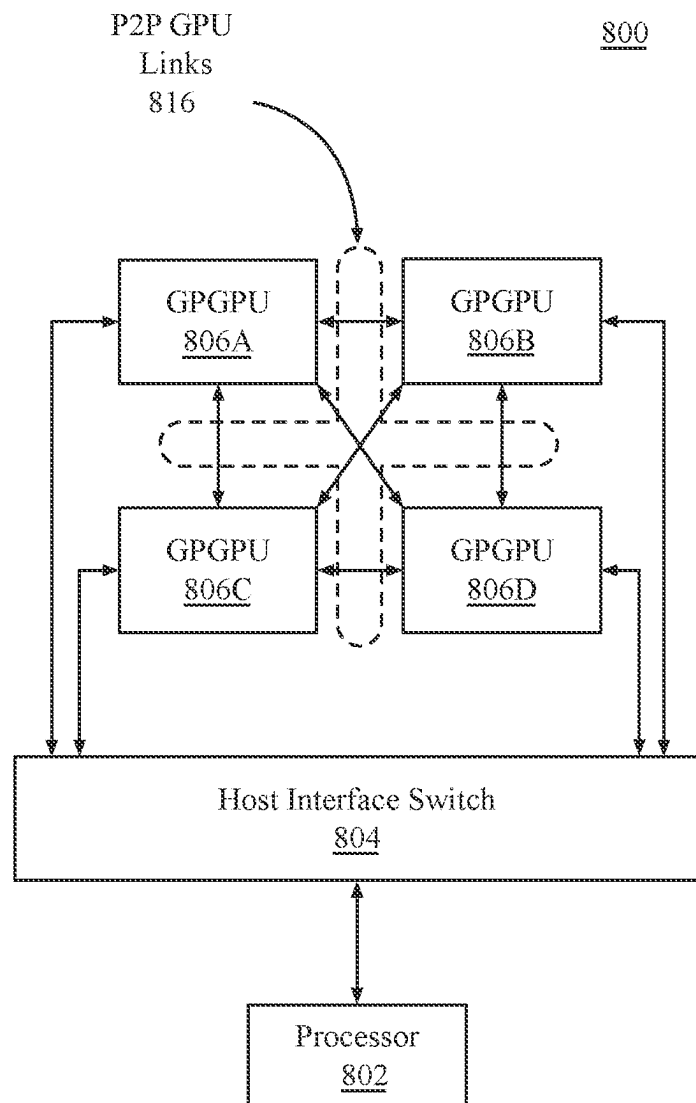


FIG. 8

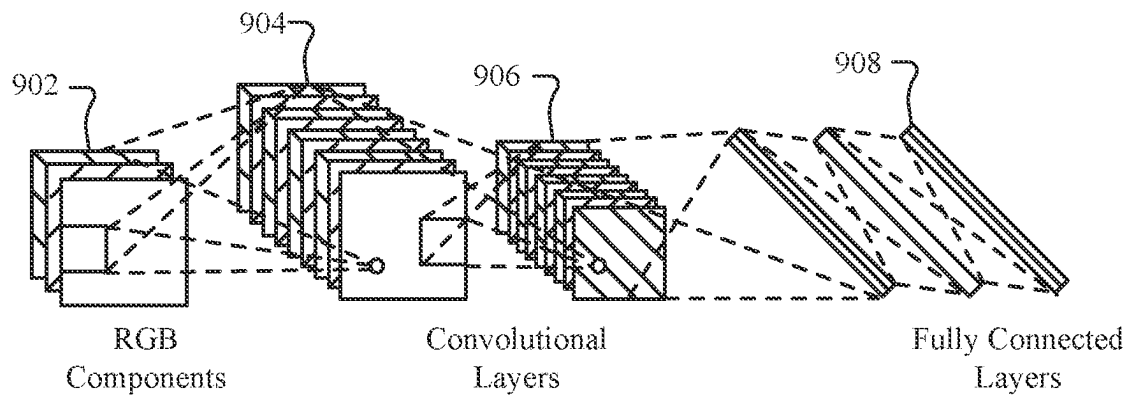


FIG. 9A

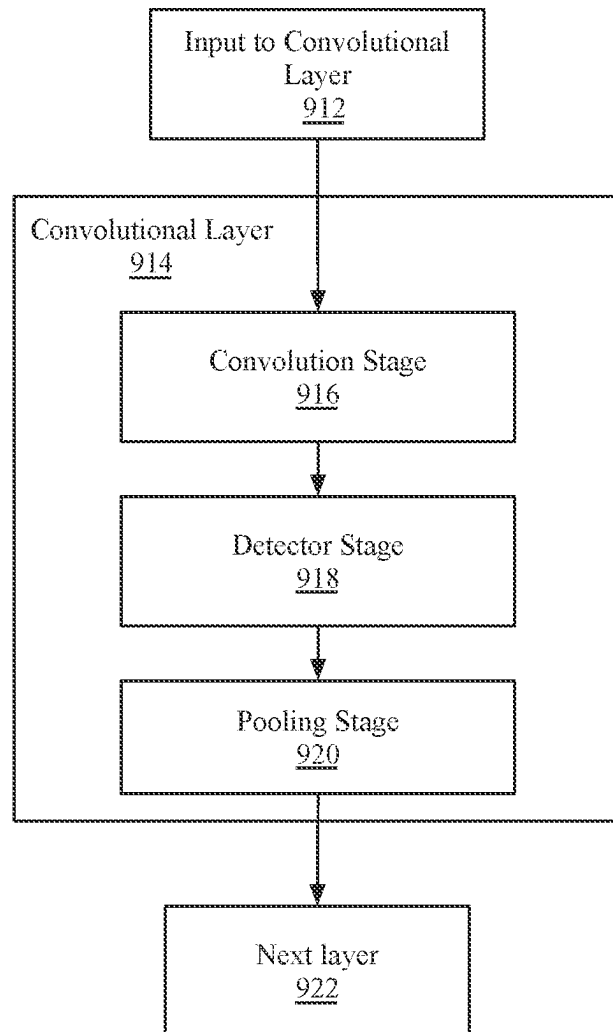


FIG. 9B

1000

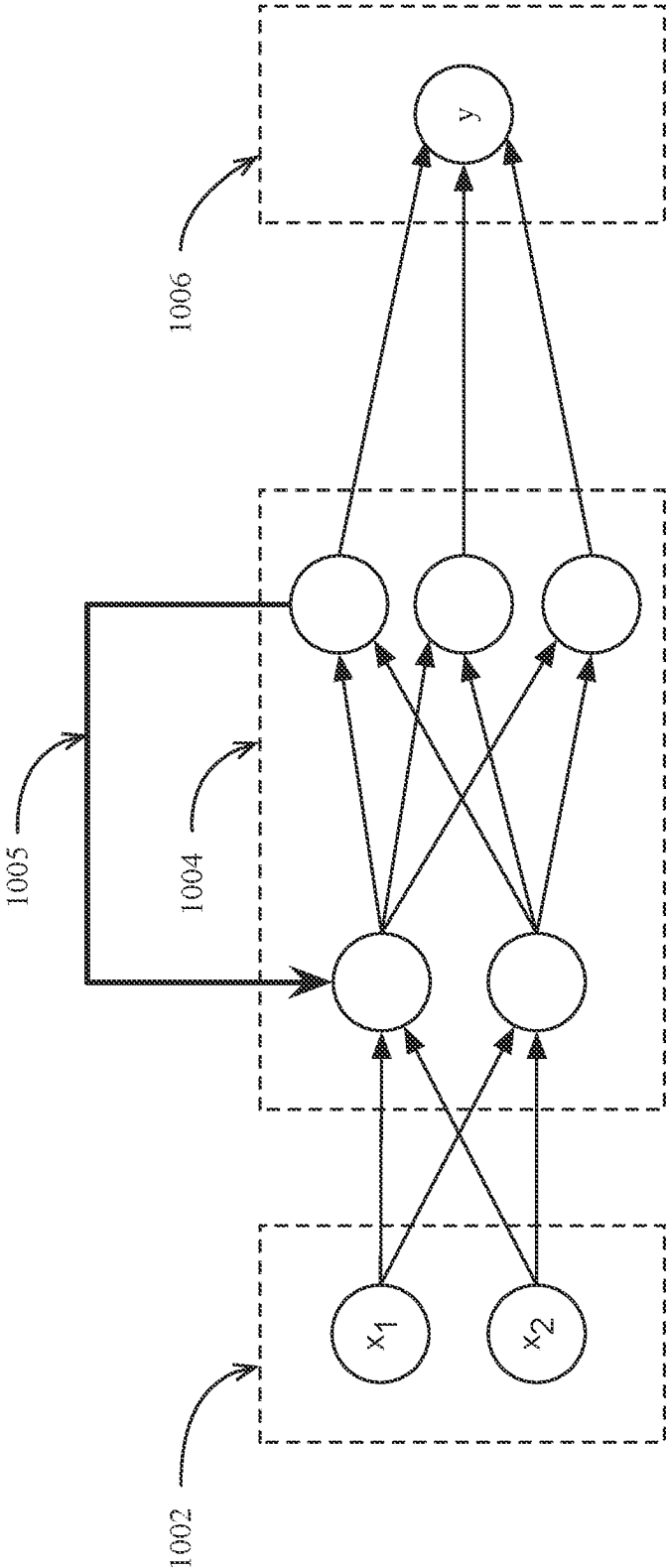
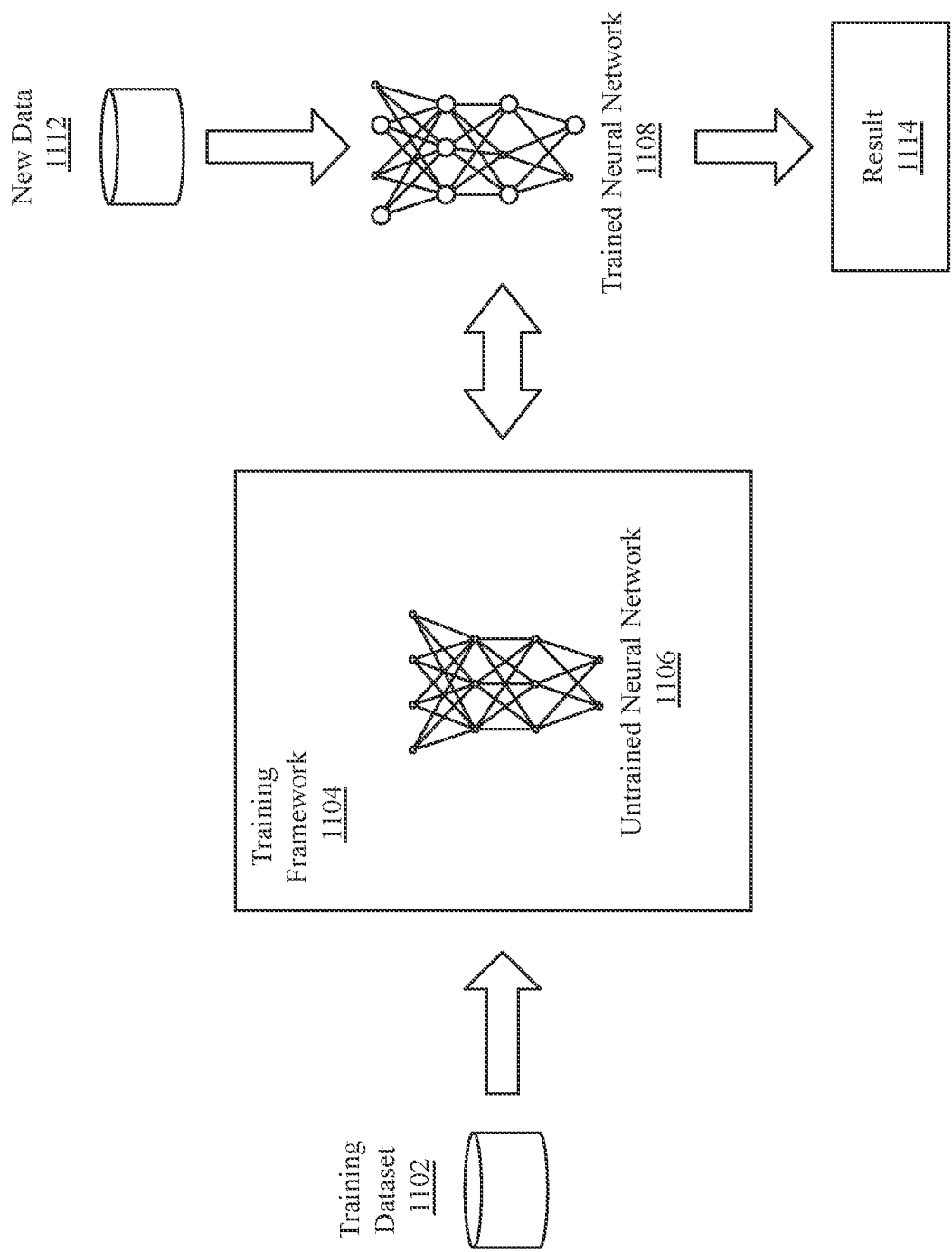


FIG. 10



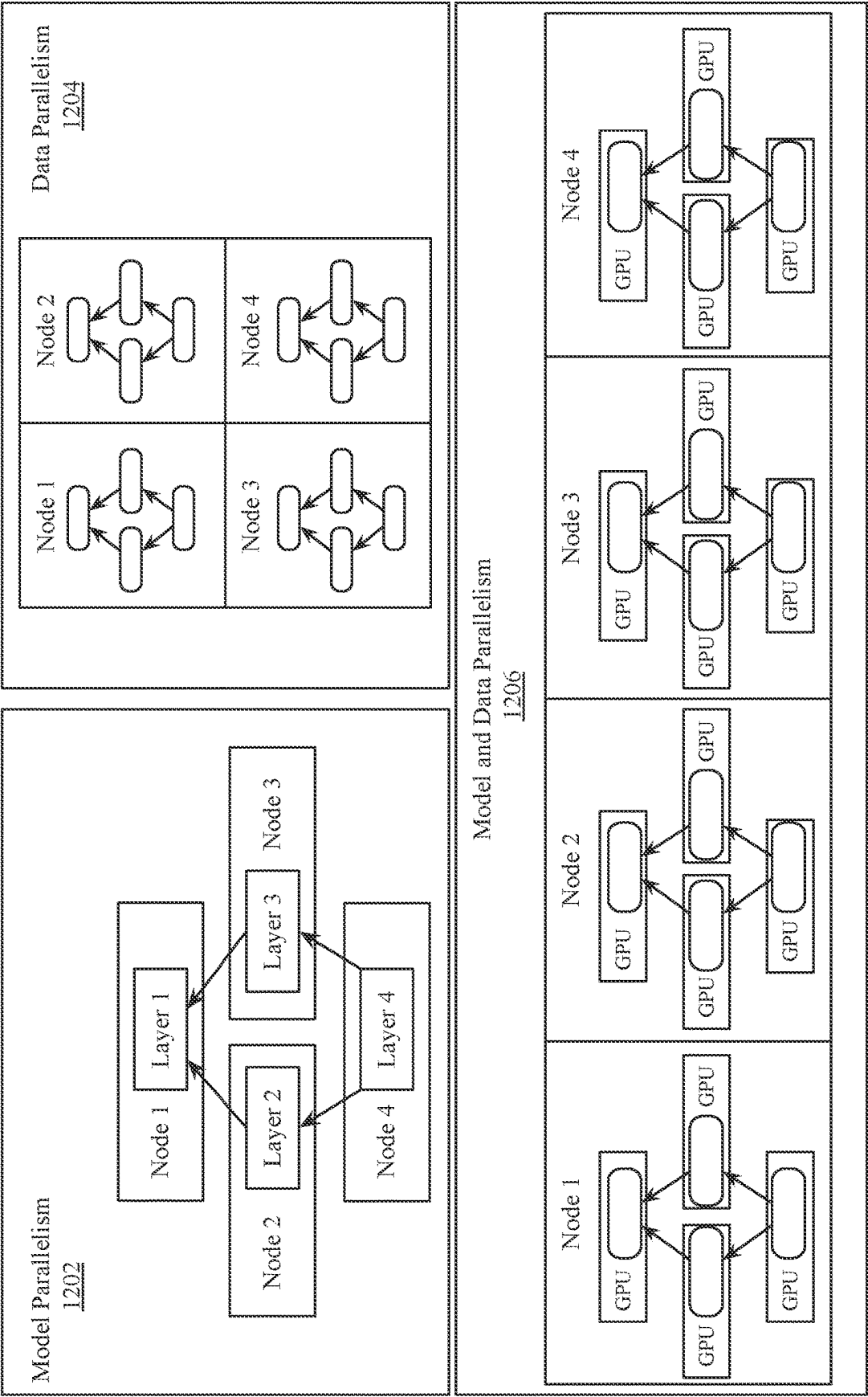


FIG. 12A

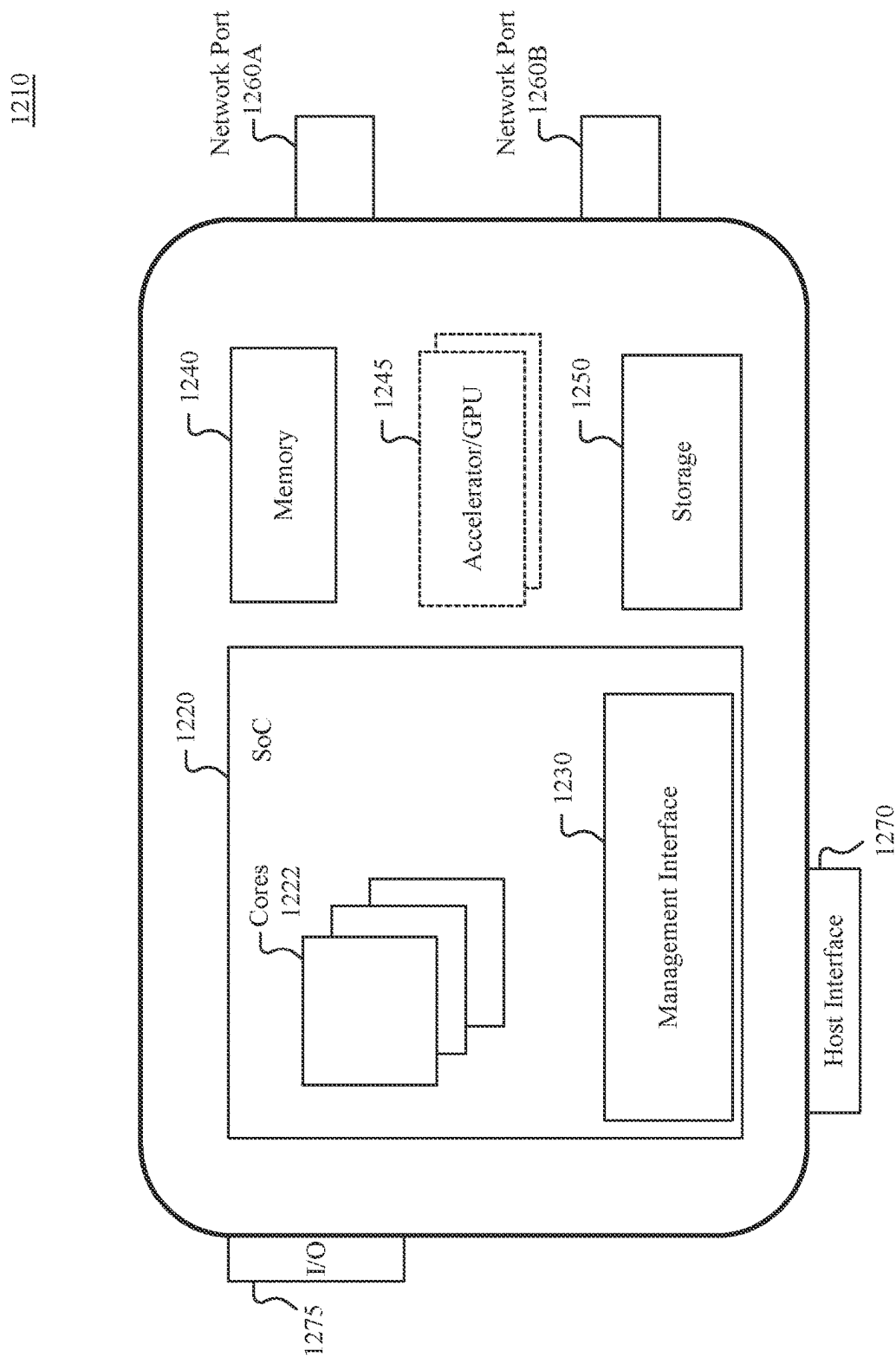


FIG. 12B

1300

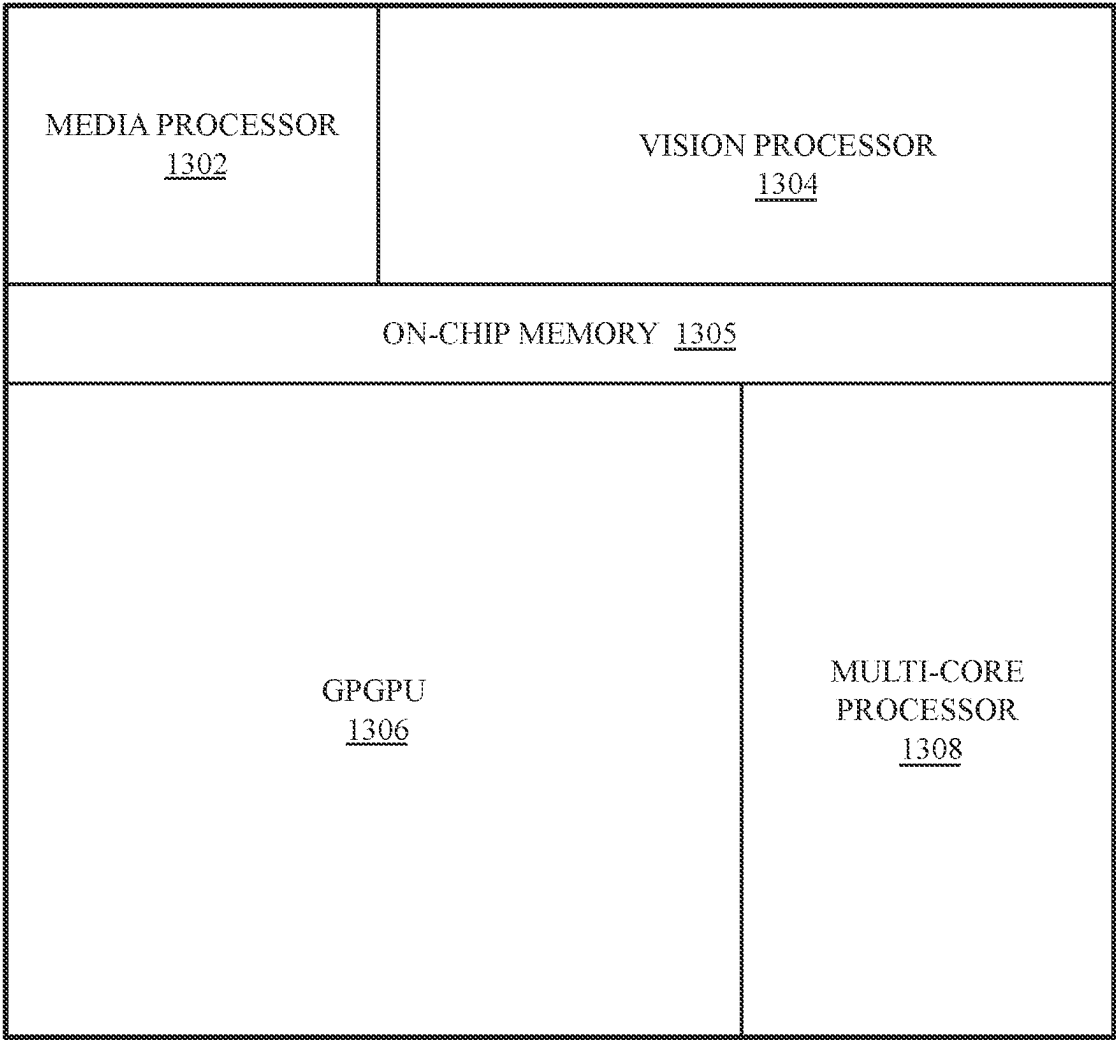


FIG. 13

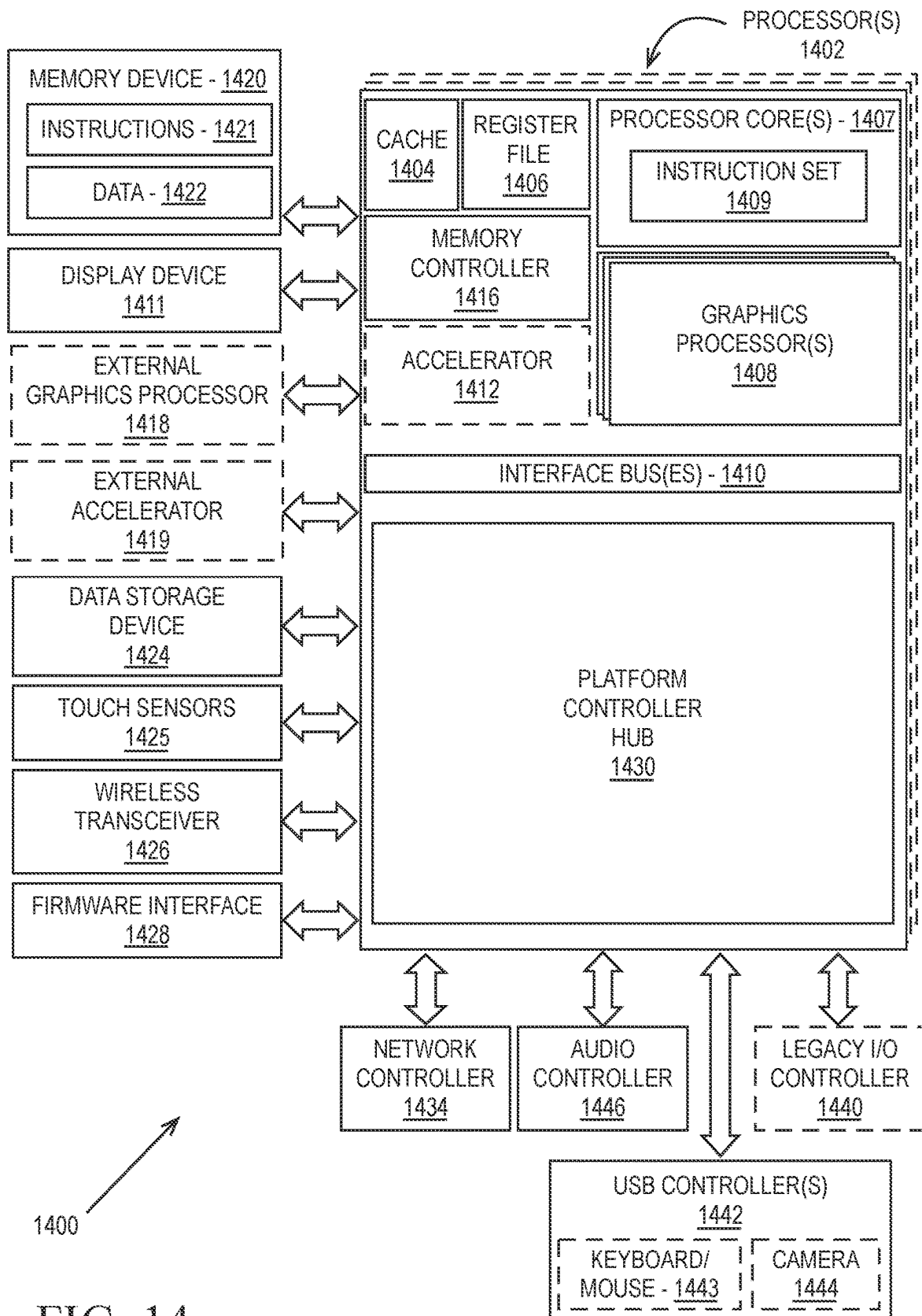


FIG. 14

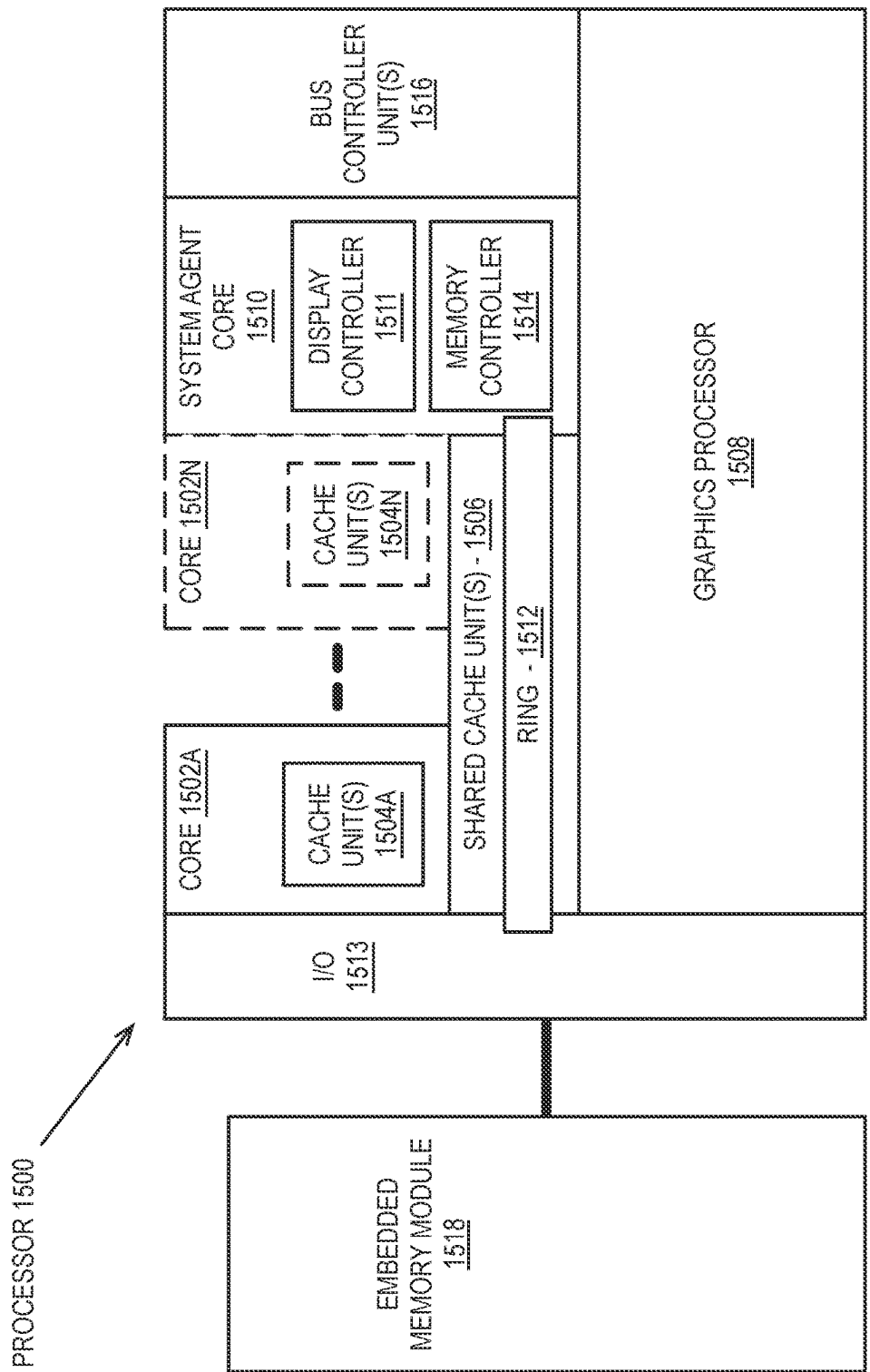


FIG. 15A

1519

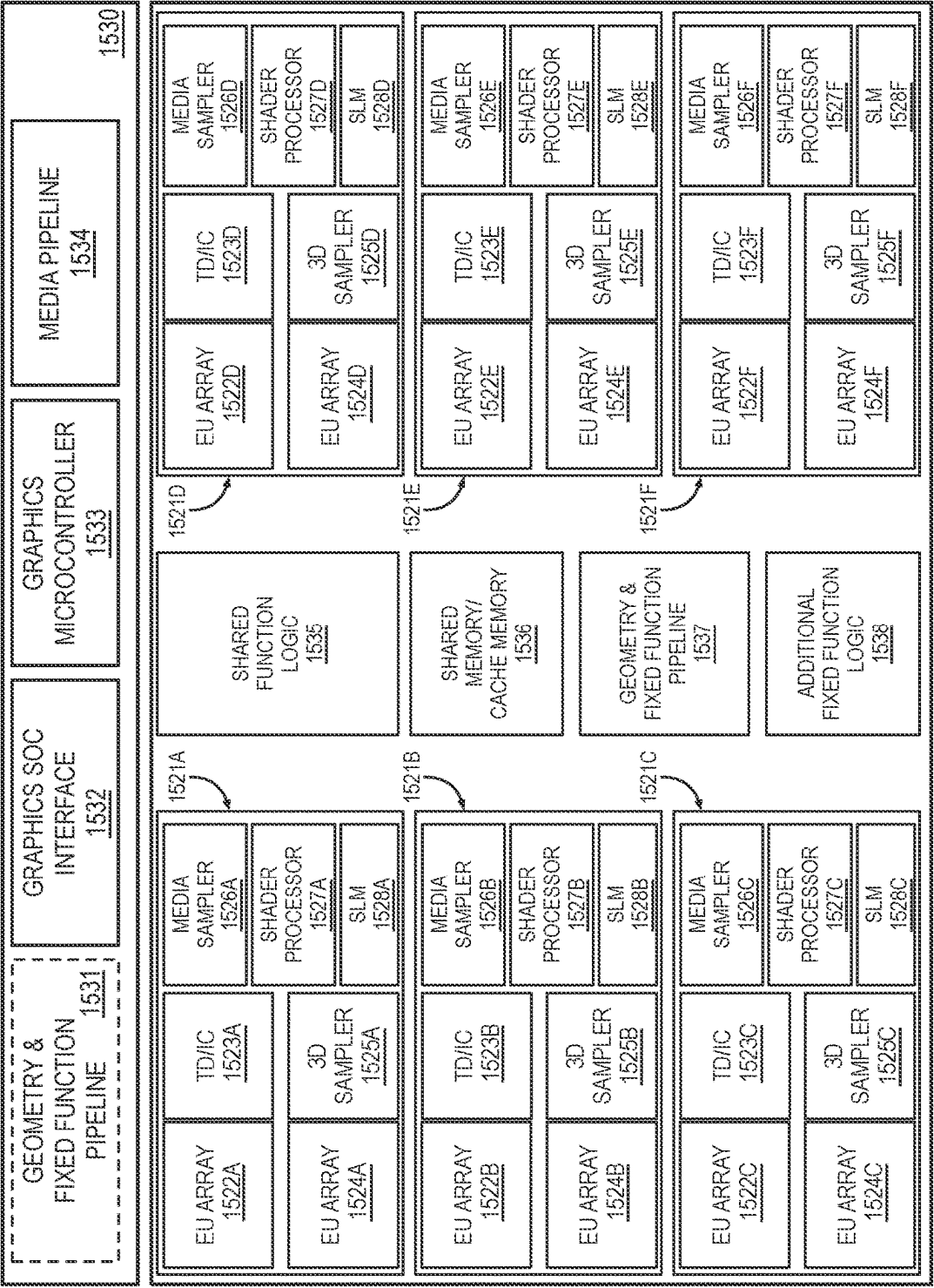


FIG. 15B

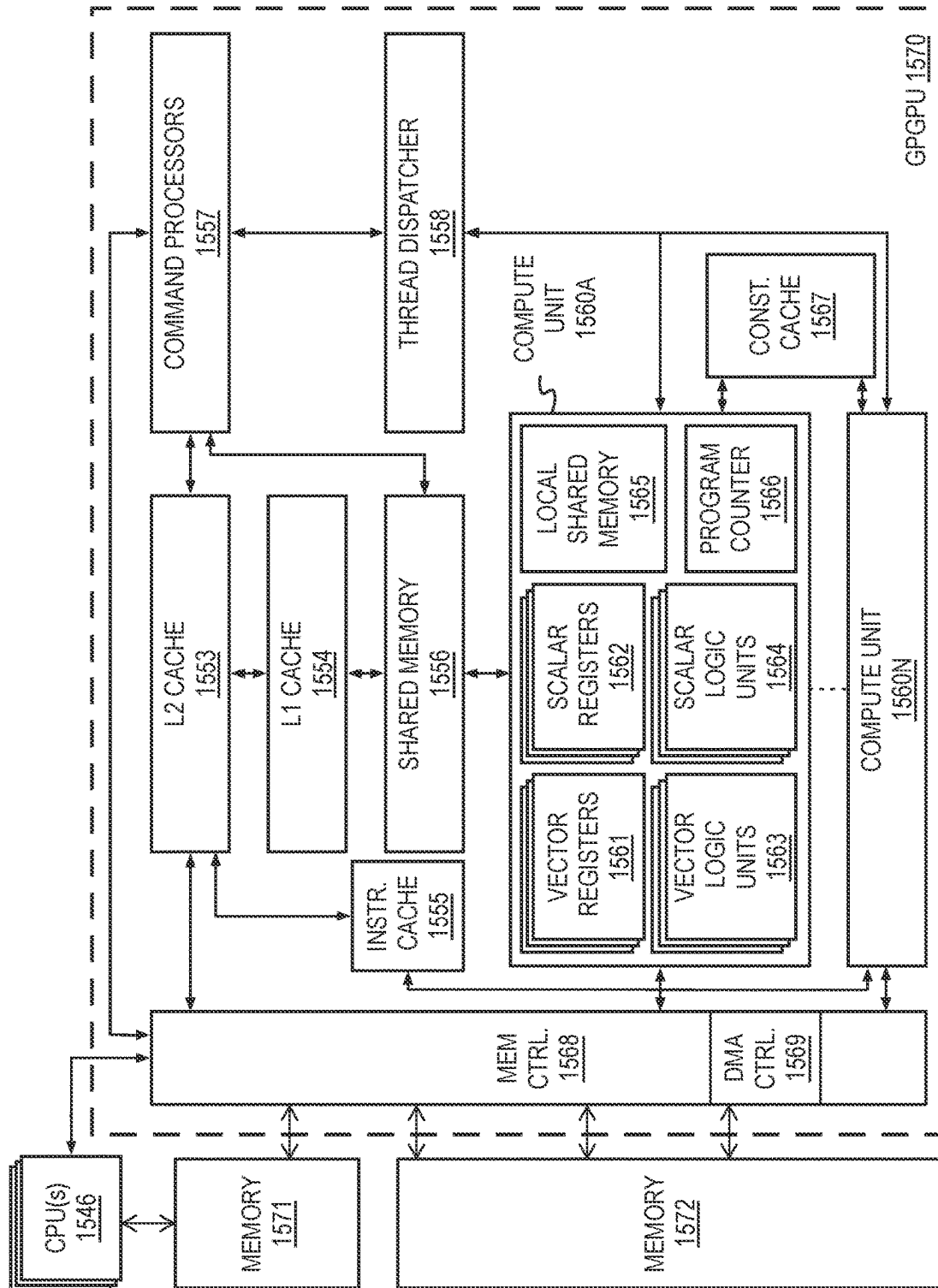


FIG. 15C

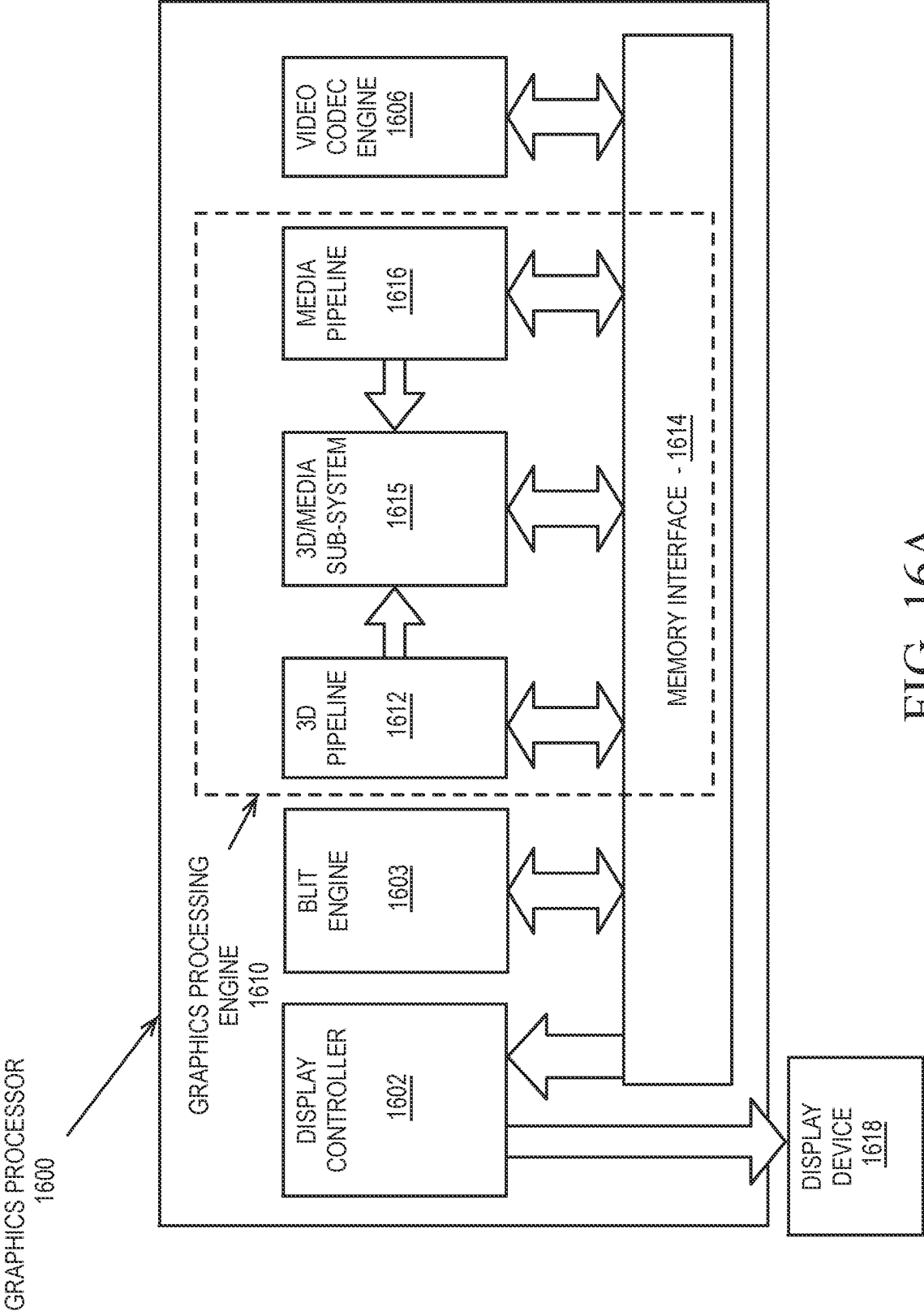


FIG. 16A

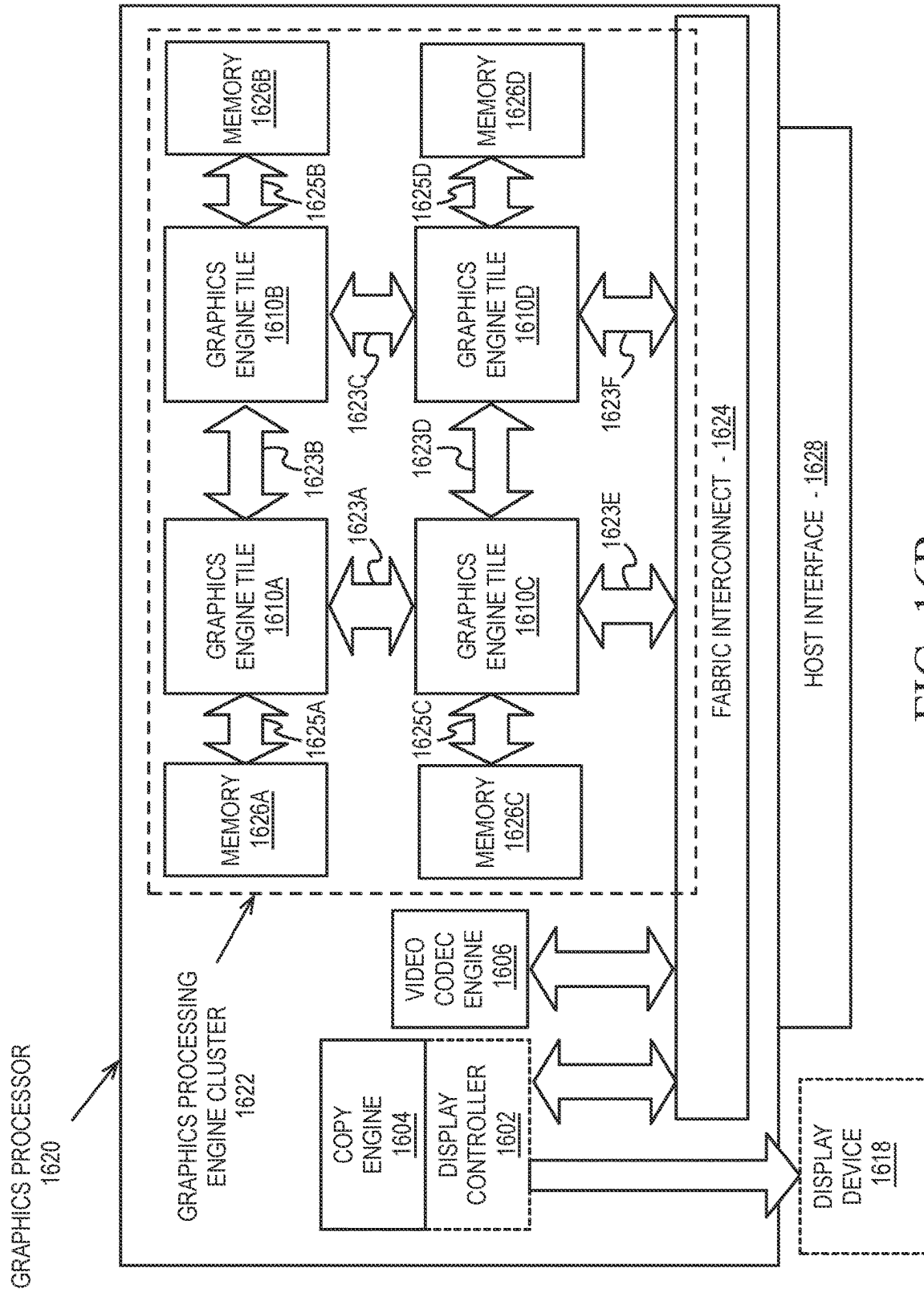


FIG. 16B

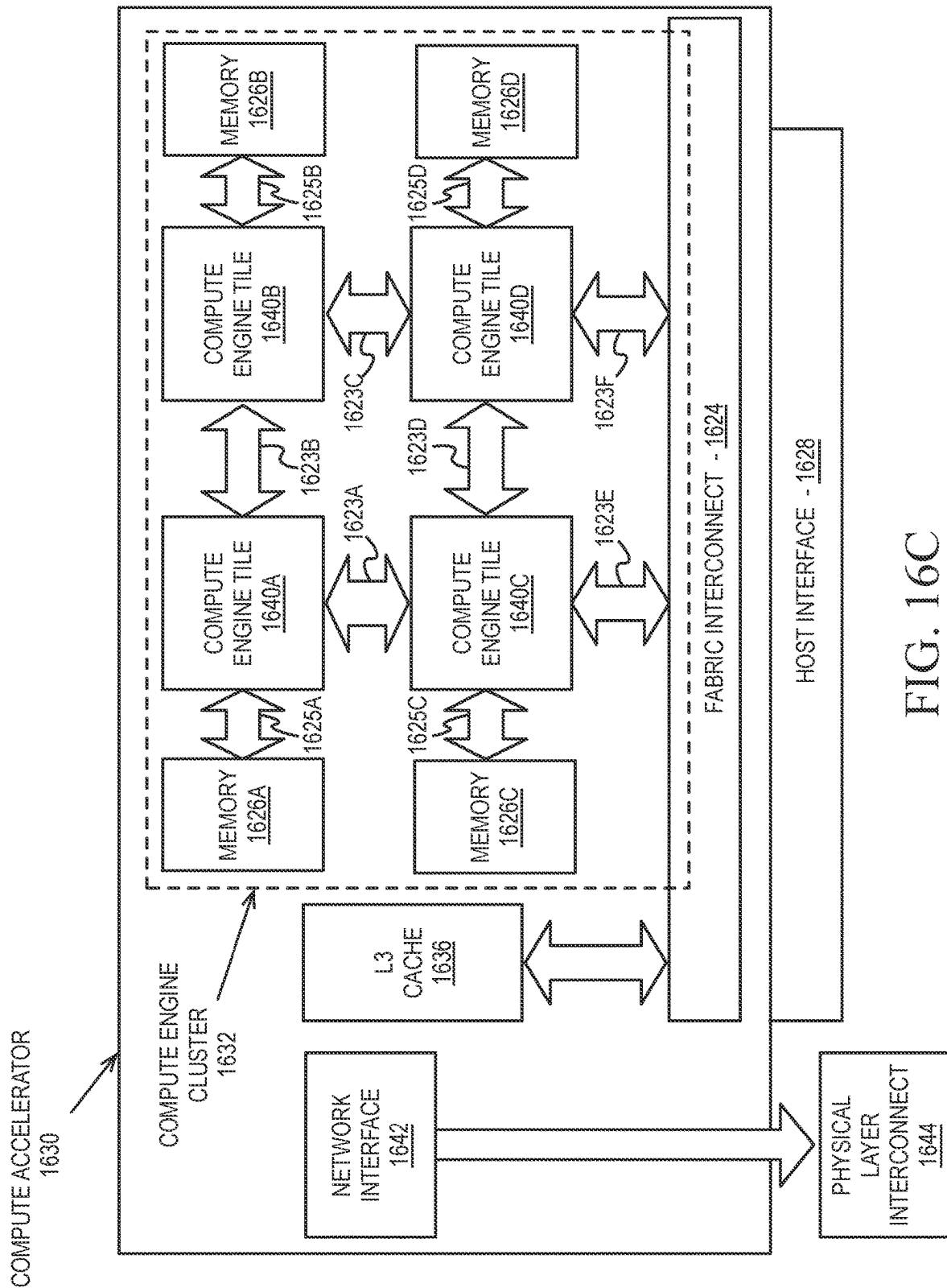


FIG. 16C

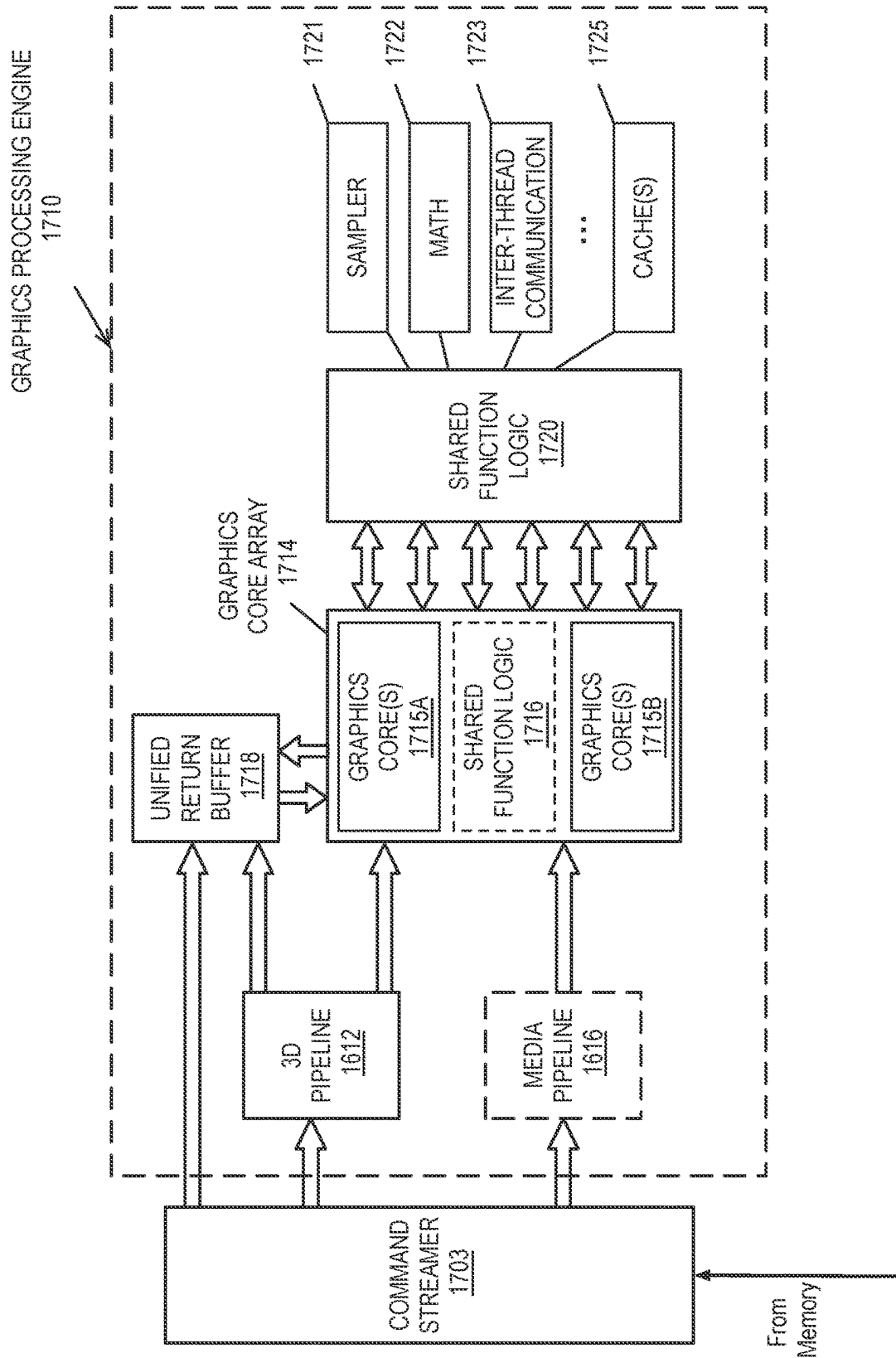


FIG. 17

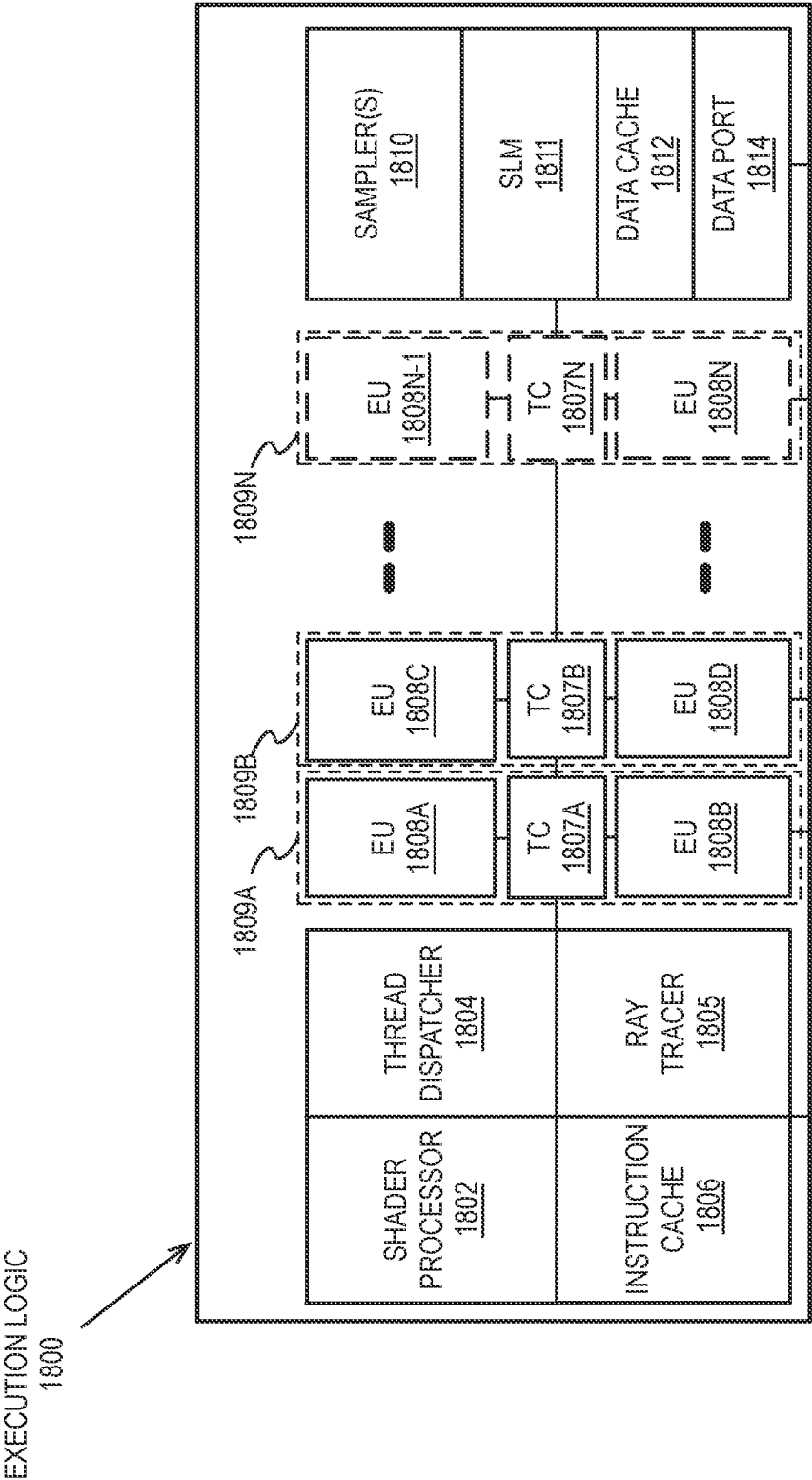


FIG. 18A

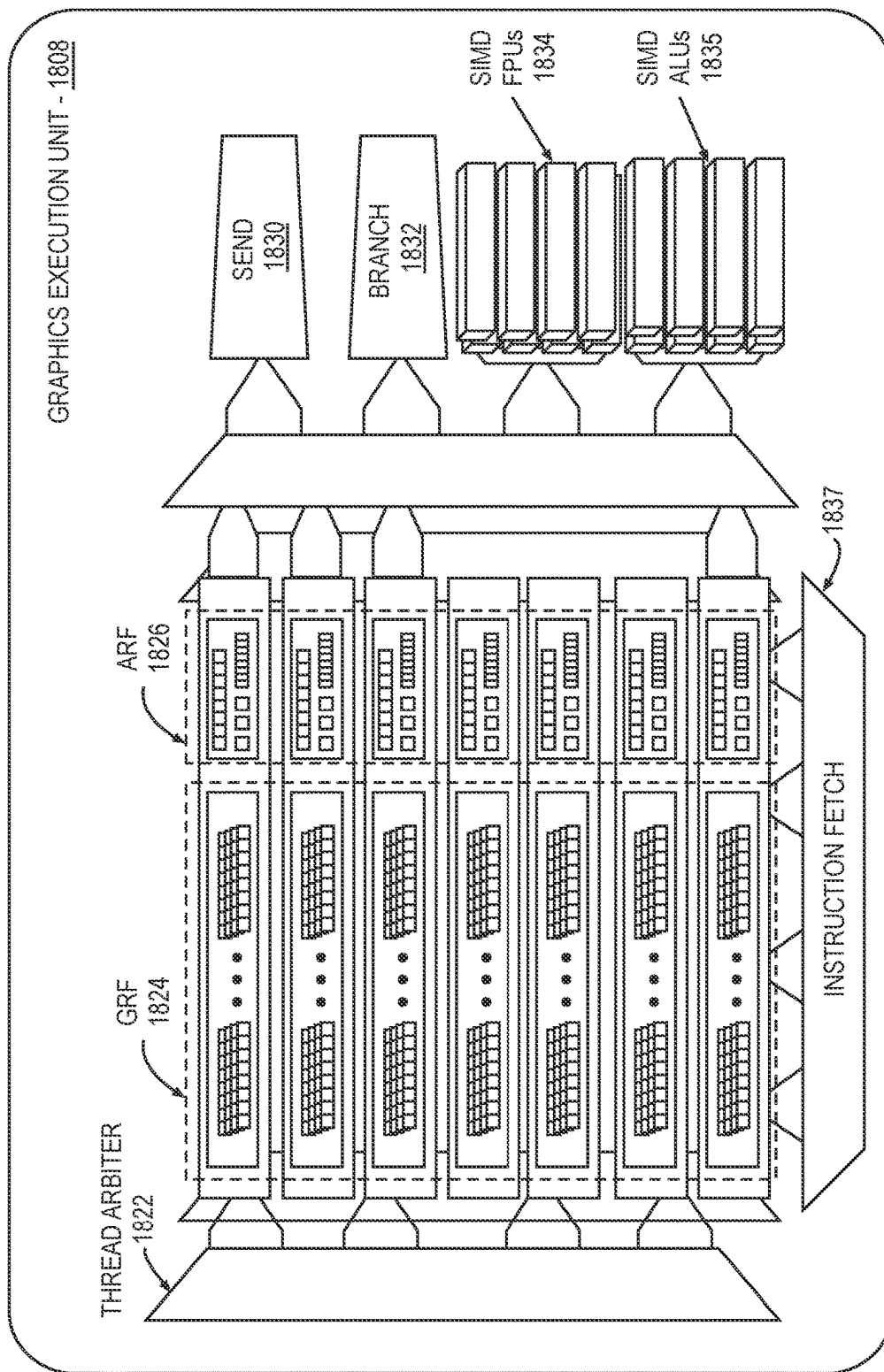


FIG. 18B

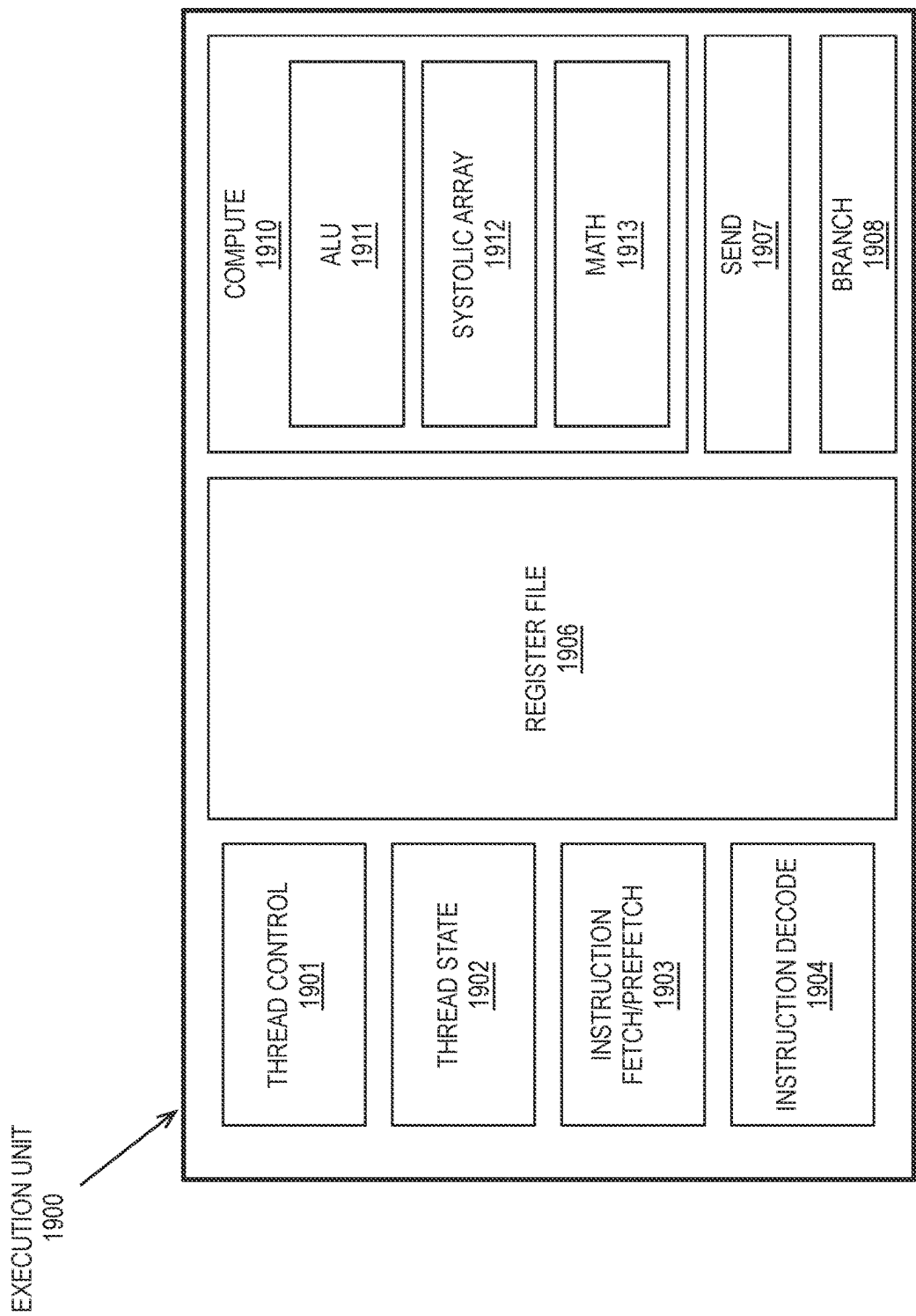


FIG. 19

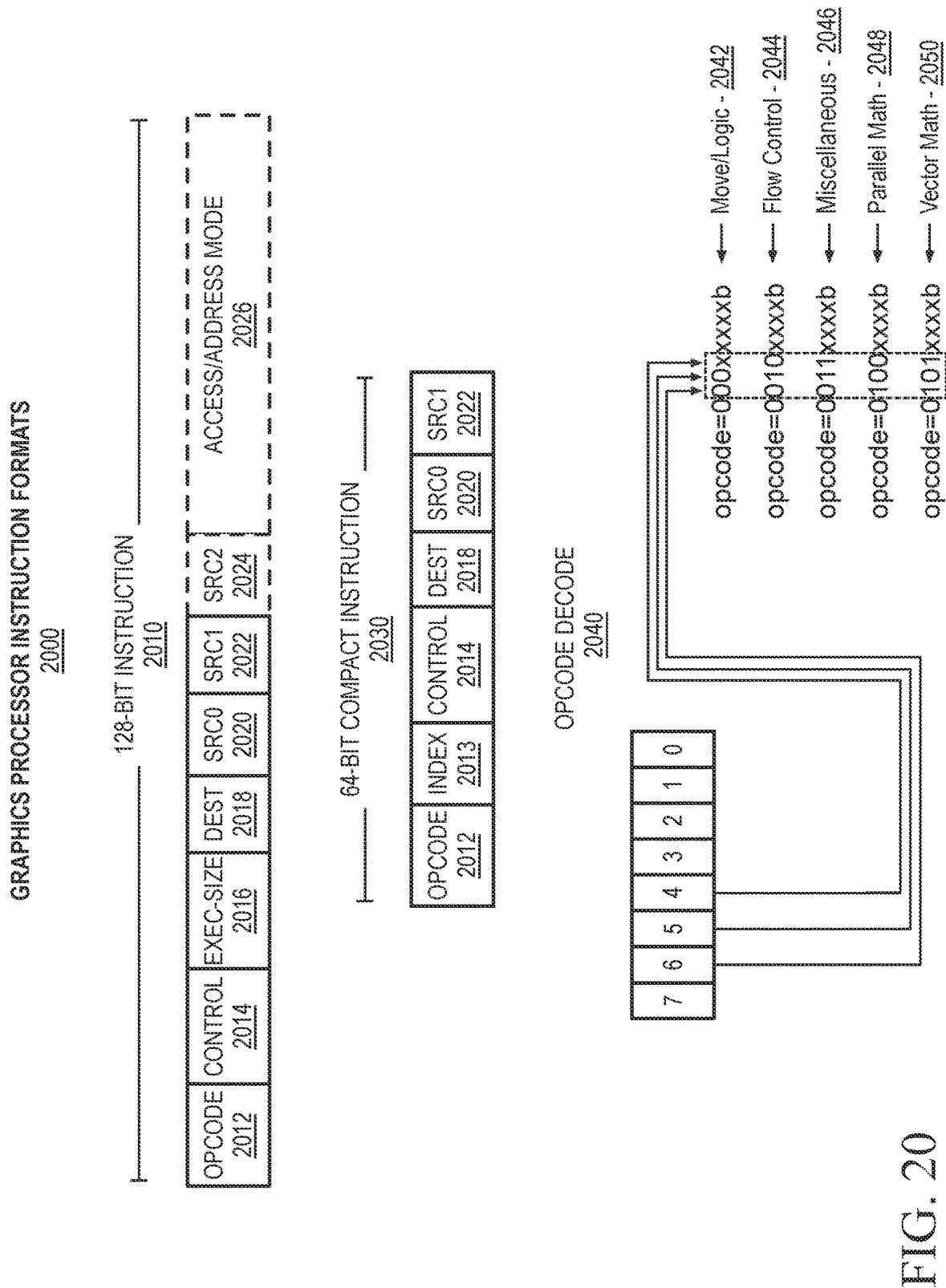


FIG. 20

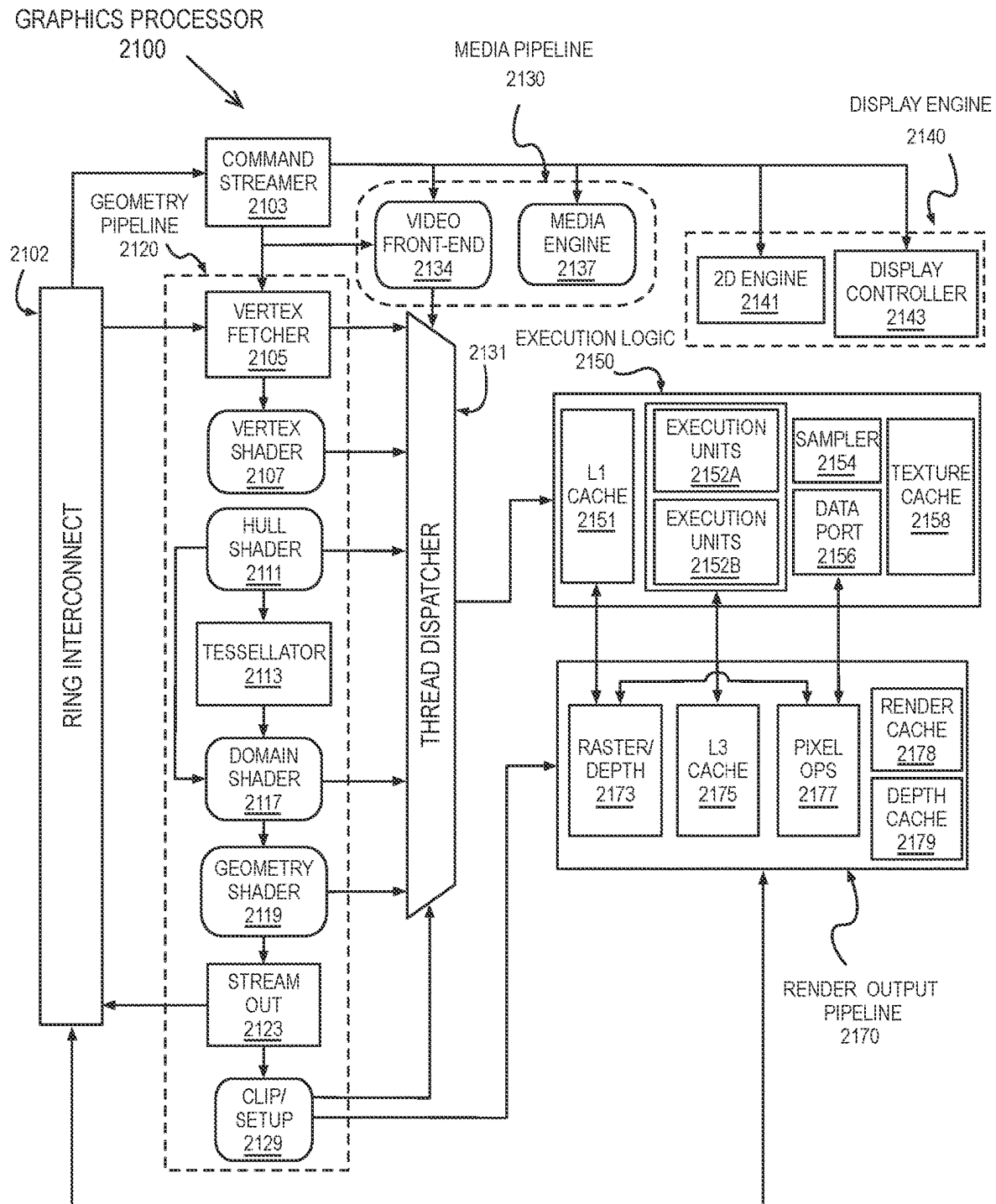


FIG. 21

FIG. 22A GRAPHICS PROCESSOR COMMAND FORMAT
2200

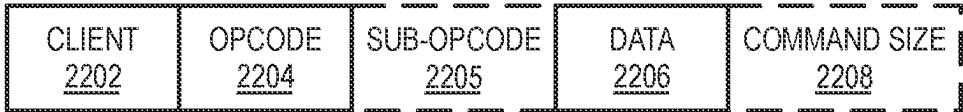
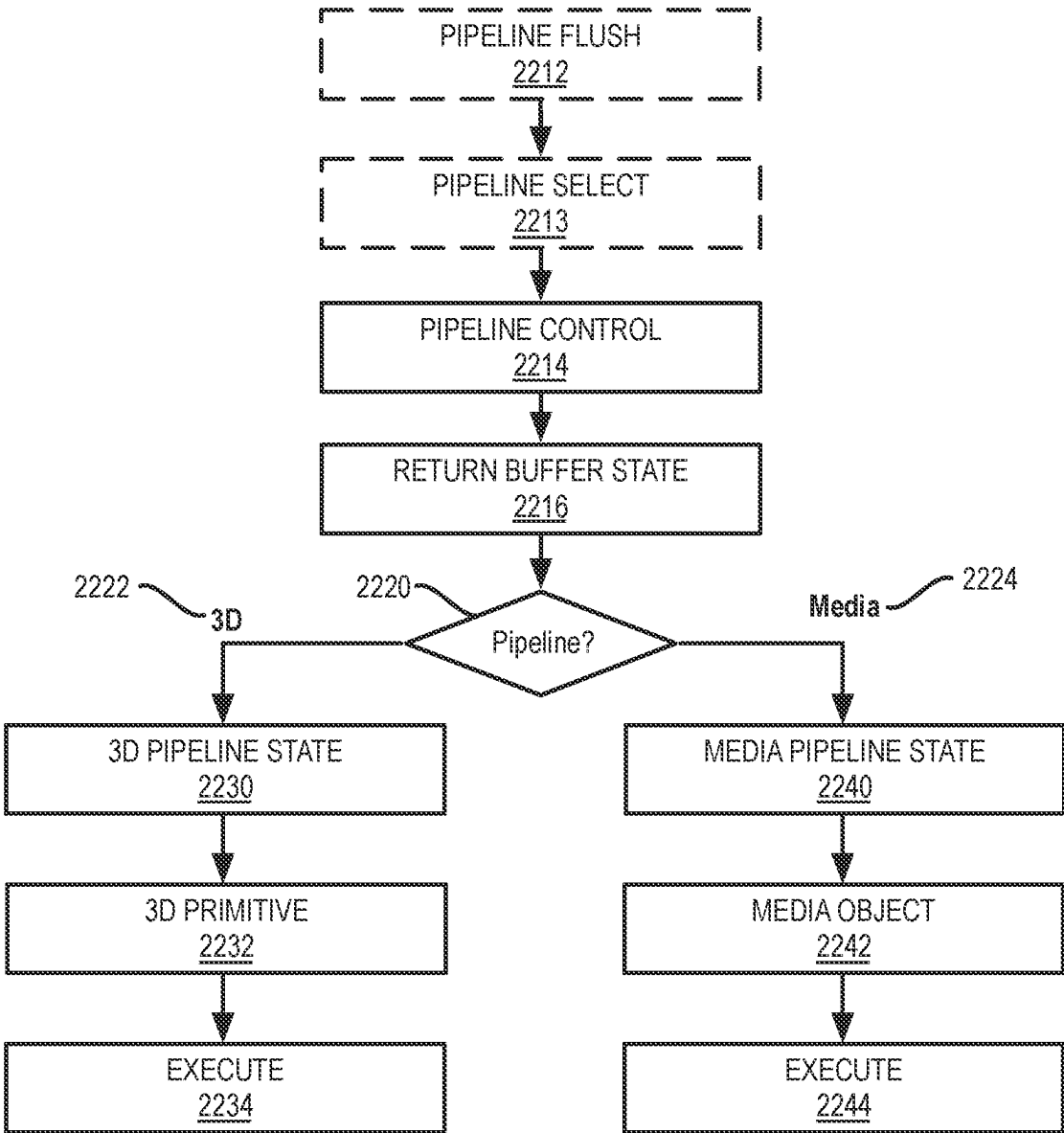


FIG. 22B GRAPHICS PROCESSOR COMMAND SEQUENCE
2210



DATA PROCESSING SYSTEM - 2300

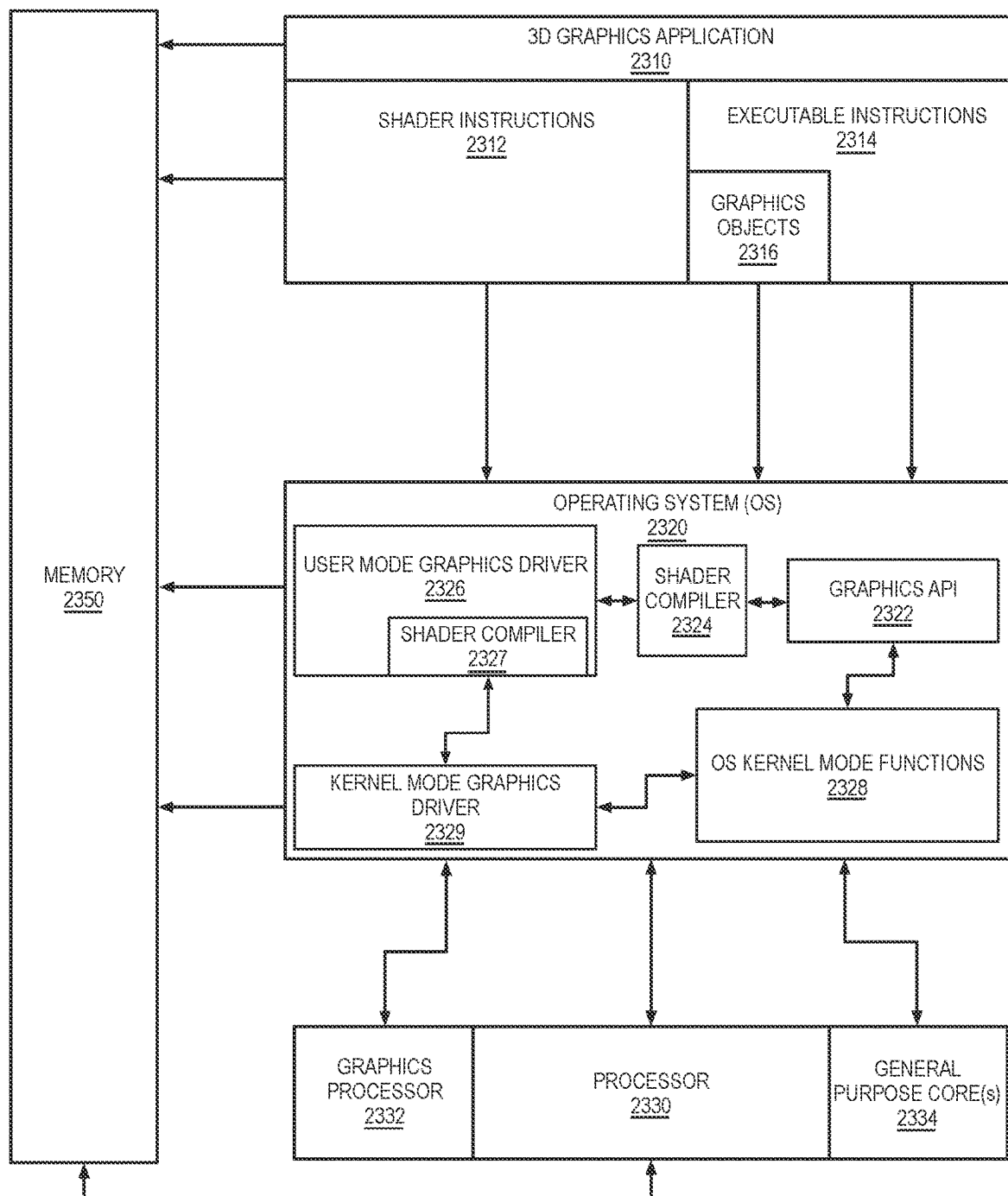


FIG. 23

IP CORE DEVELOPMENT - 2400

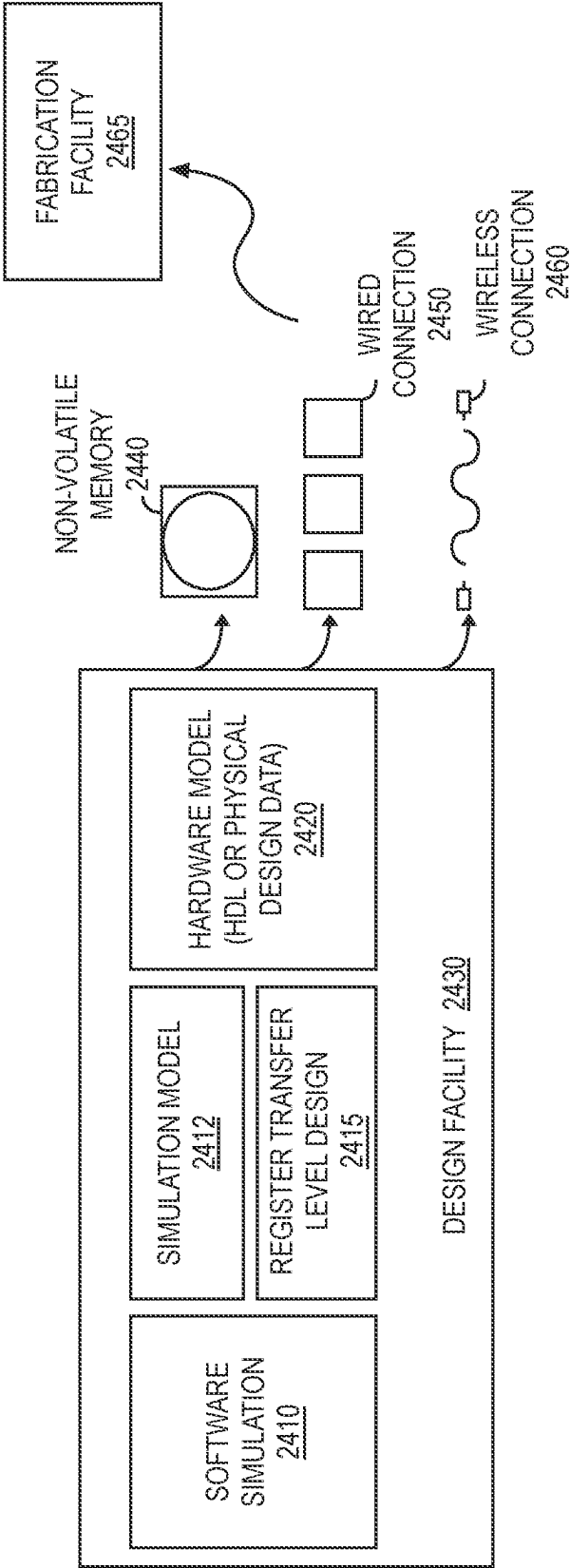


FIG. 24A

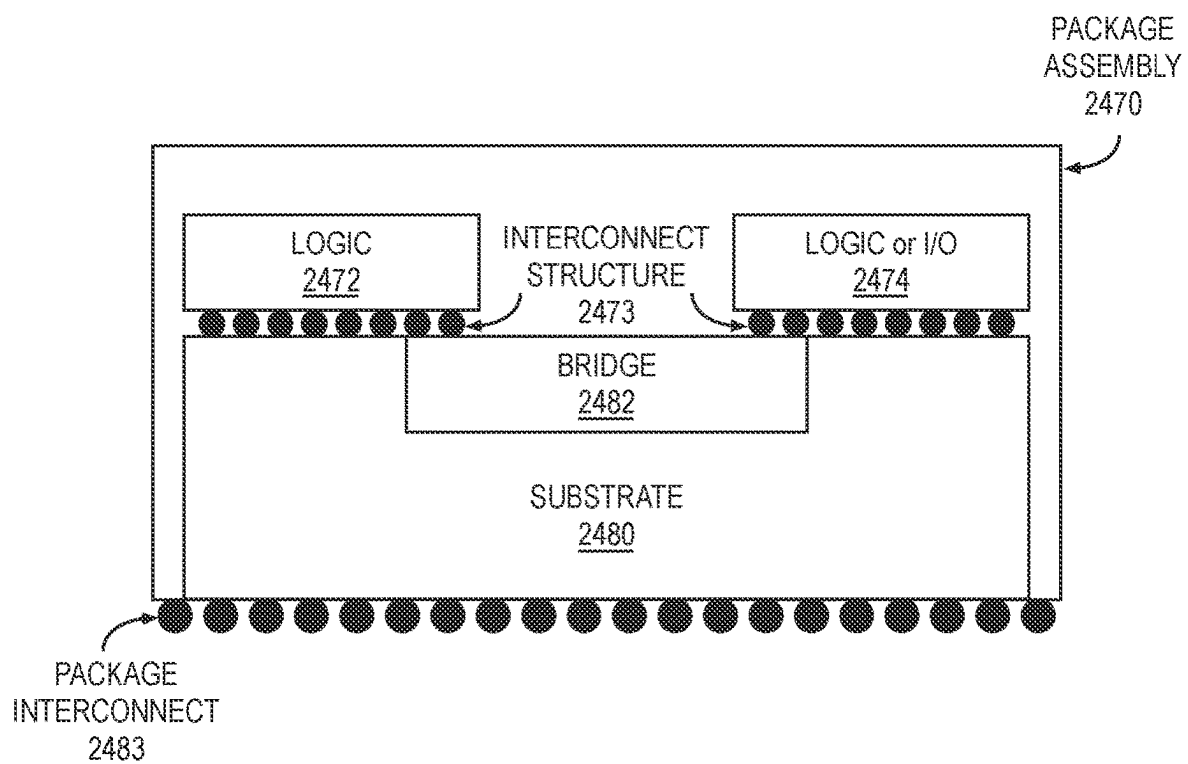


FIG. 24B

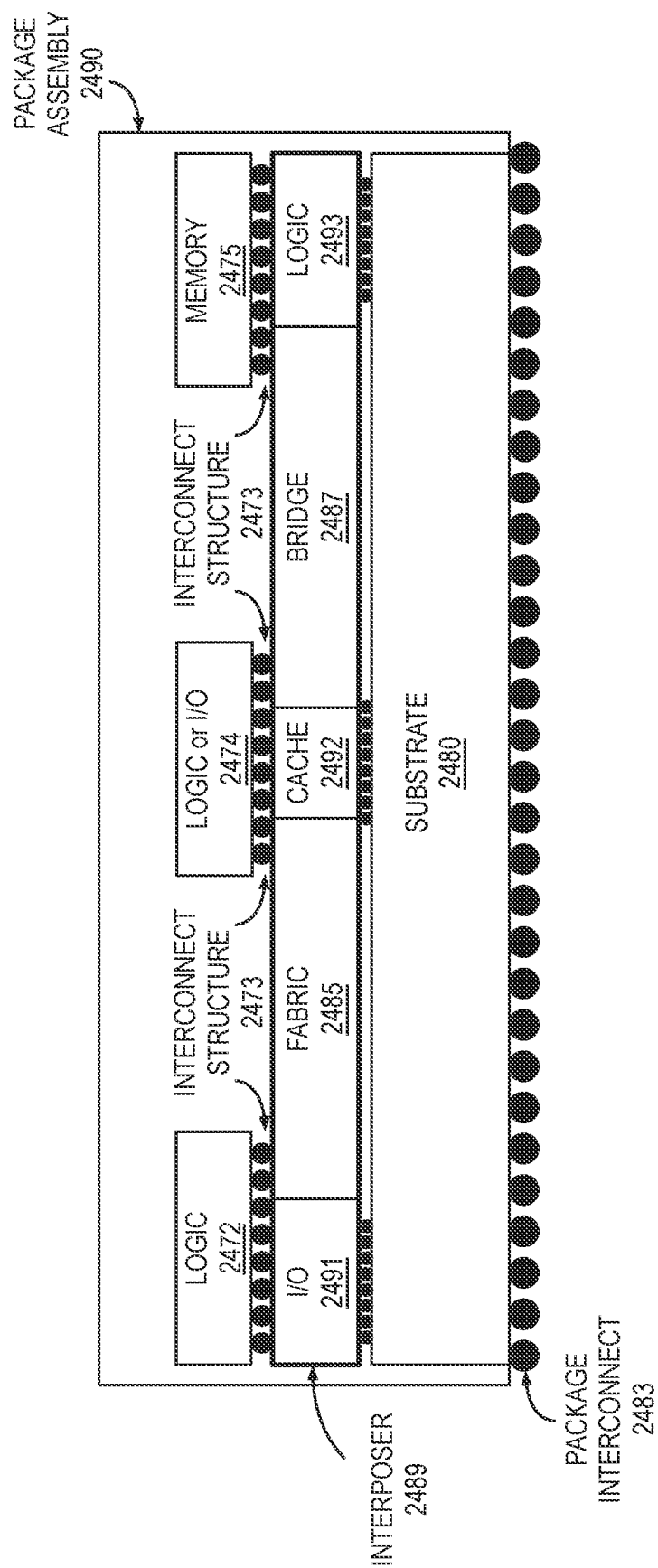


FIG. 24C

2494

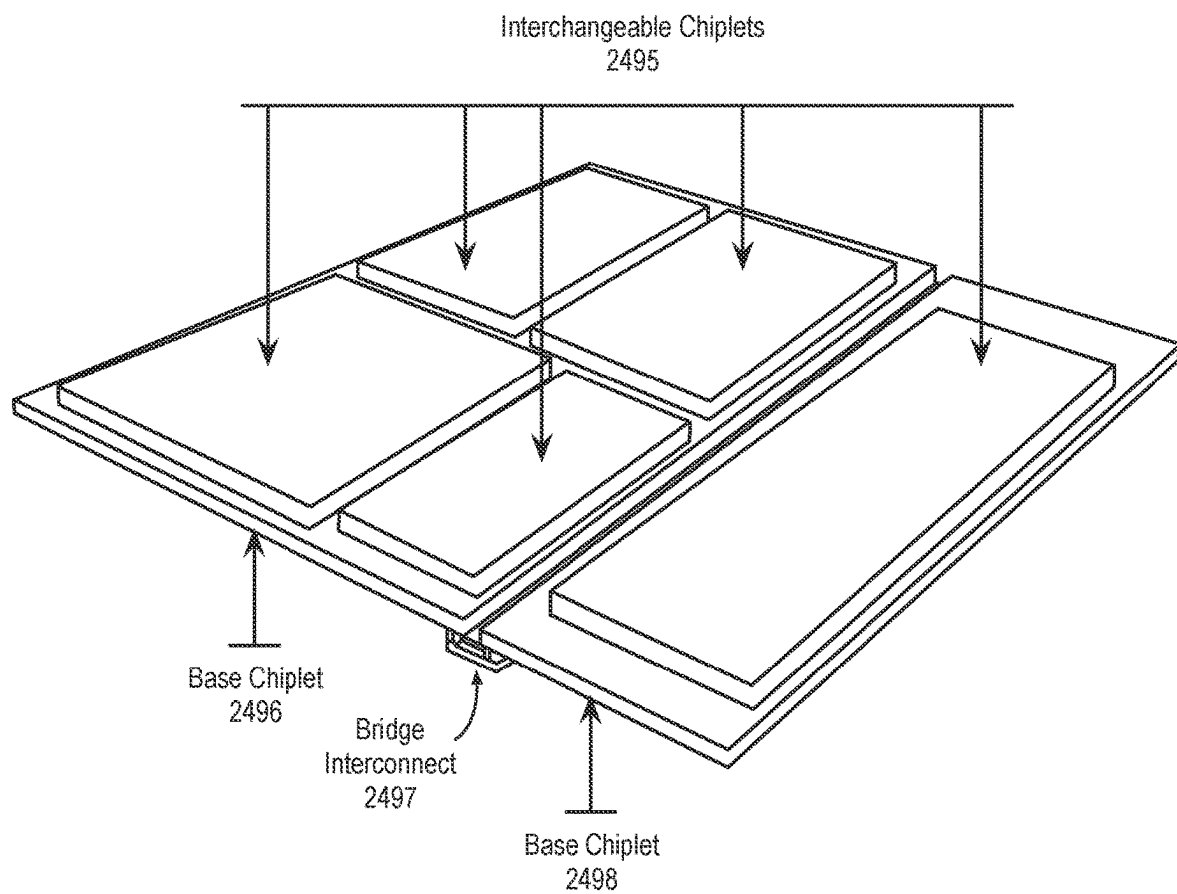


FIG. 24D

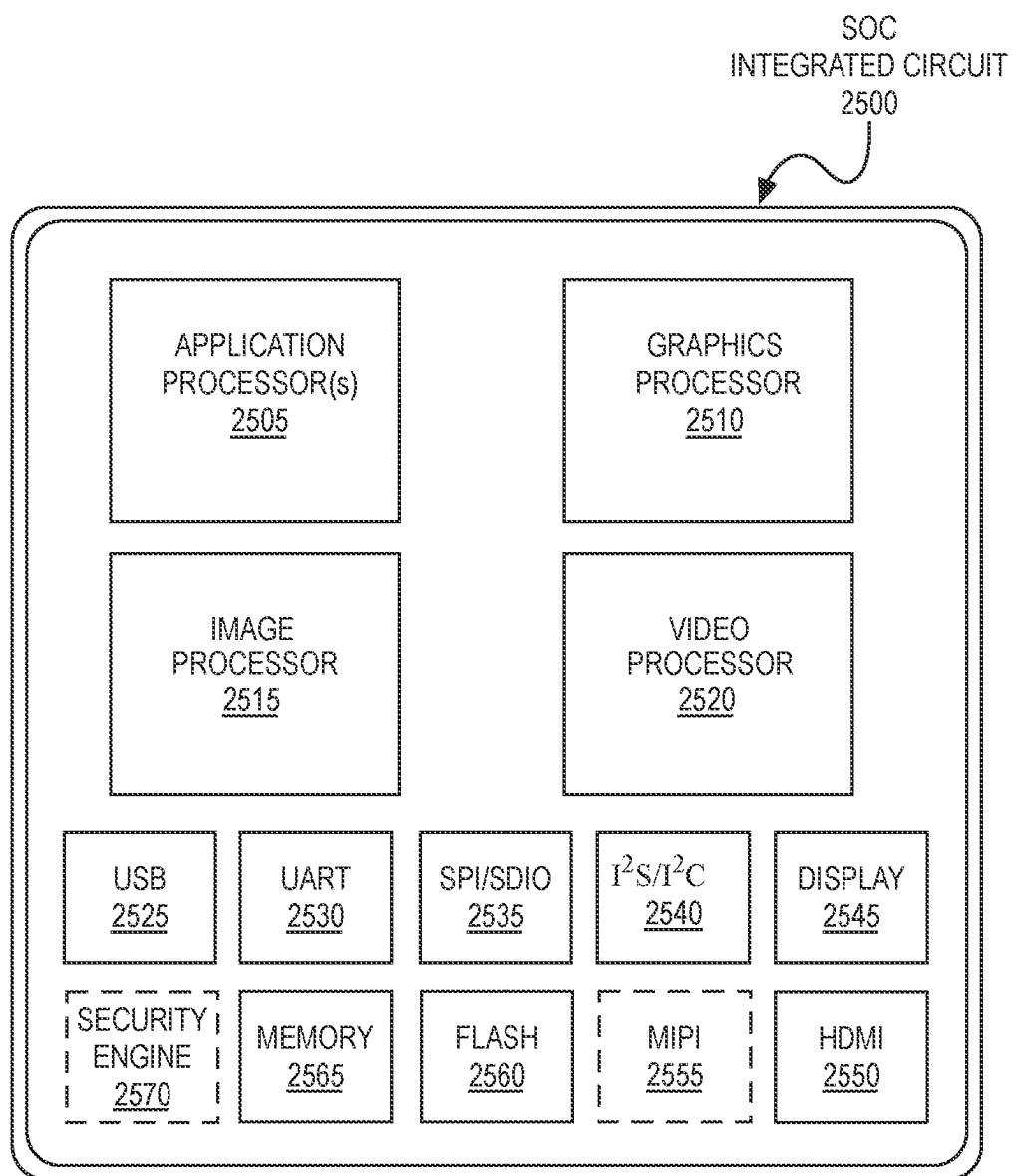


FIG. 25

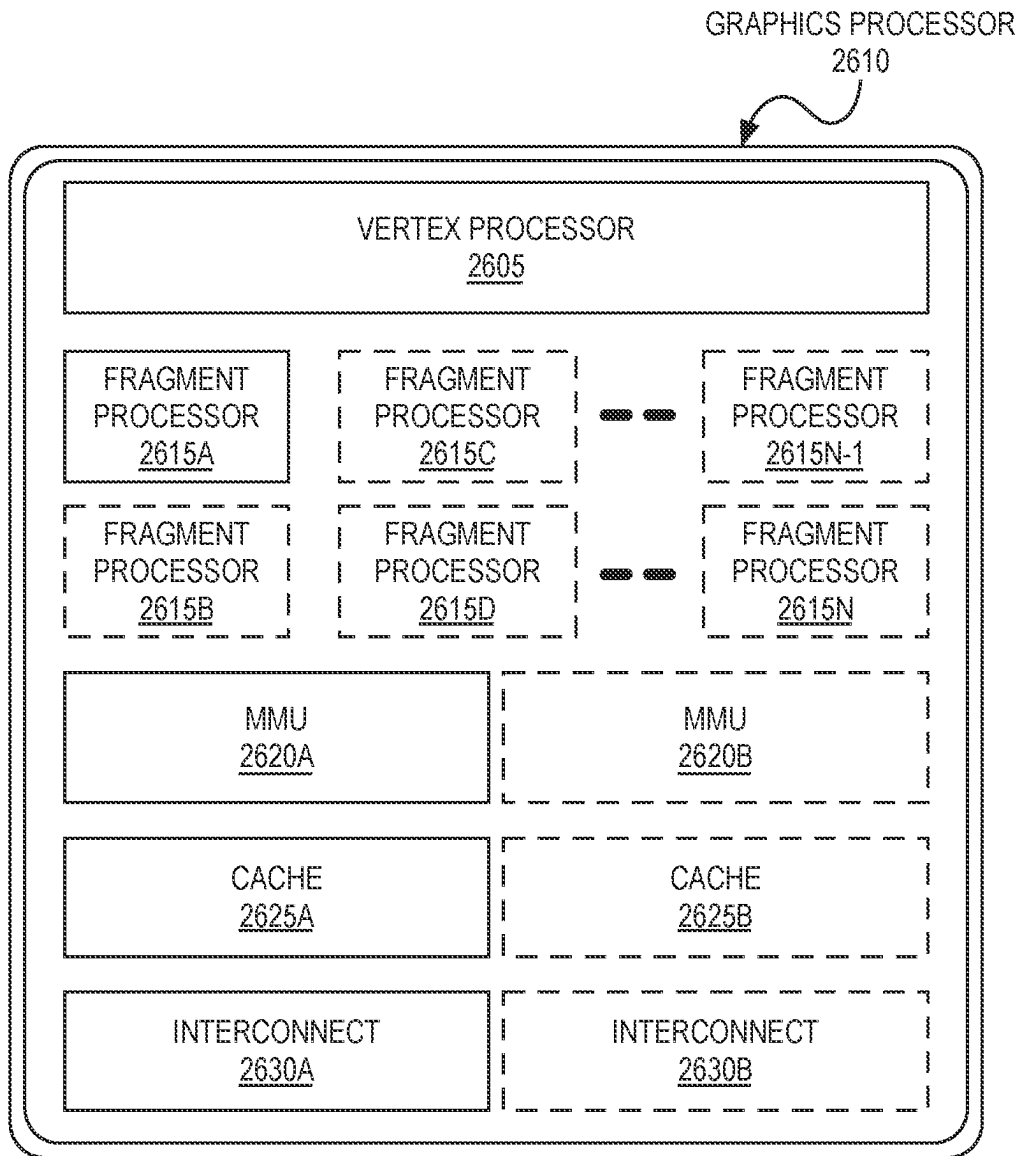


FIG. 26A

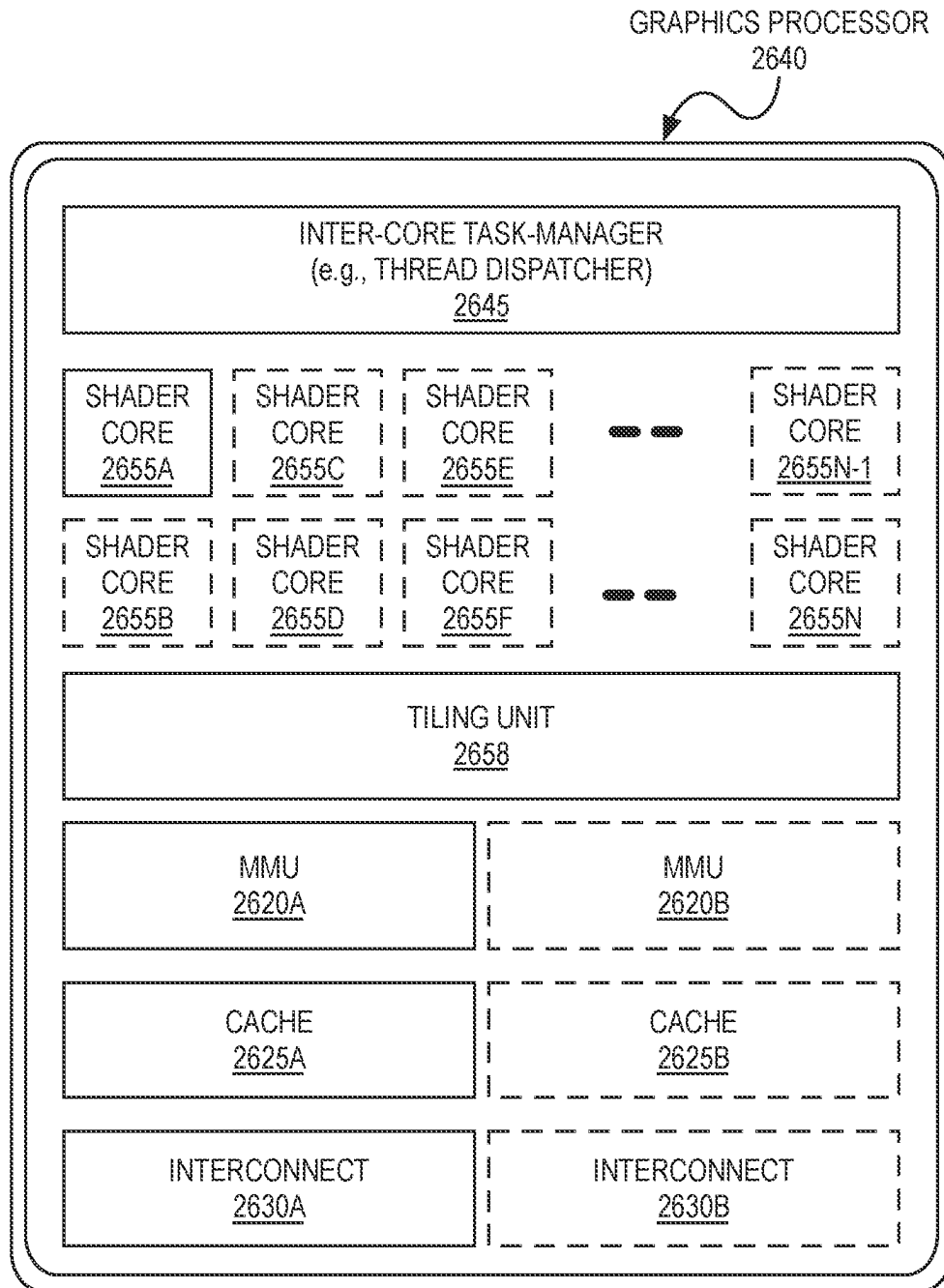


FIG. 26B

2700

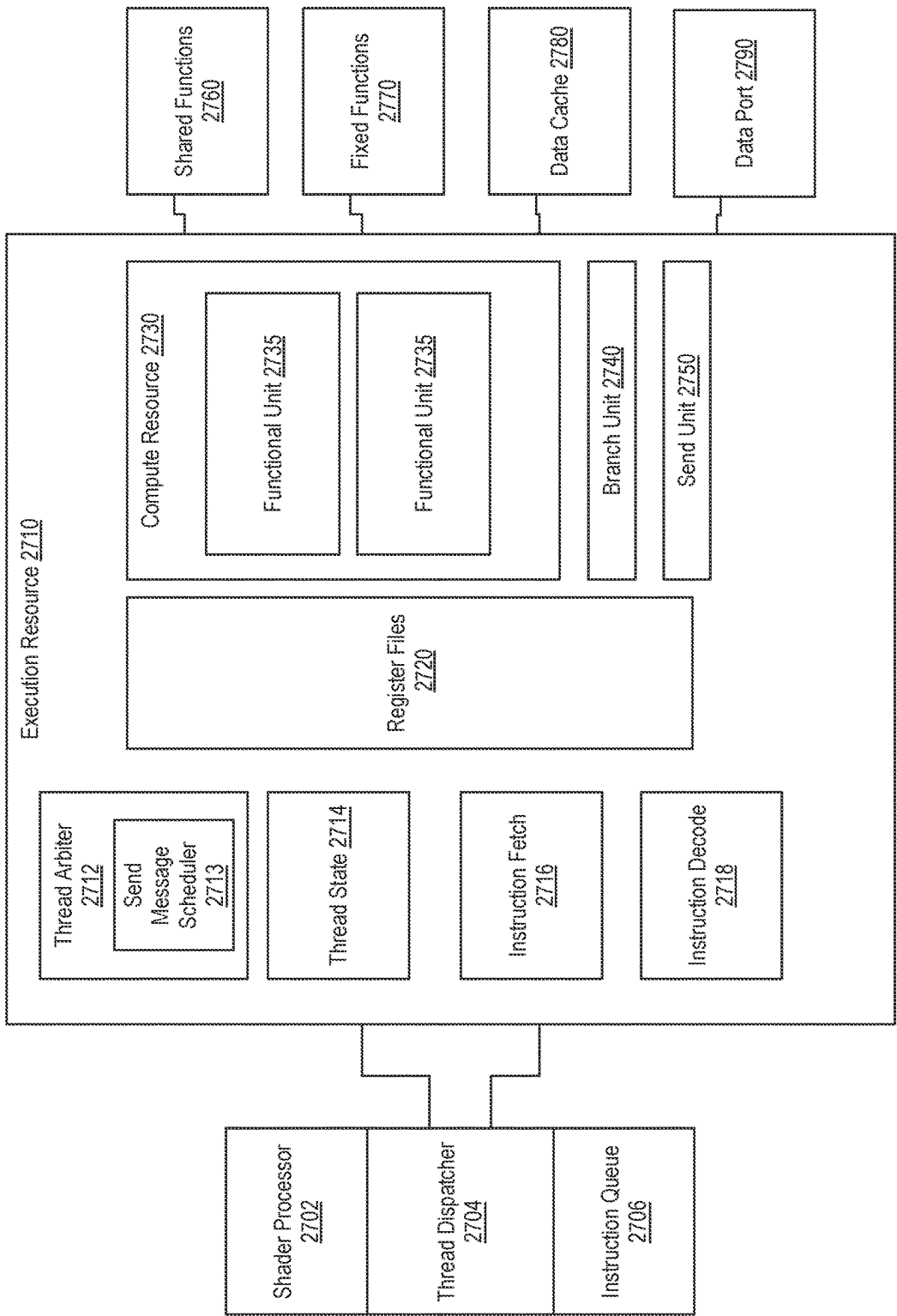
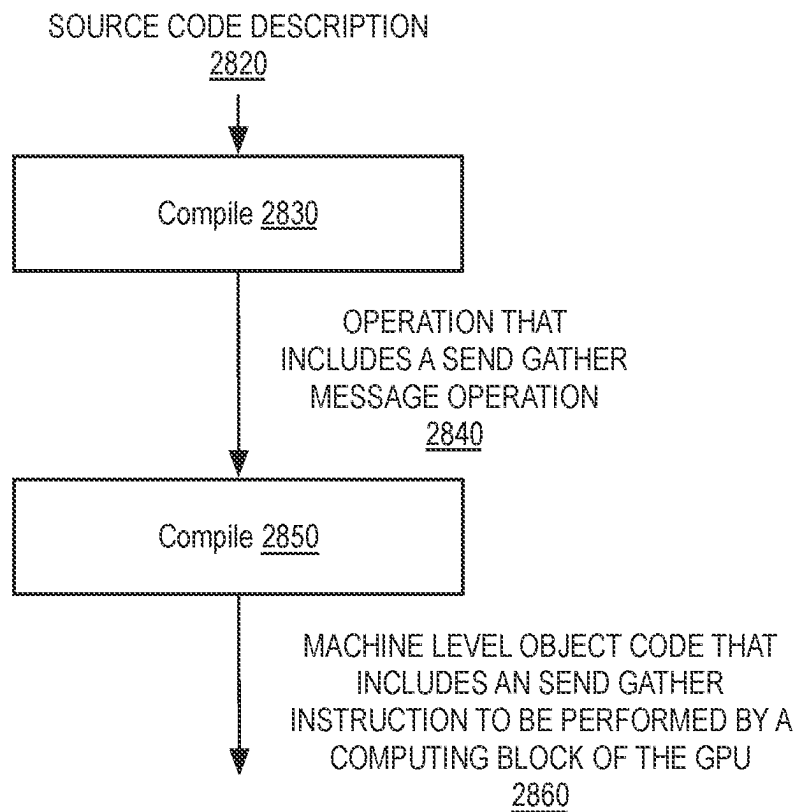


FIG. 27

2800

<u>2802</u>	<u>2804</u>	<u>2806</u>	<u>2808</u>	<u>2810</u>
sendg	dst	SrcVec Length	ex_desc	SrcVec

FIG. 28A2815**FIG. 28B**

2900

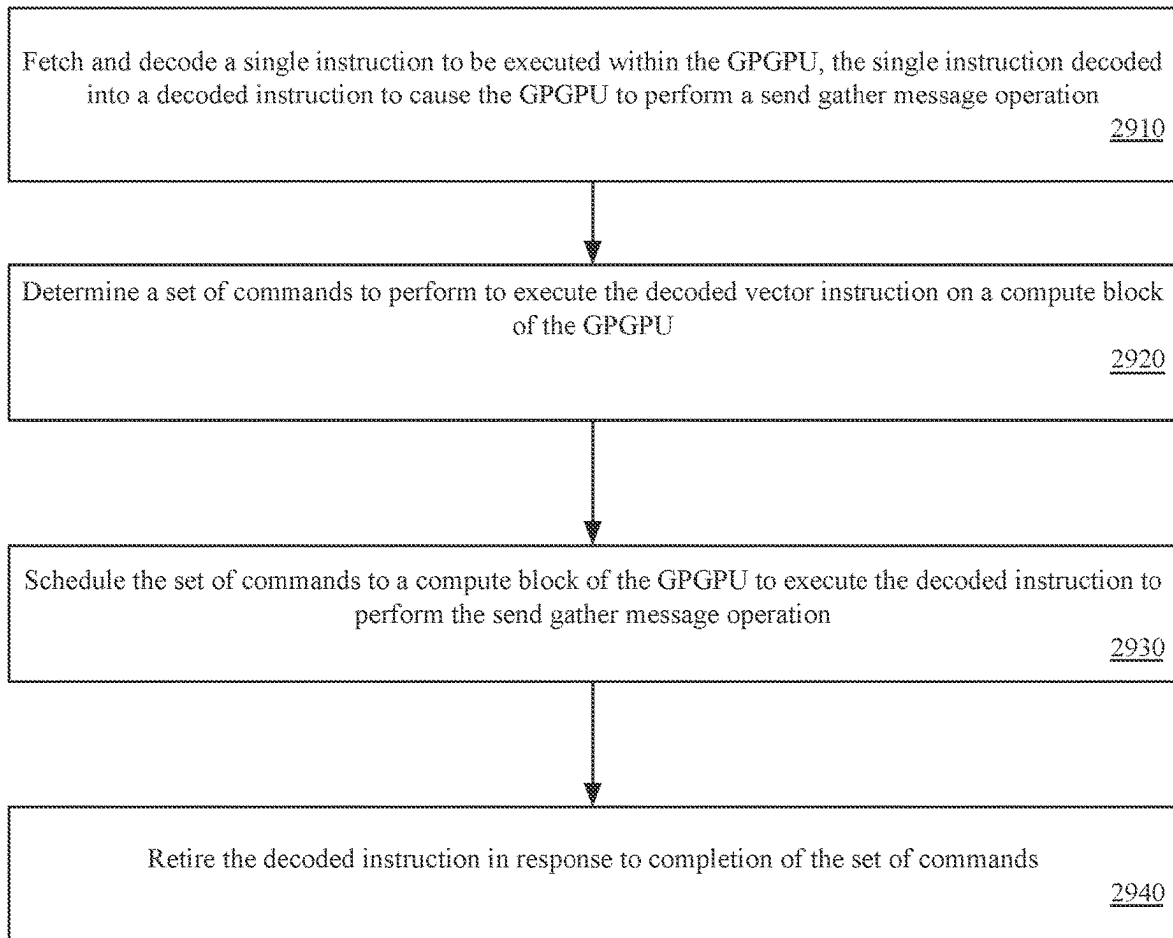
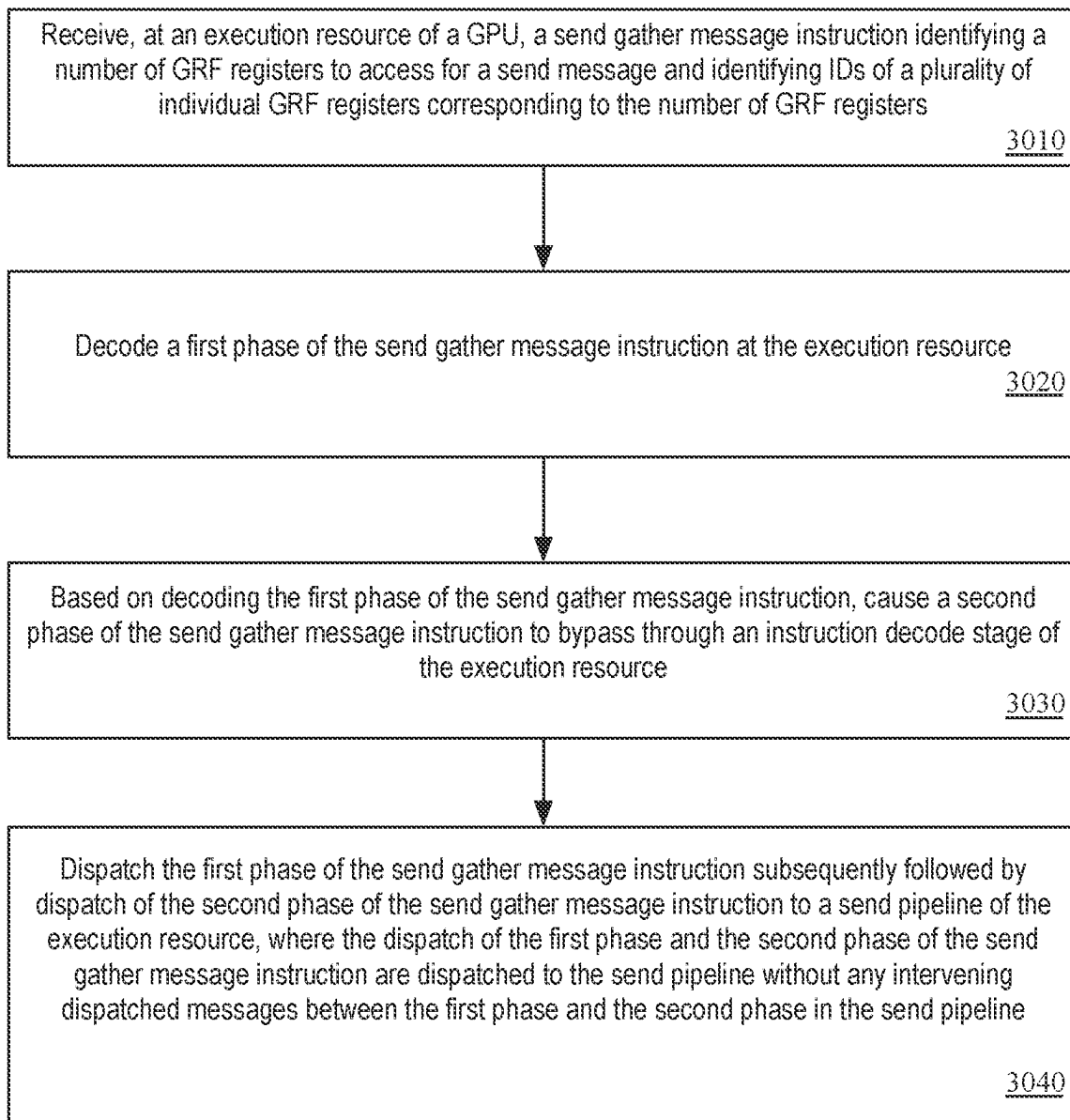
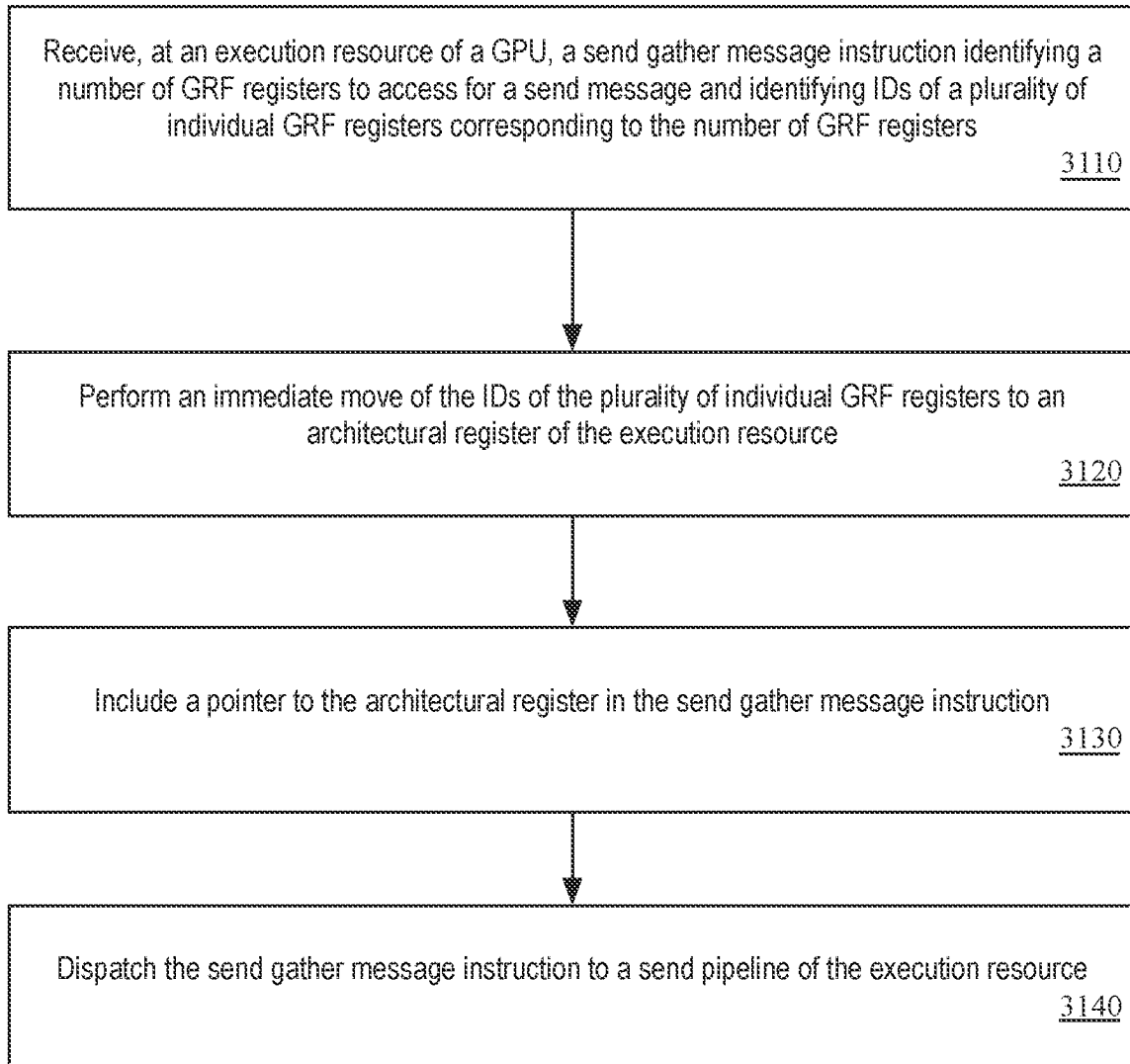


FIG. 29

3000**FIG. 30**

3100**FIG. 31**

1

GATHERING PAYLOAD FROM ARBITRARY REGISTERS FOR SEND MESSAGES IN A GRAPHICS ENVIRONMENT

FIELD

This document relates generally to data processing and more particularly to data processing via a general-purpose graphics processing unit.

BACKGROUND

Current parallel graphics data processing includes systems and methods developed to perform specific operations on graphics data such as, for example, linear interpolation, tessellation, rasterization, texture mapping, depth testing, etc. Traditionally, graphics processors used fixed function computational units to process graphics data. However, more recently, portions of graphics processors have been made programmable, enabling such processors to support a wider variety of operations for processing vertex and fragment data.

To further increase performance, graphics processors typically implement processing techniques such as pipelining that attempt to process, in parallel, as much graphics data as possible throughout the different parts of the graphics pipeline. Parallel graphics processors with single instruction, multiple thread (SIMT) architectures are designed to maximize the amount of parallel processing in the graphics pipeline. In a SIMT architecture, groups of parallel threads attempt to execute program instructions synchronously together as often as possible to increase processing efficiency. A general overview of software and hardware for SIMT architectures can be found in Shane Cook, *CUDA Programming* Chapter 3, pages 37-51 (2013).

Parallel rendering graphics architectures utilize communications between the execution resources (e.g., EUs) and the shared functions and/or fixed function pipelines to enable efficient parallel rendering computations. Such communications between the execution resources and the shared functions and/or fixed functions can be accomplished via packets of information called messages. Message transmission is requested using send message instructions. The current send message instructions require the payload of the instruction to be in contiguous registers. For example, in conventional approaches, the execution resource (e.g., EU) utilizes a send message instruction (e.g., send or sendc instruction (where sendc refers to a conditional send message)), and those send message instructions take a set of GRF registers and send that set of GRF registers out to a shared function (or fixed function).

BRIEF DESCRIPTION OF THE DRAWINGS

So that the manner in which the features of the embodiments can be understood in detail, a more particular description of the embodiments, briefly summarized above, may be had by reference to embodiments, some of which are illustrated in the appended drawings. It is to be noted, however, that the appended drawings illustrate only typical embodiments and are therefore not to be considered limiting of its scope.

FIG. 1 is a block diagram illustrating a computer system configured to implement one or more aspects of the embodiments described herein;

FIG. 2A-2D illustrate parallel processor components;

2

FIG. 3A-3C are block diagrams of graphics multiprocessors and multiprocessor-based GPUs;

FIG. 4A-4F illustrate an example architecture in which a plurality of GPUs is communicatively coupled to a plurality of multi-core processors;

FIG. 5 illustrates a graphics processing pipeline;

FIG. 6 illustrates a machine learning software stack;

FIG. 7 illustrates a general-purpose graphics processing unit;

FIG. 8 illustrates a multi-GPU computing system;

FIG. 9A-9B illustrate layers of example deep neural networks;

FIG. 10 illustrates an example recurrent neural network;

FIG. 11 illustrates training and deployment of a deep neural network;

FIG. 12A is a block diagram illustrating distributed learning;

FIG. 12B is a block diagram illustrating a programmable network interface and data processing unit;

FIG. 13 illustrates an example inferencing system on a chip (SOC) suitable for performing inferencing using a trained model;

FIG. 14 is a block diagram of a processing system;

FIG. 15A-15C illustrate computing systems and graphics processors;

FIG. 16A-16C illustrate block diagrams of additional graphics processor and compute accelerator architectures;

FIG. 17 is a block diagram of a graphics processing engine of a graphics processor;

FIG. 18A-18B illustrate thread execution logic including an array of processing elements employed in a graphics processor core;

FIG. 19 illustrates an additional execution unit;

FIG. 20 is a block diagram illustrating a graphics processor instruction formats;

FIG. 21 is a block diagram of an additional graphics processor architecture;

FIG. 22A-22B illustrate a graphics processor command format and command sequence;

FIG. 23 illustrates example graphics software architecture for a data processing system;

FIG. 24A is a block diagram illustrating an IP core development system;

FIG. 24B illustrates a cross-section side view of an integrated circuit package assembly;

FIG. 24C illustrates a package assembly that includes multiple units of hardware logic chiplets connected to a substrate (e.g., base die);

FIG. 24D illustrates a package assembly including interchangeable chiplets;

FIG. 25 is a block diagram illustrating an example system on a chip integrated circuit;

FIG. 26A-26B are block diagrams illustrating example graphics processors for use within an SoC;

FIG. 27 is a block diagram illustrating an example integrated circuit graphics processor having an execution resource for providing a send gather message instruction to gather payload from arbitrary registers for a send message in a graphics environment, according to embodiments.

FIG. 28A illustrates a set of instructions executable by a processing unit, according to embodiments described herein

FIG. 28B illustrates a program code compilation process, according to an embodiment.

FIG. 29 is a flow diagram illustrating an embodiment of a method for executing an instruction to perform gathering payload from arbitrary registers for send messages in a graphics environment.

FIG. 30 is a flow diagram illustrating an embodiment of a method for scheduling a send gather message that gathers payload from arbitrary registers in a graphics environment.

FIG. 31 is a flow diagram illustrating an embodiment of a method for scheduling a send gather message using an architectural register to gather payload from arbitrary registers in a graphics environment.

DETAILED DESCRIPTION

Implementations are directed to gathering payload from arbitrary registers for send messages in a graphics environment, such as in a graphics processing unit (GPU).

A graphics processing unit (GPU) is communicatively coupled to host/processor cores to accelerate, for example, graphics operations, machine-learning operations, pattern analysis operations, and/or various general-purpose GPU (GPGPU) functions. The GPU may be communicatively coupled to the host processor/cores over a bus or another interconnect (e.g., a high-speed interconnect such as PCIe or NVLink). Alternatively, the GPU may be integrated on the same package or chip as the cores and communicatively coupled to the cores over an internal processor bus/interconnect (i.e., internal to the package or chip). Regardless of the manner in which the GPU is connected, the processor cores may allocate work to the GPU in the form of sequences of commands/instructions contained in a work descriptor. The GPU then uses dedicated circuitry/logic for efficiently processing these commands/instructions.

In the following description, numerous specific details are set forth to provide a more thorough understanding. However, it can be apparent to one of skill in the art that the embodiments described herein may be practiced without one or more of these specific details. In other instances, well-known features have not been described to avoid obscuring the details of the present embodiments.

System Overview

FIG. 1 is a block diagram illustrating a computing system 100 configured to implement one or more aspects of the embodiments described herein. The computing system 100 includes a processing subsystem 101 having one or more processor(s) 102 and a system memory 104 communicating via an interconnection path that may include a memory hub 105. The memory hub 105 may be a separate component within a chipset component or may be integrated within the one or more processor(s) 102. The memory hub 105 couples with an I/O subsystem 111 via a communication link 106. The I/O subsystem 111 includes an I/O hub 107 that can enable the computing system 100 to receive input from one or more input device(s) 108. Additionally, the I/O hub 107 can enable a display controller, which may be included in the one or more processor(s) 102, to provide outputs to one or more display device(s) 110A. In one embodiment the one or more display device(s) 110A coupled with the I/O hub 107 can include a local, internal, or embedded display device.

The processing subsystem 101, for example, includes one or more parallel processor(s) 112 coupled to memory hub 105 via a bus or other communication link 113. The communication link 113 may be one of any number of standards-based communication link technologies or protocols, such as, but not limited to PCI Express, or may be a vendor specific communications interface or communications fabric. The one or more parallel processor(s) 112 may form a computationally focused parallel or vector processing system that can include a large number of processing cores and/or processing clusters, such as a many integrated core (MIC) processor. For example, the one or more parallel

processor(s) 112 form a graphics processing subsystem that can output pixels to one of the one or more display device(s) 110A coupled via the I/O hub 107. The one or more parallel processor(s) 112 can also include a display controller and display interface (not shown) to enable a direct connection to one or more display device(s) 110B.

Within the I/O subsystem 111, a system storage unit 114 can connect to the I/O hub 107 to provide a storage mechanism for the computing system 100. An I/O switch 116 can be used to provide an interface mechanism to enable connections between the I/O hub 107 and other components, such as a network adapter 118 and/or wireless network adapter 119 that may be integrated into the platform, and various other devices that can be added via one or more add-in device(s) 120. The add-in device(s) 120 may also include, for example, one or more external graphics processor devices, graphics cards, and/or compute accelerators. The network adapter 118 can be an Ethernet adapter or another wired network adapter. The wireless network adapter 119 can include one or more of a Wi-Fi, Bluetooth, near field communication (NFC), or other network device that includes one or more wireless radios.

The computing system 100 can include other components not explicitly shown, including USB or other port connections, optical storage drives, video capture devices, and the like, which may also be connected to the I/O hub 107. Communication paths interconnecting the various components in FIG. 1 may be implemented using any suitable protocols, such as PCI (Peripheral Component Interconnect) based protocols (e.g., PCI-Express), or any other bus or point-to-point communication interfaces and/or protocol(s), such as the NVLink high-speed interconnect, Compute Express Link™ (CXL™) (e.g., CXL.mem), Infinity Fabric (IF), Ethernet (IEEE 802.3), remote direct memory access (RDMA), InfiniBand, Internet Wide Area RDMA Protocol (iWARP), Transmission Control Protocol (TCP), User Datagram Protocol (UDP), quick UDP Internet Connections (QUIC), RDMA over Converged Ethernet (RoCE), Intel QuickPath Interconnect (QPI), Intel Ultra Path Interconnect (UPI), Intel On-Chip System Fabric (IOSF), Omnipath, HyperTransport, Advanced Microcontroller Bus Architecture (AMBA) interconnect, OpenCAPI, Gen-Z, Cache Coherent Interconnect for Accelerators (CCIX), 3GPP Long Term Evolution (LTE) (4G), 3GPP 5G, and variations thereof, or wired or wireless interconnect protocols known in the art. In some examples, data can be copied or stored to virtualized storage nodes using a protocol such as non-volatile memory express (NVMe) over Fabrics (NVMe-oF) or NVMe.

The one or more parallel processor(s) 112 may incorporate circuitry optimized for graphics and video processing, including, for example, video output circuitry, and constitutes a graphics processing unit (GPU). Alternatively or additionally, the one or more parallel processor(s) 112 can incorporate circuitry optimized for general purpose processing, while preserving the underlying computational architecture, described in greater detail herein. Components of the computing system 100 may be integrated with one or more other system elements on a single integrated circuit. For example, the one or more parallel processor(s) 112, memory hub 105, processor(s) 102, and I/O hub 107 can be integrated into a system on chip (SoC) integrated circuit. Alternatively, the components of the computing system 100 can be integrated into a single package to form a system in package (SIP) configuration. In one embodiment at least a portion of the components of the computing system 100 can

be integrated into a multi-chip module (MCM), which can be interconnected with other multi-chip modules into a modular computing system.

It can be appreciated that the computing system **100** shown herein is illustrative and that variations and modifications are possible. The connection topology, including the number and arrangement of bridges, the number of processor(s) **102**, and the number of parallel processor(s) **112**, may be modified as desired. For instance, system memory **104** can be connected to the processor(s) **102** directly rather than through a bridge, while other devices communicate with system memory **104** via the memory hub **105** and the processor(s) **102**. In other alternative topologies, the parallel processor(s) **112** are connected to the I/O hub **107** or directly to one of the one or more processor(s) **102**, rather than to the memory hub **105**. In other embodiments, the I/O hub **107** and memory hub **105** may be integrated into a single chip. It is also possible that two or more sets of processor(s) **102** are attached via multiple sockets, which can couple with two or more instances of the parallel processor(s) **112**.

Some of the particular components shown herein are optional and may not be included in all implementations of the computing system **100**. For example, any number of add-in cards or peripherals may be supported, or some components may be eliminated. Furthermore, some architectures may use different terminology for components similar to those illustrated in FIG. 1. For example, the memory hub **105** may be referred to as a Northbridge in some architectures, while the I/O hub **107** may be referred to as a Southbridge.

FIG. 2A illustrates a parallel processor **200**. The parallel processor **200** may be a GPU, GPGPU or the like as described herein. The various components of the parallel processor **200** may be implemented using one or more integrated circuit devices, such as programmable processors, application specific integrated circuits (ASICs), or field programmable gate arrays (FPGA). The illustrated parallel processor **200** may be one or more of the parallel processor(s) **112** shown in FIG. 1.

The parallel processor **200** includes a parallel processing unit **202**. The parallel processing unit includes an I/O unit **204** that enables communication with other devices, including other instances of the parallel processing unit **202**. The I/O unit **204** may be directly connected to other devices. For instance, the I/O unit **204** connects with other devices via the use of a hub or switch interface, such as memory hub **105**. The connections between the memory hub **105** and the I/O unit **204** form a communication link **113**. Within the parallel processing unit **202**, the I/O unit **204** connects with a host interface **206** and a memory crossbar **216**, where the host interface **206** receives commands directed to performing processing operations and the memory crossbar **216** receives commands directed to performing memory operations.

When the host interface **206** receives a command buffer via the I/O unit **204**, the host interface **206** can direct work operations to perform those commands to a front end **208**. In one embodiment the front end **208** couples with a scheduler **210**, which is configured to distribute commands or other work items to a processing cluster array **212**. The scheduler **210** ensures that the processing cluster array **212** is properly configured and in a valid state before tasks are distributed to the processing clusters of the processing cluster array **212**. The scheduler **210** may be implemented via firmware logic executing on a microcontroller. The microcontroller implemented scheduler **210** is configurable to perform complex scheduling and work distribution operations at coarse and fine granularity, enabling rapid preemption and context

switching of threads executing on the processing cluster array **212**. Preferably, the host software can prove workloads for scheduling on the processing cluster array **212** via one of multiple graphics processing doorbells. In other examples, polling for new workloads or interrupts can be used to identify or indicate availability of work to perform. The workloads can then be automatically distributed across the processing cluster array **212** by the scheduler **210** logic within the scheduler microcontroller.

The processing cluster array **212** can include up to “N” processing clusters (e.g., cluster **214A**, cluster **214B**, through cluster **214N**). Each cluster **214A-214N** of the processing cluster array **212** can execute a large number of concurrent threads. The scheduler **210** can allocate work to the clusters **214A-214N** of the processing cluster array **212** using various scheduling and/or work distribution algorithms, which may vary depending on the workload arising for each type of program or computation. The scheduling can be handled dynamically by the scheduler **210**, or can be assisted in part by compiler logic during compilation of program logic configured for execution by the processing cluster array **212**. Optionally, different clusters **214A-214N** of the processing cluster array **212** can be allocated for processing different types of programs or for performing different types of computations.

The processing cluster array **212** can be configured to perform various types of parallel processing operations. For example, the processing cluster array **212** is configured to perform general-purpose parallel compute operations. For example, the processing cluster array **212** can include logic to execute processing tasks including filtering of video and/or audio data, performing modeling operations, including physics operations, and performing data transformations.

The processing cluster array **212** is configured to perform parallel graphics processing operations. In such embodiments in which the parallel processor **200** is configured to perform graphics processing operations, the processing cluster array **212** can include additional logic to support the execution of such graphics processing operations, including, but not limited to texture sampling logic to perform texture operations, as well as tessellation logic and other vertex processing logic. Additionally, the processing cluster array **212** can be configured to execute graphics processing related shader programs such as, but not limited to vertex shaders, tessellation shaders, geometry shaders, and pixel shaders. The parallel processing unit **202** can transfer data from system memory via the I/O unit **204** for processing. During processing the transferred data can be stored to on-chip memory (e.g., parallel processor memory **222**) during processing, then written back to system memory.

In embodiments in which the parallel processing unit **202** is used to perform graphics processing, the scheduler **210** may be configured to divide the processing workload into approximately equal sized tasks, to better enable distribution of the graphics processing operations to multiple clusters **214A-214N** of the processing cluster array **212**. In some of these embodiments, portions of the processing cluster array **212** can be configured to perform different types of processing. For example a first portion may be configured to perform vertex shading and topology generation, a second portion may be configured to perform tessellation and geometry shading, and a third portion may be configured to perform pixel shading or other screen space operations, to produce a rendered image for display. Intermediate data produced by one or more of the clusters **214A-214N** may be

stored in buffers to allow the intermediate data to be transmitted between clusters **214A-214N** for further processing.

During operation, the processing cluster array **212** can receive processing tasks to be executed via the scheduler **210**, which receives commands defining processing tasks from front end **208**. For graphics processing operations, processing tasks can include indices of data to be processed, e.g., surface (patch) data, primitive data, vertex data, and/or pixel data, as well as state parameters and commands defining how the data is to be processed (e.g., what program is to be executed). The scheduler **210** may be configured to fetch the indices corresponding to the tasks or may receive the indices from the front end **208**. The front end **208** can be configured to ensure the processing cluster array **212** is configured to a valid state before the workload specified by incoming command buffers (e.g., batch-buffers, push buffers, etc.) is initiated.

Each of the one or more instances of the parallel processing unit **202** can couple with parallel processor memory **222**. The parallel processor memory **222** can be accessed via the memory crossbar **216**, which can receive memory requests from the processing cluster array **212** as well as the I/O unit **204**. The memory crossbar **216** can access the parallel processor memory **222** via a memory interface **218**. The memory interface **218** can include multiple partition units (e.g., partition unit **220A**, partition unit **220B**, through partition unit **220N**) that can each couple to a portion (e.g., memory unit) of parallel processor memory **222**. The number of partition units **220A-220N** may be configured to be equal to the number of memory units, such that a first partition unit **220A** has a corresponding first memory unit **224A**, a second partition unit **220B** has a corresponding second memory unit **224B**, and an Nth partition unit **220N** has a corresponding Nth memory unit **224N**. In other embodiments, the number of partition units **220A-220N** may not be equal to the number of memory devices.

The memory units **224A-224N** can include various types of memory devices, including dynamic random-access memory (DRAM) or graphics random access memory (SGRAM), such as synchronous graphics random access memory (SGRAM), including graphics double data rate (GDDR) memory. Optionally, the memory units **224A-224N** may also include 3D stacked memory, including but not limited to high bandwidth memory (HBM). Persons skilled in the art can appreciate that the specific implementation of the memory units **224A-224N** can vary, and can be selected from one of various conventional designs. Render targets, such as frame buffers or texture maps may be stored across the memory units **224A-224N**, allowing partition units **220A-220N** to write portions of each render target in parallel to efficiently use the available bandwidth of parallel processor memory **222**. In some embodiments, a local instance of the parallel processor memory **222** may be excluded in favor of a unified memory design that utilizes system memory in conjunction with local cache memory.

Optionally, any one of the clusters **214A-214N** of the processing cluster array **212** has the ability to process data that can be written to any of the memory units **224A-224N** within parallel processor memory **222**. The memory crossbar **216** can be configured to transfer the output of each cluster **214A-214N** to any partition unit **220A-220N** or to another cluster **214A-214N**, which can perform additional processing operations on the output. Each cluster **214A-214N** can communicate with the memory interface **218** through the memory crossbar **216** to read from or write to various external memory devices. In one of the embodiments with the memory crossbar **216** the memory crossbar

216 has a connection to the memory interface **218** to communicate with the I/O unit **204**, as well as a connection to a local instance of the parallel processor memory **222**, enabling the processing units within the different processing clusters **214A-214N** to communicate with system memory or other memory that is not local to the parallel processing unit **202**. Generally, the memory crossbar **216** may, for example, be able to use virtual channels to separate traffic streams between the clusters **214A-214N** and the partition units **220A-220N**.

While a single instance of the parallel processing unit **202** is illustrated within the parallel processor **200**, any number of instances of the parallel processing unit **202** can be included. For example, multiple instances of the parallel processing unit **202** can be provided on a single add-in card, or multiple add-in cards can be interconnected. For example, the parallel processor **200** can be an add-in device, such as add-in device **120** of FIG. 1, which may be a graphics card such as a discrete graphics card that includes one or more GPUs, one or more memory devices, and device-to-device or network or fabric interfaces. The different instances of the parallel processing unit **202** can be configured to interoperate even if the different instances have different numbers of processing cores, different amounts of local parallel processor memory, and/or other configuration differences. Optionally, some instances of the parallel processing unit **202** can include higher precision floating point units relative to other instances. Systems incorporating one or more instances of the parallel processing unit **202** or the parallel processor **200** can be implemented in a variety of configurations and form factors, including but not limited to desktop, laptop, or handheld personal computers, servers, workstations, game consoles, and/or embedded systems. An orchestrator can form composite nodes for workload performance using one or more of: disaggregated processor resources, cache resources, memory resources, storage resources, and networking resources.

FIG. 2B is a block diagram of a partition unit **220**. The partition unit **220** may be an instance of one of the partition units **220A-220N** of FIG. 2A. As illustrated, the partition unit **220** includes an L2 cache **221**, a frame buffer interface **225**, and a ROP **226** (raster operations unit). The L2 cache **221** is a read/write cache that is configured to perform load and store operations received from the memory crossbar **216** and ROP **226**. Read misses and urgent write-back requests are output by L2 cache **221** to frame buffer interface **225** for processing. Updates can also be sent to the frame buffer via the frame buffer interface **225** for processing. In one embodiment the frame buffer interface **225** interfaces with one of the memory units in parallel processor memory, such as the memory units **224A-224N** of FIG. 2A (e.g., within parallel processor memory **222**). The partition unit **220** may additionally or alternatively also interface with one of the memory units in parallel processor memory via a memory controller (not shown).

In graphics applications, the ROP **226** is a processing unit that performs raster operations such as stencil, z test, blending, and the like. The ROP **226** then outputs processed graphics data that is stored in graphics memory. In some embodiments the ROP **226** includes or couples with a CODEC **227** that includes compression logic to compress depth or color data that is written to memory or the L2 cache **221** and decompress depth or color data that is read from memory or the L2 cache **221**. The compression logic can be lossless compression logic that makes use of one or more of multiple compression algorithms. The type of compression that is performed by the CODEC **227** can vary based on the

statistical characteristics of the data to be compressed. For example, in one embodiment, delta color compression is performed on depth and color data on a per-tile basis. In one embodiment the CODEC 227 includes compression and decompression logic that can compress and decompress compute data associated with machine learning operations. The CODEC 227 can, for example, compress sparse matrix data for sparse machine learning operations. The CODEC 227 can also compress sparse matrix data that is encoded in a sparse matrix format (e.g., coordinate list encoding (COO), compressed sparse row (CSR), compress sparse column (CSC), etc.) to generate compressed and encoded sparse matrix data. The compressed and encoded sparse matrix data can be decompressed and/or decoded before being processed by processing elements or the processing elements can be configured to consume compressed, encoded, or compressed and encoded data for processing.

The ROP 226 may be included within each processing cluster (e.g., cluster 214A-214N of FIG. 2A) instead of within the partition unit 220. In such embodiment, read and write requests for pixel data are transmitted over the memory crossbar 216 instead of pixel fragment data. The processed graphics data may be displayed on a display device, such as one of the one or more display device(s) 110 of FIG. 1, routed for further processing by the processor(s) 102, or routed for further processing by one of the processing entities within the parallel processor 200 of FIG. 2A.

FIG. 2C is a block diagram of a processing cluster 214 within a parallel processing unit. For example, the processing cluster is an instance of one of the processing clusters 214A-214N of FIG. 2A. The processing cluster 214 can be configured to execute many threads in parallel, where the term "thread" refers to an instance of a particular program executing on a particular set of input data. Optionally, single-instruction, multiple-data (SIMD) instruction issue techniques may be used to support parallel execution of a large number of threads without providing multiple independent instruction units. Alternatively, single-instruction, multiple-thread (SMT) techniques may be used to support parallel execution of a large number of generally synchronized threads, using a common instruction unit configured to issue instructions to a set of processing engines within each one of the processing clusters. Unlike a SIMD execution regime, where all processing engines typically execute identical instructions, SMT execution allows different threads to more readily follow divergent execution paths through a given thread program. Persons skilled in the art can understand that a SIMD processing regime represents a functional subset of a SMT processing regime.

Operation of the processing cluster 214 can be controlled via a pipeline manager 232 that distributes processing tasks to SMT parallel processors. The pipeline manager 232 receives instructions from the scheduler 210 of FIG. 2A and manages execution of those instructions via a graphics multiprocessor 234 and/or a texture unit 236. The illustrated graphics multiprocessor 234 is an example instance of a SMT parallel processor. However, various types of SMT parallel processors of differing architectures may be included within the processing cluster 214. One or more instances of the graphics multiprocessor 234 can be included within a processing cluster 214. The graphics multiprocessor 234 can process data and a data crossbar 240 can be used to distribute the processed data to one of multiple possible destinations, including other shader units. The pipeline manager 232 can facilitate the distribution of processed data by specifying destinations for processed data to be distributed via the data crossbar 240.

Each graphics multiprocessor 234 within the processing cluster 214 can include an identical set of functional execution logic (e.g., arithmetic logic units, load-store units, etc.). The functional execution logic can be configured in a pipelined manner in which new instructions can be issued before previous instructions are complete. The functional execution logic supports a variety of operations including integer and floating-point arithmetic, comparison operations, Boolean operations, bit-shifting, and computation of various algebraic functions. The same functional-unit hardware could be leveraged to perform different operations and any combination of functional units may be present.

The instructions transmitted to the processing cluster 214 constitute a thread. A set of threads executing across the set of parallel processing engines is a thread group. A thread group executes the same program on different input data. Each thread within a thread group can be assigned to a different processing engine within a graphics multiprocessor 234. A thread group may include fewer threads than the number of processing engines within the graphics multiprocessor 234. When a thread group includes fewer threads than the number of processing engines, one or more of the processing engines may be idle during cycles in which that thread group is being processed. A thread group may also include more threads than the number of processing engines within the graphics multiprocessor 234. When the thread group includes more threads than the number of processing engines within the graphics multiprocessor 234, processing can be performed over consecutive clock cycles. Optionally, multiple thread groups can be executed concurrently on the graphics multiprocessor 234.

The graphics multiprocessor 234 may include an internal cache memory to perform load and store operations. Optionally, the graphics multiprocessor 234 can forego an internal cache and use a cache memory (e.g., level 1 (L1) cache 248) within the processing cluster 214. Each graphics multiprocessor 234 also has access to level 2 (L2) caches within the partition units (e.g., partition units 220A-220N of FIG. 2A) that are shared among all processing clusters 214 and may be used to transfer data between threads. The graphics multiprocessor 234 may also access off-chip global memory, which can include one or more of local parallel processor memory and/or system memory. Any memory external to the parallel processing unit 202 may be used as global memory. Embodiments in which the processing cluster 214 includes multiple instances of the graphics multiprocessor 234 can share common instructions and data, which may be stored in the L1 cache 248.

Each processing cluster 214 may include an MMU 245 (memory management unit) that is configured to map virtual addresses into physical addresses. In other embodiments, one or more instances of the MMU 245 may reside within the memory interface 218 of FIG. 2A. The MMU 245 includes a set of page table entries (PTEs) used to map a virtual address to a physical address of a tile and optionally a cache line index. The MMU 245 may include address translation lookaside buffers (TLB) or caches that may reside within the graphics multiprocessor 234 or the L1 cache or processing cluster 214. The physical address is processed to distribute surface data access locality to allow efficient request interleaving among partition units. The cache line index may be used to determine whether a request for a cache line is a hit or miss.

In graphics and computing applications, a processing cluster 214 may be configured such that each graphics multiprocessor 234 is coupled to a texture unit 236 for performing texture mapping operations, e.g., determining

11

texture sample positions, reading texture data, and filtering the texture data. Texture data is read from an internal texture L1 cache (not shown) or in some embodiments from the L1 cache within graphics multiprocessor 234 and is fetched from an L2 cache, local parallel processor memory, or system memory, as needed. Each graphics multiprocessor 234 outputs processed tasks to the data crossbar 240 to provide the processed task to another processing cluster 214 for further processing or to store the processed task in an L2 cache, local parallel processor memory, or system memory via the memory crossbar 216. A preROP 242 (pre-raster operations unit) is configured to receive data from graphics multiprocessor 234, direct data to ROP units, which may be located with partition units as described herein (e.g., partition units 220A-220N of FIG. 2A). The preROP 242 unit can perform optimizations for color blending, organize pixel color data, and perform address translations.

It can be appreciated that the core architecture described herein is illustrative and that variations and modifications are possible. Any number of processing units, e.g., graphics multiprocessor 234, texture units 236, preROPs 242, etc., may be included within a processing cluster 214. Further, while only one processing cluster 214 is shown, a parallel processing unit as described herein may include any number of instances of the processing cluster 214. Optionally, each processing cluster 214 can be configured to operate independently of other processing clusters 214 using separate and distinct processing units, L1 caches, L2 caches, etc.

FIG. 2D shows an example of the graphics multiprocessor 234 in which the graphics multiprocessor 234 couples with the pipeline manager 232 of the processing cluster 214. The graphics multiprocessor 234 has an execution pipeline including but not limited to an instruction cache 252, an instruction unit 254, an address mapping unit 256, a register file 258, one or more general purpose graphics processing unit (GPGPU) cores 262, and one or more load/store units 266. The GPGPU cores 262 and load/store units 266 are coupled with cache memory 272 and shared memory 270 via a memory and cache interconnect 268. The graphics multiprocessor 234 may additionally include tensor and/or ray-tracing cores 263 that include hardware logic to accelerate matrix and/or ray-tracing operations.

The instruction cache 252 may receive a stream of instructions to execute from the pipeline manager 232. The instructions are cached in the instruction cache 252 and dispatched for execution by the instruction unit 254. The instruction unit 254 can dispatch instructions as thread groups (e.g., warps), with each thread of the thread group assigned to a different execution unit within GPGPU core 262. An instruction can access any of a local, shared, or global address space by specifying an address within a unified address space. The address mapping unit 256 can be used to translate addresses in the unified address space into a distinct memory address that can be accessed by the load/store units 266.

The register file 258 provides a set of registers for the functional units of the graphics multiprocessor 234. The register file 258 provides temporary storage for operands connected to the data paths of the functional units (e.g., GPGPU cores 262, load/store units 266) of the graphics multiprocessor 234. The register file 258 may be divided between each of the functional units such that each functional unit is allocated a dedicated portion of the register file 258. For example, the register file 258 may be divided between the different warps being executed by the graphics multiprocessor 234.

12

The GPGPU cores 262 can each include floating point units (FPUs) and/or integer arithmetic logic units (ALUs) that are used to execute instructions of the graphics multiprocessor 234. In some implementations, the GPGPU cores 262 can include hardware logic that may otherwise reside within the tensor and/or ray-tracing cores 263. The GPGPU cores 262 can be similar in architecture or can differ in architecture. For example and in one embodiment, a first portion of the GPGPU cores 262 include a single precision FPU and an integer ALU while a second portion of the GPGPU cores include a double precision FPU. Optionally, the FPUs can implement the IEEE 754-2008 standard for floating point arithmetic or enable variable precision floating point arithmetic. The graphics multiprocessor 234 can additionally include one or more fixed function or special function units to perform specific functions such as copy rectangle or pixel blending operations. One or more of the GPGPU cores can also include fixed or special function logic.

The GPGPU cores 262 may include SIMD logic capable of performing a single instruction on multiple sets of data. Optionally, GPGPU cores 262 can physically execute SIMD4, SIMD8, and SIMD16 instructions and logically execute SIMD1, SIMD2, and SIMD32 instructions. The SIMD instructions for the GPGPU cores can be generated at compile time by a shader compiler or automatically generated when executing programs written and compiled for single program multiple data (SPMD) or SIMT architectures. Multiple threads of a program configured for the SIMT execution model can be executed via a single SIMD instruction. For example and in one embodiment, eight SIMT threads that perform the same or similar operations can be executed in parallel via a single SIMD8 logic unit.

The memory and cache interconnect 268 is an interconnect network that connects each of the functional units of the graphics multiprocessor 234 to the register file 258 and to the shared memory 270. For example, the memory and cache interconnect 268 is a crossbar interconnect that allows the load/store unit 266 to implement load and store operations between the shared memory 270 and the register file 258. The register file 258 can operate at the same frequency as the GPGPU cores 262, thus data transfer between the GPGPU cores 262 and the register file 258 is low latency. The shared memory 270 can be used to enable communication between threads that execute on the functional units within the graphics multiprocessor 234. The cache memory 272 can be used as a data cache for example, to cache texture data communicated between the functional units and the texture unit 236. The shared memory 270 can also be used as a program managed cache. The shared memory 270 and the cache memory 272 can couple with the data crossbar 240 to enable communication with other components of the processing cluster. Threads executing on the GPGPU cores 262 can programmatically store data within the shared memory in addition to the automatically cached data that is stored within the cache memory 272.

FIG. 3A-3C illustrate additional graphics multiprocessors, according to embodiments. FIG. 3A-3B illustrate graphics multiprocessors 325, 350, which are related to the graphics multiprocessor 234 of FIG. 2C and may be used in place of one of those. Therefore, the discussion of any features in combination with the graphics multiprocessor 234 herein also discloses a corresponding combination with the graphics multiprocessor(s) 325, 350, but is not limited to such. FIG. 3C illustrates a graphics processing unit (GPU) 380 which includes dedicated sets of graphics processing resources arranged into multi-core groups 365A-365N,

which correspond to the graphics multiprocessors **325**, **350**. The illustrated graphics multiprocessors **325**, **350** and the multi-core groups **365A-365N** can be streaming multiprocessors (SM) capable of simultaneous execution of a large number of execution threads.

The graphics multiprocessor **325** of FIG. 3A includes multiple additional instances of execution resource units relative to the graphics multiprocessor **234** of FIG. 2D. For example, the graphics multiprocessor **325** can include multiple instances of the instruction unit **332A-332B**, register file **334A-334B**, and texture unit(s) **344A-344B**. The graphics multiprocessor **325** also includes multiple sets of graphics or compute execution units (e.g., GPGPU core **336A-336B**, tensor core **337A-337B**, ray-tracing core **338A-338B**) and multiple sets of load/store units **340A-340B**. The execution resource units have a common instruction cache **330**, texture and/or data cache memory **342**, and shared memory **346**.

The various components can communicate via an interconnect fabric **327**. The interconnect fabric **327** may include one or more crossbar switches to enable communication between the various components of the graphics multiprocessor **325**. The interconnect fabric **327** may be a separate, high-speed network fabric layer upon which each component of the graphics multiprocessor **325** is stacked. The components of the graphics multiprocessor **325** communicate with remote components via the interconnect fabric **327**. For example, the cores **336A-336B**, **337A-337B**, and **338A-338B** can each communicate with shared memory **346** via the interconnect fabric **327**. The interconnect fabric **327** can arbitrate communication within the graphics multiprocessor **325** to ensure a fair bandwidth allocation between components.

The graphics multiprocessor **350** of FIG. 3B includes multiple sets of execution resources **356A-356D**, where each set of execution resource includes multiple instruction units, register files, GPGPU cores, and load store units, as illustrated in FIG. 2D and FIG. 3A. The execution resources **356A-356D** can work in concert with texture unit(s) **360A-360D** for texture operations, while sharing an instruction cache **354**, and shared memory **353**. For example, the execution resources **356A-356D** can share an instruction cache **354** and shared memory **353**, as well as multiple instances of a texture and/or data cache memory **358A-358B**. The various components can communicate via an interconnect fabric **352** similar to the interconnect fabric **327** of FIG. 3A.

Persons skilled in the art can understand that the architecture described in FIGS. 1, 2A-2D, and 3A-3B are descriptive and not limiting as to the scope of the present embodiments. Thus, the techniques described herein may be implemented on any properly configured processing unit, including, without limitation, one or more mobile application processors, one or more desktop or server central processing units (CPUs) including multi-core CPUs, one or more parallel processing units, such as the parallel processing unit **202** of FIG. 2A, as well as one or more graphics processors or special purpose processing units, without departure from the scope of the embodiments described herein.

The parallel processor or GPGPU as described herein may be communicatively coupled to host/processor cores to accelerate graphics operations, machine-learning operations, pattern analysis operations, and various general-purpose GPU (GPGPU) functions. The GPU may be communicatively coupled to the host processor/cores over a bus or other interconnect (e.g., a high-speed interconnect such as

PCIe, NVLink, or other known protocols, standardized protocols, or proprietary protocols). In other embodiments, the GPU may be integrated on the same package or chip as the cores and communicatively coupled to the cores over an internal processor bus/interconnect (i.e., internal to the package or chip). Regardless of the manner in which the GPU is connected, the processor cores may allocate work to the GPU in the form of sequences of commands/instructions contained in a work descriptor. The GPU then uses dedicated circuitry/logic for efficiently processing these commands/instructions.

FIG. 3C illustrates a graphics processing unit (GPU) **380** which includes dedicated sets of graphics processing resources arranged into multi-core groups **365A-365N**. While the details of only a single multi-core group **365A** are provided, it can be appreciated that the other multi-core groups **365B-365N** may be equipped with the same or similar sets of graphics processing resources. Details described with respect to the multi-core groups **365A-365N** may also apply to any graphics multiprocessor **234**, **325**, **350** described herein.

As illustrated, a multi-core group **365A** may include a set of graphics cores **370**, a set of tensor cores **371**, and a set of ray tracing cores **372**. A scheduler/dispatcher **368** schedules and dispatches the graphics threads for execution on the various cores **370**, **371**, **372**. A set of register files **369** store operand values used by the cores **370**, **371**, **372** when executing the graphics threads. These may include, for example, integer registers for storing integer values, floating point registers for storing floating point values, vector registers for storing packed data elements (integer and/or floating-point data elements) and tile registers for storing tensor/matrix values. The tile registers may be implemented as combined sets of vector registers.

One or more combined level 1 (L1) caches and shared memory units **373** store graphics data such as texture data, vertex data, pixel data, ray data, bounding volume data, etc., locally within each multi-core group **365A**. One or more texture units **374** can also be used to perform texturing operations, such as texture mapping and sampling. A Level 2 (L2) cache **375** shared by all or a subset of the multi-core groups **365A-365N** stores graphics data and/or instructions for multiple concurrent graphics threads. As illustrated, the L2 cache **375** may be shared across a plurality of multi-core groups **365A-365N**. One or more memory controllers **367** couple the GPU **380** to a memory **366** which may be a system memory (e.g., DRAM) and/or a dedicated graphics memory (e.g., GDDR6 memory).

Input/output (I/O) circuitry **363** couples the GPU **380** to one or more I/O devices **362** such as digital signal processors (DSPs), network controllers, or user input devices. An on-chip interconnect may be used to couple the I/O devices **362** to the GPU **380** and memory **366**. One or more I/O memory management units (IOMMUs) **364** of the I/O circuitry **363** couple the I/O devices **362** directly to the system memory **366**. Optionally, the IOMMU **364** manages multiple sets of page tables to map virtual addresses to physical addresses in system memory **366**. The I/O devices **362**, CPU(s) **361**, and GPU(s) **380** may then share the same virtual address space.

In one implementation of the IOMMU **364**, the IOMMU **364** supports virtualization. In this case, it may manage a first set of page tables to map guest/graphics virtual addresses to guest/graphics physical addresses and a second set of page tables to map the guest/graphics physical addresses to system/host physical addresses (e.g., within system memory **366**). The base addresses of each of the first

15

and second sets of page tables may be stored in control registers and swapped out on a context switch (e.g., so that the new context is provided with access to the relevant set of page tables). While not illustrated in FIG. 3C, each of the cores 370, 371, 372 and/or multi-core groups 365A-365N may include translation lookaside buffers (TLBs) to cache guest virtual to guest physical translations, guest physical to host physical translations, and guest virtual to host physical translations.

The CPU(s) 361, GPUs 380, and I/O devices 362 may be integrated on a single semiconductor chip and/or chip package. The illustrated memory 366 may be integrated on the same chip or may be coupled to the memory controllers 367 via an off-chip interface. In one implementation, the memory 366 comprises GDDR6 memory which shares the same virtual address space as other physical system-level memories, although the underlying principles described herein are not limited to this specific implementation.

The tensor cores 371 may include a plurality of execution units specifically designed to perform matrix operations, which are the core compute operation used to perform deep learning operations. For example, simultaneous matrix multiplication operations may be used for neural network training and inferencing. The tensor cores 371 may perform matrix processing using a variety of operand precisions including single precision floating-point (e.g., 32 bits), half-precision floating point (e.g., 16 bits), integer words (16 bits), bytes (8 bits), and half-bytes (4 bits). For example, a neural network implementation extracts features of each rendered scene, potentially combining details from multiple frames, to construct a high-quality final image.

In deep learning implementations, parallel matrix multiplication work may be scheduled for execution on the tensor cores 371. The training of neural networks, in particular, utilizes a significant number matrix dot product operations. In order to process an inner-product formulation of an $N \times N \times N$ matrix multiply, the tensor cores 371 may include at least N dot-product processing elements. Before the matrix multiply begins, one entire matrix is loaded into tile registers and at least one column of a second matrix is loaded each cycle for N cycles. Each cycle, there are N dot products that are processed.

Matrix elements may be stored at different precisions depending on the particular implementation, including 16-bit words, 8-bit bytes (e.g., INT8) and 4-bit half-bytes (e.g., INT4). Different precision modes may be specified for the tensor cores 371 to ensure that an efficient precision is used for different workloads (e.g., such as inferencing workloads which can tolerate quantization to bytes and half-bytes). Supported formats additionally include 64-bit floating point (FP64) and non-IEEE floating point formats such as the bfloat16 format (e.g., Brain floating point), a 16-bit floating point format with one sign bit, eight exponent bits, and eight significand bits, of which seven are explicitly stored. One embodiment includes support for a reduced precision tensor-float format (TF32), which has the range of FP32 (8-bits) with the precision of FP16 (10-bits). Reduced precision TF32 operations can be performed on FP32 inputs and produce FP32 outputs at higher performance relative to FP32 and increased precision relative to FP16.

In one embodiment the tensor cores 371 support a sparse mode of operation for matrices in which the vast majority of values are zero. The tensor cores 371 include support for sparse input matrices that are encoded in a sparse matrix representation (e.g., coordinate list encoding (COO), compressed sparse row (CSR), compress sparse column (CSC), etc.). The tensor cores 371 also include support for com-

16

pressed sparse matrix representations in the event that the sparse matrix representation may be further compressed. Compressed, encoded, and/or compressed and encoded matrix data, along with associated compression and/or encoding metadata, can be read by the tensor cores 371 and the non-zero values can be extracted. For example, for a given input matrix A, a non-zero value can be loaded from the compressed and/or encoded representation of at least a portion of matrix A. Based on the location in matrix A for the non-zero value, which may be determined from index or coordinate metadata associated with the non-zero value, a corresponding value in input matrix B may be loaded. Depending on the operation to be performed (e.g., multiply), the load of the value from input matrix B may be bypassed if the corresponding value is a zero value. In one embodiment, the pairings of values for certain operations, such as multiply operations, may be pre-scanned by scheduler logic and only operations between non-zero inputs are scheduled. Depending on the dimensions of matrix A and matrix B and the operation to be performed, output matrix C may be dense or sparse. Where output matrix C is sparse, and depending on the configuration of the tensor cores 371, output matrix C may be output in a compressed format, a sparse encoding, or a compressed sparse encoding.

The ray tracing cores 372 may accelerate ray tracing operations for both real-time ray tracing and non-real-time ray tracing implementations. In particular, the ray tracing cores 372 may include ray traversal/intersection circuitry for performing ray traversal using bounding volume hierarchies (BVHs) and identifying intersections between rays and primitives enclosed within the BVH volumes. The ray tracing cores 372 may also include circuitry for performing depth testing and culling (e.g., using a Z buffer or similar arrangement). In one implementation, the ray tracing cores 372 perform traversal and intersection operations in concert with the image denoising techniques described herein, at least a portion of which may be executed on the tensor cores 371. For example, the tensor cores 371 may implement a deep learning neural network to perform denoising of frames generated by the ray tracing cores 372. However, the CPU(s) 361, graphics cores 370, and/or ray tracing cores 372 may also implement all or a portion of the denoising and/or deep learning algorithms.

In addition, as described above, a distributed approach to denoising may be employed in which the GPU 380 is in a computing device coupled to other computing devices over a network or high-speed interconnect. In this distributed approach, the interconnected computing devices may share neural network learning/training data to improve the speed with which the overall system learns to perform denoising for different types of image frames and/or different graphics applications.

The ray tracing cores 372 may process all BVH traversal and/or ray-primitive intersections, saving the graphics cores 370 from being overloaded with thousands of instructions per ray. For example, each ray tracing core 372 includes a first set of specialized circuitry for performing bounding box tests (e.g., for traversal operations) and/or a second set of specialized circuitry for performing the ray-triangle intersection tests (e.g., intersecting rays which have been traversed). Thus, for example, the multi-core group 365A can simply launch a ray probe, and the ray tracing cores 372 independently perform ray traversal and intersection and return hit data (e.g., a hit, no hit, multiple hits, etc.) to the thread context. The other cores 370, 371 are freed to perform other graphics or compute work while the ray tracing cores 372 perform the traversal and intersection operations.

Optionally, each ray tracing core 372 may include a traversal unit to perform BVH testing operations and/or an intersection unit which performs ray-primitive intersection tests. The intersection unit generates a “hit”, “no hit”, or “multiple hit” response, which it provides to the appropriate thread. During the traversal and intersection operations, the execution resources of the other cores (e.g., graphics cores 370 and tensor cores 371) are freed to perform other forms of graphics work.

In one optional embodiment described below, a hybrid rasterization/ray tracing approach is used in which work is distributed between the graphics cores 370 and ray tracing cores 372.

The ray tracing cores 372 (and/or other cores 370, 371) may include hardware support for a ray tracing instruction set such as Microsoft’s DirectX Ray Tracing (DXR) which includes a DispatchRays command, as well as ray-generation, closest-hit, any-hit, and miss shaders, which enable the assignment of unique sets of shaders and textures for each object. Another ray tracing platform which may be supported by the ray tracing cores 372, graphics cores 370 and tensor cores 371 is Vulkan 1.1.85. Note, however, that the underlying principles described herein are not limited to any particular ray tracing ISA.

In general, the various cores 372, 371, 370 may support a ray tracing instruction set that includes instructions/functions for one or more of ray generation, closest hit, any hit, ray-primitive intersection, per-primitive and hierarchical bounding box construction, miss, visit, and exceptions. More specifically, an embodiment includes ray tracing instructions to perform one or more of the following functions:

Ray Generation—Ray generation instructions may be executed for each pixel, sample, or other user-defined work assignment.

Closest Hit—A closest hit instruction may be executed to locate the closest intersection point of a ray with primitives within a scene.

Any Hit—An any hit instruction identifies multiple intersections between a ray and primitives within a scene, potentially to identify a new closest intersection point.

Intersection—An intersection instruction performs a ray-primitive intersection test and outputs a result.

Per-primitive Bounding box Construction—This instruction builds a bounding box around a given primitive or group of primitives (e.g., when building a new BVH or other acceleration data structure).

Miss—Indicates that a ray misses all geometry within a scene, or specified region of a scene.

Visit—Indicates the children volumes a ray can traverse.

Exceptions—Includes various types of exception handlers (e.g., invoked for various error conditions).

In one embodiment the ray tracing cores 372 may be adapted to accelerate general-purpose compute operations that can be accelerated using computational techniques that are analogous to ray intersection tests. A compute framework can be provided that enables shader programs to be compiled into low level instructions and/or primitives that perform general-purpose compute operations via the ray tracing cores. Example computational problems that can benefit from compute operations performed on the ray tracing cores 372 include computations involving beam, wave, ray, or particle propagation within a coordinate space. Interactions associated with that propagation can be computed relative to a geometry or mesh within the coordinate space. For example, computations associated with electromagnetic signal propagation through an environment can be

accelerated via the use of instructions or primitives that are executed via the ray tracing cores. Diffraction and reflection of the signals by objects in the environment can be computed as direct ray-tracing analogies.

Ray tracing cores 372 can also be used to perform computations that are not directly analogous to ray tracing. For example, mesh projection, mesh refinement, and volume sampling computations can be accelerated using the ray tracing cores 372. Generic coordinate space calculations, such as nearest neighbor calculations can also be performed. For example, the set of points near a given point can be discovered by defining a bounding box in the coordinate space around the point. BVH and ray probe logic within the ray tracing cores 372 can then be used to determine the set of point intersections within the bounding box. The intersections constitute the origin point and the nearest neighbors to that origin point. Computations that are performed using the ray tracing cores 372 can be performed in parallel with computations performed on the graphics cores 372 and tensor cores 371. A shader compiler can be configured to compile a compute shader or other general-purpose graphics processing program into low level primitives that can be parallelized across the graphics cores 370, tensor cores 371, and ray tracing cores 372.

Techniques for GPU to Host Processor Interconnection

FIG. 4A illustrates an example architecture in which a plurality of GPUs 410-413, e.g. such as the parallel processors 200 shown in FIG. 2A, are communicatively coupled to a plurality of multi-core processors 405-406 over high-speed links 440A-440D (e.g., buses, point-to-point interconnects, etc.). The high-speed links 440A-440D may support a communication throughput of 4 GB/s, 30 GB/s, 80 GB/s or higher, depending on the implementation. Various interconnect protocols may be used including, but not limited to, PCIe 4.0 or 5.0 and NVLink 2.0. However, the underlying principles described herein are not limited to any particular communication protocol or throughput.

Two or more of the GPUs 410-413 may be interconnected over high-speed links 442A-442B, which may be implemented using the same or different protocols/links than those used for high-speed links 440A-440D. Similarly, two or more of the multi-core processors 405-406 may be connected over high speed link 443 which may be symmetric multi-processor (SMP) buses operating at 20 GB/s, 30 GB/s, 120 GB/s or lower or higher speeds. Alternatively, all communication between the various system components shown in FIG. 4A may be accomplished using the same protocols/links (e.g., over a common interconnection fabric). As mentioned, however, the underlying principles described herein are not limited to any particular type of interconnect technology.

Each multi-core processor 405-406 may be communicatively coupled to a processor memory 401-402, via memory interconnects 430A-430B, respectively, and each GPU 410-413 is communicatively coupled to GPU memory 420-423 over GPU memory interconnects 450A-450D, respectively. The memory interconnects 430A-430B and 450A-450D may utilize the same or different memory access technologies. By way of example, and not limitation, the processor memories 401-402 and GPU memories 420-423 may be volatile memories such as dynamic random-access memories (DRAMs) (including stacked DRAMs), Graphics DDR SDRAM (GDDR) (e.g., GDDR5, GDDR6), or High Bandwidth Memory (HBM) and/or may be non-volatile memories such as 3D XPoint/Optane or Nano-Ram. For example, some portion of the memories may be volatile memory and another portion may be non-volatile memory (e.g., using a

two-level memory (2LM) hierarchy). A memory subsystem as described herein may be compatible with a number of memory technologies, such as Double Data Rate versions released by JEDEC (Joint Electronic Device Engineering Council).

As described below, although the various processors **405-406** and GPUs **410-413** may be physically coupled to a particular memory **401-402**, **420-423**, respectively, a unified memory architecture may be implemented in which the same virtual system address space (also referred to as the “effective address” space) is distributed among all of the various physical memories. For example, processor memories **401-402** may each comprise 64 GB of the system memory address space and GPU memories **420-423** may each comprise 32 GB of the system memory address space (resulting in a total of 256 GB addressable memory in this example).

FIG. **4B** illustrates additional optional details for an interconnection between a multi-core processor **407** and a graphics acceleration module **446**. The graphics acceleration module **446** may include one or more GPU chips integrated on a line card which is coupled to the processor **407** via the high-speed link **440**. Alternatively, the graphics acceleration module **446** may be integrated on the same package or chip as the processor **407**.

The illustrated processor **407** includes a plurality of cores **460A-460D**, each with a translation lookaside buffer **461A-461D** and one or more caches **462A-462D**. The cores may include various other components for executing instructions and processing data which are not illustrated to avoid obscuring the underlying principles of the components described herein (e.g., instruction fetch units, branch prediction units, decoders, execution units, reorder buffers, etc.). The caches **462A-462D** may comprise level 1 (L1) and level 2 (L2) caches. In addition, one or more shared caches **456** may be included in the caching hierarchy and shared by sets of the cores **460A-460D**. For example, one embodiment of the processor **407** includes 24 cores, each with its own L1 cache, twelve shared L2 caches, and twelve shared L3 caches. In this embodiment, one of the L2 and L3 caches are shared by two adjacent cores. The processor **407** and the graphics accelerator integration module **446** connect with system memory **441**, which may include processor memories **401-402**.

Coherency is maintained for data and instructions stored in the various caches **462A-462D**, **456** and system memory **441** via inter-core communication over a coherence bus **464**. For example, each cache may have cache coherency logic/circuitry associated therewith to communicate to over the coherence bus **464** in response to detected reads or writes to particular cache lines. In one implementation, a cache snooping protocol is implemented over the coherence bus **464** to snoop cache accesses. Cache snooping/coherency techniques are well understood by those of skill in the art and cannot be described in detail here to avoid obscuring the underlying principles described herein.

A proxy circuit **425** may be provided that communicatively couples the graphics acceleration module **446** to the coherence bus **464**, allowing the graphics acceleration module **446** to participate in the cache coherence protocol as a peer of the cores. In particular, an interface **435** provides connectivity to the proxy circuit **425** over high-speed link **440** (e.g., a PCIe bus, NVLink, etc.) and an interface **437** connects the graphics acceleration module **446** to the high-speed link **440**.

In one implementation, an accelerator integration circuit **436** provides cache management, memory access, context

management, and interrupt management services on behalf of a plurality of graphics processing engines **431**, **432**, N of the graphics acceleration module **446**. The graphics processing engines **431**, **432**, N may each comprise a separate graphics processing unit (GPU). Alternatively, the graphics processing engines **431**, **432**, N may comprise different types of graphics processing engines within a GPU such as graphics execution units, media processing engines (e.g., video encoders/decoders), samplers, and blit engines. In other words, the graphics acceleration module may be a GPU with a plurality of graphics processing engines **431-432**, N or the graphics processing engines **431-432**, N may be individual GPUs integrated on a common package, line card, or chip.

The accelerator integration circuit **436** may include a memory management unit (MMU) **439** for performing various memory management functions such as virtual-to-physical memory translations (also referred to as effective-to-real memory translations) and memory access protocols for accessing system memory **441**. The MMU **439** may also include a translation lookaside buffer (TLB) (not shown) for caching the virtual/effective to physical/real address translations. In one implementation, a cache **438** stores commands and data for efficient access by the graphics processing engines **431**, **432**, N. The data stored in cache **438** and graphics memories **433-434**, M may be kept coherent with the core caches **462A-462D**, **456** and system memory **441**. As mentioned, this may be accomplished via proxy circuit **425** which takes part in the cache coherency mechanism on behalf of cache **438** and memories **433-434**, M (e.g., sending updates to the cache **438** related to modifications/accesses of cache lines on processor caches **462A-462D**, **456** and receiving updates from the cache **438**).

A set of registers **445** store context data for threads executed by the graphics processing engines **431-432**, N and a context management circuit **448** manages the thread contexts. For example, the context management circuit **448** may perform save and restore operations to save and restore contexts of the various threads during contexts switches (e.g., where a first thread is saved and a second thread is restored so that the second thread can be executed by a graphics processing engine). For example, on a context switch, the context management circuit **448** may store current register values to a designated region in memory (e.g., identified by a context pointer). It may then restore the register values when returning to the context. An interrupt management circuit **447**, for example, may receive and processes interrupts received from system devices.

In one implementation, virtual/effective addresses from a graphics processing engine **431** are translated to real/physical addresses in system memory **441** by the MMU **439**. Optionally, the accelerator integration circuit **436** supports multiple (e.g., 4, 8, 16) graphics accelerator modules **446** and/or other accelerator devices. The graphics accelerator module **446** may be dedicated to a single application executed on the processor **407** or may be shared between multiple applications. Optionally, a virtualized graphics execution environment is provided in which the resources of the graphics processing engines **431-432**, N are shared with multiple applications, virtual machines (VMs), or containers. The resources may be subdivided into “slices” which are allocated to different VMs and/or applications based on the processing requirements and priorities associated with the VMs and/or applications. VMs and containers can be used interchangeably herein.

A virtual machine (VM) can be software that runs an operating system and one or more applications. A VM can be

defined by specification, configuration files, virtual disk file, non-volatile random access memory (NVRAM) setting file, and the log file and is backed by the physical resources of a host computing platform. A VM can include an operating system (OS) or application environment that is installed on software, which imitates dedicated hardware. The end user has the same experience on a virtual machine as they would have on dedicated hardware. Specialized software, called a hypervisor, emulates the PC client or server's CPU, memory, hard disk, network and other hardware resources completely, enabling virtual machines to share the resources. The hypervisor can emulate multiple virtual hardware platforms that are isolated from each other, allowing virtual machines to run Linux®, Windows® Server, VMware ESXi, and other operating systems on the same underlying physical host.

A container can be a software package of applications, configurations and dependencies so the applications run reliably on one computing environment to another. Containers can share an operating system installed on the server platform and run as isolated processes. A container can be a software package that contains components the software utilizes to run such as system tools, libraries, and settings. Containers are not installed like traditional software programs, which allows them to be isolated from the other software and the operating system itself. The isolated nature of containers provides several benefits. First, the software in a container can run the same in different environments. For example, a container that includes PHP and MySQL can run identically on both a Linux® computer and a Windows® machine. Second, containers provide added security since the software can not affect the host operating system. While an installed application may alter system settings and modify resources, such as the Windows registry, a container can only modify settings within the container.

Thus, the accelerator integration circuit 436 acts as a bridge to the system for the graphics acceleration module 446 and provides address translation and system memory cache services. In one embodiment, to facilitate the bridging functionality, the accelerator integration circuit 436 may also include shared I/O 497 (e.g., PCIe, USB, or others) and hardware to enable system control of voltage, clocking, performance, thermals, and security. The shared I/O 497 may utilize separate physical connections or may traverse the high-speed link 440. In addition, the accelerator integration circuit 436 may provide virtualization facilities for the host processor to manage virtualization of the graphics processing engines, interrupts, and memory management.

Because hardware resources of the graphics processing engines 431-432, N are mapped explicitly to the real address space seen by the host processor 407, any host processor can address these resources directly using an effective address value. One optional function of the accelerator integration circuit 436 is the physical separation of the graphics processing engines 431-432, N so that they appear to the system as independent units.

One or more graphics memories 433-434, M may be coupled to each of the graphics processing engines 431-432, N, respectively. The graphics memories 433-434, M store instructions and data being processed by each of the graphics processing engines 431-432, N. The graphics memories 433-434, M may be volatile memories such as DRAMs (including stacked DRAMs), GDDR memory (e.g., GDDR5, GDDR6), or HBM, and/or may be non-volatile memories such as 3D XPoint/Optane, Samsung Z-NAND, or Nano-Ram.

To reduce data traffic over the high-speed link 440, biasing techniques may be used to ensure that the data stored in graphics memories 433-434, M is data which can be used frequently by the graphics processing engines 431-432, N and preferably not used by the cores 460A-460D (at least not frequently). Similarly, the biasing mechanism attempts to keep data utilized by the cores (and preferably not the graphics processing engines 431-432, N) within the caches 462A-462D, 456 of the cores and system memory 441.

According to a variant shown in FIG. 4C the accelerator integration circuit 436 is integrated within the processor 407. The graphics processing engines 431-432, N communicate directly over the high-speed link 440 to the accelerator integration circuit 436 via interface 437 and interface 435 (which, again, may utilize any form of bus or interface protocol). The accelerator integration circuit 436 may perform the same operations as those described with respect to FIG. 4B, but potentially at a higher throughput given its close proximity to the coherence bus 464 and caches 462A-462D, 456.

The embodiments described may support different programming models including a dedicated-process programming model (no graphics acceleration module virtualization) and shared programming models (with virtualization). The latter may include programming models which are controlled by the accelerator integration circuit 436 and programming models which are controlled by the graphics acceleration module 446.

In the embodiments of the dedicated process model, graphics processing engines 431, 432, . . . N may be dedicated to a single application or process under a single operating system. The single application can funnel other application requests to the graphics engines 431, 432, . . . N, providing virtualization within a VM/partition.

In the dedicated-process programming models, the graphics processing engines 431, 432, N, may be shared by multiple VM/application partitions. The shared models utilize a system hypervisor to virtualize the graphics processing engines 431-432, N to allow access by each operating system. For single-partition systems without a hypervisor, the graphics processing engines 431-432, N are owned by the operating system. In both cases, the operating system can virtualize the graphics processing engines 431-432, N to provide access to each process or application.

For the shared programming model, the graphics acceleration module 446 or an individual graphics processing engine 431-432, N selects a process element using a process handle. The process elements may be stored in system memory 441 and be addressable using the effective address to real address translation techniques described herein. The process handle may be an implementation-specific value provided to the host process when registering its context with the graphics processing engine 431-432, N (that is, calling system software to add the process element to the process element linked list). The lower 16-bits of the process handle may be the offset of the process element within the process element linked list.

FIG. 4D illustrates an example accelerator integration slice 490. As used herein, a "slice" comprises a specified portion of the processing resources of the accelerator integration circuit 436. Application effective address space 482 within system memory 441 stores process elements 483. The process elements 483 may be stored in response to GPU invocations 481 from applications 480 executed on the processor 407. A process element 483 contains the process state for the corresponding application 480. A work descriptor (WD) 484 contained in the process element 483 can be

23

a single job requested by an application or may contain a pointer to a queue of jobs. In the latter case, the WD **484** is a pointer to the job request queue in the application's address space **482**.

The graphics acceleration module **446** and/or the individual graphics processing engines **431-432**, **N** can be shared by all or a subset of the processes in the system. For example, the technologies described herein may include an infrastructure for setting up the process state and sending a WD **484** to a graphics acceleration module **446** to start a job in a virtualized environment.

In one implementation, the dedicated-process programming model is implementation-specific. In this model, a single process owns the graphics acceleration module **446** or an individual graphics processing engine **431**. Because the graphics acceleration module **446** is owned by a single process, the hypervisor initializes the accelerator integration circuit **436** for the owning partition and the operating system initializes the accelerator integration circuit **436** for the owning process at the time when the graphics acceleration module **446** is assigned.

In operation, a WD fetch unit **491** in the accelerator integration slice **490** fetches the next WD **484** which includes an indication of the work to be done by one of the graphics processing engines of the graphics acceleration module **446**. Data from the WD **484** may be stored in registers **445** and used by the MMU **439**, interrupt management circuit **447** and/or context management circuit **448** as illustrated. For example, the MMU **439** may include segment/page walk circuitry for accessing segment/page tables **486** within the OS virtual address space **485**. The interrupt management circuit **447** may process interrupt events **492** received from the graphics acceleration module **446**. When performing graphics operations, an effective address **493** generated by a graphics processing engine **431-432**, **N** is translated to a real address by the MMU **439**.

The same set of registers **445** may be duplicated for each graphics processing engine **431-432**, **N** and/or graphics acceleration module **446** and may be initialized by the hypervisor or operating system. Each of these duplicated registers may be included in an accelerator integration slice **490**. In one embodiment, each graphics processing engine **431-432**, **N** may be presented to the hypervisor **496** as a distinct graphics processor device. QoS settings can be configured for clients of a specific graphics processing engine **431-432**, **N** and data isolation between the clients of each engine can be enabled. Example registers that may be initialized by the hypervisor are shown in Table 1.

TABLE 1

Hypervisor Initialized Registers	
1	Slice Control Register
2	Real Address (RA) Scheduled Processes Area Pointer
3	Authority Mask Override Register
4	Interrupt Vector Table Entry Offset
5	Interrupt Vector Table Entry Limit
6	State Register
7	Logical Partition ID
8	Real address (RA) Hypervisor Accelerator Utilization Record Pointer
9	Storage Description Register

24

Example registers that may be initialized by the operating system are shown in Table 2.

TABLE 2

Operating System Initialized Registers	
1	Process and Thread Identification
2	Effective Address (EA) Context Save/Restore Pointer
3	Virtual Address (VA) Accelerator Utilization Record Pointer
4	Virtual Address (VA) Storage Segment Table Pointer
5	Authority Mask
6	Work descriptor

Each WD **484** may be specific to a particular graphics acceleration module **446** and/or graphics processing engine **431-432**, **N**. It contains all the information a graphics processing engine **431-432**, **N** utilizes to do its work or it can be a pointer to a memory location where the application has set up a command queue of work to be completed.

FIG. 4E illustrates additional optional details of a shared model. It includes a hypervisor real address space **498** in which a process element list **499** is stored. The hypervisor real address space **498** is accessible via a hypervisor **496** which virtualizes the graphics acceleration module engines for the operating system **495**.

The shared programming models allow for all or a subset of processes from all or a subset of partitions in the system to use a graphics acceleration module **446**. There are two programming models where the graphics acceleration module **446** is shared by multiple processes and partitions: time-sliced shared and graphics directed shared.

In this model, the system hypervisor **496** owns the graphics acceleration module **446** and makes its function available to all operating systems **495**. For a graphics acceleration module **446** to support virtualization by the system hypervisor **496**, the graphics acceleration module **446** may adhere to the following requirements: 1) An application's job request should be autonomous (that is, the state does not have to be maintained between jobs), or the graphics acceleration module **446** should provide a context save and restore mechanism. 2) An application's job request is guaranteed by the graphics acceleration module **446** to complete in a specified amount of time, including any translation faults, or the graphics acceleration module **446** provides the ability to preempt the processing of the job. 3) The graphics acceleration module **446** should be guaranteed fairness between processes when operating in the directed shared programming model.

For the shared model, the application **480** may be utilized to make an operating system **495** system call with a graphics acceleration module **446** type, a work descriptor (WD), an authority mask register (AMR) value, and a context save/restore area pointer (CSRP). The graphics acceleration module **446** type describes the targeted acceleration function for the system call. The graphics acceleration module **446** type may be a system-specific value. The WD is formatted specifically for the graphics acceleration module **446** and can be in the form of a graphics acceleration module **446** command, an effective address pointer to a user-defined structure, an effective address pointer to a queue of commands, or any other data structure to describe the work to be done by the graphics acceleration module **446**. In one embodiment, the AMR value is the AMR state to use for the current process. The value passed to the operating system is similar to an application setting the AMR. If the accelerator integration circuit **436** and graphics acceleration module **446** implementations do not support a User Authority Mask Override Register (UAMOR), the operating system may apply the current UAMOR value to the AMR value before passing the AMR in the hypervisor call. The hypervisor **496**

25

may optionally apply the current Authority Mask Override Register (AMOR) value before placing the AMR into the process element **483**. The CSRP may be one of the registers **445** containing the effective address of an area in the application's address space **482** for the graphics acceleration module **446** to save and restore the context state. This pointer is optional if no state is utilized to be saved between jobs or when a job is preempted. The context save/restore area may be pinned system memory.

Upon receiving the system call, the operating system **495** may verify that the application **480** has registered and been given the authority to use the graphics acceleration module **446**. The operating system **495** then calls the hypervisor **496** with the information shown in Table 3.

TABLE 3

OS to Hypervisor Call Parameters	
1	A work descriptor (WD)
2	An Authority Mask Register (AMR) value (potentially masked)
3	An effective address (EA) Context Save/Restore Area Pointer (CSRP)
4	A process ID (PID) and optional thread ID (TID)
5	A virtual address (VA) accelerator utilization record pointer (AURP)
6	Virtual address of storage segment table pointer (SSTP)
7	A logical interrupt service number (LISN)

Upon receiving the hypervisor call, the hypervisor **496** verifies that the operating system **495** has registered and been given the authority to use the graphics acceleration module **446**. The hypervisor **496** then puts the process element **483** into the process element linked list for the corresponding graphics acceleration module **446** type. The process element may include the information shown in Table 4.

TABLE 4

Process Element Information	
1	A work descriptor (WD)
2	An Authority Mask Register (AMR) value (potentially masked).
3	An effective address (EA) Context Save/Restore Area Pointer (CSRP)
4	A process ID (PID) and optional thread ID (TID)
5	A virtual address (VA) accelerator utilization record pointer (AURP)
6	Virtual address of storage segment table pointer (SSTP)
7	A logical interrupt service number (LISN)
8	Interrupt vector table, derived from hypervisor call parameters
9	A state register (SR) value
10	A logical partition ID (LPID)
11	A real address (RA) hypervisor accelerator utilization record pointer
12	Storage Descriptor Register (SDR)

The hypervisor may initialize a plurality of accelerator integration slice **490** registers **445**.

As illustrated in FIG. 4F, in one optional implementation a unified memory addressable via a common virtual memory address space used to access the physical processor memories **401-402** and GPU memories **420-423** is employed. In this implementation, operations executed on the GPUs **410-413** utilize the same virtual/effective memory address space to access the processors memories **401-402** and vice versa, thereby simplifying programmability. A first portion of the virtual/effective address space may be allocated to the processor memory **401**, a second portion to the second processor memory **402**, a third portion to the GPU memory **420**, and so on. The entire virtual/effective memory space

26

(sometimes referred to as the effective address space) may thereby be distributed across each of the processor memories **401-402** and GPU memories **420-423**, allowing any processor or GPU to access any physical memory with a virtual address mapped to that memory.

Bias/coherence management circuitry **494A-494E** within one or more of the MMUs **439A-439E** may be provided that ensures cache coherence between the caches of the host processors (e.g., **405**) and the GPUs **410-413** and implements biasing techniques indicating the physical memories in which certain types of data should be stored. While multiple instances of bias/coherence management circuitry **494A-494E** are illustrated in FIG. 4F, the bias/coherence circuitry may be implemented within the MMU of one or more host processors **405** and/or within the accelerator integration circuit **436**.

The GPU-attached memory **420-423** may be mapped as part of system memory, and accessed using shared virtual memory (SVM) technology, but without suffering the typical performance drawbacks associated with full system cache coherence. The ability to GPU-attached memory **420-423** to be accessed as system memory without onerous cache coherence overhead provides a beneficial operating environment for GPU offload. This arrangement allows the host processor **405** software to setup operands and access computation results, without the overhead of tradition I/O DMA data copies. Such traditional copies involve driver calls, interrupts and memory mapped I/O (MMIO) accesses that are all inefficient relative to simple memory accesses. At the same time, the ability to access GPU attached memory **420-423** without cache coherence overheads can be contributory to the execution time of an offloaded computation. In cases with substantial streaming write memory traffic, for example, cache coherence overhead can significantly reduce the effective write bandwidth seen by a GPU **410-413**. The efficiency of operand setup, the efficiency of results access, and the efficiency of GPU computation all play a role in determining the effectiveness of GPU offload.

A selection between GPU bias and host processor bias may be driven by a bias tracker data structure. A bias table may be used, for example, which may be a page-granular structure (i.e., controlled at the granularity of a memory page) that includes 1 or 2 bits per GPU-attached memory page. The bias table may be implemented in a stolen memory range of one or more GPU-attached memories **420-423**, with or without a bias cache in the GPU **410-413** (e.g., to cache frequently/recently used entries of the bias table). Alternatively, the entire bias table may be maintained within the GPU.

In one implementation, the bias table entry associated with each access to the GPU-attached memory **420-423** is accessed prior the actual access to the GPU memory, causing the following operations. First, local requests from the GPU **410-413** that find their page in GPU bias are forwarded directly to a corresponding GPU memory **420-423**. Local requests from the GPU that find their page in host bias are forwarded to the processor **405** (e.g., over a high-speed link as discussed above). Optionally, requests from the processor **405** that find the requested page in host processor bias complete the request like a normal memory read. Alternatively, requests directed to a GPU-biased page may be forwarded to the GPU **410-413**. The GPU may then transition the page to a host processor bias if it is not currently using the page.

The bias state of a page can be changed either by a software-based mechanism, a hardware-assisted software-based mechanism, or, for a limited set of cases, a purely hardware-based mechanism.

One mechanism for changing the bias state employs an API call (e.g., OpenCL), which, in turn, calls the GPU's device driver which, in turn, sends a message (or enqueues a command descriptor) to the GPU directing it to change the bias state and, for some transitions, perform a cache flushing operation in the host. The cache flushing operation is utilized for a transition from host processor **405** bias to GPU bias, but is not utilized for the opposite transition.

Cache coherency may be maintained by temporarily rendering GPU-biased pages uncacheable by the host processor **405**. To access these pages, the processor **405** may request access from the GPU **410** which may or may not grant access right away, depending on the implementation. Thus, to reduce communication between the host processor **405** and GPU **410** it is beneficial to ensure that GPU-biased pages are those which are utilized by the GPU but not the host processor **405** and vice versa.

Graphics Processing Pipeline

FIG. 5 illustrates a graphics processing pipeline **500**. A graphics multiprocessor, such as graphics multiprocessor **234** as in FIG. 2D, graphics multiprocessor **325** of FIG. 3A, graphics multiprocessor **350** of FIG. 3B can implement the illustrated graphics processing pipeline **500**. The graphics multiprocessor can be included within the parallel processing subsystems as described herein, such as the parallel processor **200** of FIG. 2A, which may be related to the parallel processor(s) **112** of FIG. 1 and may be used in place of one of those. The various parallel processing systems can implement the graphics processing pipeline **500** via one or more instances of the parallel processing unit (e.g., parallel processing unit **202** of FIG. 2A) as described herein. For example, a shader unit (e.g., graphics multiprocessor **234** of FIG. 2C) may be configured to perform the functions of one or more of a vertex processing unit **504**, a tessellation control processing unit **508**, a tessellation evaluation processing unit **512**, a geometry processing unit **516**, and a fragment/pixel processing unit **524**. The functions of data assembler **502**, primitive assemblers **506**, **514**, **518**, tessellation unit **510**, rasterizer **522**, and raster operations unit **526** may also be performed by other processing engines within a processing cluster (e.g., processing cluster **214** of FIG. 2A) and a corresponding partition unit (e.g., partition unit **220A-220N** of FIG. 2A). The graphics processing pipeline **500** may also be implemented using dedicated processing units for one or more functions. It is also possible that one or more portions of the graphics processing pipeline **500** are performed by parallel processing logic within a general-purpose processor (e.g., CPU). Optionally, one or more portions of the graphics processing pipeline **500** can access on-chip memory (e.g., parallel processor memory **222** as in FIG. 2A) via a memory interface **528**, which may be an instance of the memory interface **218** of FIG. 2A. The graphics processor pipeline **500** may also be implemented via a multi-core group **365A** as in FIG. 3C.

The data assembler **502** is a processing unit that may collect vertex data for surfaces and primitives. The data assembler **502** then outputs the vertex data, including the vertex attributes, to the vertex processing unit **504**. The vertex processing unit **504** is a programmable execution unit that executes vertex shader programs, lighting and transforming vertex data as specified by the vertex shader programs. The vertex processing unit **504** reads data that is stored in cache, local or system memory for use in process-

ing the vertex data and may be programmed to transform the vertex data from an object-based coordinate representation to a world space coordinate space or a normalized device coordinate space.

A first instance of a primitive assembler **506** receives vertex attributes from the vertex processing unit **504**. The primitive assembler **506** readings stored vertex attributes as needed and constructs graphics primitives for processing by tessellation control processing unit **508**. The graphics primitives include triangles, line segments, points, patches, and so forth, as supported by various graphics processing application programming interfaces (APIs).

The tessellation control processing unit **508** treats the input vertices as control points for a geometric patch. The control points are transformed from an input representation from the patch (e.g., the patch's bases) to a representation that is suitable for use in surface evaluation by the tessellation evaluation processing unit **512**. The tessellation control processing unit **508** can also compute tessellation factors for edges of geometric patches. A tessellation factor applies to a single edge and quantifies a view-dependent level of detail associated with the edge. A tessellation unit **510** is configured to receive the tessellation factors for edges of a patch and to tessellate the patch into multiple geometric primitives such as line, triangle, or quadrilateral primitives, which are transmitted to a tessellation evaluation processing unit **512**. The tessellation evaluation processing unit **512** operates on parameterized coordinates of the subdivided patch to generate a surface representation and vertex attributes for each vertex associated with the geometric primitives.

A second instance of a primitive assembler **514** receives vertex attributes from the tessellation evaluation processing unit **512**, reading stored vertex attributes as needed, and constructs graphics primitives for processing by the geometry processing unit **516**. The geometry processing unit **516** is a programmable execution unit that executes geometry shader programs to transform graphics primitives received from primitive assembler **514** as specified by the geometry shader programs. The geometry processing unit **516** may be programmed to subdivide the graphics primitives into one or more new graphics primitives and calculate parameters used to rasterize the new graphics primitives.

The geometry processing unit **516** may be able to add or delete elements in the geometry stream. The geometry processing unit **516** outputs the parameters and vertices specifying new graphics primitives to primitive assembler **518**. The primitive assembler **518** receives the parameters and vertices from the geometry processing unit **516** and constructs graphics primitives for processing by a viewport scale, cull, and clip unit **520**. The geometry processing unit **516** reads data that is stored in parallel processor memory or system memory for use in processing the geometry data. The viewport scale, cull, and clip unit **520** performs clipping, culling, and viewport scaling and outputs processed graphics primitives to a rasterizer **522**.

The rasterizer **522** can perform depth culling and other depth-based optimizations. The rasterizer **522** also performs scan conversion on the new graphics primitives to generate fragments and output those fragments and associated coverage data to the fragment/pixel processing unit **524**. The fragment/pixel processing unit **524** is a programmable execution unit that is configured to execute fragment shader programs or pixel shader programs. The fragment/pixel processing unit **524** transforming fragments or pixels received from rasterizer **522**, as specified by the fragment or pixel shader programs. For example, the fragment/pixel

processing unit **524** may be programmed to perform operations included but not limited to texture mapping, shading, blending, texture correction and perspective correction to produce shaded fragments or pixels that are output to a raster operations unit **526**. The fragment/pixel processing unit **524** can read data that is stored in either the parallel processor memory or the system memory for use when processing the fragment data. Fragment or pixel shader programs may be configured to shade at sample, pixel, tile, or other granularities depending on the sampling rate configured for the processing units.

The raster operations unit **526** is a processing unit that performs raster operations including, but not limited to stencil, z-test, blending, and the like, and outputs pixel data as processed graphics data to be stored in graphics memory (e.g., parallel processor memory **222** as in FIG. 2A, and/or system memory **104** as in FIG. 1), to be displayed on the one or more display device(s) **110** or for further processing by one of the one or more processor(s) **102** or parallel processor(s) **112**. The raster operations unit **526** may be configured to compress z or color data that is written to memory and decompress z or color data that is read from memory.

Machine Learning Overview

The architecture described above can be applied to perform training and inference operations using machine learning models. Machine learning has been successful at solving many kinds of tasks. The computations that arise when training and using machine learning algorithms (e.g., neural networks) lend themselves naturally to efficient parallel implementations. Accordingly, parallel processors such as general-purpose graphics processing units (GPGPUs) have played a significant role in the practical implementation of deep neural networks. Parallel graphics processors with single instruction, multiple thread (SIMT) architectures are designed to maximize the amount of parallel processing in the graphics pipeline. In an SIMT architecture, groups of parallel threads attempt to execute program instructions synchronously together as often as possible to increase processing efficiency. The efficiency provided by parallel machine learning algorithm implementations allows the use of high capacity networks and enables those networks to be trained on larger datasets.

A machine learning algorithm is an algorithm that can learn based on a set of data. For example, machine learning algorithms can be designed to model high-level abstractions within a data set. For example, image recognition algorithms can be used to determine which of several categories to which a given input belong; regression algorithms can output a numerical value given an input; and pattern recognition algorithms can be used to generate translated text or perform text to speech and/or speech recognition.

An example type of machine learning algorithm is a neural network. There are many types of neural networks; a simple type of neural network is a feedforward network. A feedforward network may be implemented as an acyclic graph in which the nodes are arranged in layers. Typically, a feedforward network topology includes an input layer and an output layer that are separated by at least one hidden layer. The hidden layer transforms input received by the input layer into a representation that is useful for generating output in the output layer. The network nodes are fully connected via edges to the nodes in adjacent layers, but there are no edges between nodes within each layer. Data received at the nodes of an input layer of a feedforward network are propagated (i.e., “fed forward”) to the nodes of the output layer via an activation function that calculates the states of the nodes of each successive layer in the network based on

coefficients (“weights”) respectively associated with each of the edges connecting the layers. Depending on the specific model being represented by the algorithm being executed, the output from the neural network algorithm can take various forms.

Before a machine learning algorithm can be used to model a particular problem, the algorithm is trained using a training data set. Training a neural network involves selecting a network topology, using a set of training data representing a problem being modeled by the network, and adjusting the weights until the network model performs with a minimal error for all instances of the training data set. For example, during a supervised learning training process for a neural network, the output produced by the network in response to the input representing an instance in a training data set is compared to the “correct” labeled output for that instance, an error signal representing the difference between the output and the labeled output is calculated, and the weights associated with the connections are adjusted to minimize that error as the error signal is backward propagated through the layers of the network. The network is considered “trained” when the errors for each of the outputs generated from the instances of the training data set are minimized.

The accuracy of a machine learning algorithm can be affected significantly by the quality of the data set used to train the algorithm. The training process can be computationally intensive and may utilize a significant amount of time on a conventional general-purpose processor. Accordingly, parallel processing hardware is used to train many types of machine learning algorithms. This is particularly useful for optimizing the training of neural networks, as the computations performed in adjusting the coefficients in neural networks lend themselves naturally to parallel implementations. Specifically, many machine learning algorithms and software applications have been adapted to make use of the parallel processing hardware within general-purpose graphics processing devices.

FIG. 6 is a generalized diagram of a machine learning software stack **600**. A machine learning application **602** is any logic that can be configured to train a neural network using a training dataset or to use a trained deep neural network to implement machine intelligence. The machine learning application **602** can include training and inference functionality for a neural network and/or specialized software that can be used to train a neural network before deployment. The machine learning application **602** can implement any type of machine intelligence including but not limited to image recognition, mapping and localization, autonomous navigation, speech synthesis, medical imaging, or language translation. Example machine learning applications **602** include, but are not limited to, voice-based virtual assistants, image or facial recognition algorithms, autonomous navigation, and the software tools that are used to train the machine learning models used by the machine learning applications **602**.

Hardware acceleration for the machine learning application **602** can be enabled via a machine learning framework **604**. The machine learning framework **604** can provide a library of machine learning primitives. Machine learning primitives are basic operations that are commonly performed by machine learning algorithms. Without the machine learning framework **604**, developers of machine learning algorithms would be utilized to create and optimize the main computational logic associated with the machine learning algorithm, then re-optimize the computational logic as new parallel processors are developed. Instead, the machine learning application can be configured to perform

the computations using the primitives provided by the machine learning framework **604**. Example primitives include tensor convolutions, activation functions, and pooling, which are computational operations that are performed while training a convolutional neural network (CNN). The machine learning framework **604** can also provide primitives to implement basic linear algebra subprograms performed by many machine-learning algorithms, such as matrix and vector operations. Examples of a machine learning framework **604** include, but are not limited to, TensorFlow, TensorRT, PyTorch, MXNet, Caffe, and other high-level machine learning frameworks.

The machine learning framework **604** can process input data received from the machine learning application **602** and generate the appropriate input to a compute framework **606**. The compute framework **606** can abstract the underlying instructions provided to the GPGPU driver **608** to enable the machine learning framework **604** to take advantage of hardware acceleration via the GPGPU hardware **610** without requiring the machine learning framework **604** to have intimate knowledge of the architecture of the GPGPU hardware **610**. Additionally, the compute framework **606** can enable hardware acceleration for the machine learning framework **604** across a variety of types and generations of the GPGPU hardware **610**. Example compute frameworks **606** include the CUDA compute framework and associated machine learning libraries, such as the CUDA Deep Neural Network (cuDNN) library. The machine learning software stack **600** can also include communication libraries or frameworks to facilitate multi-GPU and multi-node compute.

GPGPU Machine Learning Acceleration

FIG. 7 illustrates a general-purpose graphics processing unit **700**, which may be the parallel processor **200** of FIG. 2A or the parallel processor(s) **112** of FIG. 1. The general-purpose processing unit (GPGPU) **700** may be configured to provide support for hardware acceleration of primitives provided by a machine learning framework to accelerate the processing the type of computational workloads associated with training deep neural networks. Additionally, the GPGPU **700** can be linked directly to other instances of the GPGPU to create a multi-GPU cluster to improve training speed for particularly deep neural networks. Primitives are also supported to accelerate inference operations for deployed neural networks.

The GPGPU **700** includes a host interface **702** to enable a connection with a host processor. The host interface **702** may be a PCI Express interface. However, the host interface can also be a vendor specific communications interface or communications fabric. The GPGPU **700** receives commands from the host processor and uses a global scheduler **704** to distribute execution threads associated with those commands to a set of processing clusters **706A-706H**. The processing clusters **706A-706H** share a cache memory **708**. The cache memory **708** can serve as a higher-level cache for cache memories within the processing clusters **706A-706H**. The illustrated processing clusters **706A-706H** may correspond with processing clusters **214A-214N** as in FIG. 2A.

The GPGPU **700** includes memory **714A-714B** coupled with the processing clusters **706A-706H** via a set of memory controllers **712A-712B**. The memory **714A-714B** can include various types of memory devices including dynamic random-access memory (DRAM) or graphics random access memory, such as synchronous graphics random access memory (SGRAM), including graphics double data rate (GDDR) memory. The memory **714A-714B** may also

include 3D stacked memory, including but not limited to high bandwidth memory (HBM).

Each of the processing clusters **706A-706H** may include a set of graphics multiprocessors, such as the graphics multiprocessor **234** of FIG. 2D, graphics multiprocessor **325** of FIG. 3A, graphics multiprocessor **350** of FIG. 3B, or may include a multi-core group **365A-365N** as in FIG. 3C. The graphics multiprocessors of the compute cluster include multiple types of integer and floating-point logic units that can perform computational operations at a range of precisions including suited for machine learning computations. For example, at least a subset of the floating-point units in each of the processing clusters **706A-706H** can be configured to perform 16-bit or 32-bit floating point operations, while a different subset of the floating-point units can be configured to perform 64-bit floating point operations.

Multiple instances of the GPGPU **700** can be configured to operate as a compute cluster. The communication mechanism used by the compute cluster for synchronization and data exchange varies across embodiments. For example, the multiple instances of the GPGPU **700** communicate over the host interface **702**. In one embodiment the GPGPU **700** includes an I/O hub **709** that couples the GPGPU **700** with a GPU link **710** that enables a direct connection to other instances of the GPGPU. The GPU link **710** may be coupled to a dedicated GPU-to-GPU bridge that enables communication and synchronization between multiple instances of the GPGPU **700**. Optionally, the GPU link **710** couples with a high-speed interconnect to transmit and receive data to other GPGPUs or parallel processors. The multiple instances of the GPGPU **700** may be located in separate data processing systems and communicate via a network device that is accessible via the host interface **702**. The GPU link **710** may be configured to enable a connection to a host processor in addition to or as an alternative to the host interface **702**.

While the illustrated configuration of the GPGPU **700** can be configured to train neural networks, an alternate configuration of the GPGPU **700** can be configured for deployment within a high performance or low power inferencing platform. In an inferencing configuration, the GPGPU **700** includes fewer of the processing clusters **706A-706H** relative to the training configuration. Additionally, memory technology associated with the memory **714A-714B** may differ between inferencing and training configurations. In one embodiment, the inferencing configuration of the GPGPU **700** can support inferencing specific instructions. For example, an inferencing configuration can provide support for one or more 8-bit integer dot product instructions, which are commonly used during inferencing operations for deployed neural networks.

FIG. 8 illustrates a multi-GPU computing system **800**. The multi-GPU computing system **800** can include a processor **802** coupled to multiple GPGPUs **806A-806D** via a host interface switch **804**. The host interface switch **804** may be a PCI express switch device that couples the processor **802** to a PCI express bus over which the processor **802** can communicate with the set of GPGPUs **806A-806D**. Each of the multiple GPGPUs **806A-806D** can be an instance of the GPGPU **700** of FIG. 7. The GPGPUs **806A-806D** can interconnect via a set of high-speed point to point GPU to GPU links **816**. The high-speed GPU to GPU links can connect to each of the GPGPUs **806A-806D** via a dedicated GPU link, such as the GPU link **710** as in FIG. 7. The P2P GPU links **816** enable direct communication between each of the GPGPUs **806A-806D** without requiring communication over the host interface bus to which the processor **802** is connected. With GPU-to-GPU traffic directed to the P2P

GPU links, the host interface bus remains available for system memory access or to communicate with other instances of the multi-GPU computing system **800**, for example, via one or more network devices. While in FIG. **8** the GPGPUs **806A-806D** connect to the processor **802** via the host interface switch **804**, the processor **802** may alternatively include direct support for the P2P GPU links **816** and connect directly to the GPGPUs **806A-806D**. In one embodiment the P2P GPU link **816** enable the multi-GPU computing system **800** to operate as a single logical GPU. Machine Learning Neural Network Implementations

The computing architecture described herein can be configured to perform the types of parallel processing that is particularly suited for training and deploying neural networks for machine learning. A neural network can be generalized as a network of functions having a graph relationship. As is well-known in the art, there are a variety of types of neural network implementations used in machine learning. One example type of neural network is the feed-forward network, as previously described.

A second example type of neural network is the Convolutional Neural Network (CNN). A CNN is a specialized feedforward neural network for processing data having a known, grid-like topology, such as image data. Accordingly, CNNs are commonly used for compute vision and image recognition applications, but they also may be used for other types of pattern recognition such as speech and language processing. The nodes in the CNN input layer are organized into a set of “filters” (feature detectors inspired by the receptive fields found in the retina), and the output of each set of filters is propagated to nodes in successive layers of the network. The computations for a CNN include applying the convolution mathematical operation to each filter to produce the output of that filter. Convolution is a specialized kind of mathematical operation performed by two functions to produce a third function that is a modified version of one of the two original functions. In convolutional network terminology, the first function to the convolution can be referred to as the input, while the second function can be referred to as the convolution kernel. The output may be referred to as the feature map. For example, the input to a convolution layer can be a multidimensional array of data that defines the various color components of an input image. The convolution kernel can be a multidimensional array of parameters, where the parameters are adapted by the training process for the neural network.

Recurrent neural networks (RNNs) are a family of feed-forward neural networks that include feedback connections between layers. RNNs enable modeling of sequential data by sharing parameter data across different parts of the neural network. The architecture for an RNN includes cycles. The cycles represent the influence of a present value of a variable on its own value at a future time, as at least a portion of the output data from the RNN is used as feedback for processing subsequent input in a sequence. This feature makes RNNs particularly useful for language processing due to the variable nature in which language data can be composed.

The figures described below present example feedforward, CNN, and RNN networks, as well as describe a general process for respectively training and deploying each of those types of networks. It can be understood that these descriptions are example and non-limiting as to any specific embodiment described herein and the concepts illustrated can be applied generally to deep neural networks and machine learning techniques in general.

The example neural networks described above can be used to perform deep learning. Deep learning is machine

learning using deep neural networks. The deep neural networks used in deep learning are artificial neural networks composed of multiple hidden layers, as opposed to shallow neural networks that include only a single hidden layer. Deeper neural networks are generally more computationally intensive to train. However, the additional hidden layers of the network enable multistep pattern recognition that results in reduced output error relative to shallow machine learning techniques.

Deep neural networks used in deep learning typically include a front-end network to perform feature recognition coupled to a back-end network which represents a mathematical model that can perform operations (e.g., object classification, speech recognition, etc.) based on the feature representation provided to the model. Deep learning enables machine learning to be performed without requiring hand crafted feature engineering to be performed for the model. Instead, deep neural networks can learn features based on statistical structure or correlation within the input data. The learned features can be provided to a mathematical model that can map detected features to an output. The mathematical model used by the network is generally specialized for the specific task to be performed, and different models can be used to perform different task.

Once the neural network is structured, a learning model can be applied to the network to train the network to perform specific tasks. The learning model describes how to adjust the weights within the model to reduce the output error of the network. Backpropagation of errors is a common method used to train neural networks. An input vector is presented to the network for processing. The output of the network is compared to the desired output using a loss function and an error value is calculated for each of the neurons in the output layer. The error values are then propagated backwards until each neuron has an associated error value which roughly represents its contribution to the original output. The network can then learn from those errors using an algorithm, such as the stochastic gradient descent algorithm, to update the weights of the of the neural network.

FIG. **9A-9B** illustrate an example convolutional neural network. FIG. **9A** illustrates various layers within a CNN. As shown in FIG. **9A**, an example CNN used to model image processing can receive input **902** describing the red, green, and blue (RGB) components of an input image. The input **902** can be processed by multiple convolutional layers (e.g., convolutional layer **904**, convolutional layer **906**). The output from the multiple convolutional layers may optionally be processed by a set of fully connected layers **908**. Neurons in a fully connected layer have full connections to all activations in the previous layer, as previously described for a feedforward network. The output from the fully connected layers **908** can be used to generate an output result from the network. The activations within the fully connected layers **908** can be computed using matrix multiplication instead of convolution. Not all CNN implementations make use of fully connected layers **908**. For example, in some implementations the convolutional layer **906** can generate output for the CNN.

The convolutional layers are sparsely connected, which differs from traditional neural network configuration found in the fully connected layers **908**. Traditional neural network layers are fully connected, such that every output unit interacts with every input unit. However, the convolutional layers are sparsely connected because the output of the convolution of a field is input (instead of the respective state value of each of the nodes in the field) to the nodes of the subsequent layer, as illustrated. The kernels associated with

35

the convolutional layers perform convolution operations, the output of which is sent to the next layer. The dimensionality reduction performed within the convolutional layers is one aspect that enables the CNN to scale to process large images.

FIG. 9B illustrates example computation stages within a convolutional layer of a CNN. Input to a convolutional layer **912** of a CNN can be processed in three stages of a convolutional layer **914**. The three stages can include a convolution stage **916**, a detector stage **918**, and a pooling stage **920**. The convolutional layer **914** can then output data to a successive convolutional layer. The final convolutional layer of the network can generate output feature map data or provide input to a fully connected layer, for example, to generate a classification value for the input to the CNN.

In the convolution stage **916** performs several convolutions in parallel to produce a set of linear activations. The convolution stage **916** can include an affine transformation, which is any transformation that can be specified as a linear transformation plus a translation. Affine transformations include rotations, translations, scaling, and combinations of these transformations. The convolution stage computes the output of functions (e.g., neurons) that are connected to specific regions in the input, which can be determined as the local region associated with the neuron. The neurons compute a dot product between the weights of the neurons and the region in the local input to which the neurons are connected. The output from the convolution stage **916** defines a set of linear activations that are processed by successive stages of the convolutional layer **914**.

The linear activations can be processed by a detector stage **918**. In the detector stage **918**, each linear activation is processed by a non-linear activation function. The non-linear activation function increases the nonlinear properties of the overall network without affecting the receptive fields of the convolution layer. Several types of non-linear activation functions may be used. One particular type is the rectified linear unit (ReLU), which uses an activation function defined as $f(x)=\max(0, x)$, such that the activation is thresholded at zero.

The pooling stage **920** uses a pooling function that replaces the output of the convolutional layer **906** with a summary statistic of the nearby outputs. The pooling function can be used to introduce translation invariance into the neural network, such that small translations to the input do not change the pooled outputs. Invariance to local translation can be useful in scenarios where the presence of a feature in the input data is more important than the precise location of the feature. Various types of pooling functions can be used during the pooling stage **920**, including max pooling, average pooling, and l2-norm pooling. Additionally, some CNN implementations do not include a pooling stage. Instead, such implementations substitute and additional convolution stage having an increased stride relative to previous convolution stages.

The output from the convolutional layer **914** can then be processed by the next layer **922**. The next layer **922** can be an additional convolutional layer or one of the fully connected layers **908**. For example, the first convolutional layer **904** of FIG. 9A can output to the second convolutional layer **906**, while the second convolutional layer can output to a first layer of the fully connected layers **908**.

FIG. 10 illustrates an example recurrent neural network **1000**. In a recurrent neural network (RNN), the previous state of the network influences the output of the current state of the network. RNNs can be built in a variety of ways using a variety of functions. The use of RNNs generally revolves around using mathematical models to predict the future

36

based on a prior sequence of inputs. For example, an RNN may be used to perform statistical language modeling to predict an upcoming word given a previous sequence of words. The illustrated RNN **1000** can be described as having an input layer **1002** that receives an input vector, hidden layers **1004** to implement a recurrent function, a feedback mechanism **1005** to enable a 'memory' of previous states, and an output layer **1006** to output a result. The RNN **1000** operates based on time-steps. The state of the RNN at a given time step is influenced based on the previous time step via the feedback mechanism **1005**. For a given time step, the state of the hidden layers **1004** is defined by the previous state and the input at the current time step. An initial input (x_1) at a first time step can be processed by the hidden layer **1004**. A second input (x_2) can be processed by the hidden layer **1004** using state information that is determined during the processing of the initial input (x_1). A given state can be computed as $s_t=f(Ux_t+Ws_{t-1})$, where U and W are parameter matrices. The function f is generally a non-linearity, such as the hyperbolic tangent function ($\tanh$) or a variant of the rectifier function $f(x)=\max(0, x)$. However, the specific mathematical function used in the hidden layers **1004** can vary depending on the specific implementation details of the RNN **1000**.

In addition to the basic CNN and RNN networks described, acceleration for variations on those networks may be enabled. One example RNN variant is the long short term memory (LSTM) RNN. LSTM RNNs are capable of learning long-term dependencies that may be utilized for processing longer sequences of language. A variant on the CNN is a convolutional deep belief network, which has a structure similar to a CNN and is trained in a manner similar to a deep belief network. A deep belief network (DBN) is a generative neural network that is composed of multiple layers of stochastic (random) variables. DBNs can be trained layer-by-layer using greedy unsupervised learning. The learned weights of the DBN can then be used to provide pre-train neural networks by determining an initial set of weights for the neural network. In further embodiments, acceleration for reinforcement learning is enabled. In reinforcement learning, an artificial agent learn by interacting with its environment. The agent is configured to optimize certain objectives to maximize cumulative rewards.

FIG. 11 illustrates training and deployment of a deep neural network. Once a given network has been structured for a task the neural network is trained using a training dataset **1102**. Various training frameworks **1104** have been developed to enable hardware acceleration of the training process. For example, the machine learning framework **604** of FIG. 6 may be configured as a training framework **1104**. The training framework **1104** can hook into an untrained neural network **1106** and enable the untrained neural net to be trained using the parallel processing resources described herein to generate a trained neural network **1108**.

To start the training process the initial weights may be chosen randomly or by pre-training using a deep belief network. The training cycle then be performed in either a supervised or unsupervised manner.

Supervised learning is a learning method in which training is performed as a mediated operation, such as when the training dataset **1102** includes input paired with the desired output for the input, or where the training dataset includes input having known output and the output of the neural network is manually graded. The network processes the inputs and compares the resulting outputs against a set of expected or desired outputs. Errors are then propagated back through the system. The training framework **1104** can adjust

to adjust the weights that control the untrained neural network **1106**. The training framework **1104** can provide tools to monitor how well the untrained neural network **1106** is converging towards a model suitable to generating correct answers based on known input data. The training process occurs repeatedly as the weights of the network are adjusted to refine the output generated by the neural network. The training process can continue until the neural network reaches a statistically desired accuracy associated with a trained neural net **1108**. The trained neural network **1108** can then be deployed to implement any number of machine learning operations to generate an inference result **1114** based on input of new data **1112**.

Unsupervised learning is a learning method in which the network attempts to train itself using unlabeled data. Thus, for unsupervised learning the training dataset **1102** can include input data without any associated output data. The untrained neural network **1106** can learn groupings within the unlabeled input and can determine how individual inputs are related to the overall dataset. Unsupervised training can be used to generate a self-organizing map, which is a type of trained neural network **1108** capable of performing operations useful in reducing the dimensionality of data. Unsupervised training can also be used to perform anomaly detection, which allows the identification of data points in an input dataset that deviate from the normal patterns of the data.

Variations on supervised and unsupervised training may also be employed. Semi-supervised learning is a technique in which in the training dataset **1102** includes a mix of labeled and unlabeled data of the same distribution. Incremental learning is a variant of supervised learning in which input data is continuously used to further train the model. Incremental learning enables the trained neural network **1108** to adapt to the new data **1112** without forgetting the knowledge instilled within the network during initial training.

Whether supervised or unsupervised, the training process for particularly deep neural networks may be too computationally intensive for a single compute node. Instead of using a single compute node, a distributed network of computational nodes can be used to accelerate the training process.

FIG. **12A** is a block diagram illustrating distributed learning. Distributed learning is a training model that uses multiple distributed computing nodes to perform supervised or unsupervised training of a neural network. The distributed computational nodes can each include one or more host processors and one or more of the general-purpose processing nodes, such as the highly parallel general-purpose graphics processing unit **700** as in FIG. **7**. As illustrated, distributed learning can be performed with model parallelism **1202**, data parallelism **1204**, or a combination of model and data parallelism **1206**.

In model parallelism **1202**, different computational nodes in a distributed system can perform training computations for different parts of a single network. For example, each layer of a neural network can be trained by a different processing node of the distributed system. The benefits of model parallelism include the ability to scale to particularly large models. Splitting the computations associated with different layers of the neural network enables the training of very large neural networks in which the weights of all layers would not fit into the memory of a single computational node. In some instances, model parallelism can be particularly useful in performing unsupervised training of large neural networks.

In data parallelism **1204**, the different nodes of the distributed network have a complete instance of the model and each node receives a different portion of the data. The results from the different nodes are then combined. While different approaches to data parallelism are possible, data parallel training approaches all utilize a technique of combining results and synchronizing the model parameters between each node. Example approaches to combining data include parameter averaging and update based data parallelism. Parameter averaging trains each node on a subset of the training data and sets the global parameters (e.g., weights, biases) to the average of the parameters from each node. Parameter averaging uses a central parameter server that maintains the parameter data. Update based data parallelism is similar to parameter averaging except that instead of transferring parameters from the nodes to the parameter server, the updates to the model are transferred. Additionally, update based data parallelism can be performed in a decentralized manner, where the updates are compressed and transferred between nodes.

Combined model and data parallelism **1206** can be implemented, for example, in a distributed system in which each computational node includes multiple GPUs. Each node can have a complete instance of the model with separate GPUs within each node are used to train different portions of the model.

Distributed training has increased overhead relative to training on a single machine. However, the parallel processors and GPGPUs described herein can each implement various techniques to reduce the overhead of distributed training, including techniques to enable high bandwidth GPU-to-GPU data transfer and accelerated remote data synchronization.

FIG. **12B** is a block diagram illustrating a programmable network interface **1210** and data processing unit. The programmable network interface **1210** is a programmable network engine that can be used to accelerate network-based compute tasks within a distributed environment. The programmable network interface **1210** can couple with a host system via host interface **1270**. The programmable network interface **1210** can be used to accelerate network or storage operations for CPUs or GPUs of the host system. The host system can be, for example, a node of a distributed learning system used to perform distributed training, for example, as shown in FIG. **12A**. The host system can also be a data center node within a data center.

In one embodiment, access to remote storage containing model data can be accelerated by the programmable network interface **1210**. For example, the programmable network interface **1210** can be configured to present remote storage devices as local storage devices to the host system. The programmable network interface **1210** can also accelerate remote direct memory access (RDMA) operations performed between GPUs of the host system with GPUs of remote systems. In one embodiment, the programmable network interface **1210** can enable storage functionality such as, but not limited to NVME-oF. The programmable network interface **1210** can also accelerate encryption, data integrity, compression, and other operations for remote storage on behalf of the host system, allowing remote storage to approach the latencies of storage devices that are directly attached to the host system.

The programmable network interface **1210** can also perform resource allocation and management on behalf of the host system. Storage security operations can be offloaded to the programmable network interface **1210** and performed in concert with the allocation and management of remote

storage resources. Network-based operations to manage access to the remote storage that would otherwise be performed by a processor of the host system can instead be performed by the programmable network interface **1210**.

In one embodiment, network and/or data security operations can be offloaded from the host system to the programmable network interface **1210**. Data center security policies for a data center node can be handled by the programmable network interface **1210** instead of the processors of the host system. For example, the programmable network interface **1210** can detect and mitigate against an attempted network-based attack (e.g., DDoS) on the host system, preventing the attack from compromising the availability of the host system.

The programmable network interface **1210** can include a system on a chip (SoC **1220**) that executes an operating system via multiple processor cores **1222**. The processor cores **1222** can include general-purpose processor (e.g., CPU) cores. In one embodiment the processor cores **1222** can also include one or more GPU cores. The SoC **1220** can execute instructions stored in a memory device **1240**. A storage device **1250** can store local operating system data. The storage device **1250** and memory device **1240** can also be used to cache remote data for the host system. Network ports **1260A-1260B** enable a connection to a network or fabric and facilitate network access for the SoC **1220** and, via the host interface **1270**, for the host system. The programmable network interface **1210** can also include an I/O interface **1275**, such as a USB interface. The I/O interface **1275** can be used to couple external devices to the programmable network interface **1210** or as a debug interface. The programmable network interface **1210** also includes a management interface **1230** that enables software on the host device to manage and configure the programmable network interface **1210** and/or SoC **1220**. In one embodiment the programmable network interface **1210** may also include one or more accelerators or GPUs **1245** to accept offload of parallel compute tasks from the SoC **1220**, host system, or remote systems coupled via the network ports **1260A-1260B**.

Example Machine Learning Applications

Machine learning can be applied to solve a variety of technological problems, including but not limited to computer vision, autonomous driving and navigation, speech recognition, and language processing. Computer vision has traditionally been an active research area for machine learning applications. Applications of computer vision range from reproducing human visual abilities, such as recognizing faces, to creating new categories of visual abilities. For example, computer vision applications can be configured to recognize sound waves from the vibrations induced in objects visible in a video. Parallel processor accelerated machine learning enables computer vision applications to be trained using significantly larger training dataset than previously feasible and enables inferencing systems to be deployed using low power parallel processors.

Parallel processor accelerated machine learning has autonomous driving applications including lane and road sign recognition, obstacle avoidance, navigation, and driving control. Accelerated machine learning techniques can be used to train driving models based on datasets that define the appropriate responses to specific training input. The parallel processors described herein can enable rapid training of the increasingly complex neural networks used for autonomous driving solutions and enables the deployment of low power inferencing processors in a mobile platform suitable for integration into autonomous vehicles.

Parallel processor accelerated deep neural networks have enabled machine learning approaches to automatic speech recognition (ASR). ASR includes the creation of a function that computes a probable linguistic sequence given an input acoustic sequence. Accelerated machine learning using deep neural networks have enabled the replacement of the hidden Markov models (HMMs) and Gaussian mixture models (GMMs) previously used for ASR.

Parallel processor accelerated machine learning can also be used to accelerate natural language processing. Automatic learning procedures can make use of statistical inference algorithms to produce models that are robust to erroneous or unfamiliar input. Example natural language processor applications include automatic machine translation between human languages.

The parallel processing platforms used for machine learning can be divided into training platforms and deployment platforms. Training platforms are generally highly parallel and include optimizations to accelerate multi-GPU single node training and multi-node, multi-GPU training. Example parallel processors suited for training include the general-purpose graphics processing unit **700** of FIG. 7 and the multi-GPU computing system **800** of FIG. 8. On the contrary, deployed machine learning platforms generally include lower power parallel processors suitable for use in products such as cameras, autonomous robots, and autonomous vehicles.

Additionally, machine learning techniques can be applied to accelerate or enhance graphics processing activities. For example, a machine learning model can be trained to recognize output generated by a GPU accelerated application and generate an upscaled version of that output. Such techniques can be applied to accelerate the generation of high resolution images for a gaming application. Various other graphics pipeline activities can benefit from the use of machine learning. For example, machine learning models can be trained to perform tessellation operations on geometry data to increase the complexity of geometric models, allowing fine-detailed geometry to be automatically generated from geometry of relatively lower detail.

FIG. 13 illustrates an example inferencing system on a chip (SOC) **1300** suitable for performing inferencing using a trained model. The SOC **1300** can integrate processing components including a media processor **1302**, a vision processor **1304**, a GPGPU **1306** and a multi-core processor **1308**. The GPGPU **1306** may be a GPGPU as described herein, such as the GPGPU **700**, and the multi-core processor **1308** may be a multi-core processor described herein, such as the multi-core processors **405-406**. The SOC **1300** can additionally include on-chip memory **1305** that can enable a shared on-chip data pool that is accessible by each of the processing components. The processing components can be optimized for low power operation to enable deployment to a variety of machine learning platforms, including autonomous vehicles and autonomous robots. For example, one implementation of the SOC **1300** can be used as a portion of the main control system for an autonomous vehicle. Where the SOC **1300** is configured for use in autonomous vehicles the SOC is designed and configured for compliance with the relevant functional safety standards of the deployment jurisdiction.

During operation, the media processor **1302** and vision processor **1304** can work in concert to accelerate computer vision operations. The media processor **1302** can enable low latency decode of multiple high-resolution (e.g., 4K, 8K) video streams. The decoded video streams can be written to a buffer in the on-chip memory **1305**. The vision processor

1304 can then parse the decoded video and perform preliminary processing operations on the frames of the decoded video in preparation of processing the frames using a trained image recognition model. For example, the vision processor **1304** can accelerate convolution operations for a CNN that is used to perform image recognition on the high-resolution video data, while back end model computations are performed by the GPGPU **1306**.

The multi-core processor **1308** can include control logic to assist with sequencing and synchronization of data transfers and shared memory operations performed by the media processor **1302** and the vision processor **1304**. The multi-core processor **1308** can also function as an application processor to execute software applications that can make use of the inferencing compute capability of the GPGPU **1306**. For example, at least a portion of the navigation and driving logic can be implemented in software executing on the multi-core processor **1308**. Such software can directly issue computational workloads to the GPGPU **1306** or the computational workloads can be issued to the multi-core processor **1308**, which can offload at least a portion of those operations to the GPGPU **1306**.

The GPGPU **1306** can include compute clusters such as a low power configuration of the processing clusters **706A-706H** within general-purpose graphics processing unit **700**. The compute clusters within the GPGPU **1306** can support instruction that are specifically optimized to perform inferencing computations on a trained neural network. For example, the GPGPU **1306** can support instructions to perform low precision computations such as 8-bit and 4-bit integer vector operations.

Additional System Overview

FIG. **14** is a block diagram of a processing system **1400**. The elements of FIG. **14** having the same or similar names as the elements of any other figure herein describe the same elements as in the other figures, can operate or function in a manner similar to that, can comprise the same components, and can be linked to other entities, as those described elsewhere herein, but are not limited to such. System **1400** may be used in a single processor desktop system, a multiprocessor workstation system, or a server system having a large number of processors **1402** or processor cores **1407**. The system **1400** may be a processing platform incorporated within a system-on-a-chip (SoC) integrated circuit for use in mobile, handheld, or embedded devices such as within Internet-of-things (IoT) devices with wired or wireless connectivity to a local or wide area network.

The system **1400** may be a processing system having components that correspond with those of FIG. **1**. For example, in different configurations, processor(s) **1402** or processor core(s) **1407** may correspond with processor(s) **102** of FIG. **1**. Graphics processor(s) **1408** may correspond with parallel processor(s) **112** of FIG. **1**. External graphics processor **1418** may be one of the add-in device(s) **120** of FIG. **1**.

The system **1400** can include, couple with, or be integrated within: a server-based gaming platform; a game console, including a game and media console; a mobile gaming console, a handheld game console, or an online game console. The system **1400** may be part of a mobile phone, smart phone, tablet computing device or mobile Internet-connected device such as a laptop with low internal storage capacity. Processing system **1400** can also include, couple with, or be integrated within: a wearable device, such as a smart watch wearable device; smart eyewear or clothing enhanced with augmented reality (AR) or virtual reality (VR) features to provide visual, audio or tactile outputs to

supplement real world visual, audio or tactile experiences or otherwise provide text, audio, graphics, video, holographic images or video, or tactile feedback; other augmented reality (AR) device; or other virtual reality (VR) device. The processing system **1400** may include or be part of a television or set top box device. The system **1400** can include, couple with, or be integrated within a self-driving vehicle such as a bus, tractor trailer, car, motor or electric power cycle, plane or glider (or any combination thereof). The self-driving vehicle may use system **1400** to process the environment sensed around the vehicle.

The one or more processors **1402** may include one or more processor cores **1407** to process instructions which, when executed, perform operations for system or user software. The least one of the one or more processor cores **1407** may be configured to process a specific instruction set **1409**. The instruction set **1409** may facilitate Complex Instruction Set Computing (CISC), Reduced Instruction Set Computing (RISC), or computing via a Very Long Instruction Word (VLIW). One or more processor cores **1407** may process a different instruction set **1409**, which may include instructions to facilitate the emulation of other instruction sets. Processor core **1407** may also include other processing devices, such as a Digital Signal Processor (DSP).

The processor **1402** may include cache memory **1404**. Depending on the architecture, the processor **1402** can have a single internal cache or multiple levels of internal cache. In some embodiments, the cache memory is shared among various components of the processor **1402**. In some embodiments, the processor **1402** also uses an external cache (e.g., a Level-3 (L3) cache or Last Level Cache (LLC)) (not shown), which may be shared among processor cores **1407** using known cache coherency techniques. A register file **1406** can be additionally included in processor **1402** and may include different types of registers for storing different types of data (e.g., integer registers, floating point registers, status registers, and an instruction pointer register). Some registers may be general-purpose registers, while other registers may be specific to the design of the processor **1402**.

The one or more processor(s) **1402** may be coupled with one or more interface bus(es) **1410** to transmit communication signals such as address, data, or control signals between processor **1402** and other components in the system **1400**. The interface bus **1410**, in one of these embodiments, can be a processor bus, such as a version of the Direct Media Interface (DMI) bus. However, processor busses are not limited to the DMI bus, and may include one or more Peripheral Component Interconnect buses (e.g., PCI, PCI express), memory busses, or other types of interface busses. For example, the processor(s) **1402** may include an integrated memory controller **1416** and a platform controller hub **1430**. The memory controller **1416** facilitates communication between a memory device and other components of the system **1400**, while the platform controller hub (PCH) **1430** provides connections to I/O devices via a local I/O bus.

The memory device **1420** can be a dynamic random-access memory (DRAM) device, a static random-access memory (SRAM) device, flash memory device, phase-change memory device, or some other memory device having suitable performance to serve as process memory. The memory device **1420** can, for example, operate as system memory for the system **1400**, to store data **1422** and instructions **1421** for use when the one or more processors **1402** executes an application or process. Memory controller **1416** also couples with an optional external graphics processor **1418**, which may communicate with the one or more graphics processors **1408** in processors **1402** to perform

graphics and media operations. In some embodiments, graphics, media, and or compute operations may be assisted by an accelerator **1412** which is a coprocessor that can be configured to perform a specialized set of graphics, media, or compute operations. For example, the accelerator **1412** may be a matrix multiplication accelerator used to optimize machine learning or compute operations. The accelerator **1412** can be a ray-tracing accelerator that can be used to perform ray-tracing operations in concert with the graphics processor **1408**. In one embodiment, an external accelerator **1419** may be used in place of or in concert with the accelerator **1412**.

A display device **1411** may be provided that can connect to the processor(s) **1402**. The display device **1411** can be one or more of an internal display device, as in a mobile electronic device or a laptop device or an external display device attached via a display interface (e.g., DisplayPort, etc.). The display device **1411** can be a head mounted display (HMD) such as a stereoscopic display device for use in virtual reality (VR) applications or augmented reality (AR) applications.

The platform controller hub **1430** may enable peripherals to connect to memory device **1420** and processor **1402** via a high-speed I/O bus. The I/O peripherals include, but are not limited to, an audio controller **1446**, a network controller **1434**, a firmware interface **1428**, a wireless transceiver **1426**, touch sensors **1425**, a data storage device **1424** (e.g., non-volatile memory, volatile memory, hard disk drive, flash memory, NAND, 3D NAND, 3D XPoint/Optane, etc.). The data storage device **1424** can connect via a storage interface (e.g., SATA) or via a peripheral bus, such as a Peripheral Component Interconnect bus (e.g., PCI, PCI express). The touch sensors **1425** can include touch screen sensors, pressure sensors, or fingerprint sensors. The wireless transceiver **1426** can be a Wi-Fi transceiver, a Bluetooth transceiver, or a mobile network transceiver such as a 3G, 4G, 5G, or Long-Term Evolution (LTE) transceiver. The firmware interface **1428** enables communication with system firmware, and can be, for example, a unified extensible firmware interface (UEFI). The network controller **1434** can enable a network connection to a wired network. In some embodiments, a high-performance network controller (not shown) couples with the interface bus **1410**. The audio controller **1446** may be a multi-channel high definition audio controller. In some of these embodiments the system **1400** includes an optional legacy I/O controller **1440** for coupling legacy (e.g., Personal System 2 (PS/2)) devices to the system. The platform controller hub **1430** can also connect to one or more Universal Serial Bus (USB) controllers **1442** connect input devices, such as keyboard and mouse **1443** combinations, a camera **1444**, or other USB input devices.

It can be appreciated that the system **1400** shown is example and not limiting, as other types of data processing systems that are differently configured may also be used. For example, an instance of the memory controller **1416** and platform controller hub **1430** may be integrated into a discrete external graphics processor, such as the external graphics processor **1418**. The platform controller hub **1430** and/or memory controller **1416** may be external to the one or more processor(s) **1402**. For example, the system **1400** can include an external memory controller **1416** and platform controller hub **1430**, which may be configured as a memory controller hub and peripheral controller hub within a system chipset that is in communication with the processor(s) **1402**.

For example, circuit boards (“sleds”) can be used on which components such as CPUs, memory, and other com-

ponents are placed are designed for increased thermal performance. Processing components such as the processors may be located on a top side of a sled while near memory, such as DIMMs, are located on a bottom side of the sled. As a result of the enhanced airflow provided by this design, the components may operate at higher frequencies and power levels than in typical systems, thereby increasing performance. Furthermore, the sleds are configured to blindly mate with power and data communication cables in a rack, thereby enhancing their ability to be quickly removed, upgraded, reinstalled, and/or replaced. Similarly, individual components located on the sleds, such as processors, accelerators, memory, and data storage drives, are configured to be easily upgraded due to their increased spacing from each other. In the illustrative embodiment, the components additionally include hardware attestation features to prove their authenticity.

A data center can utilize a single network architecture (“fabric”) that supports multiple other network architectures including Ethernet and Omni-Path. The sleds can be coupled to switches via optical fibers, which provide higher bandwidth and lower latency than typical twisted pair cabling (e.g., Category 5, Category 5e, Category 6, etc.). Due to the high bandwidth, low latency interconnections and network architecture, the data center may, in use, pool resources, such as memory, accelerators (e.g., GPUs, graphics accelerators, FPGAs, ASICs, neural network and/or artificial intelligence accelerators, etc.), and data storage drives that are physically disaggregated, and provide them to compute resources (e.g., processors) on an as needed basis, enabling the compute resources to access the pooled resources as if they were local.

A power supply or source can provide voltage and/or current to system **1400** or any component or system described herein. In one example, the power supply includes an AC to DC (alternating current to direct current) adapter to plug into a wall outlet. Such AC power can be renewable energy (e.g., solar power) power source. In one example, the power source includes a DC power source, such as an external AC to DC converter. A power source or power supply may also include wireless charging hardware to charge via proximity to a charging field. The power source can include an internal battery, alternating current supply, motion-based power supply, solar power supply, or fuel cell source.

FIG. **15A-15C** illustrate computing systems and graphics processors. The elements of FIG. **15A-15C** having the same or similar names as the elements of any other figure herein describe the same elements as in the other figures, can operate or function in a manner similar to that, can comprise the same components, and can be linked to other entities, as those described elsewhere herein, but are not limited to such.

FIG. **15A** is a block diagram of a processor **1500**, which may be a variant of one of the processors **1402** and may be used in place of one of those. Therefore, the discussion of any features in combination with the processor **1500** herein also discloses a corresponding combination with the processor(s) **1402**, but is not limited to such. The processor **1500** may have one or more processor cores **1502A-1502N**, an integrated memory controller **1514**, and an integrated graphics processor **1508**. Where an integrated graphics processor **1508** is excluded, the system that includes the processor can include a graphics processor device within a system chipset or coupled via a system bus. Processor **1500** can include additional cores up to and including additional core **1502N** represented by the dashed lined boxes. Each of processor cores **1502A-1502N** includes one or more internal

cache units **1504A-1504N**. In some embodiments each processor core **1502A-1502N** also has access to one or more shared cache units **1506**. The internal cache units **1504A-1504N** and shared cache units **1506** represent a cache memory hierarchy within the processor **1500**. The cache memory hierarchy may include at least one level of instruction and data cache within each processor core and one or more levels of shared mid-level cache, such as a Level 2 (L2), Level 3 (L3), Level 4 (L4), or other levels of cache, where the highest level of cache before external memory is classified as the LLC. In some embodiments, cache coherency logic maintains coherency between the various cache units **1506** and **1504A-1504N**.

The processor **1500** may also include a set of one or more bus controller units **1516** and a system agent core **1510**. The one or more bus controller units **1516** manage a set of peripheral buses, such as one or more PCI or PCI express busses. System agent core **1510** provides management functionality for the various processor components. The system agent core **1510** may include one or more integrated memory controllers **1514** to manage access to various external memory devices (not shown).

For example, one or more of the processor cores **1502A-1502N** may include support for simultaneous multi-threading. The system agent core **1510** includes components for coordinating and operating cores **1502A-1502N** during multi-threaded processing. System agent core **1510** may additionally include a power control unit (PCU), which includes logic and components to regulate the power state of processor cores **1502A-1502N** and graphics processor **1508**.

The processor **1500** may additionally include graphics processor **1508** to execute graphics processing operations. In some of these embodiments, the graphics processor **1508** couples with the set of shared cache units **1506**, and the system agent core **1510**, including the one or more integrated memory controllers **1514**. The system agent core **1510** may also include a display controller **1511** to drive graphics processor output to one or more coupled displays. The display controller **1511** may also be a separate module coupled with the graphics processor via at least one interconnect, or may be integrated within the graphics processor **1508**.

A ring-based interconnect unit **1512** may be used to couple the internal components of the processor **1500**. However, an alternative interconnect unit may be used, such as a point-to-point interconnect, a switched interconnect, or other techniques, including techniques well known in the art. In some of these embodiments with a ring-based interconnect **1512**, the graphics processor **1508** couples with the ring-based interconnect **1512** via an I/O link **1513**.

The example I/O link **1513** represents at least one of multiple varieties of I/O interconnects, including an on package I/O interconnect which facilitates communication between various processor components and a high-performance embedded memory module **1518**, such as an eDRAM module. Optionally, each of the processor cores **1502A-1502N** and graphics processor **1508** can use embedded memory modules **1518** as a shared Last Level Cache.

The processor cores **1502A-1502N** may, for example, be homogenous cores executing the same instruction set architecture. Alternatively, the processor cores **1502A-1502N** are heterogeneous in terms of instruction set architecture (ISA), where one or more of processor cores **1502A-1502N** execute a first instruction set, while at least one of the other cores executes a subset of the first instruction set or a different instruction set. The processor cores **1502A-1502N** may be heterogeneous in terms of microarchitecture, where one or

more cores having a relatively higher power consumption couple with one or more power cores having a lower power consumption. As another example, the processor cores **1502A-1502N** are heterogeneous in terms of computational capability. Additionally, processor **1500** can be implemented on one or more chips or as an SoC integrated circuit having the illustrated components, in addition to other components.

FIG. **15B** is a block diagram of hardware logic of a graphics processor core **1519**, according to some embodiments described herein. The graphics processor core **1519**, sometimes referred to as a core slice, can be one or multiple graphics cores within a modular graphics processor. The graphics processor core **1519** is example of one graphics core slice, and a graphics processor as described herein may include multiple graphics core slices based on target power and performance envelopes. Each graphics processor core **1519** can include a fixed function block **1530** coupled with multiple sub-cores **1521A-1521F**, also referred to as sub-slices, that include modular blocks of general-purpose and fixed function logic. In one configuration, a sub-core (sub-slice) of the multiple sub-cores **1521A-1521F** is an architectural equivalent to a graphics multiprocessor **234** of FIG. **2D**, graphics multiprocessor **325** of FIG. **3A**, and/or a multi-core group of the multi-core groups **365A-365N** of FIG. **3C**.

The fixed function block **1530** may include a geometry/fixed function pipeline **1531** that can be shared by all sub-cores in the graphics processor core **1519**, for example, in lower performance and/or lower power graphics processor implementations. The geometry/fixed function pipeline **1531** may include a 3D fixed function pipeline (e.g., 3D pipeline **1612** as in FIG. **16A** described below) a video front-end unit, a thread spawner and thread dispatcher, and a unified return buffer manager, which manages unified return buffers (e.g., unified return buffer **1718** in FIG. **17**, as described below).

The fixed function block **1530** may also include a graphics SoC interface **1532**, a graphics microcontroller **1533**, and a media pipeline **1534**. The graphics SoC interface **1532** provides an interface between the graphics processor core **1519** and other processor cores within a system on a chip integrated circuit. The graphics microcontroller **1533** is a programmable sub-processor that is configurable to manage various functions of the graphics processor core **1519**, including thread dispatch, scheduling, and pre-emption. The media pipeline **1534** (e.g., media pipeline **1616** of FIG. **16A** and FIG. **17**) includes logic to facilitate the decoding, encoding, pre-processing, and/or post-processing of multimedia data, including image and video data. The media pipeline **1534** implement media operations via requests to compute or sampling logic within the sub-cores **1521-1521F**.

The SoC interface **1532** may enable the graphics processor core **1519** to communicate with general-purpose application processor cores (e.g., CPUs) and/or other components within an SoC, including memory hierarchy elements such as a shared last level cache memory, the system RAM, and/or embedded on-chip or on-package DRAM. The SoC interface **1532** can also enable communication with fixed function devices within the SoC, such as camera imaging pipelines, and enables the use of and/or implements global memory atomics that may be shared between the graphics processor core **1519** and CPUs within the SoC. The SoC interface **1532** can also implement power management controls for the graphics processor core **1519** and enable an interface between a clock domain of the graphics processor core **1519** and other clock domains within the SoC. Option-

ally, the SoC interface **1532** enables receipt of command buffers from a command streamer and global thread dispatcher that are configured to provide commands and instructions to each of one or more graphics cores within a graphics processor. The commands and instructions can be dispatched to the media pipeline **1534**, when media operations are to be performed, or a geometry and fixed function pipeline (e.g., geometry and fixed function pipeline **1531**, geometry and fixed function pipeline **1537**) when graphics processing operations are to be performed.

The graphics microcontroller **1533** can be configured to perform various scheduling and management tasks for the graphics processor core **1519**. In one configuration the graphics microcontroller **1533** can, for example, perform graphics and/or compute workload scheduling on the various graphics parallel engines within execution unit (EU) arrays **1522A-1522F**, **1524A-1524F** within the sub-cores **1521A-1521F**. In this workload scheduling, host software executing on a CPU core of an SoC including the graphics processor core **1519** can submit workloads to one of multiple graphics processor doorbells, which invokes a scheduling operation on the appropriate graphics engine. Scheduling operations include determining which workload to run next, submitting a workload to a command streamer, preempting existing workloads running on an engine, monitoring progress of a workload, and notifying host software when a workload is complete. Optionally, the graphics microcontroller **1533** can also facilitate low-power or idle states for the graphics processor core **1519**, providing the graphics processor core **1519** with the ability to save and restore registers within the graphics processor core **1519** across low-power state transitions independently from the operating system and/or graphics driver software on the system.

The graphics processor core **1519** may have more than or fewer than the illustrated sub-cores **1521A-1521F**, up to N modular sub-cores. For each set of N sub-cores, the graphics processor core **1519** can also include shared function logic **1535**, shared and/or cache memory **1536**, a geometry/fixed function pipeline **1537**, as well as additional fixed function logic **1538** to accelerate various graphics and compute processing operations. The shared function logic **1535** can include logic units associated with the shared function logic **1720** of FIG. 17 (e.g., sampler, math, and/or inter-thread communication logic) that can be shared by each N sub-cores within the graphics processor core **1519**. The shared and/or cache memory **1536** can be a last-level cache for the set of N sub-cores **1521A-1521F** within the graphics processor core **1519**, and can also serve as shared memory that is accessible by multiple sub-cores. The geometry/fixed function pipeline **1537** can be included instead of the geometry/fixed function pipeline **1531** within the fixed function block **1530** and can include the same or similar logic units.

The graphics processor core **1519** may include additional fixed function logic **1538** that can include various fixed function acceleration logic for use by the graphics processor core **1519**. Optionally, the additional fixed function logic **1538** includes an additional geometry pipeline for use in position only shading. In position-only shading, two geometry pipelines exist, the full geometry pipeline within the geometry/fixed function pipeline **1538**, **1531**, and a cull pipeline, which is an additional geometry pipeline which may be included within the additional fixed function logic **1538**. For example, the cull pipeline may be a trimmed down version of the full geometry pipeline. The full pipeline and the cull pipeline can execute different instances of the same

application, each instance having a separate context. Position only shading can hide long cull runs of discarded triangles, enabling shading to be completed earlier in some instances. For example, the cull pipeline logic within the additional fixed function logic **1538** can execute position shaders in parallel with the main application and generally generates critical results faster than the full pipeline, as the cull pipeline fetches and shades only the position attribute of the vertices, without performing rasterization and rendering of the pixels to the frame buffer. The cull pipeline can use the generated critical results to compute visibility information for all the triangles without regard to whether those triangles are culled. The full pipeline (which in this instance may be referred to as a replay pipeline) can consume the visibility information to skip the culled triangles to shade only the visible triangles that are finally passed to the rasterization phase.

Optionally, the additional fixed function logic **1538** can also include machine-learning acceleration logic, such as fixed function matrix multiplication logic, for implementations including optimizations for machine learning training or inferring.

Within each graphics sub-core **1521A-1521F** a set of execution resources is included that may be used to perform graphics, media, and compute operations in response to requests by graphics pipeline, media pipeline, or shader programs. The graphics sub-cores **1521A-1521F** include multiple EU arrays **1522A-1522F**, **1524A-1524F**, thread dispatch and inter-thread communication (TD/IC) logic **1523A-1523F**, a 3D (e.g., texture) sampler **1525A-1525F**, a media sampler **1526A-1526F**, a shader processor **1527A-1527F**, and shared local memory (SLM) **1528A-1528F**. The EU arrays **1522A-1522F**, **1524A-1524F** each include multiple execution units, which are general-purpose graphics processing units capable of performing floating-point and integer/fixed-point logic operations in service of a graphics, media, or compute operation, including graphics, media, or compute shader programs. The TD/IC logic **1523A-1523F** performs local thread dispatch and thread control operations for the execution units within a sub-core and facilitate communication between threads executing on the execution units of the sub-core. The 3D sampler **1525A-1525F** can read texture or other 3D graphics related data into memory. The 3D sampler can read texture data differently based on a configured sample state and the texture format associated with a given texture. The media sampler **1526A-1526F** can perform similar read operations based on the type and format associated with media data. For example, each graphics sub-core **1521A-1521F** can alternately include a unified 3D and media sampler. Threads executing on the execution units within each of the sub-cores **1521A-1521F** can make use of shared local memory **1528A-1528F** within each sub-core, to enable threads executing within a thread group to execute using a common pool of on-chip memory.

FIG. 15C is a block diagram of general-purpose graphics processing unit (GPGPU) **1570** that can be configured as a graphics processor, e.g. the graphics processor **1508**, and/or compute accelerator, according to embodiments described herein. The GPGPU **1570** can interconnect with host processors (e.g., one or more CPU(s) **1546**) and memory **1571**, **1572** via one or more system and/or memory busses. Memory **1571** may be system memory that can be shared with the one or more CPU(s) **1546**, while memory **1572** is device memory that is dedicated to the GPGPU **1570**. For example, components within the GPGPU **1570** and memory **1572** may be mapped into memory addresses that are accessible to the one or more CPU(s) **1546**. Access to

memory **1571** and **1572** may be facilitated via a memory controller **1568**. The memory controller **1568** may include an internal direct memory access (DMA) controller **1569** or can include logic to perform operations that would otherwise be performed by a DMA controller.

The GPGPU **1570** includes multiple cache memories, including an L2 cache **1553**, L1 cache **1554**, an instruction cache **1555**, and shared memory **1556**, at least a portion of which may also be partitioned as a cache memory. The GPGPU **1570** also includes multiple compute units **1560A-1560N**. Each compute unit **1560A-1560N** includes a set of vector registers **1561**, scalar registers **1562**, vector logic units **1563**, and scalar logic units **1564**. The compute units **1560A-1560N** can also include local shared memory **1565** and a program counter **1566**. The compute units **1560A-1560N** can couple with a constant cache **1567**, which can be used to store constant data, which is data that cannot change during the run of kernel or shader program that executes on the GPGPU **1570**. The constant cache **1567** may be a scalar data cache and cached data can be fetched directly into the scalar registers **1562**.

During operation, the one or more CPU(s) **1546** can write commands into registers or memory in the GPGPU **1570** that has been mapped into an accessible address space. The command processors **1557** can read the commands from registers or memory and determine how those commands can be processed within the GPGPU **1570**. A thread dispatcher **1558** can then be used to dispatch threads to the compute units **1560A-1560N** to perform those commands. Each compute unit **1560A-1560N** can execute threads independently of the other compute units. Additionally, each compute unit **1560A-1560N** can be independently configured for conditional computation and can conditionally output the results of computation to memory. The command processors **1557** can interrupt the one or more CPU(s) **1546** when the submitted commands are complete.

FIG. **16A-16C** illustrate block diagrams of additional graphics processor and compute accelerator architectures provided by embodiments described herein, e.g. in accordance with FIG. **15A-15C**. The elements of FIG. **16A-16C** having the same or similar names as the elements of any other figure herein describe the same elements as in the other figures, can operate or function in a manner similar to that, can comprise the same components, and can be linked to other entities, as those described elsewhere herein, but are not limited to such.

FIG. **16A** is a block diagram of a graphics processor **1600**, which may be a discrete graphics processing unit, or may be a graphics processor integrated with a plurality of processing cores, or other semiconductor devices such as, but not limited to, memory devices or network interfaces. The graphics processor **1600** may be a variant of the graphics processor **1508** and may be used in place of the graphics processor **1508**. Therefore, the discussion of any features in combination with the graphics processor **1508** herein also discloses a corresponding combination with the graphics processor **1600**, but is not limited to such. The graphics processor may communicate via a memory mapped I/O interface to registers on the graphics processor and with commands placed into the processor memory. Graphics processor **1600** may include a memory interface **1614** to access memory. Memory interface **1614** can be an interface to local memory, one or more internal caches, one or more shared external caches, and/or to system memory.

Optionally, graphics processor **1600** also includes a display controller **1602** to drive display output data to a display device **1618**. Display controller **1602** includes hardware for

one or more overlay planes for the display and composition of multiple layers of video or user interface elements. The display device **1618** can be an internal or external display device. In one embodiment the display device **1618** is a head mounted display device, such as a virtual reality (VR) display device or an augmented reality (AR) display device. Graphics processor **1600** may include a video codec engine **1606** to encode, decode, or transcode media to, from, or between one or more media encoding formats, including, but not limited to Moving Picture Experts Group (MPEG) formats such as MPEG-2, Advanced Video Coding (AVC) formats such as H.264/MPEG-4 AVC, H.265/HEVC, Alliance for Open Media (AOMedia) VP8, VP9, as well as the Society of Motion Picture & Television Engineers (SMPTE) 421M/VC-1, and Joint Photographic Experts Group (JPEG) formats such as JPEG, and Motion JPEG (MJPEG) formats.

Graphics processor **1600** may include a block image transfer (BLIT) engine **1603** to perform two-dimensional (2D) rasterizer operations including, for example, bit-boundary block transfers. However, alternatively, 2D graphics operations may be performed using one or more components of graphics processing engine (GPE) **1610**. In some embodiments, GPE **1610** is a compute engine for performing graphics operations, including three-dimensional (3D) graphics operations and media operations.

GPE **1610** may include a 3D pipeline **1612** for performing 3D operations, such as rendering three-dimensional images and scenes using processing functions that act upon 3D primitive shapes (e.g., rectangle, triangle, etc.). The 3D pipeline **1612** includes programmable and fixed function elements that perform various tasks within the element and/or spawn execution threads to a 3D/Media subsystem **1615**. While 3D pipeline **1612** can be used to perform media operations, an embodiment of GPE **1610** also includes a media pipeline **1616** that is specifically used to perform media operations, such as video post-processing and image enhancement.

Media pipeline **1616** may include fixed function or programmable logic units to perform one or more specialized media operations, such as video decode acceleration, video de-interlacing, and video encode acceleration in place of, or on behalf of video codec engine **1606**. Media pipeline **1616** may additionally include a thread spawning unit to spawn threads for execution on 3D/Media subsystem **1615**. The spawned threads perform computations for the media operations on one or more graphics execution units included in 3D/Media subsystem **1615**.

The 3D/Media subsystem **1615** may include logic for executing threads spawned by 3D pipeline **1612** and media pipeline **1616**. The pipelines may send thread execution requests to 3D/Media subsystem **1615**, which includes thread dispatch logic for arbitrating and dispatching the various requests to available thread execution resources. The execution resources include an array of graphics execution units to process the 3D and media threads. The 3D/Media subsystem **1615** may include one or more internal caches for thread instructions and data. Additionally, the 3D/Media subsystem **1615** may also include shared memory, including registers and addressable memory, to share data between threads and to store output data.

FIG. **16B** illustrates a graphics processor **1620**, being a variant of the graphics processor **1600** and may be used in place of the graphics processor **1600** and vice versa. Therefore, the discussion of any features in combination with the graphics processor **1600** herein also discloses a corresponding combination with the graphics processor **1620**, but is not limited to such. The graphics processor **1620** has a tiled

51

architecture, according to embodiments described herein. The graphics processor **1620** may include a graphics processing engine cluster **1622** having multiple instances of the graphics processing engine **1610** of FIG. **16A** within a graphics engine tile **1610A-1610D**. Each graphics engine tile **1610A-1610D** can be interconnected via a set of tile interconnects **1623A-1623F**. Each graphics engine tile **1610A-1610D** can also be connected to a memory module or memory device **1626A-1626D** via memory interconnects **1625A-1625D**. The memory devices **1626A-1626D** can use any graphics memory technology. For example, the memory devices **1626A-1626D** may be graphics double data rate (GDDR) memory. The memory devices **1626A-1626D** may be high-bandwidth memory (HBM) modules that can be on-die with their respective graphics engine tile **1610A-1610D**. The memory devices **1626A-1626D** may be stacked memory devices that can be stacked on top of their respective graphics engine tile **1610A-1610D**. Each graphics engine tile **1610A-1610D** and associated memory **1626A-1626D** may reside on separate chiplets, which are bonded to a base die or base substrate, as described in further detail in FIG. **24B-24D**.

The graphics processor **1620** may be configured with a non-uniform memory access (NUMA) system in which memory devices **1626A-1626D** are coupled with associated graphics engine tiles **1610A-1610D**. A given memory device may be accessed by graphics engine tiles other than the tile to which it is directly connected. However, access latency to the memory devices **1626A-1626D** may be lowest when accessing a local tile. In one embodiment, a cache coherent NUMA (ccNUMA) system is enabled that uses the tile interconnects **1623A-1623F** to enable communication between cache controllers within the graphics engine tiles **1610A-1610D** to keep a consistent memory image when more than one cache stores the same memory location.

The graphics processing engine cluster **1622** can connect with an on-chip or on-package fabric interconnect **1624**. In one embodiment the fabric interconnect **1624** includes a network processor, network on a chip (NoC), or another switching processor to enable the fabric interconnect **1624** to act as a packet switched fabric interconnect that switches data packets between components of the graphics processor **1620**. The fabric interconnect **1624** can enable communication between graphics engine tiles **1610A-1610D** and components such as the video codec engine **1606** and one or more copy engines **1604**. The copy engines **1604** can be used to move data out of, into, and between the memory devices **1626A-1626D** and memory that is external to the graphics processor **1620** (e.g., system memory). The fabric interconnect **1624** can also be used to interconnect the graphics engine tiles **1610A-1610D**. The graphics processor **1620** may optionally include a display controller **1602** to enable a connection with an external display device **1618**. The graphics processor may also be configured as a graphics or compute accelerator. In the accelerator configuration, the display controller **1602** and display device **1618** may be omitted.

The graphics processor **1620** can connect to a host system via a host interface **1628**. The host interface **1628** can enable communication between the graphics processor **1620**, system memory, and/or other system components. The host interface **1628** can be, for example, a PCI express bus or another type of host system interface. For example, the host interface **1628** may be an NVLink or NVSwitch interface. The host interface **1628** and fabric interconnect **1624** can cooperate to enable multiple instances of the graphics processor **1620** to act as single logical device. Cooperation

52

between the host interface **1628** and fabric interconnect **1624** can also enable the individual graphics engine tiles **1610A-1610D** to be presented to the host system as distinct logical graphics devices.

FIG. **16C** illustrates a compute accelerator **1630**, according to embodiments described herein. The compute accelerator **1630** can include architectural similarities with the graphics processor **1620** of FIG. **16B** and is optimized for compute acceleration. A compute engine cluster **1632** can include a set of compute engine tiles **1640A-1640D** that include execution logic that is optimized for parallel or vector-based general-purpose compute operations. The compute engine tiles **1640A-1640D** may not include fixed function graphics processing logic, although in some embodiments one or more of the compute engine tiles **1640A-1640D** can include logic to perform media acceleration. The compute engine tiles **1640A-1640D** can connect to memory **1626A-1626D** via memory interconnects **1625A-1625D**. The memory **1626A-1626D** and memory interconnects **1625A-1625D** may be similar technology as in graphics processor **1620**, or can be different. The graphics compute engine tiles **1640A-1640D** can also be interconnected via a set of tile interconnects **1623A-1623F** and may be connected with and/or interconnected by a fabric interconnect **1624**. In one embodiment the compute accelerator **1630** includes a large L3 cache **1636** that can be configured as a device-wide cache. The compute accelerator **1630** can also connect to a host processor and memory via a host interface **1628** in a similar manner as the graphics processor **1620** of FIG. **16B**.

The compute accelerator **1630** can also include an integrated network interface **1642**. In one embodiment the integrated network interface **1642** includes a network processor and controller logic that enables the compute engine cluster **1632** to communicate over a physical layer interconnect **1644** without requiring data to traverse memory of a host system. In one embodiment, one of the compute engine tiles **1640A-1640D** is replaced by network processor logic and data to be transmitted or received via the physical layer interconnect **1644** may be transmitted directly to or from memory **1626A-1626D**. Multiple instances of the compute accelerator **1630** may be joined via the physical layer interconnect **1644** into a single logical device. Alternatively, the various compute engine tiles **1640A-1640D** may be presented as distinct network accessible compute accelerator devices.

Graphics Processing Engine

FIG. **17** is a block diagram of a graphics processing engine **1710** of a graphics processor in accordance with some embodiments. The graphics processing engine (GPE) **1710** may be a version of the GPE **1610** shown in FIG. **16A**, and may also represent a graphics engine tile **1610A-1610D** of FIG. **16B**. The elements of FIG. **17** having the same or similar names as the elements of any other figure herein describe the same elements as in the other figures, can operate or function in a manner similar to that, can comprise the same components, and can be linked to other entities, as those described elsewhere herein, but are not limited to such. For example, the 3D pipeline **1612** and media pipeline **1616** of FIG. **16A** are also illustrated in FIG. **17**. The media pipeline **1616** is optional in some embodiments of the GPE **1710** and may not be explicitly included within the GPE **1710**. For example and in at least one embodiment, a separate media and/or image processor is coupled to the GPE **1710**.

GPE **1710** may couple with or include a command streamer **1703**, which provides a command stream to the 3D pipeline **1612** and/or media pipelines **1616**. Alternatively or

additionally, the command streamer **1703** may be directly coupled to a unified return buffer **1718**. The unified return buffer **1718** may be communicatively coupled to a graphics core array **1714**. Optionally, the command streamer **1703** is coupled with memory, which can be system memory, or one or more of internal cache memory and shared cache memory. The command streamer **1703** may receive commands from the memory and sends the commands to 3D pipeline **1612** and/or media pipeline **1616**. The commands are directives fetched from a ring buffer, which stores commands for the 3D pipeline **1612** and media pipeline **1616**. The ring buffer can additionally include batch command buffers storing batches of multiple commands. The commands for the 3D pipeline **1612** can also include references to data stored in memory, such as but not limited to vertex and geometry data for the 3D pipeline **1612** and/or image data and memory objects for the media pipeline **1616**. The 3D pipeline **1612** and media pipeline **1616** process the commands and data by performing operations via logic within the respective pipelines or by dispatching one or more execution threads to the graphics core array **1714**. The graphics core array **1714** may include one or more blocks of graphics cores (e.g., graphics core(s) **1715A**, graphics core(s) **1715B**), each block including one or more graphics cores. Each graphics core includes a set of graphics execution resources that includes general-purpose and graphics specific execution logic to perform graphics and compute operations, as well as fixed function texture processing and/or machine learning and artificial intelligence acceleration logic.

In various embodiments the 3D pipeline **1612** can include fixed function and programmable logic to process one or more shader programs, such as vertex shaders, geometry shaders, pixel shaders, fragment shaders, compute shaders, or other shader programs, by processing the instructions and dispatching execution threads to the graphics core array **1714**. The graphics core array **1714** provides a unified block of execution resources for use in processing these shader programs. Multi-purpose execution logic (e.g., execution units) within the graphics core(s) **1715A-1715B** of the graphics core array **1714** includes support for various 3D API shader languages and can execute multiple simultaneous execution threads associated with multiple shaders.

The graphics core array **1714** may include execution logic to perform media functions, such as video and/or image processing. The execution units may include general-purpose logic that is programmable to perform parallel general-purpose computational operations, in addition to graphics processing operations. The general-purpose logic can perform processing operations in parallel or in conjunction with general-purpose logic within the processor core(s) **1407** of FIG. **14** or core **1502A-1502N** as in FIG. **15A**.

Output data generated by threads executing on the graphics core array **1714** can output data to memory in a unified return buffer (URB) **1718**. The URB **1718** can store data for multiple threads. The URB **1718** may be used to send data between different threads executing on the graphics core array **1714**. The URB **1718** may additionally be used for synchronization between threads on the graphics core array **1714** and fixed function logic within the shared function logic **1720**.

Optionally, the graphics core array **1714** may be scalable, such that the array includes a variable number of graphics cores, each having a variable number of execution units based on the target power and performance level of GPE

1710. The execution resources may be dynamically scalable, such that execution resources may be enabled or disabled as needed.

The graphics core array **1714** couples with shared function logic **1720** that includes multiple resources that are shared between the graphics cores in the graphics core array. The shared functions within the shared function logic **1720** are hardware logic units that provide specialized supplemental functionality to the graphics core array **1714**. In various embodiments, shared function logic **1720** includes but is not limited to sampler **1721**, math **1722**, and inter-thread communication (ITC) **1723** logic. Additionally, one or more cache(s) **1725** within the shared function logic **1720** may be implemented.

A shared function is implemented at least in a case where the demand for a given specialized function is insufficient for inclusion within the graphics core array **1714**. Instead a single instantiation of that specialized function is implemented as a stand-alone entity in the shared function logic **1720** and shared among the execution resources within the graphics core array **1714**. The precise set of functions that are shared between the graphics core array **1714** and included within the graphics core array **1714** varies across embodiments. Specific shared functions within the shared function logic **1720** that are used extensively by the graphics core array **1714** may be included within shared function logic **1716** within the graphics core array **1714**. Optionally, the shared function logic **1716** within the graphics core array **1714** can include some or all logic within the shared function logic **1720**. All logic elements within the shared function logic **1720** may be duplicated within the shared function logic **1716** of the graphics core array **1714**. Alternatively, the shared function logic **1720** is excluded in favor of the shared function logic **1716** within the graphics core array **1714**.

Execution Units

FIG. **18A-18B** illustrate thread execution logic **1800** including an array of processing elements employed in a graphics processor core according to embodiments described herein. The elements of FIG. **18A-18B** having the same or similar names as the elements of any other figure herein describe the same elements as in the other figures, can operate or function in a manner similar to that, can comprise the same components, and can be linked to other entities, as those described elsewhere herein, but are not limited to such. FIG. **18A-18B** illustrates an overview of thread execution logic **1800**, which may be representative of hardware logic illustrated with each sub-core **1521A-1521F** of FIG. **15B**. FIG. **18A** is representative of an execution unit within a general-purpose graphics processor, while FIG. **18B** is representative of an execution unit that may be used within a compute accelerator.

As illustrated in FIG. **18A**, thread execution logic **1800** may include a shader processor **1802**, a thread dispatcher **1804**, instruction cache **1806**, a scalable execution unit array including a plurality of graphics execution units **1808A-1808N**, a sampler **1810**, shared local memory **1811**, a data cache **1812**, and a data port **1814**. Optionally, the scalable execution unit array can dynamically scale by enabling or disabling one or more execution units (e.g., any of graphics execution units **1808A**, **1808B**, **1808C**, **1808D**, through **1808N-1** and **1808N**) based on the computational requirements of a workload. The included components may be interconnected via an interconnect fabric that links to each of the components. Thread execution logic **1800** may include one or more connections to memory, such as system memory or cache memory, through one or more of instruction cache **1806**, data port **1814**, sampler **1810**, and graphics

execution units **1808A-1808N**. Each execution unit (e.g. **1808A**) may be a stand-alone programmable general-purpose computational unit that is capable of executing multiple simultaneous hardware threads while processing multiple data elements in parallel for each thread. In various embodiments, the array of execution units **1808A-1808N** is scalable to include any number individual execution units.

In some embodiments the graphics execution units **1808A-1808N** may be primarily used to execute shader programs. A shader processor **1802** can process the various shader programs and dispatch execution threads associated with the shader programs via a thread dispatcher **1804**. The thread dispatcher may include logic to arbitrate thread initiation requests from the graphics and media pipelines and instantiate the requested threads on one or more execution units in the graphics execution units **1808A-1808N**. For example, a geometry pipeline can dispatch vertex, tessellation, or geometry shaders to the thread execution logic for processing. Optionally, the thread dispatcher **1804** can also process runtime thread spawning requests from the executing shader programs.

In some embodiments, the graphics execution units **1808A-1808N** may support an instruction set that includes native support for many standard 3D graphics shader instructions, such that shader programs from graphics libraries (e.g., Direct 3D and OpenGL) are executed with a minimal translation. The execution units support vertex and geometry processing (e.g., vertex programs, geometry programs, vertex shaders), pixel processing (e.g., pixel shaders, fragment shaders) and general-purpose processing (e.g., compute and media shaders). Each of the graphics execution units **1808A-1808N** is capable of multi-issue single instruction multiple data (SIMD) execution and multi-threaded operation enables an efficient execution environment in the face of higher latency memory accesses. Each hardware thread within each execution unit has a dedicated high-bandwidth register file and associated independent thread-state. Execution is multi-issue per clock to pipelines capable of integer, single and double precision floating point operations, SIMD branch capability, logical operations, transcendental operations, and other miscellaneous operations. While waiting for data from memory or one of the shared functions, dependency logic within the execution units **1808A-1808N** causes a waiting thread to sleep until the requested data has been returned. While the waiting thread is sleeping, hardware resources may be devoted to processing other threads. For example, during a delay associated with a vertex shader operation, an execution unit can perform operations for a pixel shader, fragment shader, or another type of shader program, including a different vertex shader, such as vertex shader **2107** illustrated in FIG. **21**. Various embodiments can apply to use execution by use of Single Instruction Multiple Thread (SIMT) as an alternate to use of SIMD or in addition to use of SIMD. Reference to a SIMD core or operation can apply also to SIMT or apply to SIMD in combination with SIMT.

Each execution unit in graphics execution units **1808A-1808N** operates on arrays of data elements. The number of data elements is the "execution size," or the number of channels for the instruction. An execution channel is a logical unit of execution for data element access, masking, and flow control within instructions. The number of channels may be independent of the number of physical Arithmetic Logic Units (ALUs), Floating-Point Units (FPUs), or other logic units (e.g., tensor cores, ray tracing cores, etc.)

for a particular graphics processor. Additionally, the graphics execution units **1808A-1808N** may support integer and floating-point data types.

The execution unit instruction set includes SIMD instructions. The various data elements can be stored as a packed data type in a register and the execution unit can process the various elements based on the data size of the elements. For example, when operating on a 256-bit wide vector, the 256 bits of the vector are stored in a register and the execution unit operates on the vector as four separate 64-bit packed data elements (Quad-Word (QW) size data elements), eight separate 32-bit packed data elements (Double Word (DW) size data elements), sixteen separate 16-bit packed data elements (Word (W) size data elements), or thirty-two separate 8-bit data elements (byte (B) size data elements). However, different vector widths and register sizes are possible.

Optionally, one or more execution units can be combined into a fused graphics execution unit **1809A-1809N** having thread control logic (**1807A-1807N**) that is common to the fused EUs. Multiple EUs can be fused into an EU group. Each EU in the fused EU group can be configured to execute a separate SIMD hardware thread. The number of EUs in a fused EU group can vary according to embodiments. Additionally, various SIMD widths can be performed per-EU, including but not limited to SIMD8, SIMD16, and SIMD32. Each fused graphics execution unit **1809A-1809N** includes at least two execution units. For example, fused execution unit **1809A** includes a first EU **1808A**, second EU **1808B**, and thread control logic **1807A** that is common to the first EU **1808A** and the second EU **1808B**. The thread control logic **1807A** controls threads executed on the fused graphics execution unit **1809A**, allowing each EU within the fused execution units **1809A-1809N** to execute using a common instruction pointer register.

One or more internal instruction caches (e.g., **1806**) are included in the thread execution logic **1800** to cache thread instructions for the execution units. One or more data caches (e.g., **1812**) may be included in the thread execution logic **1800** to cache thread data during thread execution. Threads executing on the execution logic **1800** can also store explicitly managed data in the shared local memory **1811**. A sampler **1810** may be included to provide texture sampling for 3D operations and media sampling for media operations. Sampler **1810** may include specialized texture or media sampling functionality to process texture or media data during the sampling process before providing the sampled data to an execution unit.

During execution, the graphics and media pipelines send thread initiation requests to thread execution logic **1800** via thread spawning and dispatch logic. Once a group of geometric objects has been processed and rasterized into pixel data, pixel processor logic (e.g., pixel shader logic, fragment shader logic, etc.) within the shader processor **1802** is invoked to further compute output information and cause results to be written to output surfaces (e.g., color buffers, depth buffers, stencil buffers, etc.). A pixel shader or fragment shader may calculate the values of the various vertex attributes that are to be interpolated across the rasterized object. The pixel processor logic within the shader processor **1802** may then execute an application programming interface (API)-supplied pixel or fragment shader program. To execute the shader program, the shader processor **1802** dispatches threads to an execution unit (e.g., **1808A**) via thread dispatcher **1804**. Shader processor **1802** may use texture sampling logic in the sampler **1810** to access texture data in texture maps stored in memory. Arithmetic opera-

tions on the texture data and the input geometry data compute pixel color data for each geometric fragment, or discards one or more pixels from further processing.

In addition, the data port **1814** may provide a memory access mechanism for the thread execution logic **1800** to output processed data to memory for further processing on a graphics processor output pipeline. The data port **1814** may include or couple to one or more cache memories (e.g., data cache **1812**) to cache data for memory access via the data port **1814**.

Optionally, the execution logic **1800** can also include a ray tracer **1805** that can provide ray tracing acceleration functionality. The ray tracer **1805** can support a ray tracing instruction set that includes instructions/functions for ray generation. The ray tracing instruction set can be similar to or different from the ray-tracing instruction set supported by the ray tracing cores **372** in FIG. 3C.

FIG. 18B illustrates example internal details of an execution unit **1808**. A graphics execution unit **1808** can include an instruction fetch unit **1837**, a general register file (GRF) array **1824**, an architectural register file array (ARF) **1826**, a thread arbiter **1822**, a send unit **1830**, a branch unit **1832**, a set of SIMD floating point units (FPU(s)) **1834**, and optionally a set of dedicated integer SIMD ALUs **1835**. The GRF **1824** and ARF **1826** includes the set of general register files and architecture register files associated with each simultaneous hardware thread that may be active in the graphics execution unit **1808**. Per thread architectural state may be maintained in the ARF **1826**, while data used during thread execution is stored in the GRF **1824**. The execution state of each thread, including the instruction pointers for each thread, can be held in thread-specific registers in the ARF **1826**.

The graphics execution unit **1808** may have an architecture that is a combination of Simultaneous Multi-Threading (SMT) and fine-grained Interleaved Multi-Threading (IMT). The architecture may have a modular configuration that can be fine-tuned at design time based on a target number of simultaneous threads and number of registers per execution unit, where execution unit resources are divided across logic used to execute multiple simultaneous threads. The number of logical threads that may be executed by the graphics execution unit **1808** is not limited to the number of hardware threads, and multiple logical threads can be assigned to each hardware thread.

Optionally, the graphics execution unit **1808** can co-issue multiple instructions, which may each be different instructions. The thread arbiter **1822** of the graphics execution unit **1808** can dispatch the instructions to one of the send unit **1830**, branch unit **1832**, or SIMD FPU(s) **1834** for execution. Each execution thread can access **128** general-purpose registers within the GRF **1824**, where each register can store 32 bytes, accessible as a SIMD 8-element vector of 32-bit data elements. Each execution unit thread may have access to 4 Kbytes within the GRF **1824**, although embodiments are not so limited, and greater or fewer register resources may be provided in other embodiments. The graphics execution unit **1808** may be partitioned into seven hardware threads that can independently perform computational operations, although the number of threads per execution unit can also vary according to embodiments, for example, up to 16 hardware threads may be supported. In an example embodiment, in which seven threads may access 4 Kbytes, the GRF **1824** can store a total of 28 Kbytes. In another example embodiment, where 16 threads may access 4 Kbytes, the GRF **1824** can store a total of 64 Kbytes. The number of threads per execution unit are, however, not limited to those

examples and may be more or less than the given numbers. Flexible addressing modes can permit registers to be addressed together to build effectively wider registers or to represent strided rectangular block data structures.

Additionally or alternatively, memory operations, sampler operations, and other longer-latency system communications may be dispatched via “send” instructions that are executed by the message passing send unit **1830**. Branch instructions may be dispatched to a dedicated branch unit **1832** to facilitate SIMD divergence and eventual convergence.

The graphics execution unit **1808** may include one or more SIMD floating point units (FPU(s)) **1834** to perform floating-point operations. The FPU(s) **1834** may also support integer computation. In some instances, the FPU(s) **1834** can SIMD execute up to M number of 32-bit floating-point (or integer) operations, or SIMD execute up to 2M 16-bit integer or 16-bit floating-point operations. Optionally, at least one of the FPU(s) provides extended math capability to support high-throughput transcendental math functions and double precision 64-bit floating-point. A set of 8-bit integer SIMD ALUs **1835** may also be present, and may be specifically optimized to perform operations associated with machine learning computations.

Optionally, arrays of multiple instances of the graphics execution unit **1808** can be instantiated in a graphics sub-core grouping (e.g., a sub-slice). For scalability, product architects can choose the exact number of execution units per sub-core grouping. The execution unit **1808** may execute instructions across a plurality of execution channels. In addition, each thread executed on the graphics execution unit **1808** may be executed on a different channel.

FIG. 19 illustrates a further example execution unit **1900**. The elements of FIG. 19 having the same or similar names as the elements of any other figure herein describe the same elements as in the other figures, can operate or function in a manner similar to that, can comprise the same components, and can be linked to other entities, as those described elsewhere herein, but are not limited to such. The execution unit **1900** may be a compute-optimized execution unit for use in, for example, a compute engine tile **1640A-1640D** as in FIG. 16C, but is not limited as such. The execution unit **1900** may also be used in a graphics engine tile **1610A-1610D** as in FIG. 16B. The execution unit **1900** may include a thread control unit **1901**, a thread state unit **1902**, an instruction fetch/prefetch unit **1903**, and an instruction decode unit **1904**. The execution unit **1900** may additionally include a register file **1906** that stores registers that can be assigned to hardware threads within the execution unit. The execution unit **1900** may additionally include a send unit **1907** and a branch unit **1908**. The send unit **1907** and branch unit **1908** may operate similarly as the send unit **1830** and a branch unit **1832** of the graphics execution unit **1808** of FIG. 18B.

The execution unit **1900** can also include a compute unit **1910** that includes multiple different types of functional units. The compute unit **1910** may also include an ALU **1911**, a systolic array **1912**, and a math unit **1913**. The ALU **1911** includes an array of arithmetic logic units. The ALU **1911** can be configured to perform 64-bit, 32-bit, and 16-bit integer and floating-point operations across multiple processing lanes and data channels and for multiple hardware and/or software threads. The ALU **1911** can perform integer and floating-point operations simultaneously (e.g., within the same clock cycle).

The systolic array **1912** includes a W wide and D deep network of data processing units that can be used to perform

vector or other data-parallel operations in a systolic manner. The systolic array **1912** can be configured to perform various matrix operations, including as dot product, outer product, and general matrix-matrix multiplication (GEMM) operations. The systolic array **1912** may support 16-bit floating point operations, as well as 8-bit, 4-bit, 2-bit, and binary integer operations. The systolic array **1912** may be configured to accelerate machine learning operations. The systolic array **1912** can be configured with support for bfloat16, (brain floating point) 16-bit floating point format or a tensor float 32-bit floating point format (TF32) that have different numbers of mantissa and exponent bits relative to Institute of Electrical and Electronics Engineers (IEEE) 754 formats. FP64 formats can also be supported.

In one embodiment, the systolic array **1912** includes hardware to accelerate sparse-matrix operations. Multiplication operations for sparse regions of input data can be bypassed without sacrificing throughput. Block sparsity within input matrices can be detected and operations having known output values can be bypassed. In one embodiment, the systolic array **1912** includes hardware to enable operations on sparse data having a compressed representation. A compressed representation of a sparse matrix stores non-zero values and metadata that defines the position of the non-zero values within the matrix. Example compressed representations include but are not limited to compressed tensor representations such as compressed sparse row (CSR), compressed sparse column (CSC), compressed sparse fiber (CSF) representations. Support for compressed representations enable operations to be performed on input in a compressed tensor format without requiring the compressed representation to be decompressed or decoded. In such embodiment, operations can be performed only on non-zero input values and the resulting non-zero output values can be mapped into an output matrix. In some embodiments, hardware support is also provided for machine-specific lossless data compression formats that are used when transmitting data within hardware or across system busses. Such data may be retained in a compressed format for sparse input data and the systolic array **1912** can use the compression metadata for the compressed data to enable operations to be performed on only non-zero values, or to enable blocks of zero data input to be bypassed for multiply operations.

The math unit **1913** can be configured to perform a specific subset of mathematical operations in an efficient and lower-power manner than then ALU unit **1911**. The math unit **1913** can include math logic found in shared function logic of a graphics processing engine provided by other embodiments described, e.g., the math logic **1722** of the shared function logic **1720** of FIG. 17. The math unit **1913** can be configured to perform 32-bit and 64-bit floating point operations.

The thread control unit **1901** includes logic to control the execution of threads within the execution unit. The thread control unit **1901** can include thread arbitration logic to start, stop, and preempt execution of threads within the execution unit **1900**. The thread state unit **1902** can be used to store thread state for threads assigned to execute on the execution unit **1900**. Storing the thread state within the execution unit **1900** enables the rapid pre-emption of threads when those threads become blocked or idle. The instruction fetch/prefetch unit **1903** can fetch instructions from an instruction cache of higher-level execution logic (e.g., instruction cache **1806** as in FIG. 18A). The instruction fetch/prefetch unit **1903** can also issue prefetch requests for instructions to be loaded into the instruction cache based on an analysis of

currently executing threads. The instruction decode unit **1904** can be used to decode instructions to be executed by the compute units. The instruction decode unit **1904** can be used as a secondary decoder to decode complex instructions into constituent micro-operations.

The execution unit **1900** additionally includes a register file **1906** that can be used by hardware threads executing on the execution unit **1900**. Registers in the register file **1906** can be divided across the logic used to execute multiple simultaneous threads within the compute unit **1910** of the execution unit **1900**. The number of logical threads that may be executed by the graphics execution unit **1900** is not limited to the number of hardware threads, and multiple logical threads can be assigned to each hardware thread. The size of the register file **1906** can vary across embodiments based on the number of supported hardware threads. Register renaming may be used to dynamically allocate registers to hardware threads.

FIG. 20 is a block diagram illustrating graphics processor instruction formats **2000**. The graphics processor execution units support an instruction set having instructions in multiple formats. The solid lined boxes illustrate the components that are generally included in an execution unit instruction, while the dashed lines include components that are optional or that are only included in a sub-set of the instructions. In some embodiments the graphics processor instruction formats **2000** described and illustrated are macro-instructions, in that they are instructions supplied to the execution unit, as opposed to micro-operations resulting from instruction decode once the instruction is processed. Thus, a single instructions may cause hardware to perform multiple micro-operations

The graphics processor execution units as described herein may natively support instructions in a 128-bit instruction format **2010**. A 64-bit compacted instruction format **2030** is available for some instructions based on the selected instruction, instruction options, and number of operands. The native 128-bit instruction format **2010** provides access to all instruction options, while some options and operations are restricted in the 64-bit format **2030**. The native instructions available in the 64-bit format **2030** vary by embodiment. The instruction is compacted in part using a set of index values in an index field **2013**. The execution unit hardware references a set of compaction tables based on the index values and uses the compaction table outputs to reconstruct a native instruction in the 128-bit instruction format **2010**. Other sizes and formats of instruction can be used.

For each format, instruction opcode **2012** defines the operation that the execution unit is to perform. The execution units execute each instruction in parallel across the multiple data elements of each operand. For example, in response to an add instruction the execution unit performs a simultaneous add operation across each color channel representing a texture element or picture element. By default, the execution unit performs each instruction across all data channels of the operands. Instruction control field **2014** may enable control over certain execution options, such as channels selection (e.g., predication) and data channel order (e.g., swizzle). For instructions in the 128-bit instruction format **2010** an exec-size field **2016** limits the number of data channels that can be executed in parallel. An exec-size field **2016** may not be available for use in the 64-bit compact instruction format **2030**.

Some execution unit instructions have up to three operands including two source operands, src0 **2020**, src1 **2022**, and one destination operand (dest **2018**). Other instructions,

61

such as, for example, data manipulation instructions, dot product instructions, multiply-add instructions, or multiply-accumulate instructions, can have a third source operand (e.g., SRC2 **2024**). The instruction opcode **2012** determines the number of source operands. An instruction's last source operand can be an immediate (e.g., hard-coded) value passed with the instruction. The execution units may also support multiple destination instructions, where one or more of the destinations is implied or implicit based on the instruction and/or the specified destination.

The 128-bit instruction format **2010** may include an access/address mode field **2026** specifying, for example, whether direct register addressing mode or indirect register addressing mode is used. When direct register addressing mode is used, the register address of one or more operands is directly provided by bits in the instruction.

The 128-bit instruction format **2010** may also include an access/address mode field **2026**, which specifies an address mode and/or an access mode for the instruction. The access mode may be used to define a data access alignment for the instruction. Access modes including a 16-byte aligned access mode and a 1-byte aligned access mode may be supported, where the byte alignment of the access mode determines the access alignment of the instruction operands. For example, when in a first mode, the instruction may use byte-aligned addressing for source and destination operands and when in a second mode, the instruction may use 16-byte-aligned addressing for all source and destination operands.

The address mode portion of the access/address mode field **2026** may determine whether the instruction is to use direct or indirect addressing. When direct register addressing mode is used bits in the instruction directly provide the register address of one or more operands. When indirect register addressing mode is used, the register address of one or more operands may be computed based on an address register value and an address immediate field in the instruction.

Instructions may be grouped based on opcode **2012** bit-fields to simplify Opcode decode **2040**. For an 8-bit opcode, bits **4**, **5**, and **6** allow the execution unit to determine the type of opcode. The precise opcode grouping shown is merely an example. A move and logic opcode group **2042** may include data movement and logic instructions (e.g., move (mov), compare (cmp)). Move and logic group **2042** may share the five least significant bits (LSB), where move (mov) instructions are in the form of 0000xxxxb and logic instructions are in the form of 0001xxxxb. A flow control instruction group **2044** (e.g., call, jump (jmp)) includes instructions in the form of 0010xxxxb (e.g., 0x20). A miscellaneous instruction group **2046** includes a mix of instructions, including synchronization instructions (e.g., wait, send) in the form of 0011xxxxb (e.g., 0x30). A parallel math instruction group **2048** includes component-wise arithmetic instructions (e.g., add, multiply (mul)) in the form of 0100xxxxb (e.g., 0x40). The parallel math instruction group **2048** performs the arithmetic operations in parallel across data channels. The vector math group **2050** includes arithmetic instructions (e.g., dp4) in the form of 0101xxxxb (e.g., 0x50). The vector math group performs arithmetic such as dot product calculations on vector operands. The illustrated opcode decode **2040**, in one embodiment, can be used to determine which portion of an execution unit can be used to execute a decoded instruction. For example, some instructions may be designated as systolic instructions that can be performed by a systolic array. Other instructions, such as

62

ray-tracing instructions (not shown) can be routed to a ray-tracing core or ray-tracing logic within a slice or partition of execution logic.

Graphics Pipeline

FIG. **21** is a block diagram of graphics processor **2100**, according to another embodiment. The elements of FIG. **21** having the same or similar names as the elements of any other figure herein describe the same elements as in the other figures, can operate or function in a manner similar to that, can comprise the same components, and can be linked to other entities, as those described elsewhere herein, but are not limited to such.

The graphics processor **2100** may include different types of graphics processing pipelines, such as a geometry pipeline **2120**, a media pipeline **2130**, a display engine **2140**, thread execution logic **2150**, and a render output pipeline **2170**. Graphics processor **2100** may be a graphics processor within a multi-core processing system that includes one or more general-purpose processing cores. The graphics processor may be controlled by register writes to one or more control registers (not shown) or via commands issued to graphics processor **2100** via a ring interconnect **2102**. Ring interconnect **2102** may couple graphics processor **2100** to other processing components, such as other graphics processors or general-purpose processors. Commands from ring interconnect **2102** are interpreted by a command streamer **2103**, which supplies instructions to individual components of the geometry pipeline **2120** or the media pipeline **2130**.

Command streamer **2103** may direct the operation of a vertex fetcher **2105** that reads vertex data from memory and executes vertex-processing commands provided by command streamer **2103**. The vertex fetcher **2105** may provide vertex data to a vertex shader **2107**, which performs coordinate space transformation and lighting operations to each vertex. Vertex fetcher **2105** and vertex shader **2107** may execute vertex-processing instructions by dispatching execution threads to execution units **2152A-2152B** via a thread dispatcher **2131**.

The execution units **2152A-2152B** may be an array of vector processors having an instruction set for performing graphics and media operations. The execution units **2152A-2152B** may have an attached L1 cache **2151** that is specific for each array or shared between the arrays. The cache can be configured as a data cache, an instruction cache, or a single cache that is partitioned to contain data and instructions in different partitions.

A geometry pipeline **2120** may include tessellation components to perform hardware-accelerated tessellation of 3D objects. A programmable hull shader **2111** may configure the tessellation operations. A programmable domain shader **2117** may provide back-end evaluation of tessellation output. A tessellator **2113** may operate at the direction of hull shader **2111** and contain special purpose logic to generate a set of detailed geometric objects based on a coarse geometric model that is provided as input to geometry pipeline **2120**. In addition, if tessellation is not used, tessellation components (e.g., hull shader **2111**, tessellator **2113**, and domain shader **2117**) can be bypassed. The tessellation components can operate based on data received from the vertex shader **2107**.

Complete geometric objects may be processed by a geometry shader **2119** via one or more threads dispatched to execution units **2152A-2152B**, or can proceed directly to the clipper **2129**. The geometry shader may operate on entire geometric objects, rather than vertices or patches of vertices as in previous stages of the graphics pipeline. If the tessellation is disabled the geometry shader **2119** receives input

from the vertex shader **2107**. The geometry shader **2119** may be programmable by a geometry shader program to perform geometry tessellation if the tessellation units are disabled.

Before rasterization, a clipper **2129** processes vertex data. The clipper **2129** may be a fixed function clipper or a programmable clipper having clipping and geometry shader functions. A rasterizer and depth test component **2173** in the render output pipeline **2170** may dispatch pixel shaders to convert the geometric objects into per pixel representations. The pixel shader logic may be included in thread execution logic **2150**. Optionally, an application can bypass the rasterizer and depth test component **2173** and access unrasterized vertex data via a stream out unit **2123**.

The graphics processor **2100** has an interconnect bus, interconnect fabric, or some other interconnect mechanism that allows data and message passing amongst the components of the processor. In some embodiments, execution units **2152A-2152B** and associated logic units (e.g., L1 cache **2151**, sampler **2154**, texture cache **2158**, etc.) interconnect via a data port **2156** to perform memory access and communicate with render output pipeline components of the processor. A sampler **2154**, caches **2151**, **2158** and execution units **2152A-2152B** each may have separate memory access paths. Optionally, the texture cache **2158** can also be configured as a sampler cache.

The render output pipeline **2170** may contain a rasterizer and depth test component **2173** that converts vertex-based objects into an associated pixel-based representation. The rasterizer logic may include a windower/masker unit to perform fixed function triangle and line rasterization. An associated render cache **2178** and depth cache **2179** are also available in some embodiments. A pixel operations component **2177** performs pixel-based operations on the data, though in some instances, pixel operations associated with 2D operations (e.g. bit block image transfers with blending) are performed by the 2D engine **2141**, or substituted at display time by the display controller **2143** using overlay display planes. A shared L3 cache **2175** may be available to all graphics components, allowing the sharing of data without the use of main system memory.

The media pipeline **2130** may include a media engine **2137** and a video front-end **2134**. Video front-end **2134** may receive pipeline commands from the command streamer **2103**. The media pipeline **2130** may include a separate command streamer. Video front-end **2134** may process media commands before sending the command to the media engine **2137**. Media engine **2137** may include thread spawning functionality to spawn threads for dispatch to thread execution logic **2150** via thread dispatcher **2131**.

The graphics processor **2100** may include a display engine **2140**. This display engine **2140** may be external to processor **2100** and may couple with the graphics processor via the ring interconnect **2102**, or some other interconnect bus or fabric. Display engine **2140** may include a 2D engine **2141** and a display controller **2143**. Display engine **2140** may contain special purpose logic capable of operating independently of the 3D pipeline. Display controller **2143** may couple with a display device (not shown), which may be a system integrated display device, as in a laptop computer, or an external display device attached via a display device connector.

The geometry pipeline **2120** and media pipeline **2130** may be configurable to perform operations based on multiple graphics and media programming interfaces and are not specific to any one application programming interface (API). A driver software for the graphics processor may translate API calls that are specific to a particular graphics or

media library into commands that can be processed by the graphics processor. Support may be provided for the Open Graphics Library (OpenGL), Open Computing Language (OpenCL), and/or Vulkan graphics and compute API, all from the Khronos Group. Support may also be provided for the Direct3D library from the Microsoft Corporation. A combination of these libraries may be supported. Support may also be provided for the Open Source Computer Vision Library (OpenCV). A future API with a compatible 3D pipeline would also be supported if a mapping can be made from the pipeline of the future API to the pipeline of the graphics processor.

Graphics Pipeline Programming

FIG. **22A** is a block diagram illustrating a graphics processor command format **2200** used for programming graphics processing pipelines, such as, for example, the pipelines described herein in conjunction with FIG. **16A**, **17**, **21**. FIG. **22B** is a block diagram illustrating a graphics processor command sequence **2210** according to an embodiment. The solid lined boxes in FIG. **22A** illustrate the components that are generally included in a graphics command while the dashed lines include components that are optional or that are only included in a sub-set of the graphics commands. The example graphics processor command format **2200** of FIG. **22A** includes data fields to identify a client **2202**, a command operation code (opcode) **2204**, and data **2206** for the command. A sub-opcode **2205** and a command size **2208** are also included in some commands.

Client **2202** may specify the client unit of the graphics device that processes the command data. A graphics processor command parser may examine the client field of each command to condition the further processing of the command and route the command data to the appropriate client unit. The graphics processor client units may include a memory interface unit, a render unit, a 2D unit, a 3D unit, and a media unit. Each client unit may have a corresponding processing pipeline that processes the commands. Once the command is received by the client unit, the client unit reads the opcode **2204** and, if present, sub-opcode **2205** to determine the operation to perform. The client unit performs the command using information in data field **2206**. For some commands an explicit command size **2208** is expected to specify the size of the command. The command parser may automatically determine the size of at least some of the commands based on the command opcode. Commands may be aligned via multiples of a double word. Other command formats can also be used.

The flow diagram in FIG. **22B** illustrates an example graphics processor command sequence **2210**. Software or firmware of a data processing system that features an example graphics processor may use a version of the command sequence shown to set up, execute, and terminate a set of graphics operations. A sample command sequence is shown and described for purposes of example only and is not limited to these specific commands or to this command sequence. Moreover, the commands may be issued as batch of commands in a command sequence, such that the graphics processor can process the sequence of commands in at least partially concurrence.

The graphics processor command sequence **2210** may begin with a pipeline flush command **2212** to cause any active graphics pipeline to complete the currently pending commands for the pipeline. Optionally, the 3D pipeline **2222** and the media pipeline **2224** may not operate concurrently. The pipeline flush is performed to cause the active graphics pipeline to complete any pending commands. In response to a pipeline flush, the command parser for the graphics

processor can pause command processing until the active drawing engines complete pending operations and the relevant read caches are invalidated. Optionally, any data in the render cache that is marked 'dirty' can be flushed to memory. Pipeline flush command **2212** can be used for pipeline synchronization or before placing the graphics processor into a low power state.

A pipeline select command **2213** may be used when a command sequence requests the graphics processor to explicitly switch between pipelines. A pipeline select command **2213** may be utilized only once within an execution context before issuing pipeline commands unless the context is to issue commands for both pipelines. A pipeline flush command **2212** may be utilized immediately before a pipeline switch via the pipeline select command **2213**.

A pipeline control command **2214** may configure a graphics pipeline for operation and may be used to program the 3D pipeline **2222** and the media pipeline **2224**. The pipeline control command **2214** may configure the pipeline state for the active pipeline. The pipeline control command **2214** may be used for pipeline synchronization and to clear data from one or more cache memories within the active pipeline before processing a batch of commands.

Commands related to the return buffer state **2216** may be used to configure a set of return buffers for the respective pipelines to write data. Some pipeline operations utilize the allocation, selection, or configuration of one or more return buffers into which the operations write intermediate data during processing. The graphics processor may also use one or more return buffers to store output data and to perform cross thread communication. The return buffer state **2216** may include selecting the size and number of return buffers to use for a set of pipeline operations.

The remaining commands in the command sequence differ based on the active pipeline for operations. Based on a pipeline determination **2220**, the command sequence is tailored to the 3D pipeline **2222** beginning with the 3D pipeline state **2230** or the media pipeline **2224** beginning at the media pipeline state **2240**.

The commands to configure the 3D pipeline state **2230** include 3D state setting commands for vertex buffer state, vertex element state, constant color state, depth buffer state, and other state variables that are to be configured before 3D primitive commands are processed. The values of these commands are determined at least in part based on the particular 3D API in use. The 3D pipeline state **2230** commands may also be able to selectively disable or bypass certain pipeline elements if those elements cannot be used.

A 3D primitive **2232** command may be used to submit 3D primitives to be processed by the 3D pipeline. Commands and associated parameters that are passed to the graphics processor via the 3D primitive **2232** command are forwarded to the vertex fetch function in the graphics pipeline. The vertex fetch function uses the 3D primitive **2232** command data to generate vertex data structures. The vertex data structures are stored in one or more return buffers. The 3D primitive **2232** command may be used to perform vertex operations on 3D primitives via vertex shaders. To process vertex shaders, 3D pipeline **2222** dispatches shader execution threads to graphics processor execution units.

The 3D pipeline **2222** may be triggered via an execute **2234** command or event. A register may write trigger command executions. An execution may be triggered via a 'go' or 'kick' command in the command sequence. Command execution may be triggered using a pipeline synchronization command to flush the command sequence through the graphics pipeline. The 3D pipeline can perform geom-

etry processing for the 3D primitives. Once operations are complete, the resulting geometric objects are rasterized and the pixel engine colors the resulting pixels. Additional commands to control pixel shading and pixel back end operations may also be included for those operations.

The graphics processor command sequence **2210** may follow the media pipeline **2224** path when performing media operations. In general, the specific use and manner of programming for the media pipeline **2224** depends on the media or compute operations to be performed. Specific media decode operations may be offloaded to the media pipeline during media decode. The media pipeline can also be bypassed and media decode can be performed in whole or in part using resources provided by one or more general-purpose processing cores. The media pipeline may also include elements for general-purpose graphics processor unit (GPGPU) operations, where the graphics processor is used to perform SIMD vector operations using computational shader programs that are not explicitly related to the rendering of graphics primitives.

Media pipeline **2224** may be configured in a similar manner as the 3D pipeline **2222**. A set of commands to configure the media pipeline state **2240** are dispatched or placed into a command queue before the media object commands **2242**. Commands for the media pipeline state **2240** may include data to configure the media pipeline elements that can be used to process the media objects. This includes data to configure the video decode and video encode logic within the media pipeline, such as encode or decode format. Commands for the media pipeline state **2240** may also support the use of one or more pointers to "indirect" state elements that contain a batch of state settings.

Media object commands **2242** may supply pointers to media objects for processing by the media pipeline. The media objects include memory buffers containing video data to be processed. Optionally, all media pipeline states should be valid before issuing a media object command **2242**. Once the pipeline state is configured and media object commands **2242** are queued, the media pipeline **2224** is triggered via an execute command **2244** or an equivalent execute event (e.g., register write). Output from media pipeline **2224** may then be post processed by operations provided by the 3D pipeline **2222** or the media pipeline **2224**. GPGPU operations may be configured and executed in a similar manner as media operations.

Graphics Software Architecture

FIG. 23 illustrates an example graphics software architecture for a data processing system **2300**. Such a software architecture may include a 3D graphics application **2310**, an operating system **2320**, and at least one processor **2330**. Processor **2330** may include a graphics processor **2332** and one or more general-purpose processor core(s) **2334**. The processor **2330** may be a variant of the processor **1402** or any other of the processors described herein. The processor **2330** may be used in place of the processor **1402** or any other of the processors described herein. Therefore, the discussion of any features in combination with the processor **1402** or any other of the processors described herein also discloses a corresponding combination with the graphics processor **2332**, but is not limited to such. Moreover, the elements of FIG. 23 having the same or similar names as the elements of any other figure herein describe the same elements as in the other figures, can operate or function in a manner similar to that, can comprise the same components, and can be linked to other entities, as those described elsewhere herein, but are not limited to such. The graphics application **2310** and

operating system **2320** are each executed in the system memory **2350** of the data processing system.

3D graphics application **2310** may contain one or more shader programs including shader instructions **2312**. The shader language instructions may be in a high-level shader language, such as the High-Level Shader Language (HLSL) of Direct3D, the OpenGL Shader Language (GLSL), and so forth. The application may also include executable instructions **2314** in a machine language suitable for execution by the general-purpose processor core **2334**. The application may also include graphics objects **2316** defined by vertex data.

The operating system **2320** may be a Microsoft® Windows® operating system from the Microsoft Corporation, a proprietary UNIX-like operating system, or an open source UNIX-like operating system using a variant of the Linux kernel. The operating system **2320** can support a graphics API **2322** such as the Direct3D API, the OpenGL API, or the Vulkan API. When the Direct3D API is in use, the operating system **2320** uses a front-end shader compiler **2324** to compile any shader instructions **2312** in HLSL into a lower-level shader language. The compilation may be a just-in-time (JIT) compilation or the application can perform shader pre-compilation. High-level shaders may be compiled into low-level shaders during the compilation of the 3D graphics application **2310**. The shader instructions **2312** may be provided in an intermediate form, such as a version of the Standard Portable Intermediate Representation (SPIR) used by the Vulkan API.

User mode graphics driver **2326** may contain a back-end shader compiler **2327** to convert the shader instructions **2312** into a hardware specific representation. When the OpenGL API is in use, shader instructions **2312** in the GLSL high-level language are passed to a user mode graphics driver **2326** for compilation. The user mode graphics driver **2326** may use operating system kernel mode functions **2328** to communicate with a kernel mode graphics driver **2329**. The kernel mode graphics driver **2329** may communicate with graphics processor **2332** to dispatch commands and instructions.

IP Core Implementations

One or more aspects may be implemented by representative code stored on a machine-readable medium (e.g., a non-transitory computer-readable (or machine-readable) storage medium) that represents and/or defines logic within an integrated circuit such as a processor. For example, the machine-readable medium may include instructions which represent various logic within the processor. When read by a machine, the instructions may cause the machine to fabricate the logic to perform the techniques described herein. Such representations, known as “IP cores,” are reusable units of logic for an integrated circuit that may be stored on a tangible, machine-readable medium as a hardware model that describes the structure of the integrated circuit. The hardware model may be supplied to various customers or manufacturing facilities, which load the hardware model on fabrication machines that manufacture the integrated circuit. The integrated circuit may be fabricated such that the circuit performs operations described in association with any of the embodiments described herein.

FIG. 24A is a block diagram illustrating an IP core development system **2400** that may be used to manufacture an integrated circuit to perform operations according to an embodiment. The IP core development system **2400** may be used to generate modular, re-usable designs that can be incorporated into a larger design or used to construct an entire integrated circuit (e.g., an SOC integrated circuit). A

design facility **2430** can generate a software simulation **2410** of an IP core design in a high-level programming language (e.g., C/C++). The software simulation **2410** can be used to design, test, and verify the behavior of the IP core using a simulation model **2412**. The simulation model **2412** may include functional, behavioral, and/or timing simulations. A register transfer level (RTL) design **2415** can then be created or synthesized from the simulation model **2412**. The RTL design **2415** is an abstraction of the behavior of the integrated circuit that models the flow of digital signals between hardware registers, including the associated logic performed using the modeled digital signals. In addition to an RTL design **2415**, lower-level designs at the logic level or transistor level may also be created, designed, or synthesized. Thus, the particular details of the initial design and simulation may vary.

The RTL design **2415** or equivalent may be further synthesized by the design facility into a hardware model **2420**, which may be in a hardware description language (HDL), or some other representation of physical design data. The HDL may be further simulated or tested to verify the IP core design. The IP core design can be stored for delivery to a 3rd party fabrication facility **2465** using non-volatile memory **2440** (e.g., hard disk, flash memory, or any non-volatile storage medium). Alternatively, the IP core design may be transmitted (e.g., via the Internet) over a wired connection **2450** or wireless connection **2460**. The fabrication facility **2465** may then fabricate an integrated circuit that is based at least in part on the IP core design. The fabricated integrated circuit can be configured to perform operations in accordance with at least one embodiment described herein.

FIG. 24B illustrates a cross-section side view of an integrated circuit package assembly **2470**. The integrated circuit package assembly **2470** illustrates an implementation of one or more processor or accelerator devices as described herein. The package assembly **2470** includes multiple units of hardware logic **2472**, **2474** connected to a substrate **2480**. The logic **2472**, **2474** may be implemented at least partly in configurable logic or fixed-functionality logic hardware, and can include one or more portions of any of the processor core(s), graphics processor(s), or other accelerator devices described herein. Each unit of logic **2472**, **2474** can be implemented within a semiconductor die and coupled with the substrate **2480** via an interconnect structure **2473**. The interconnect structure **2473** may be configured to route electrical signals between the logic **2472**, **2474** and the substrate **2480**, and can include interconnects such as, but not limited to bumps or pillars. The interconnect structure **2473** may be configured to route electrical signals such as, for example, input/output (I/O) signals and/or power or ground signals associated with the operation of the logic **2472**, **2474**. Optionally, the substrate **2480** may be an epoxy-based laminate substrate. The substrate **2480** may also include other suitable types of substrates. The package assembly **2470** can be connected to other electrical devices via a package interconnect **2483**. The package interconnect **2483** may be coupled to a surface of the substrate **2480** to route electrical signals to other electrical devices, such as a motherboard, other chipset, or multi-chip module.

The units of logic **2472**, **2474** may be electrically coupled with a bridge **2482** that is configured to route electrical signals between the logic **2472**, **2474**. The bridge **2482** may be a dense interconnect structure that provides a route for electrical signals. The bridge **2482** may include a bridge substrate composed of glass or a suitable semiconductor material. Electrical routing features can be formed on the

bridge substrate to provide a chip-to-chip connection between the logic **2472**, **2474**.

Although two units of logic **2472**, **2474** and a bridge **2482** are illustrated, embodiments described herein may include more or fewer logic units on one or more dies. The one or more dies may be connected by zero or more bridges, as the bridge **2482** may be excluded when the logic is included on a single die. Alternatively, multiple dies or units of logic can be connected by one or more bridges. Additionally, multiple logic units, dies, and bridges can be connected together in other possible configurations, including three-dimensional configurations.

FIG. **24C** illustrates a package assembly **2490** that includes multiple units of hardware logic chiplets connected to a substrate **2480** (e.g., base die). A graphics processing unit, parallel processor, and/or compute accelerator as described herein can be composed from diverse silicon chiplets that are separately manufactured. In this context, a chiplet is an at least partially packaged integrated circuit that includes distinct units of logic that can be assembled with other chiplets into a larger package. A diverse set of chiplets with different IP core logic can be assembled into a single device. Additionally the chiplets can be integrated into a base die or base chiplet using active interposer technology. The concepts described herein enable the interconnection and communication between the different forms of IP within the GPU. IP cores can be manufactured using different process technologies and composed during manufacturing, which avoids the complexity of converging multiple IPs, especially on a large SoC with several flavors IPs, to the same manufacturing process. Enabling the use of multiple process technologies improves the time to market and provides a cost-effective way to create multiple product SKUs. Additionally, the disaggregated IPs are more amenable to being power gated independently, components that are not in use on a given workload can be powered off, reducing overall power consumption.

In various embodiments a package assembly **2490** can include fewer or greater number of components and chiplets that are interconnected by a fabric **2485** or one or more bridges **2487**. The chiplets within the package assembly **2490** may have a 2.5D arrangement using Chip-on-Wafer-on-Substrate stacking in which multiple dies are stacked side-by-side on a silicon interposer that includes through-silicon vias (TSVs) to couple the chiplets with the substrate **2480**, which includes electrical connections to the package interconnect **2483**.

In one embodiment, silicon interposer is an active interposer **2489** that includes embedded logic in addition to TSVs. In such embodiment, the chiplets within the package assembly **2490** are arranged using 3D face to face die stacking on top of the active interposer **2489**. The active interposer **2489** can include hardware logic for I/O **2491**, cache memory **2492**, and other hardware logic **2493**, in addition to interconnect fabric **2485** and a silicon bridge **2487**. The fabric **2485** enables communication between the various logic chiplets **2472**, **2474** and the logic **2491**, **2493** within the active interposer **2489**. The fabric **2485** may be an NoC interconnect or another form of packet switched fabric that switches data packets between components of the package assembly. For complex assemblies, the fabric **2485** may be a dedicated chiplet enables communication between the various hardware logic of the package assembly **2490**.

Bridge structures **2487** within the active interposer **2489** may be used to facilitate a point to point interconnect between, for example, logic or I/O chiplets **2474** and

memory chiplets **2475**. In some implementations, bridge structures **2487** may also be embedded within the substrate **2480**.

The hardware logic chiplets can include special purpose hardware logic chiplets **2472**, logic or I/O chiplets **2474**, and/or memory chiplets **2475**. The hardware logic chiplets **2472** and logic or I/O chiplets **2474** may be implemented at least partly in configurable logic or fixed-functionality logic hardware and can include one or more portions of any of the processor core(s), graphics processor(s), parallel processors, or other accelerator devices described herein. The memory chiplets **2475** can be DRAM (e.g., GDDR, HBM) memory or cache (SRAM) memory. Cache memory **2492** within the active interposer **2489** (or substrate **2480**) can act as a global cache for the package assembly **2490**, part of a distributed global cache, or as a dedicated cache for the fabric **2485**.

Each chiplet can be fabricated as separate semiconductor die and coupled with a base die that is embedded within or coupled with the substrate **2480**. The coupling with the substrate **2480** can be performed via an interconnect structure **2473**. The interconnect structure **2473** may be configured to route electrical signals between the various chiplets and logic within the substrate **2480**. The interconnect structure **2473** can include interconnects such as, but not limited to bumps or pillars. In some embodiments, the interconnect structure **2473** may be configured to route electrical signals such as, for example, input/output (U/O) signals and/or power or ground signals associated with the operation of the logic, U/O and memory chiplets. In one embodiment, an additional interconnect structure couples the active interposer **2489** with the substrate **2480**.

The substrate **2480** may be an epoxy-based laminate substrate, however, it is not limited to that and the substrate **2480** may also include other suitable types of substrates. The package assembly **2490** can be connected to other electrical devices via a package interconnect **2483**. The package interconnect **2483** may be coupled to a surface of the substrate **2480** to route electrical signals to other electrical devices, such as a motherboard, other chipset, or multi-chip module.

A logic or I/O chiplet **2474** and a memory chiplet **2475** may be electrically coupled via a bridge **2487** that is configured to route electrical signals between the logic or I/O chiplet **2474** and a memory chiplet **2475**. The bridge **2487** may be a dense interconnect structure that provides a route for electrical signals. The bridge **2487** may include a bridge substrate composed of glass or a suitable semiconductor material. Electrical routing features can be formed on the bridge substrate to provide a chip-to-chip connection between the logic or I/O chiplet **2474** and a memory chiplet **2475**. The bridge **2487** may also be referred to as a silicon bridge or an interconnect bridge. For example, the bridge **2487** is an Embedded Multi-die Interconnect Bridge (EMIB). Alternatively, the bridge **2487** may simply be a direct connection from one chiplet to another chiplet.

FIG. **24D** illustrates a package assembly **2494** including interchangeable chiplets **2495**, according to an embodiment. The interchangeable chiplets **2495** can be assembled into standardized slots on one or more base chiplets **2496**, **2498**. The base chiplets **2496**, **2498** can be coupled via a bridge interconnect **2497**, which can be similar to the other bridge interconnects described herein and may be, for example, an EMIB. Memory chiplets can also be connected to logic or I/O chiplets via a bridge interconnect. I/O and logic chiplets can communicate via an interconnect fabric. The base chiplets can each support one or more slots in a standardized format for one of logic or I/O or memory/cache.

SRAM and power delivery circuits may be fabricated into one or more of the base chiplets **2496**, **2498**, which can be fabricated using a different process technology relative to the interchangeable chiplets **2495** that are stacked on top of the base chiplets. For example, the base chiplets **2496**, **2498** can be fabricated using a larger process technology, while the interchangeable chiplets can be manufactured using a smaller process technology. One or more of the interchangeable chiplets **2495** may be memory (e.g., DRAM) chiplets. Different memory densities can be selected for the package assembly **2494** based on the power, and/or performance targeted for the product that uses the package assembly **2494**. Additionally, logic chiplets with a different number of type of functional units can be selected at time of assembly based on the power, and/or performance targeted for the product. Additionally, chiplets containing IP logic cores of differing types can be inserted into the interchangeable chiplet slots, enabling hybrid processor designs that can mix and match different technology IP blocks.

Example System on a Chip Integrated Circuit

FIG. **25-26B** illustrate example integrated circuits and associated graphics processors that may be fabricated using one or more IP cores. In addition to what is illustrated, other logic and circuits may be included, including additional graphics processors/cores, peripheral interface controllers, or general-purpose processor cores. The elements of FIG. **25-26B** having the same or similar names as the elements of any other figure herein describe the same elements as in the other figures, can operate or function in a manner similar to that, can comprise the same components, and can be linked to other entities, as those described elsewhere herein, but are not limited to such.

FIG. **25** is a block diagram illustrating an example system on a chip integrated circuit **2500** that may be fabricated using one or more IP cores. Example integrated circuit **2500** includes one or more application processor(s) **2505** (e.g., CPUs), at least one graphics processor **2510**, which may be a variant of the graphics processor **1408**, **1508**, **2510**, or of any graphics processor described herein and may be used in place of any graphics processor described. Therefore, the discussion of any features in combination with a graphics processor herein also discloses a corresponding combination with the graphics processor **2510**, but is not limited to such. The integrated circuit **2500** may additionally include an image processor **2515** and/or a video processor **2520**, any of which may be a modular IP core from the same or multiple different design facilities. Integrated circuit **2500** may include peripheral or bus logic including a USB controller **2525**, UART controller **2530**, an SPI/SDIO controller **2535**, and an I²S/I²C controller **2540**. Additionally, the integrated circuit can include a display device **2545** coupled to one or more of a high-definition multimedia interface (HDMI) controller **2550** and a mobile industry processor interface (MIPI) display interface **2555**. Storage may be provided by a flash memory subsystem **2560** including flash memory and a flash memory controller. Memory interface may be provided via a memory controller **2565** for access to SDRAM or SRAM memory devices. Some integrated circuits additionally include an embedded security engine **2570**.

FIG. **26A-26B** are block diagrams illustrating example graphics processors for use within an SoC, according to embodiments described herein. The graphics processors may be variants of the graphics processor **1408**, **1508**, **2510**, or any other graphics processor described herein. The graphics processors may be used in place of the graphics processor **1408**, **1508**, **2510**, or any other of the graphics processors described herein. Therefore, the discussion of any features in

combination with the graphics processor **1408**, **1508**, **2510**, or any other of the graphics processors described herein also discloses a corresponding combination with the graphics processors of FIG. **26A-26B**, but is not limited to such. FIG. **26A** illustrates an example graphics processor **2610** of a system on a chip integrated circuit that may be fabricated using one or more IP cores, according to an embodiment. FIG. **26B** illustrates an additional example graphics processor **2640** of a system on a chip integrated circuit that may be fabricated using one or more IP cores, according to an embodiment. Graphics processor **2610** of FIG. **26A** is an example of a low power graphics processor core. Graphics processor **2640** of FIG. **26B** is an example of a higher performance graphics processor core. For example, each of graphics processor **2610** and graphics processor **2640** can be a variant of the graphics processor **2510** of FIG. **25**, as mentioned at the outset of this paragraph.

As shown in FIG. **26A**, graphics processor **2610** includes a vertex processor **2605** and one or more fragment processor(s) **2615A-2615N** (e.g., **2615A**, **2615B**, **2615C**, **2615D**, through **2615N-1**, and **2615N**). Graphics processor **2610** can execute different shader programs via separate logic, such that the vertex processor **2605** is optimized to execute operations for vertex shader programs, while the one or more fragment processor(s) **2615A-2615N** execute fragment (e.g., pixel) shading operations for fragment or pixel shader programs. The vertex processor **2605** performs the vertex processing stage of the 3D graphics pipeline and generates primitives and vertex data. The fragment processor(s) **2615A-2615N** use the primitive and vertex data generated by the vertex processor **2605** to produce a frame-buffer that is displayed on a display device. The fragment processor(s) **2615A-2615N** may be optimized to execute fragment shader programs as provided for in the OpenGL API, which may be used to perform similar operations as a pixel shader program as provided for in the Direct 3D API.

Graphics processor **2610** additionally includes one or more memory management units (MMUs) **2620A-2620B**, cache(s) **2625A-2625B**, and circuit interconnect(s) **2630A-2630B**. The one or more MMU(s) **2620A-2620B** provide for virtual to physical address mapping for the graphics processor **2610**, including for the vertex processor **2605** and/or fragment processor(s) **2615A-2615N**, which may reference vertex or image/texture data stored in memory, in addition to vertex or image/texture data stored in the one or more cache(s) **2625A-2625B**. The one or more MMU(s) **2620A-2620B** may be synchronized with other MMUs within the system, including one or more MMUs associated with the one or more application processor(s) **2505**, image processor **2515**, and/or video processor **2520** of FIG. **25**, such that each processor **2505-2520** can participate in a shared or unified virtual memory system. Components of graphics processor **2610** may correspond with components of other graphics processors described herein. The one or more MMU(s) **2620A-2620B** may correspond with MMU **245** of FIG. **2C**. Vertex processor **2605** and fragment processor **2615A-2615N** may correspond with graphics multiprocessor **234**. The one or more circuit interconnect(s) **2630A-2630B** enable graphics processor **2610** to interface with other IP cores within the SoC, either via an internal bus of the SoC or via a direct connection, according to embodiments. The one or more circuit interconnect(s) **2630A-2630B** may correspond with the data crossbar **240** of FIG. **2C**. Further correspondence may be found between analogous components of the graphics processor **2610** and the various graphics processor architectures described herein.

As shown FIG. 26B, graphics processor 2640 includes the one or more MMU(s) 2620A-2620B, cache(s) 2625A-2625B, and circuit interconnect(s) 2630A-2630B of the graphics processor 2610 of FIG. 26A. Graphics processor 2640 includes one or more shader cores 2655A-2655N (e.g., 2655A, 2655B, 2655C, 2655D, 2655E, 2655F, through 2655N-1, and 2655N), which provides for a unified shader core architecture in which a single core or type or core can execute all types of programmable shader code, including shader program code to implement vertex shaders, fragment shaders, and/or compute shaders. The exact number of shader cores present can vary among embodiments and implementations. Additionally, graphics processor 2640 includes an inter-core task manager 2645, which acts as a thread dispatcher to dispatch execution threads to one or more shader cores 2655A-2655N and a tiling unit 2658 to accelerate tiling operations for tile-based rendering, in which rendering operations for a scene are subdivided in image space, for example to exploit local spatial coherence within a scene or to optimize use of internal caches. Shader cores 2655A-2655N may correspond with, for example, graphics multiprocessor 234 as in FIG. 2D, or graphics multiprocessors 325, 350 of FIGS. 3A and 3B respectively, or multi-core group 365A of FIG. 3C.

In some embodiments, a processing resource represents a processing element (e.g., GPGPU core, ray-tracing core, tensor core, execution resource, execution unit (EU), stream processor, streaming multiprocessor (SM), graphics multiprocessor) associated with a graphics processor or graphics processor structure (e.g., parallel processing unit, graphics processing engine, multi-core group, compute unit, compute unit of graphics core next) in a GPU as described herein. For example, the processing resource may be one of the GPGPU cores, or tensor/ray-tracing cores of graphics multiprocessor; a ray-tracing core, tensor core or GPGPU core of graphics multiprocessor; execution resources of graphics multiprocessor; one of GFX cores, tensor cores, or ray tracing cores of a multi-core group; one of vector logic units or scalar logic units of a compute unit; execution unit with EU array or EU array; an execution unit of execution logic; and/or execution unit. The processing resource may also be an execution resource within, for example, a graphics processing engine, processing cluster, GPGPU, GPGPU, graphics processing engine, graphics processing engine cluster, and/or graphics processing engine. The processing resource may also be a processing resource within graphics processor, graphics processor, and/or graphics processor. Gathering Payload from Arbitrary Registers for Send Messages in a Graphics Environment

Parallel computing is a type of computation in which many calculations or the execution of processes are carried out simultaneously. Parallel computing may come in a variety of forms, including, but not limited to, SIMD or SIMT. SIMD describes computers with multiple processing elements that perform the same operation on multiple data points simultaneously. In one example, the figures discussed above refer to SIMD and its implementation in a general processor in terms of EUs, FPU, and ALUs. In a common SIMD machine, data is packaged into registers, each containing an array of channels. Instructions operate on the data found in channel n of a register with the data found in the same channel of another register. SIMD machines are advantageous in areas where a single sequence of instructions can be simultaneously applied to high amounts of data. For example, in one embodiment, a graphics processor (e.g., GPGPU, GPU, etc.) can be used to perform SIMD vector operations using computational shader programs.

Various embodiments can also apply to use execution by use of Single Instruction Multiple Thread (SIMT) as an alternate to use of SIMD or in addition to use of SIMD. Reference to a SIMD core or operation can apply also to SIMT or apply to SIMD in combination with SIMT. The following description is discussed in terms of SIMD machines. However, embodiments herein are not solely limited to application in the SIMD context and may apply in other parallel computing paradigms, such as SIMT, for example. For ease of discussion and explanation, the following description generally focuses on a SIMD implementation. However, embodiments can similarly apply to SIMT machines with no modifications to the described techniques and methodologies. With respect to SIMT machines, similar patterns as discussed below can be followed to provide instructions to the systolic array and execute the instructions on the SIMT machine. Other types of parallel computing machines may also utilize embodiments herein as well.

Parallel rendering graphics architectures utilize communications between the execution resources (e.g., EUs) and the shared functions and/or fixed function pipelines to enable efficient parallel rendering computations. Such communications between the execution resources and the shared functions and/or fixed functions can be accomplished via packets of information called messages. Message transmission is requested using send message instructions. The current send message instructions require the payload of the instruction to be in contiguous registers. For example, in conventional approaches, the execution resource (e.g., EU) utilizes a send message instruction (e.g., send or sendc instruction (where sendc refers to a conditional send message)), and those send message instructions take a set of GRF registers and send that set of GRF registers out to a shared function (or fixed function).

In order to ensure that the set of GRF registers is contiguous, conventional approaches implement additional move (e.g., mov) instructions in the processor (e.g., 3D shaders) to cause the payload to be in contiguous registers (e.g., as a block). However, the data that is utilized in the block often comes from different computation outputs found in different computation output registers that are not contiguous. For example, different FPU can be writing to different registers. The move instructions of the conventional approaches are inefficient as the move instructions are not doing anything except putting data together in a contiguous block of GRF registers. Such inefficiency consumes additional energy in the processor and reduces execution efficiency.

Embodiments address the above-noted drawbacks by providing a send gather message instruction to gather payload from arbitrary registers for a send message in a graphics environment. In implementations herein, arbitrary registers can refer to non-contiguous registers. The move instructions of the conventional approach can be avoided by implementations herein by removing the contiguous register requirement. Implementations herein provide a send gather message instruction (e.g., sendg/sendgc), which can gather payload from any arbitrary register that does not have to be contiguous. For example, the send gather message instruction of implementations herein can gather up to 16 registers for a payload of the send message. This send gather message instruction can also provide up to 16 pointers and can collect 16 GRFs from different arbitrary locations and send them out.

Embodiments provide a technical advantage of improving performance of the processor. The approaches discussed to provide for a send gather message instruction to gather

payload from arbitrary registers for a send message in a graphics environment reduce the amount of instructions in the execution resources (e.g., EUs) of the graphics environment, which reduces the total energy consumed by avoiding extraneous move instructions. In some implementations, the energy savings can depend on a dynamic count of these move instructions.

FIG. 27 is a block diagram illustrating an example integrated circuit graphics processor 2700 having an execution resource for providing a send gather message instruction to gather payload from arbitrary registers for a send message in a graphics environment, according to embodiments. In one implementation, graphics processor 2700 may include a GPGPU or GPU, such as the example GPGPUs and/or GPUs described herein with respect to FIGS. 1-26. The elements of FIG. 27 having the same or similar names as the elements of any other figure herein describe the same elements as in the other figures, can operate or function in a manner similar to that, can comprise the same components, and can be linked to other entities, as those described elsewhere herein, but are not limited to such. Example graphics processor 2700 may be a variant of the graphics processor 1408, 1508, 2510, or of any graphics processor described herein and may be used in place of any graphics processor described. Example execution resource 2710 may be a variant of the execution unit 1900 or of any execution unit or processing resource described herein and may be used in place of any execution resource described. Therefore, the discussion of any features in combination with a graphics processor herein also discloses a corresponding combination with the graphics processor 2700, but is not limited to such.

The graphics processor 2700 illustrated in FIG. 27 may include a shader processor 2702, a thread dispatcher 2704, an instruction queue 2706, one or more execution resources 2710, shared functions 2760, fixed functions 2770, a data cache 2780, and a data port 2790. The graphics processor 2700 may include similar components as described above with respect to execution logic 1800 described in FIG. 18A. The elements of FIG. 27 having the same or similar names as the elements of FIG. 18A herein describe the same elements as in FIG. 18A, can operate or function in a manner similar to that, can comprise the same components, and can be linked to other entities, as those described elsewhere herein, but are not limited to such. For ease of description and explanation, the description of the same or similar elements as provided above in FIG. 18A similarly applies to the description of the same or similar elements of FIG. 27 herein and may not necessarily be repeated.

The graphics processor 2700 illustrated in FIG. 27 may include one or more execution resources 2710. The execution resource 2710 may be a compute-optimized processing resources, such as an EU, for use in, for example, a compute engine tile 1640A-1640D as in FIG. 16C, but is not limited as such. The execution resource 2710 may also be used in a graphics engine tile 1610A-1610D as in FIG. 16B. The execution resource 2710 may include a thread arbiter 2712, a thread state 2714, instruction fetch 2716, instruction decode 2718, register files 2720, compute resource 2730 having functional units 2735, branch unit 2740, and send unit 2750. The execution resources 2710 may include execution circuitry to execute operations of the graphics processor 2700. The execution resource 2710 may include similar components as described above with respect to execution unit 1900 described in FIG. 19. The elements of FIG. 27 having the same or similar names as the elements of FIG. 19 herein describe the same elements as in FIG. 19, can operate

or function in a manner similar to that, can comprise the same components, and can be linked to other entities, as those described elsewhere herein, but are not limited to such. For ease of description and explanation, the description of the same or similar elements as provided above in FIG. 19 similarly applies to the description of the same or similar elements of FIG. 27 herein and may not necessarily be repeated.

The execution resource 2710 may include a register file 2720 that stores registers that can be assigned to hardware threads within the execution resource 2710. Registers in the register file 2720 can be divided across the logic used to execute multiple simultaneous threads within the compute resource 2730 of the execution resource 2710. The number of logical threads that may be executed by the execution resource 2710 is not limited to the number of hardware threads, and multiple logical threads can be assigned to each hardware thread. The size of the register file 2720 can vary across embodiments based on the number of supported hardware threads. Register renaming may be used to dynamically allocate registers to hardware threads.

The execution resource 2710 can also include a compute resource 2730 that includes multiple different types of functional units. The compute resource 2730 may include one or more functional units 2735 that includes an array of arithmetic logic units (ALUs), for example. The functional units 2735 and/or ALUs can be configured to perform 64-bit, 32-bit, and 16-bit integer and floating-point operations across multiple processing lanes and data channels and for multiple hardware and/or software threads. The functional unit 2735 can perform integer and floating-point operations simultaneously (e.g., within the same clock cycle).

In implementations herein, the graphics processor 2700 is enhanced to provide send gather message instruction to gather payload from arbitrary registers for a send message in a graphics environment, such as in the GPU environment discussed herein. Implementations herein provide for a send gather message instruction. In some implementations, two send gather message instructions, one for a regular send gather message (e.g., sendg) and one for a conditional send gather message (e.g., sendgc) are provided. A message is a self-contained packet of information created by a kernel and directed to a specific shared function, such as shared functions 2760. The message can be defined by a set of GRF registers or message register files (MRF) registers (part of register files 2720) that hold message operands, a destination shared function ID, a function-specific encoding of a target operation, and a set of destination GRF registers (part of register files 2720) to which can writeback response is to be directed. The GRF or MRF registers may also be referred to as "individual registers" herein. Messages are dispatched to the shared function 2760 under software control via a send instruction, such as the send gather message instruction discussed herein. The send instruction identifies the contents of the message and the GRF registers locations to direct any response.

The send gather message instruction provided by implementations herein can gather payload from arbitrary registers of the register files 2720 of the execution resource 2710. In one example embodiment, the send gather message instruction can utilize a 128-bit source vector (SrcVec) field in the instruction to represent up to 16 different registers (8bits per register). The size of the send gather message instruction becomes 256 bits with the SrcVec field. The compiler sees the message as a 256-bit instruction. However, in some implementations herein, the thread dispatcher 2704 and the execution resource 2710 can be enhanced to read the

send gather message instruction in two phases in the hardware, instead of adding infrastructure to read 256-bit instruction from the instruction queue **2706**. In other implementations, the thread dispatcher **2704** and the execution resource **2710** are enhanced to read the send gather message instruction in a single phase in the hardware, where the instruction and source vector are read and processed at the same time.

The send gather message instruction is formatted so that the request message does not utilize contiguous registers **2720**. The payload of the send gather message instruction provided herein can be stored in any set of arbitrary GRF registers **2720**. A source vector length field (e.g., SrcVec.Length field) of the send gather message instruction can indicate a number of GRF registers and a source vector field (e.g., SrcVec field) can indicate the set of individual GRF registers for the request message.

In one embodiment, the send gather message instruction is formatted to utilize an architectural register, such as a scalar register of register files **2720** of the execution resource **2710**, to store the list of GRF registers **2720** (e.g., store IDs of the GRF registers in the scalar register). In one embodiment, the execution resource **2710** may execute one or more immediate move instruction(s) to move the IDs of the arbitrary GRF registers to the scalar register. A single immediate move instruction may be able to move 8 bytes to the scalar register at a time. This means that 8 IDs or pointers to the GRF registers **2720** can be moved to the scalar register using a single immediate move instruction. A pointer to this architectural register is then included in the send gather message instruction. In one example, the source vector field (e.g., SrcVec field) of the send gather message instruction may include the pointer to the scalar register.

In implementations herein, the thread dispatcher **2704** may receive the send gather message instruction from, for example, a shader processor **2702**. The thread dispatcher **2704** can cause this send gather message instruction to be dispatched to the execution resource **2710** for processing.

In one embodiment, at the execution resource **2710**, the thread arbiter **2712** can include a send message scheduler **2713** to read the send gather message instruction in two phases. The first phase can include, but is not limited to, an identification of the set of GRF registers or MRF registers that hold message operands, the destination shared function ID, the function-specific encoding of a target operation, and/or the source vector length (e.g., SrcVec.Length field). The first phase can be decoded, using the instruction decode **2718**, similar to a conventional send message instruction. However, in implementations herein, the send message scheduler **2713** can cause a second phase of the send gather message instruction to bypass through the instruction decode **2718** behind the first phase of the send gather message instruction. The second phase can include, but is not limited to, the source vector field (e.g., SrcVec field) that indicates the set of individual GRF registers for the request message.

The send message scheduler **2713** of the thread arbiter **2712** can treat the two phases of the send gather message instruction as two atomic instructions. These two atomic instructions of the send gather message instruction can be dispatched sequentially, one after the other, without any intervening messages between the two phases of the instruction. In some implementations, the two phases of the instruction can be in two different cache lines. The send message scheduler **2713** of the thread arbiter **2712** should schedule the send gather message instruction to the send pipe of send unit **2750** after both of the phases are read from instruction queue **2706**.

In one embodiment, a single-phase mechanism is implemented, where the thread dispatcher **2704** and the execution resource **2710** are enhanced to read the send gather message instruction in a single phase in the hardware. In the single phase approach, the instruction and source vector are read and processed at the same time.

In embodiments utilizing the architectural register (e.g., scalar register) to store the IDs (pointers) of the arbitrary GRF registers, a single phase mechanism is implemented to read the send gather message instruction in a single phase in the hardware.

FIG. **28A** illustrates a set of instructions **2800** executable by a processing unit, according to embodiments described herein. FIG. **28A** illustrates fields of the instruction **2800** to perform a send gather message operation. The instruction **2800** includes, but is not limited to, a send gather message instruction. Other send gather message instructions may also be implemented by embodiments.

The send gather message instruction **2800** is executable by a processing unit, such as a VPU or FPU, provided by an embodiment. FIG. **28A** illustrates fields of the send gather message instruction **2800**, which, when executed, causes a processing unit to execute an instruction to perform a send gather message operations using specified operands. In one embodiment, the instruction **2800** includes an opcode field **2802** and operand fields to specify a destination (dst) **2804**, a source vector length (SrcVec Length) **2806**, a destination shared function ID (ex_desc) **2808**, and a source vector (SrcVec) **2810**.

The opcode field **2802** can specify an opcode that identifies the instruction **2800** to execution logic. In one embodiment the opcode field **2802** includes one or more bits that, when enabled, indicate that the instruction is to be executed by a processing unit of a compute block (e.g., compute block **1424**). In one embodiment, the opcode field **2802** identifies the opcode for the send gather message (sendg) instruction.

The destination (dst) **2804** can be used to specify a destination to provide a GRF register location for a response message if present, and to provide parameters to form a channel enable sideband signals. The source vector length (SrcVec Length) **2806** can be used to indicate a number of GRF registers for the payload. The destination shared function ID (ex_desc) **2808** can be used to specify a target function ID (e.g., shared function ID (SFID)) for the send gather message. The source vector (SrcVec) **2810** can be used to indicate the actual individual GRF registers for the send gather message.

FIG. **28B** illustrates a program code compilation process **2815**, according to an embodiment. In one embodiment, a source code level description **2820** of a software program is compiled at a compiler **2830**, which can include multiple levels of compilations, to a level having an operation **2840** that includes or specifies a send gather message instruction to be performed by processing logic. The send gather message can include a send message that gathers payload from arbitrary registers that do not have to be contiguous. As discussed above, the send gather message itself identifies each arbitrary registers used to gather the payload, as well as the number of registers used for the payload gather.

The operation **2840** can be an operation specified in an intermediate language or can be program code that references a primitive of a compute framework, such as a primitive provided by a machine learning framework. The operation **2840** that includes or specifies a send gather message instruction may then be further compiled by an additional compiler **2850**, which can be a shader compiler,

into machine level object code **2860** that includes a send gather instruction to be performed by a processing unit (e.g., VPU, FPU) of a compute block, as described herein.

FIG. **29** is a flow diagram illustrating an embodiment of a method **2900** for executing an instruction to perform gathering payload from arbitrary registers for send messages in a graphics environment. Method **2900** may be performed by processing logic that may comprise hardware (e.g., circuitry, dedicated logic, programmable logic, etc.), software (such as instructions run on a processing device), or a combination thereof. The process of method **2900** is illustrated in linear sequences for brevity and clarity in presentation; however, it is contemplated that any number of them can be performed in parallel, asynchronously, or in different orders. Further, for brevity, clarity, and ease of understanding, many of the components and processes described with respect to FIGS. **1-28** may not be repeated or discussed hereafter. In one implementation, a processor, such as a processor **2700** of FIG. **27**, may perform method **2900**.

Method **2900** begins at processing block **2910** where a single instruction is fetched and decoded to be executed within a GPGPU. In one implementation, the single instruction decoded into a decoded instruction to cause the GPGPU to perform a send gather message operation. At processing block **2920**, a set of commands is determined to execute the decoded vector instruction on a compute block of the GPGPU.

Subsequently, at processing block **2930**, the set of commands is scheduled to a compute block of the GPGPU to execute the decoded instruction to perform the send gather message operation. Lastly, at processing block **2940**, the decoded instruction is retired in response to completion of the set of commands.

FIG. **30** is a flow diagram illustrating an embodiment of a method **3000** for scheduling a send gather message that gathers payload from arbitrary registers in a graphics environment. Method **3000** may be performed by processing logic that may comprise hardware (e.g., circuitry, dedicated logic, programmable logic, etc.), software (such as instructions run on a processing device), or a combination thereof. The process of method **3000** is illustrated in linear sequences for brevity and clarity in presentation; however, it is contemplated that any number of them can be performed in parallel, asynchronously, or in different orders. Further, for brevity, clarity, and ease of understanding, many of the components and processes described with respect to FIGS. **1-29** may not be repeated or discussed hereafter. In one implementation, a processor, such as a processor **2700** of FIG. **27**, may perform method **3000**.

Method **3000** begins at processing block **3010** where the processor may receive, at an execution resource of a GPU, a send gather message instruction identifying a number of GRF registers to access for a send message and identifying IDs of a plurality of individual GRF registers corresponding to the number of GRF registers. Then, at block **3020**, the processor may decode a first phase of the send gather message instruction at the execution resource.

Subsequently, at block **3030**, the processor may, based on decoding the first phase of the send gather message, cause a second phase of the send gather message to bypass through an instruction decode stage of the execution resource. Lastly, at block **3040**, the processor may dispatch the first phase of the send gather message subsequently followed by dispatch of the second phase of the send gather message to a send pipeline of the execution resource. In one embodiment, the dispatch of the first phase and the second phase of the send gather message are dispatched to the send pipeline without

any intervening dispatched messages between the first phase and the second phase in the send pipeline.

As previously noted, instead of the two-phase approach discussed above, implementations may implement a single-phase mechanism. In the single-phase approach, the thread dispatcher and the execution resource are enhanced to read the send gather message instruction in a single phase in the hardware. In the single-phase approach, the instruction and source vector are read and processed at the same time.

FIG. **31** is a flow diagram illustrating an embodiment of a method **3000** for scheduling a send gather message using an architectural register to gather payload from arbitrary registers in a graphics environment. Method **3100** may be performed by processing logic that may comprise hardware (e.g., circuitry, dedicated logic, programmable logic, etc.), software (such as instructions run on a processing device), or a combination thereof. The process of method **3100** is illustrated in linear sequences for brevity and clarity in presentation; however, it is contemplated that any number of them can be performed in parallel, asynchronously, or in different orders. Further, for brevity, clarity, and ease of understanding, many of the components and processes described with respect to FIGS. **1-20** may not be repeated or discussed hereafter. In one implementation, a processor, such as a processor **2700** of FIG. **27**, may perform method **3100**.

Method **3100** begins at processing block **3110** where the processor may receive, at an execution resource of a GPU, a send gather message instruction identifying a number of GRF registers to access for a send message and identifying IDs of a plurality of individual GRF registers corresponding to the number of GRF registers. At block **3120**, the processing may perform an immediate move of the IDs of the plurality of individual GRF registers to an architectural register of the execution resource.

Subsequently, at block **3130**, the processor may include a pointer to the architectural register in the send gather message instruction. Lastly, at block **3140**, the processor may dispatch the send gather message instruction to a send pipeline of the execution resource.

The following examples pertain to further embodiments. Example 1 is an apparatus to facilitate gathering payload from arbitrary registers for send messages in a graphics environment. The apparatus of Example 1 includes a processor to provide processing resources comprising execution circuitry to: receive a send gather message instruction identifying a number of registers to access for a send message and identifying IDs of a plurality of individual registers corresponding to the number of registers; decode a first phase of the send gather message instruction; based on decoding the first phase of the send gather message instruction, cause a second phase of the send gather message instruction to bypass an instruction decode stage of the execution circuitry; and dispatch the first phase of the send gather message instruction subsequently followed by dispatch of the second phase of the send gather message instruction to a send pipeline of the execution circuitry.

In Example 2, the subject matter of Example 1 can optionally include wherein the dispatch of the first phase and the second phase of the send gather message instruction are dispatched to the send pipeline without any intervening dispatched messages between the first phase and the second phase in the send pipeline. In Example 3, the subject matter of any one of Examples 1-2 can optionally include wherein the plurality of individual registers comprise general register file (GRF) registers. In Example 4, the subject matter of any one of Examples 1-3 can optionally include wherein the

second phase of the send gather message instruction comprises the IDs of the plurality of individual registers.

In Example 5, the subject matter of any one of Examples 1-4 can optionally include wherein the first phase of the send gather message instruction identifies the number of the registers to access for the send message. In Example 6, the subject matter of any one of Examples 1-5 can optionally include wherein the plurality of individual registers are non-contiguous. In Example 7, the subject matter of any one of Examples 1-6 can optionally include wherein the first phase of the send gather message instruction identifies a destination shared function ID, a function-specific encoding of an operation of the send gather message instruction, and a destination register for a writeback response to the send gather message instruction.

In Example 8, the subject matter of any one of Examples 1-7 can optionally include wherein the processor comprises a graphics processing unit (GPU). In Example 9, the subject matter of any one of Examples 1-8 can optionally include wherein the processor is at least one of a single instruction multiple data (SIMD) machine or a single instruction multiple thread (SMT) machine.

Example 10 is a method for facilitating gathering payload from arbitrary registers for send messages in a graphics environment. The method of Example 10 can include receiving, by execution circuitry of a graphics processor, a send gather message instruction identifying a number of registers to access for a send message and identifying IDs of a plurality of individual registers corresponding to the number of registers; decoding a first phase of the send gather message instruction; based on decoding the first phase of the send gather message instruction, causing a second phase of the send gather message instruction to bypass an instruction decode stage of the execution circuitry; and dispatching the first phase of the send gather message instruction subsequently followed by dispatch of the second phase of the send gather message instruction to a send pipeline of the execution circuitry.

In Example 11, the subject matter of Example 10 can optionally include wherein the dispatch of the first phase and the second phase of the send gather message instruction are dispatched to the send pipeline without any intervening dispatched messages between the first phase and the second phase in the send pipeline. In Example 12, the subject matter of Examples 10-11 can optionally include wherein the plurality of individual registers comprise general register file (GRF) registers.

In Example 13, the subject matter of Examples 10-12 can optionally include wherein the first phase of the send gather message instruction identifies the number of the registers to access for the send message, and wherein the second phase of the send gather message instruction comprises the IDs of the plurality of individual registers. In Example 14, the subject matter of Examples 10-13 can optionally include wherein the plurality of individual registers are non-contiguous. In Example 15, the subject matter of Examples 10-14 can optionally include wherein the first phase of the send gather message instruction identifies a destination shared function ID, a function-specific encoding of an operation of the send gather message instruction, and a destination register for a writeback response to the send gather message instruction.

Example 16 is a system for facilitating gathering payload from arbitrary registers for send messages in a graphics environment. The system of Example 21 can optionally include a memory to store a block of data; and a processor coupled to the memory, the processor comprising processing

resources, the processing resources comprising execution circuitry to: receive a send gather message instruction identifying a number of registers to access for a send message and identifying IDs of a plurality of individual registers corresponding to the number of registers; perform an immediate move of the IDs of the plurality of individual registers to an architectural register of the execution circuitry; include a pointer to the architectural register in the send gather message instruction; and dispatch the send gather message instruction to a send pipeline of the execution circuitry.

In Example 17, the subject matter of Example 16 can optionally include wherein the architectural register is a scalar register. In Example 18, the subject matter of Examples 16-17 can optionally include wherein the send gather message instruction identifies the number of the registers to access for the send gather message instruction and the architectural register.

In Example 19, the subject matter of Examples 16-18 can optionally include wherein the plurality of individual registers are non-contiguous, and wherein the IDs comprise pointers. In Example 20, the subject matter of Examples 16-19 can optionally include wherein the send gather message instruction identifies a destination shared function ID, a function-specific encoding of an operation of the send gather message instruction, and a destination register for a writeback response to the send gather message instruction.

Example 21 is a system for facilitating gathering payload from arbitrary registers for send messages in a graphics environment. The system of Example 21 can optionally include a memory to store a block of data, and a processor communicably coupled to the memory to: receive a send gather message instruction identifying a number of registers to access for a send message and identifying IDs of a plurality of individual registers corresponding to the number of registers; decode a first phase of the send gather message instruction; based on decoding the first phase of the send gather message instruction, cause a second phase of the send gather message instruction to bypass an instruction decode stage of the execution circuitry; and dispatch the first phase of the send gather message instruction subsequently followed by dispatch of the second phase of the send gather message instruction to a send pipeline of the execution circuitry.

In Example 22, the subject matter of Example 21 can optionally include wherein the dispatch of the first phase and the second phase of the send gather message instruction are dispatched to the send pipeline without any intervening dispatched messages between the first phase and the second phase in the send pipeline. In Example 23, the subject matter of any one of Examples 21-22 can optionally include wherein the plurality of individual registers comprise general register file (GRF) registers. In Example 24, the subject matter of any one of Examples 21-23 can optionally include wherein the second phase of the send gather message instruction comprises the IDs of the plurality of individual registers.

In Example 25, the subject matter of any one of Examples 21-24 can optionally include wherein the first phase of the send gather message instruction identifies the number of the registers to access for the send message. In Example 26, the subject matter of any one of Examples 21-25 can optionally include wherein the plurality of individual registers are non-contiguous. In Example 27, the subject matter of any one of Examples 21-26 can optionally include wherein the first phase of the send gather message instruction identifies a destination shared function ID, a function-specific encoding of an operation of the send gather message instruction,

and a destination register for a writeback response to the send gather message instruction.

In Example 28, the subject matter of any one of Examples 21-27 can optionally include wherein the processor comprises a graphics processing unit (GPU). In Example 29, the subject matter of any one of Examples 21-28 can optionally include wherein the processor is at least one of a single instruction multiple data (SIMD) machine or a single instruction multiple thread (SIMT) machine.

Example 30 is a method for facilitating gathering payload from arbitrary registers for send messages in a graphics environment. The method of Example 30 can include receiving a send gather message instruction identifying a number of registers to access for a send message and identifying IDs of a plurality of individual registers corresponding to the number of registers; performing an immediate move of the IDs of the plurality of individual registers to an architectural register of the execution circuitry; including a pointer to the architectural register in the send gather message instruction; and dispatching the send gather message instruction to a send pipeline of the execution circuitry.

In Example 31, the subject matter of Example 30 can optionally include wherein the architectural register is a scalar register. In Example 32, the subject matter of Examples 30-31 can optionally include wherein the send gather message instruction identifies the number of the registers to access for the send gather message instruction and the architectural register.

In Example 33, the subject matter of Examples 30-32 can optionally include wherein the plurality of individual registers are non-contiguous, and wherein the IDs comprise pointers. In Example 34, the subject matter of Examples 30-33 can optionally include wherein the send gather message instruction identifies a destination shared function ID, a function-specific encoding of an operation of the send gather message instruction, and a destination register for a writeback response to the send gather message instruction.

Example 35 is an apparatus for facilitating gathering payload from arbitrary registers for send messages in a graphics environment, comprising means for receiving, via execution circuitry of a graphics processor, a send gather message instruction identifying a number of registers to access for a send message and identifying IDs of a plurality of individual registers corresponding to the number of registers; means for decoding a first phase of the send gather message instruction; based on decoding the first phase of the send gather message instruction, means for causing a second phase of the send gather message instruction to bypass an instruction decode stage of the execution circuitry; and means for dispatching the first phase of the send gather message instruction subsequently followed by dispatch of the second phase of the send gather message instruction to a send pipeline of the execution circuitry. In Example 36, the subject matter of Example 35 can optionally include the apparatus further configured to perform the method of any one of the Examples 12 to 16.

Example 37 is at least one machine readable medium comprising a plurality of instructions that in response to being executed on a computing device, cause the computing device to carry out a method according to any one of Examples 11-16. Example 38 is an apparatus for facilitating gathering payload from arbitrary registers for send messages in a graphics environment, configured to perform the method of any one of Examples 11-16. Example 39 is an apparatus for facilitating gathering payload from arbitrary registers for send messages in a graphics environment, comprising means for performing the method of any one of

claims 11 to 16. Specifics in the Examples may be used anywhere in one or more embodiments.

Example 40 is an apparatus for facilitating gathering payload from arbitrary registers for send messages in a graphics environment comprising means for receiving a send gather message instruction identifying a number of registers to access for a send message and identifying IDs of a plurality of individual registers corresponding to the number of registers; means for performing an immediate move of the IDs of the plurality of individual registers to an architectural register of the execution circuitry; means for including a pointer to the architectural register in the send gather message instruction; and means for dispatching the send gather message instruction to a send pipeline of the execution circuitry. In Example 41, the subject matter of Example 40 can optionally include the apparatus further configured to perform the method of any one of the Examples 31 to 34.

Example 42 is at least one machine readable medium comprising a plurality of instructions that in response to being executed on a computing device, cause the computing device to carry out a method according to any one of Examples 30-34. Example 43 is an apparatus for facilitating gathering payload from arbitrary registers for send messages in a graphics environment, configured to perform the method of any one of Examples 30-34. Example 44 is an apparatus for facilitating gathering payload from arbitrary registers for send messages in a graphics environment, comprising means for performing the method of any one of claims 30 to 34. Specifics in the Examples may be used anywhere in one or more embodiments.

The foregoing description and drawings are to be regarded in an illustrative rather than a restrictive sense. Persons skilled in the art can understand that various modifications and changes may be made to the embodiments described herein without departing from the broader spirit and scope of the features set forth in the appended claims.

What is claimed is:

1. A processor comprising:

processing resources comprising execution circuitry to:

- receive a send gather message instruction identifying a number of registers to access for a send message and identifying IDs of a plurality of individual registers corresponding to the number of registers;
- decode a first phase of the send gather message instruction;
- based on decoding the first phase of the send gather message instruction, cause a second phase of the send gather message instruction to bypass an instruction decode stage of the execution circuitry; and
- dispatch the first phase of the send gather message instruction subsequently followed by dispatch of the second phase of the send gather message instruction to a send pipeline of the execution circuitry.

2. The processor of claim 1, wherein the dispatch of the first phase and the second phase of the send gather message instruction are dispatched to the send pipeline without any intervening dispatched messages between the first phase and the second phase in the send pipeline.

3. The processor of claim 1, wherein the plurality of individual registers comprise general register file (GRF) registers.

4. The processor of claim 1, wherein the second phase of the send gather message instruction comprises the IDs of the plurality of individual registers.

85

5. The processor of claim 1, wherein the first phase of the send gather message instruction identifies the number of the registers to access for the send message.

6. The processor of claim 1, wherein the plurality of individual registers are non-contiguous.

7. The processor of claim 1, wherein the first phase of the send gather message instruction identifies a destination shared function ID, a function-specific encoding of an operation of the send gather message instruction, and a destination register for a writeback response to the send gather message instruction.

8. The processor of claim 1, wherein the processor comprises a graphics processing unit (GPU).

9. The processor of claim 1, wherein the processor is at least one of a single instruction multiple data (SIMD) machine or a single instruction multiple thread (SMT) machine.

10. A method comprising:

receiving, by execution circuitry of a graphics processor, a send gather message instruction identifying a number of registers to access for a send message and identifying IDs of a plurality of individual registers corresponding to the number of registers;

decoding a first phase of the send gather message instruction;

based on decoding the first phase of the send gather message instruction, causing a second phase of the send gather message instruction to bypass an instruction decode stage of the execution circuitry; and

dispatching the first phase of the send gather message instruction subsequently followed by dispatch of the second phase of the send gather message instruction to a send pipeline of the execution circuitry.

11. The method of claim 10, wherein the dispatch of the first phase and the second phase of the send gather message instruction are dispatched to the send pipeline without any intervening dispatched messages between the first phase and the second phase in the send pipeline.

12. The method of claim 10, wherein the plurality of individual registers comprise general register file (GRF) registers.

13. The method of claim 10, wherein the first phase of the send gather message instruction identifies the number of the registers to access for the send message, and wherein the second phase of the send gather message instruction comprises the IDs of the plurality of individual registers.

14. The method of claim 10, wherein the plurality of individual registers are non-contiguous.

86

15. The method of claim 10, wherein the first phase of the send gather message instruction identifies a destination shared function ID, a function-specific encoding of an operation of the send gather message instruction, and a destination register for a writeback response to the send gather message instruction.

16. A system comprising:

a memory to store a block of data; and

a processor coupled to the memory, the processor comprising processing resources, the processing resources comprising execution circuitry to:

receive a send gather message instruction identifying a number of registers to access for a send message and identifying IDs of a plurality of individual registers corresponding to the number of registers;

decode a first phase of the send gather message instruction;

based on decoding the first phase of the send gather message instruction, cause a second phase of the send gather message instruction to bypass an instruction decode stage of the execution circuitry; and dispatch the first phase of the send gather message instruction subsequently followed by dispatch of the second phase of the send gather message instruction to a send pipeline of the execution circuitry.

17. The system of claim 16, wherein the execution circuitry is further to:

perform an immediate move of the IDs of the plurality of individual registers to an architectural register of the execution circuitry;

include a pointer to the architectural register in the send gather message instruction; and

dispatch the send gather message instruction to a send pipeline of the execution circuitry;

wherein the architectural register is a scalar register.

18. The system of claim 17, wherein the send gather message instruction identifies the number of the registers to access for the send gather message instruction and the architectural register.

19. The system of claim 16, wherein the plurality of individual registers are non-contiguous, and wherein the IDs comprise pointers.

20. The system of claim 16, wherein the send gather message instruction identifies a destination shared function ID, a function-specific encoding of an operation of the send gather message instruction, and a destination register for a writeback response to the send gather message instruction.

* * * * *